

SYLLABUS

Compiler Design (CS601PC)

UNIT - I

Introduction : Language Processors, the structure of a compiler, the science of building a compiler, programming language basics.

Lexical Analysis : The Role of the Lexical Analyzer, Input Buffering, Recognition of Tokens, The Lexical-Analyzer Generator Lex, Finite Automata, From Regular Expressions to Automata, Design of a Lexical-Analyzer Generator, Optimization of DFA-Based Pattern Matchers. **(Chapter - 1)**

UNIT - II

Syntax Analysis : Introduction, Context-Free Grammars, Writing a Grammar, Top-Down Parsing, Bottom-Up Parsing, Introduction to LR Parsing : Simple LR, More Powerful LR Parsers, Using Ambiguous Grammars, Parser Generators. **(Chapter - 2)**

UNIT - III

Syntax-Directed Translation : Syntax-Directed Definitions, Evaluation Orders for SDD's, Applications of Syntax-Directed Translation, Syntax-Directed Translation Schemes, and Implementing L-Attributed SDD's.

Intermediate-Code Generation : Variants of Syntax Trees, Three-Address Code, Types and Declarations, Type Checking, Control Flow, Back patching, Switch-Statements, Intermediate Code for Procedures. **(Chapter - 3)**

UNIT - IV

Run-Time Environments : Storage organization, Stack Allocation of Space, Access to Nonlocal Data on the Stack, Heap Management, Introduction to Garbage Collection, Introduction to Trace-Based Collection.

Code Generation : Issues in the Design of a Code Generator, The Target Language, Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, A Simple Code Generator, Peephole Optimization, Register Allocation and Assignment, Dynamic Programming Code-Generation. **(Chapter - 4)**

UNIT - V

Machine-Independent Optimizations : The Principal Sources of Optimization, Introduction to Data-Flow Analysis, Foundations of Data-Flow Analysis, Constant Propagation, Partial-Redundancy Elimination, Loops in Flow Graphs. **(Chapter - 5)**

TABLE OF CONTENTS

Unit - I

Chapter - 1 Basic Introduction and Lexical Analysis (1 - 1) to (1 - 28)

1.1	Language Processors	1 - 1
1.2	The Structure of Compiler	1 - 1
1.3	Science of Building a Compiler	1 - 5
1.4	Programming Language Basics	1 - 7
1.5	The Role of Lexical Analyzer	1 - 8
1.6	Input Buffering	1 - 10
1.7	Recognition of Tokens	1 - 11
1.8	Lexical Analyzer Generator	1 - 22
1.9	Optimization of DFA based Pattern Matchers	1 - 23
	Multiple Choice Questions for Mid Term Exam	1 - 27
	Fill in the Blanks for Mid Term Exam	1 - 27

Unit - II

Chapter - 2 Syntax Analysis (2 - 1) to (2 - 40)

2.1	Introduction to Syntax Analysis	2 - 1
2.2	Context Free Grammar	2 - 1
2.3	Writing a Grammar	2 - 3
2.4	Parsing Techniques	2 - 3
2.5	Top Down Parsing	2 - 4
2.6	Bottom Up Parsing	2 - 14
2.7	Introduction to LR Parsing	2 - 17
2.8	Using Ambiguous Grammar	2 - 34
2.9	Parser Generators	2 - 36

Multiple Choice Questions for Mid Term Exam	2 - 39
---	--------

Fill in the Blanks for Mid Term Exam	2 - 40
--	--------

Unit - III

Chapter - 3 Syntax Directed Translation and Intermediate Code Generation (3 - 1) to (3 - 32)

3.1	Syntax Directed Definition	3 - 1
3.2	Evaluation Orders for SDD	3 - 3
3.3	Applications of Syntax Directed Translation ..	3 - 6
3.4	Syntax Directed Translation Schemes	3 - 8
3.5	Implementing L Attributed SDD	3 - 10
3.6	Concept of Intermediate Code	3 - 11
3.7	Variants of Syntax Trees	3 - 12
3.8	Three Address Code	3 - 13
3.9	Types and Declarations	3 - 17
3.10	Type Checking	3 - 18
3.11	Control Flow	3 - 20
3.12	Backpatching	3 - 24
3.13	Switch Statement	3 - 30
3.14	Intermediate Code for Procedures	3 - 30

Multiple Choice Questions for Mid Term Exam	3 - 31
--	--------

Fill in the Blanks for Mid Term Exam	3 - 32
--	--------

Unit - IV

Chapter - 4 Run-Time Environment and Code Generation (4 - 1) to (4 - 32)

4.1	Storage Organization	4 - 1
4.2	Stack Allocation of Space	4 - 2

4.3	Access to Non Local Data on the Stack	4 - 5
4.4	Heap Management	4 - 7
4.5	Introduction to Garbage Collection	4 - 10
4.6	Introduction to Trace Based Collection	4 - 11
4.7	Issues in Design of Code Generator	4 - 12
4.8	The Target Language	4 - 13
4.9	Addresses in the Target Code	4 - 13
4.10	Basic Blocks and Flow Graphs	4 - 14
4.11	Optimization of Basic Blocks	4 - 15
4.12	A Simple Code Generator	4 - 21
4.13	Peephole Optimization	4 - 27
4.14	Register Allocation and Assignment	4 - 28
4.15	Dynamic Programming Method of Code Generation	4 - 29

Multiple Choice Questions for Mid Term Exam	4 - 31
Fill in the Blanks for Mid Term Exam	4 - 32

Unit - V

Chapter - 5 Machine Independent Optimization (5 - 1) to (5 - 15)

5.1	Principal Sources of Optimization	5 - 1
5.2	Introduction to Data Flow Analysis	5 - 4
5.3	Foundation Data Flow Analysis	5 - 6
5.4	Constant Propagation	5 - 6
5.5	Partial Redundancy Elimination	5 - 7
5.6	Loops in Flow Graphs	5 - 9

Multiple Choice Questions for Mid Term Exam	5 - 14
--	--------

Fill in the Blanks for Mid Term Exam	5 - 14
--	--------

Solved University Question Papers (S - 1) to (S - 9)

Solved Model Question Paper (S - 10) to (S - 11)

1

Basic Introduction and Lexical Analysis

Part I : Introduction

1.1 Language Processors

Q.1 What is translator ? Give examples of translators. [JNTU : Part A, Marks 2]

Ans. : • Translator is a kind of program that takes one form of program as input and converts it into another form.

- The input program is called **source language** and the output program is called **target language**.
- For example - There are two types of translators compiler and assembler.

Q.2 Explain the roles of linker, loader and preprocessor. [JNTU : Part B, Marks 5]

Ans. :

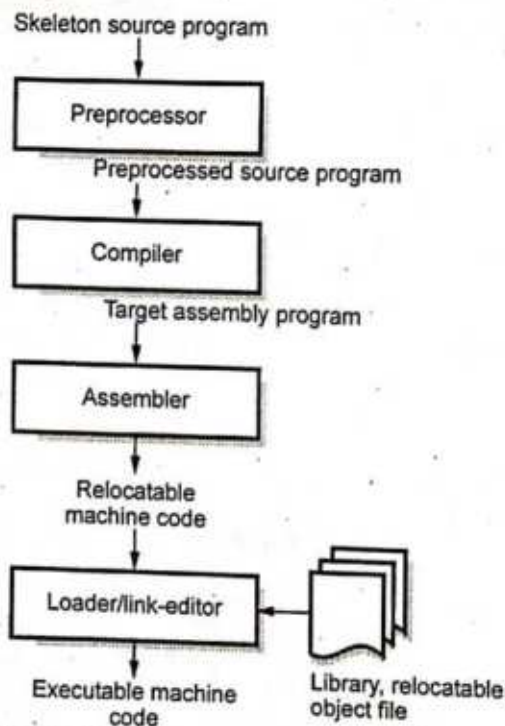


Fig. Q.2.1 (a) Process of execution of program

- The compiler takes a source program written in high level language as an input and converts it into a target assembly language. The assembler then takes this target assembly code as input and produces a relocatable machine code as an output. Then a program loader is called for performing the task of **loading and link editing**. The task of loader is to perform the relocation of an object code. **Relocation** of an object code means allocation of load time addresses which exist in the memory and placement of load time addresses and data in memory at proper locations. The link editor links the object modules and prepares a single module from several files of relocatable object modules to **resolve the mutual references**. These files may be library files and these library files may be referred by any program.

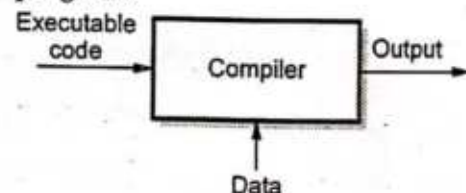


Fig. Q.2.1 (b) Process of execution of program

1.2 The Structure of Compiler

Q.3 What is compiler ? [JNTU : Part A, Marks 2]

Ans. : • Compiler is a program which takes one language (source program) as input and translates it into an equivalent another language (target program).

- During this process of translation if some errors are encountered then compiler displays them as error messages. The basic model of compiler can be represented as follows. (Refer Fig. Q.3.1)
- The compiler takes a source program as higher level languages such as C, PASCAL, FORTRAN and converts it into low level language or a machine level language such as assembly language.

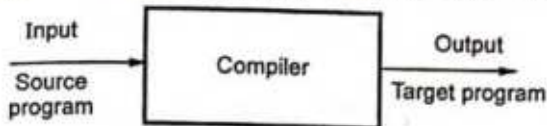


Fig. Q.3.1 Compiler

Q.4 What are two parts of a compilation ? Explain briefly. [JNTU : Part A, May-18, Marks 2]

Ans. : • The compilation is done in two parts - Analysis and Synthesis.

• In **analysis part** the source program is read and broken down into constituent pieces. The syntax and the meaning of the source string is determined and then an intermediate code is created from the input source program.

• In **synthesis part** this intermediate form of the source language is taken and converted into an equivalent target program. During this process if certain code has to be optimized for efficient execution then the required code is optimized. The analysis and synthesis model is as shown in Fig. Q.4.1.

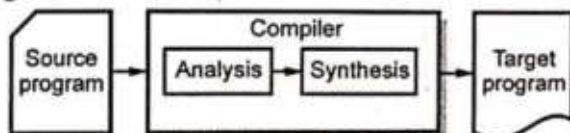


Fig. Q.4.1 Analysis and synthesis model

Q.5 Explain the various phases of a compiler with an illustrative example.

[JNTU : Part B, Nov 15, 16, Marks 10]

Ans. : 1. Lexical Analysis

- The lexical analysis is also called scanning.
- It is the phase of compilation in which the complete source code is scanned and your source program is broken up into group of strings called **token**.
- A token is a sequence of characters having a collective meaning.
- The blank characters which are used in the programming statement are eliminated during the lexical analysis phase.

2. Syntax Analysis

- The syntax analysis is also called parsing.
- In this phase the tokens generated by the lexical analyser are grouped together to form a **hierarchical structure**.
- The syntax analysis determines the structure of the source string by grouping the tokens together. The

hierarchical structure generated in this phase is called **parse tree** or **syntax tree**.

3. Semantic Analysis

- Once the syntax is checked in the syntax analyser phase the next phase i.e. the semantic analysis determines the meaning of the source string.
- For example meaning of source string means matching of parenthesis in the expression, or matching of if ...else statements or performing arithmetic operations of the expressions that are type compatible, or checking the scope of operation.

For example,

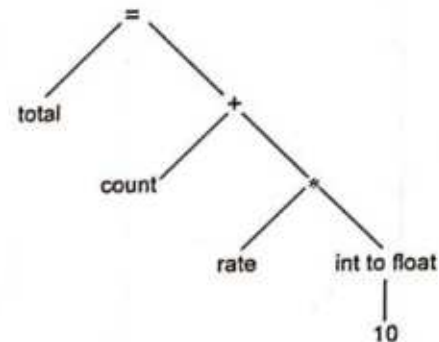


Fig. Q.5.1 Semantic analysis

- Thus these three phases are performing the task of analysis. After these phases an intermediate code gets generated.

4. Intermediate Code Generation

- The intermediate code is a kind of code which is easy to generate and this code can be easily converted to target code. This code is in variety of forms such as *three address code*, *quadruple*, *triple*, *posix*.
- Here we will consider an intermediate code in three address code form. This is like an assembly language. The **three address code** consists of instructions each of which has at the most three operands.

5. Code Optimization

- The code optimization phase attempts to improve the intermediate code.
- This is necessary to have a faster executing code or less consumption of memory.
- Thus by optimizing the code the overall running time of the target program can be improved.

Symbol table management

- To support these phases of compiler a symbol table is maintained. The task of symbol table is to store identifiers (variables) used in the program.
- The symbol table also stores information about **attributes** of each identifier. The attributes of identifiers are usually its type, its scope, information about the storage allocated for it.
- Various phases can use the symbol table in various ways. For example while doing the semantic analysis and intermediate code generation, we need to know what **type of identifiers** are. Then during code generation typically information about how much **storage is allocated** to identifier is seen.

Error detection and handling

- In compilation, each phase detects errors. These errors must be reported to error handler whose task is to handle the errors so that the compilation can proceed.

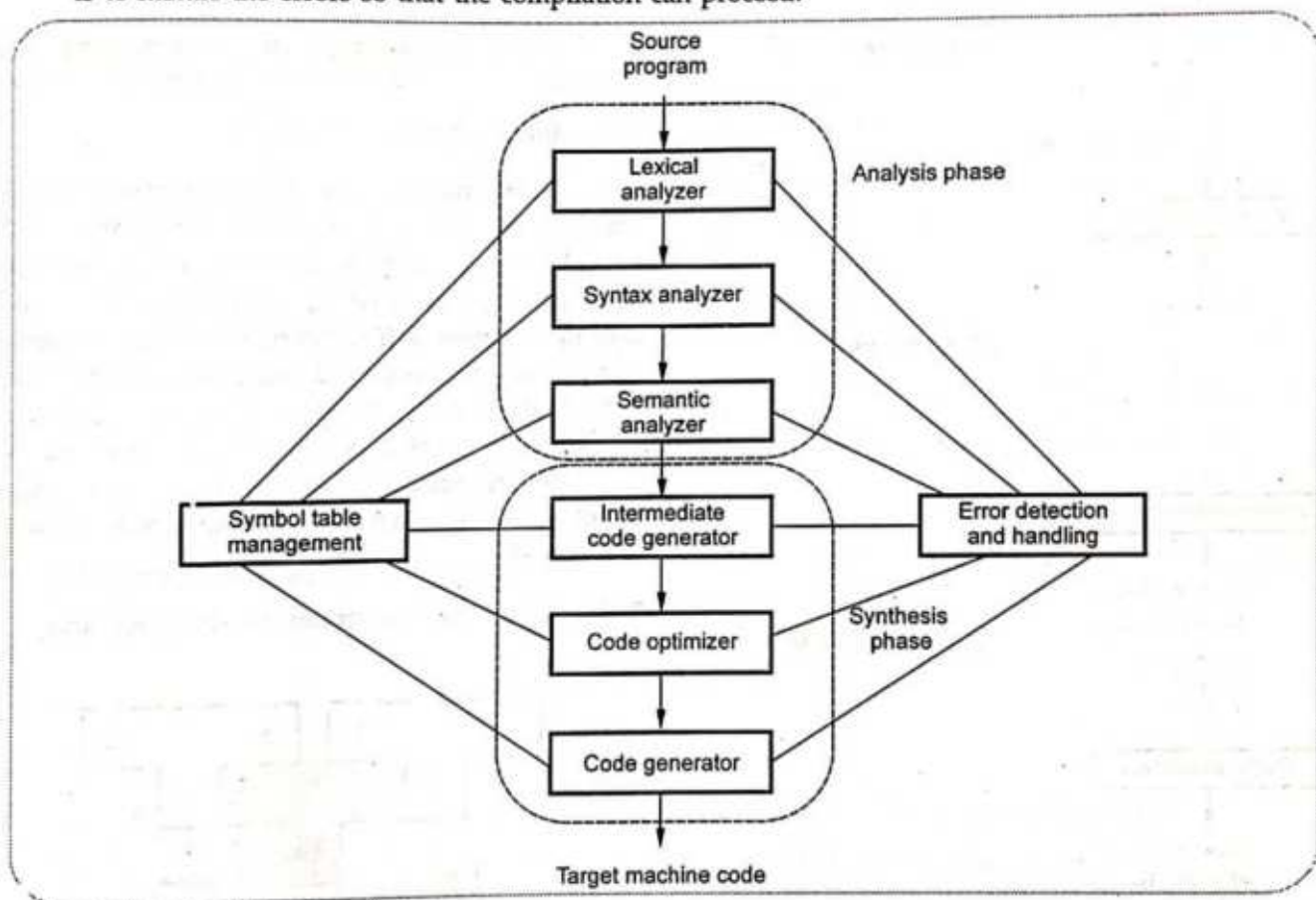


Fig. Q.5.2 Phases of compiler

- Normally, the errors are reported in the form of messages.
- Large number of errors can be detected in syntax analysis phase. Such errors are popularly called as **syntax errors**. During semantic analysis; type mismatch kind of error is usually detected.

Q.6 Define compiler. Explain brief about the syntax and semantic analysis of compiler with example.

[JNTU : Part B, March 16, Marks 5]

Ans. : Definition of compiler - Refer Q.3.

Syntax and Semantic analysis - Refer Q.6.

Q.7 Show the output generated by each phase for the following expression

$$x = (a + b) * (c + d).$$

EST [JNTU : Jan.-10, Marks 8]

Ans. :

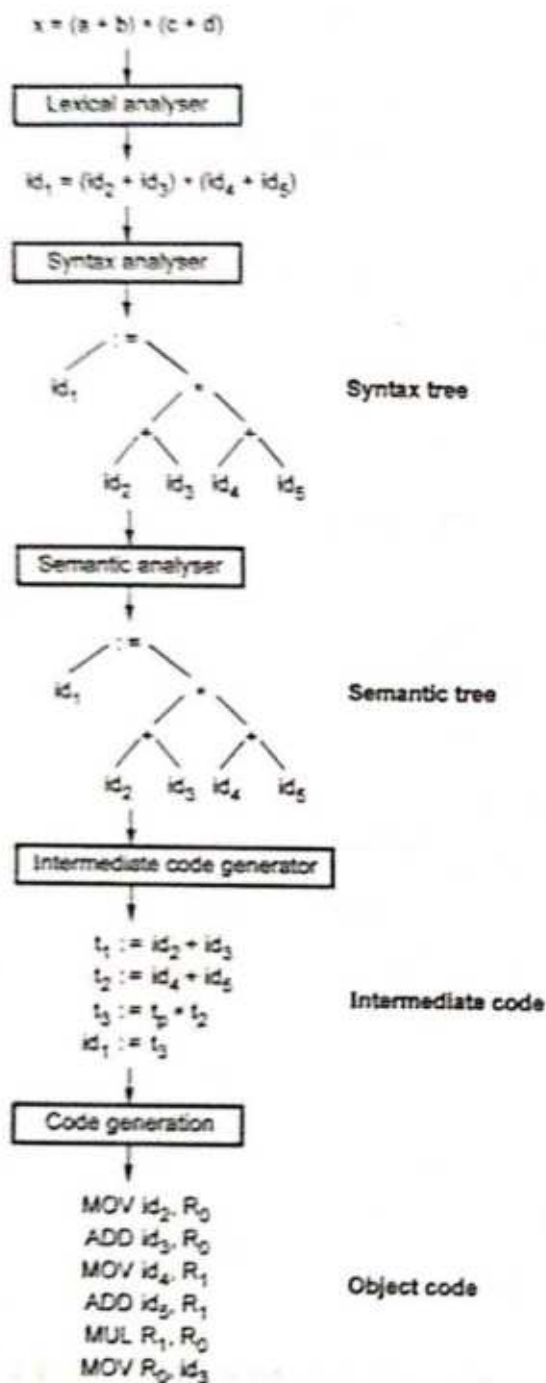


Fig. Q.7.1

The first operand represents source and second operand represents the destination.

1.3 Science of Building a Compiler

Q.8 Define bootstrapping.

EST [JNTU : Part A, March 16, Marks 2]

OR Write brief note on bootstrap process.

EST [JNTU : Part A, Dec.-16, Marks 2]

Ans. : Bootstrapping is a process in which simple language is used to translate more complicated program which in turn may handle far more complicated program. This complicated program can further handle even more complicated program and so on.

Q.9 Explain the concept of Bootstrapping with example.

EST [JNTU : Part B, March-17, Marks 4]

Ans. : Bootstrapping - Refer Q.8

Suppose we want to write a cross compiler for new language X. The implementation language of this compiler is say Y and the target code being generated is in language Z. That is, we create X_YZ . Now if existing compiler Y [i.e. compiler written in language Y] runs on machine M and generates code for M then it is denoted as Y_MM . Now if we run X_YZ using Y_MM then we get a compiler X_MMZ . That means a compiler for source language X that generates a target code in language Z and which runs on machine M.

Following diagram illustrates the above scenario.

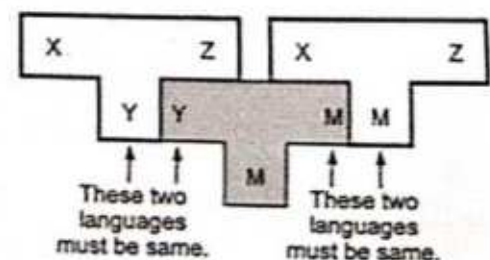
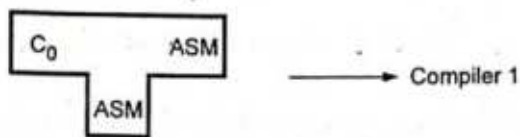


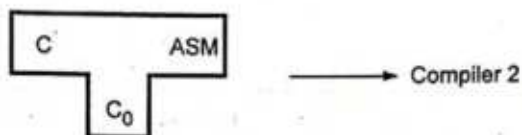
Fig. Q.9.1

Example - We can create compilers of many different forms. Now we will generate compiler which takes C language and generates an assembly language as an output with the availability of a machine of assembly language.

Step 1 : First we write a compiler for a small subset of C in assembly language.

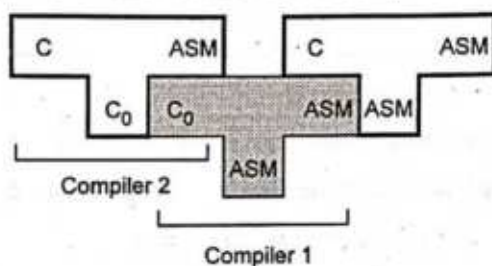


Step 2 : Then using this small subset of C i.e. C_0 , for the source language C the compiler is written.



Step 3 : Finally we compile the second compiler. Using compiler 1 the compiler 2 is compiled.

Step 4 : Thus we get a compiler written in ASM which compiles C and generates code in ASM.



Q.10 Define cross compiler.

[JNTU : Part A, Nov 15, Marks 2]

Ans. : There may be a compiler which run on one machine and produces the target code for another machine. Such a compiler is called **cross compiler**. Thus by using cross compilation technique platform independency can be achieved.

To represent cross compiler T diagram is drawn as follows

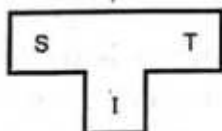


Fig. 1.10.1 (a) T diagram with S as source, T as target and I as implementation language

For source language L the target language N gets generated which runs on machine M.

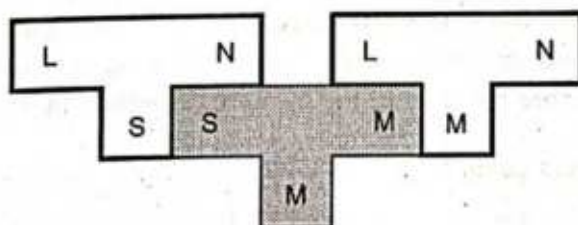


Fig. 1.10.1 (b) Cross compiler

1.4 Programming Language Basics

Q.11 What are the advantages of front end and back end model of compiler ?

Ans. :

1. By keeping the same front end and attaching different back ends one can produce a compiler for same source language on different machines.
2. By keeping different front ends and same back end one can compile several different languages on the same machine.

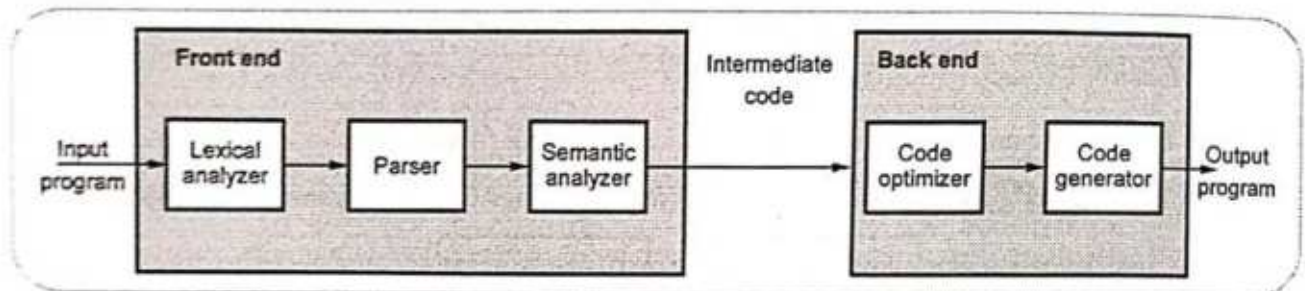


Fig. Q.11.1 Front end back end model

Q.12 What is pass of a compiler ? Explain how compiler works in passes ?

[JNTU : Part B, Marks 5]

Ans. : Definition : One complete scan of the program is called pass.

Working of Pass :

- In the first pass the source program is scanned completely and the generated output will be an easy to use form which is equivalent to the source program along with the additional information about the storage allocation.
- It is possible to leave a blank slots for missing information and fill the slot when the information becomes available. Hence there may be a requirement of more than one pass in the compilation process.
- A typical arrangement for optimizing compiler is one pass for scanning and parsing, one pass for semantic analysis and third pass for code generation and target code optimization. C and PASCAL permit one pass compilation, Modula-2 requires two passes.

Q.13 What is a pass in a compiler ? What is the effect of reducing the number of passes ?

Ans. : • One complete scan of the source language is called pass. It includes reading an input file and writing to an output file. Many phases can be grouped one pass. It is difficult to compile the source program into a single pass, because the program may have some forward references. It is desirable to have relatively few passes, because it takes time to read and write intermediate file. On the other hand if we group several phases into one pass we may be forced to keep the entire program in the memory. Therefore memory requirement may be large.

Q.14 Compare one pass and two pass compilers.

Ans. :

One pass compilers	Two pass compilers
One pass compiler scans the source program only once.	A two pass compiler does two scans over the source file. The second pass can be over a file generated in the first pass.
In this one pass Compiler has to perform some tasks such as collecting the labels, resolving forward references and doing the actual compilation.	In the first pass all it does is looks for label definitions and insert them in the symbol table. In the second pass, after the symbol table is complete, it does the actual compilation.
For example : Pascal can be implemented with single pass compiler.	For example : Java Requires multi-pass compilation.
Merit : It is faster than multi-pass compilers.	Merit : Two-pass compilers have wider scope for compilation due to two passes.
Demerit : It has limited scope for compilation.	Demerit : It is slower than one pass compilers.

Q.15 Distinguish between Pass and Phase.

[JNTU : Part A, May-13, Marks 3]

Ans. :

Phase	Pass
The processing of compilation is carried out in various steps. These steps are referred as phases. The phases of compilation are lexical analysis, syntax analysis, intermediate code generation, code generation and code optimization.	Various phases are logically grouped together to form a pass. The process of compilation can be carried out in single pass or in multiple passes.
The task of compilation is carried out in analysis and synthesis phase.	The task of compilation is carried out in single or multiple passes.
The phases namely lexical analysis, syntax analysis and semantic analysis are machine independent phases and the phases such as code generation and code optimization are machine dependant phases.	Due execution of program in passes the compilation model can be viewed as front end and back end model.

Part II : Lexical Analysis

1.5 The Role of Lexical Analyzer

Q.16 Explain the role of lexical analyzer.

[JNTU : Part A, Marks 3]

Ans. : • Lexical analyzer is the first phase of compiler.

- The lexical analyzer reads the input source program from left to right one character at a time and generates the sequence of tokens.
- Each token is a single logical cohesive unit such as identifier, keywords, operators and punctuation marks. Then the parser to determine the syntax of the source program can use these tokens.
- The role of lexical analyzer in the process of compilation is as shown below -

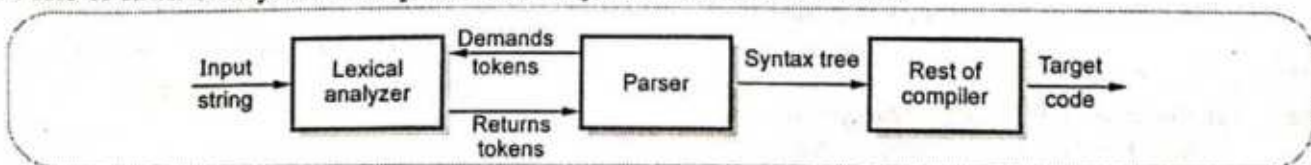


Fig. Q.16.1 Role of lexical analyzer

- As the lexical analyzer scans the source program to recognize the tokens it is also called as scanner.

Q.17 What are the features of a Lexical analyser ?

[JNTU : Part A, Dec.-17, Marks 2]

- Ans. : 1. It produces stream of tokens.
 2. It eliminates blank and comments.
 3. It generates symbol table which stores the information about identifiers, constants encountered in the input.
 4. It keeps track of line numbers.
 5. It reports the error encountered while generating the tokens.
 6. The lexical analyzer works in two phases. In first phase it performs scan and in the second phase it does lexical analysis; means it generates the series of tokens.

Q.18 Explain in detail about the role of lexical analyzer with the possible error recovery actions

[JNTU : Part B, May-18, Marks 10]

Ans. : Role of lexical analyser - Refer Q.16

Recovery actions : Various possible error recovery actions in lexical analyzer are -

- Deleting extra characters that might appear in the source statement.
- Inserting missing character
- Interchanging two adjacent characters.
- Replacing an incorrect character by the correct character.

Q.19 Define lexeme, token and pattern. Identify the lexemes that make up the tokens in the following program segment. Indicate corresponding token and pattern.

```
void swap(int a,int b)
{
    int k;
    k=a;
    a=b;
    b=k;
}
```

[JNTU : Part B, Marks 5]

Ans. :

Tokens : It describes the class or category of input string. For example, identifiers, keywords, constants are called tokens.

Patterns : Set of rules that describe the token.

Lexemes : Sequence of characters in the source program that are matched with the pattern of the token. For example, int, i, num, ans, choice.

Lexeme	Token
void	keyword
swap	identifier
(	operator
int	keyword
a	identifier
,	operator
int	keyword
b	identifier
)	operator
{	operator
int	keyword
k	identifier
k	identifier
=	operator
a	identifier
a	identifier
=	operator
b	identifier
b	identifier
=	operator
k	identifier
}	operator

Patterns

1) Identifier

- Identifier is a collection of letters.
- Identifier is a collection of alphanumeric characters.
- The first character of identifier must be a letter.

2) Operator

- Operator can be arithmetic, logical, relational operators.
- The parenthesis are considered as operators.
- Comma is treated as separation operator.
- Assignment is denoted by operator.

3) Keyword

- i) Keyword are special words to which some meaning is associated with.
- ii) int, void are keywords for denoting data types.

Q.20 Consider the following conditional statement : If($x > 3$) then $y = 5$ else $y = 10$;
How does lexical analyzer help the above statement in process of compilation ?

[JNTU : Part B, March 17, Marks 6]

Ans. : The lexical analyser will scan each programming statement and break it down into group of tokens such as identifiers, keywords, operators, constants and so on. It is as follows -

Lexeme	Token
if	keyword
(	operator
x	identifier
>	operator
3	constant
)	operator
then	keyword
y	identifier
=	operator
5	constant
else	keyword
y	identifier
=	operator
10	constant

Now we will consider encoding of tokens as follows -

Token	Code	Value
if	1	-
then	2	-
else	3	-
while	4	-
for	5	-
identifier	6	Ptr to symbol table
constant	7	Ptr to symbol table

<	8	1
<=	8	2
>	8	3
>=	8	4
=	8	5
!=	8	6
(	9	1
)	9	2
-	10	1
+	11	1
-	11	2
*	11	3
/	11	4

The symbol table for identifier and constants is as follows -

Location counter	Type	Value
100	identifier	x
101	identifier	y
103	constant	3
104	constant	5
105	constant	10

Now using hypothetical encoding we get the stream of tokens in encoded form as follows -

1, (9,1), (6,100), (8,3), 2, (6,101), (10,1), (7,104), 3, (6,101), (10,1), (7,105)

In above encoding, the value 1 indicates the if then comes the pair (9,1) in which 9 is for parenthesis and 1 is for opening bracket. Then the pair (6,100) indicates that the code 6 is for identifier and 100 is a location of identifier x in the symbol table. Continuing in this fashion, we get the above encoded form.

1.6 Input Buffering

Q.21 What is input buffering ? How is input buffering implemented ?

[JNTU : Part A, March 17, Marks 3]

Ans. : • A block of data is first read into a buffer, and then scanned by lexical analyzer. There are two

methods used in this context : one buffer scheme and two buffer scheme.

1. One buffer scheme

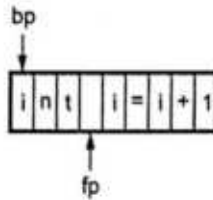


Fig. Q.21.1 One buffer scheme storing input string

In this one buffer scheme, only one buffer is used to store the input string. But the problem with this scheme is that if lexeme is very long then it crosses the buffer boundary, to scan rest of the lexeme the buffer has to be refilled, that makes overwriting the first part of lexeme.

2. Two buffer scheme

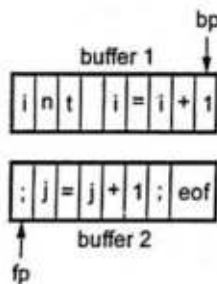


Fig. Q.21.2 Two buffer scheme storing input string

To overcome the problem of one buffer scheme, in this method two buffers are used to store the input string. The first buffer and second buffer are scanned alternately. When end of current buffer is reached the other buffer is filled. The only problem with this method is that if length of the lexeme is longer than length of the buffer then scanning input cannot be scanned completely.

- Initially both the bp and fp are pointing to the first character of first buffer.
- Then the fp moves towards right in search of end of lexeme. As soon as blank character is recognized, the string between bp and fp is identified as corresponding token.

- To identify the boundary of first buffer end of buffer character should be placed at the end of first buffer.
- Similarly end of second buffer is also recognized by the end of buffer mark present at the end of second buffer.
- When fp encounters first eof, then one can recognize end of first buffer and hence filling up of second buffer is started.
- In the same way when second eof is obtained then it indicates end of second buffer.
- Alternatively both the buffers can be filled up until end of the input program and stream of tokens is identified.
- This eof character introduced at the end is called sentinel which is used to identify the end of buffer.

1.7 Recognition of Tokens

Q.22 Define regular expression. Write about the identity rules for regular expressions.

[JNTU : Part B, Nov.-17, Marks 5]

OR Define regular expression. Explain about properties of regular expression.

[JNTU : Part B, March 16, Marks 5]

Ans. : Regular expressions are mathematical symbolisms which describe the set of strings of specific language. Here are some rules that describe definition of the regular expressions over the input set denoted by Σ .

- ϵ is a regular expression that denotes the set containing empty string.
- If R_1 and R_2 are regular expressions then $R = R_1 + R_2$ (same can also be represented as $R = R_1 | R_2$) is also regular expression which represents union operation.
- If R_1 and R_2 are regular expressions then $R = R_1.R_2$ is also a regular expression which represents concatenation operation.
- If R_1 is a regular expression then $R = R_1^*$ is also a regular expression which represents kleen closure.

Algebraic Properties :

1. $\epsilon R = R \epsilon = R$
2. $\epsilon^* = \epsilon$ ϵ is null string
3. $(\phi)^* = \epsilon$ ϕ is empty string.
4. $\phi R = R \phi = \phi$
5. $\phi + R = R$
6. $R + R = R$
7. $RR^* = R^*R = R^+$
8. $(R^*)^* = R^*$
9. $\epsilon + RR^* = R^*$
10. $(P + Q)R = PR + QR$
11. $(P + Q)^* = (P^*Q^*) = (P^* + Q^*)^*$
12. $R^*(\epsilon + R) = (\epsilon + R)R^* = R^*$
13. $(R + \epsilon)^* = R^*$
14. $\epsilon + R^* = R^*$
15. $(PQ)^*P = P(QP)^*$
16. $R^*R + R = R^*R$

Q.23 Write regular expression over alphabet {a,b,c} containing at least one 'a' and at least one 'b'.

[JNTU : Part A, March 17, Marks 2]

Ans. : Regular expression = $(a(a+b+c)^*b(a+b+c)^*c(a+b+c)^*)^+$

Q.24 Find the regular expression corresponding to given statement, subset of $\{0,1\}^*$

The language of all strings containing at least one 0 and at least one 1.

2.The language of all strings containing 0's and 1's both are even.

3.The language of all strings containing atmost one pair of consecutive 1's.

4.The language of all strings that do not end with 01.

[JNTU : Part A, Marks 4]

Ans. :

1. R.E. = $(0+1)^+$
2. R.E. = $(00+11)^*$
3. R.E. = 0^*110^*
4. R.E. = $1^* + 0^* + (10)^*$

Q.25 Write down the regular expression for the binary strings with even length.

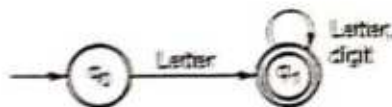
[JNTU : Part A, Marks 2]

Ans. : $(aa+ab+ba+bb)^*$

Q.26 Write a regular expression for identifier and keyword and design a transition diagram for it.

ES [JNTU : Part A, Marks 2]

Ans. : $r.e = \text{Letter} (\text{Letter} + \text{digit})^*$

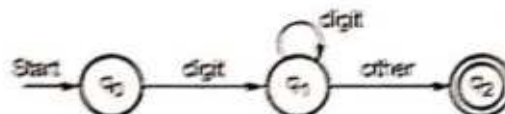


Q.27 Write a regular expression for defining constants (unsigned numbers). Design the transition graphs for them.

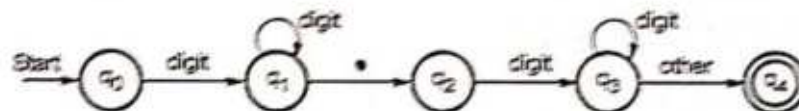
ES [JNTU : Part B, Marks 5]

Ans. : The constant can be defined by three different ways -

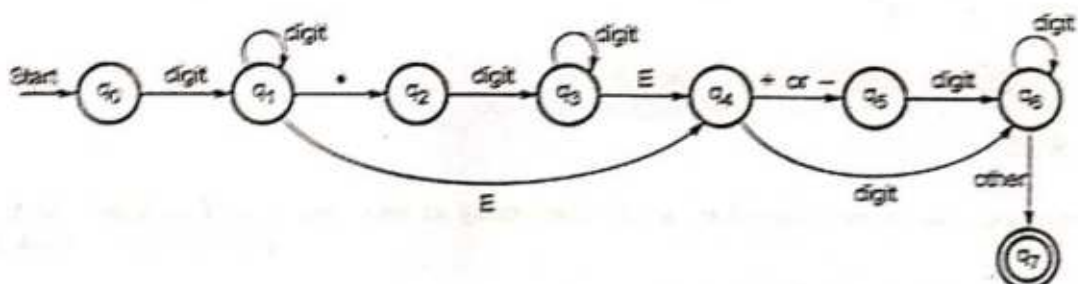
i) $r.e = \text{digit}^+$



ii) $r.e = (\text{digit})^+ \cdot (\text{digit})^+$



iii) $r.e = \text{digit}^+ (\cdot \text{digit}^+)? (E(+|-)? \text{digit}^+)?$

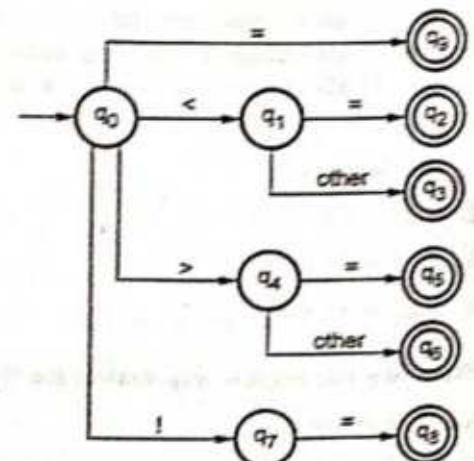


Q.28 Write a regular expression for relation operators. Design a transition diagram for them.

ES [JNTU : Part A, Marks 2]

Ans. : $r.e = (<|>|<=|>=|!=)$

During lexical analysis, the LEX program uses the regular expression and the generated lexical analyser writes the procedure or a separate function for each state of DFA. The symbols along the edges represent the input parameters for these functions. At the final state of corresponding DFA the corresponding token can be identified.



Q.29 Construct a finite automata and scanning algorithm for recognizing identifiers, numerical constants in C language.

[JNTU : Part B, Nov.-16, Marks 5]

Ans. : Refer Q.26 and Q.27.

Q.30 Specify the Thompson's constructs for building the FA from given regular expressions.

[JNTU : Part B, Marks 5]

Ans. : To construct NFA from regular expression r the Thompson's construction is used. The input string of r is parsed and following constructs are used to build an equivalent NFA.

1. For $r = \epsilon$

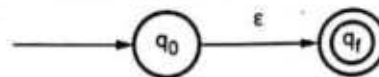


Fig. Q.30.1

2. When $r = a$ for $\Sigma = \{a\}$ the NFA is

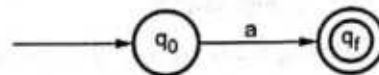


Fig. Q.30.2

3. When $r = r_1 + r_2$ then NFA can be drawn as -

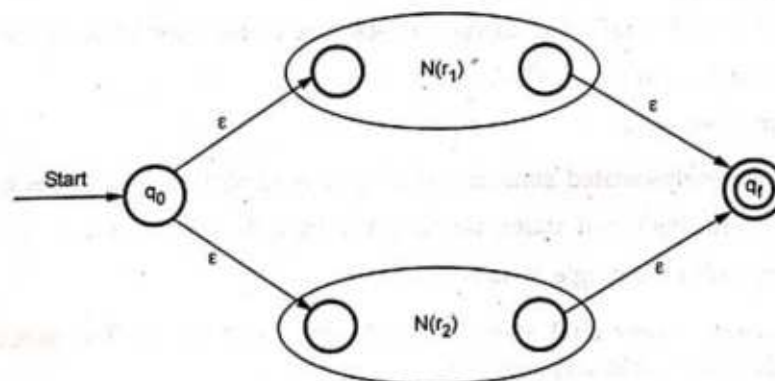


Fig. Q.30.3

Here $N(r_1)$ represents NFA for regular expression r_1 and $N(r_2)$ represents NFA for regular expression r_2 .

4. When $r = r_1 \cdot r_2$ then NFA can be drawn as -

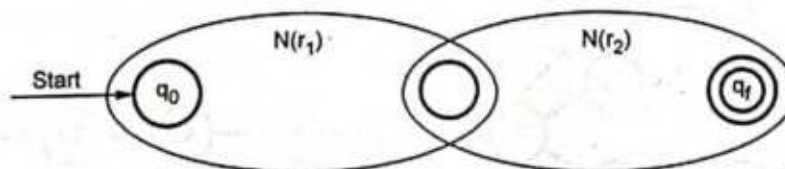


Fig. Q.30.4

5. When $r = (r_1)^*$ then the NFA can be as drawn below -

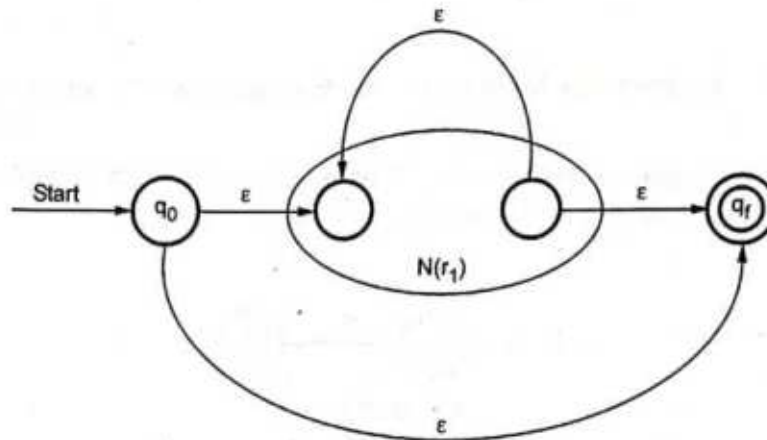


Fig. Q.30.5

Thus these constructs are useful for creating NFA equivalent to regular expression.

Q.31 Explain subset construction method for constructing DFA from NFA with an example.

[JNTU : Part B, Marks 5]

Ans. : Subset construction method :

1. Create the start state of the DFA by taking the ϵ -closure of the start state of the NFA.

2. Perform the following for the new DFA state :

For each possible input symbol :

- Apply move to the newly-created state and the input symbol; this will return a set of states.
- Apply the ϵ -closure to this set of states, possibly resulting in a new set.

This set of NFA states will be a single state in the DFA.

3. Each time we generate a new DFA state, we must apply step 2 to it. The process is complete when applying step 2 does not yield any new states.

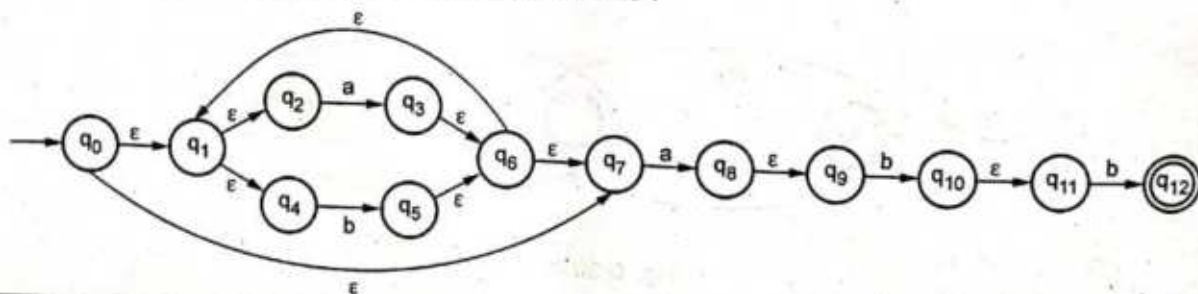
4. The finish states of the DFA are those which contain any of the finish states of the NFA.

Example - Refer Q.32.

Q.32 Construct the NFA for following Regular Expression using Thompson's construction. Apply subset construction method to convert into DFA. $(a + b)^* abb$.

[JNTU : Part B, Marks 5]

Ans. : The NFA using Thompson's construction is as follows :



$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2, q_4, q_7\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

$$\epsilon\text{-closure}(q_3) = \{q_1, q_2, q_3, q_4, q_6\}$$

$$\epsilon\text{-closure}(q_4) = \{q_4\}$$

$$\epsilon\text{-closure}(q_5) = \{q_1, q_2, q_4, q_5, q_6\}$$

$$\epsilon\text{-closure}(q_6) = \{q_1, q_2, q_6, q_7\}$$

$$\epsilon\text{-closure}(q_7) = \{q_7\}$$

$$\epsilon\text{-closure}(q_8) = \{q_8, q_9\}$$

$$\epsilon\text{-closure}(q_9) = \{q_9\}$$

$$\epsilon\text{-closure}(q_{10}) = \{q_{10}, q_{11}\}$$

$$\epsilon\text{-closure}(q_{11}) = \{q_{11}\}$$

$$\epsilon\text{-closure}(q_{12}) = \{q_{12}\}$$

Let, $\epsilon\text{-closure}(q_0) = \{q_1, q_1, q_2, q_4, q_7\}$ Call it A

$$\begin{aligned} \therefore \delta'(A, a) &= \epsilon\text{-closure}(\delta(q_0, q_1, q_2, q_4, q_7), a)) \\ &= \epsilon\text{-closure}(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) \cup \delta(q_4, a) \cup \delta(q_7, a)) \\ &= \epsilon\text{-closure}(q_3 \cup q_8) \\ &= \epsilon\text{-closure}(q_3) \cup \epsilon\text{-closure}(q_8) \\ &= \{q_1, q_2, q_3, q_4, q_6\} \cup \{q_8, q_9\} \\ &= \{q_1, q_2, q_3, q_4, q_6, q_8, q_9\} \\ &= \text{Call it as state B} \end{aligned}$$

$$\therefore \delta'(A, a) = B$$

$$\begin{aligned} \delta'(A, b) &= \epsilon\text{-closure}(\delta(q_0, q_1, q_2, q_4, q_7), b)) \\ &= \epsilon\text{-closure}(q_5) \\ &= \{q_1, q_2, q_4, q_5, q_6\} \\ &= \text{Call it as state C} \end{aligned}$$

$$\delta(A, b) = C$$

$$\begin{aligned} \delta'(B, a) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_3, q_4, q_6, q_8, q_9) a)) \\ &= \epsilon\text{-closure}(q_3) \\ &= \{q_1, q_2, q_3, q_4, q_6\} \\ &= \text{Call it as state D} \end{aligned}$$

$$\delta'(B, a) = D$$

$$\begin{aligned}
 \delta'(B, b) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_3, q_4, q_6, q_8, q_9), b)) \\
 &= \epsilon\text{-closure}(q_5 \cup q_{10}) \\
 &= \epsilon\text{-closure}(q_5) \cup \epsilon\text{-closure}(q_{10}) \\
 &= \{q_1, q_2, q_4, q_5, q_6, q_{10}, q_{11}\} \\
 &= \text{Call it as state E}
 \end{aligned}$$

$$\delta(B, b) = E$$

$$\begin{aligned}
 \delta'(C, a) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_4, q_5, q_6), a)) \\
 &= \epsilon\text{-closure}(q_3)
 \end{aligned}$$

$$\delta'(C, a) = D$$

$$\begin{aligned}
 \delta(C, b) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_4, q_5, q_6), b)) \\
 &= \epsilon\text{-closure}(q_5)
 \end{aligned}$$

$$\delta(C, a) = C$$

$$\begin{aligned}
 \delta(D, a) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_3, q_4, q_6), a)) \\
 &= \epsilon\text{-closure}(q_3)
 \end{aligned}$$

$$\delta(D, a) = D$$

$$\begin{aligned}
 \delta(D, b) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_3, q_4, q_6), b)) \\
 &= \epsilon\text{-closure}(q_5)
 \end{aligned}$$

$$\delta(D, b) = C$$

$$\begin{aligned}
 \delta(E, a) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_4, q_5, q_6, q_{10}, q_{11}), a)) \\
 &= \epsilon\text{-closure}(q_3)
 \end{aligned}$$

$$\delta(E, a) = D$$

$$\begin{aligned}
 \delta(E, b) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_4, q_5, q_6, q_{10}, q_{11}), b)) \\
 &= \epsilon\text{-closure}(q_5, q_{12}) \\
 &= \epsilon\text{-closure}(q_5) \cup \epsilon\text{-closure}(q_{12}) \\
 &= \text{Call it as state F}
 \end{aligned}$$

$$\therefore \delta(E, b) = F$$

$$\begin{aligned}
 \delta(F, a) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_4, q_5, q_6, q_{12}), a)) \\
 &= \epsilon\text{-closure}(q_3)
 \end{aligned}$$

$$\delta(F, a) = D$$

$$\begin{aligned}
 \delta(F, b) &= \epsilon\text{-closure}((\delta(q_1, q_2, q_4, q_5, q_6, q_{12}), b)) \\
 &= \epsilon\text{-closure}(q_5)
 \end{aligned}$$

$$\delta(F, b) = C$$

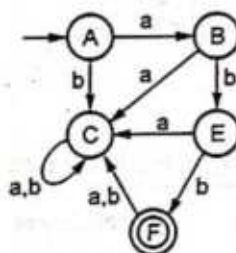
The transition table will be

State \ i/p	a	bs
A	B	C
B	D	E
C	D	C
D	D	C
E	D	F
F	D	C

State C and D are both non-final states and their input transitions are same. Hence $C = D$.

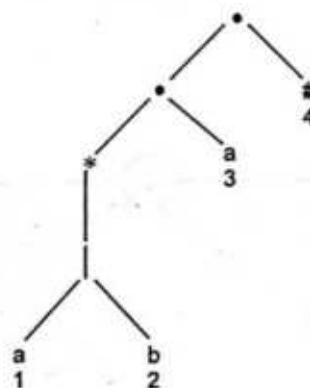
The optimized DFA will be

State \ i/p	a	b
A	B	C
B	C	E
C	C	C
E	C	F
F	C	C

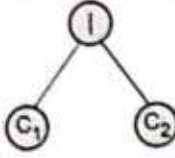
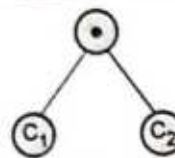


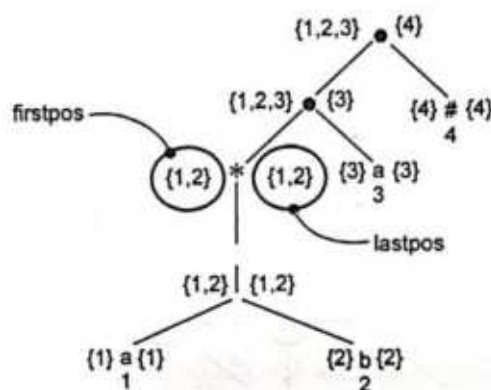
Q.33 Construct DFA for the following regular expression using syntax tree with firstpos, lastpos and followpos function. $(a|b)^*a$. [JNTU : Part B, Marks 5]

Ans. : Step 1 : We will construct syntax tree



Step 2 : We will compute nullable, firstpos and lastpos using following table.

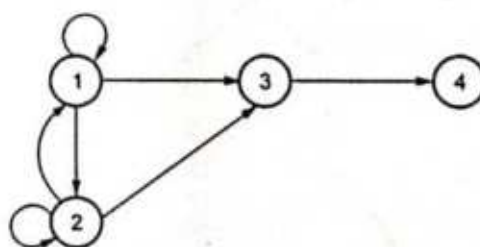
Node n	nullable (n)	firstpos (n)	lastpos (n)
Leaf ϵ	true	ϕ	ϕ
Leaf i	false	{i}	{i}
	nullable (C_1) or nullable (C_2)	firstpos (C_1) or firstpos (C_2)	lastpos (C_1) or lastpos (C_2)
	nullable (C_1) and nullable (C_2)	If nullable (C_1) then firstpos (C_1) $\cup$ firstpos (C_2) else firstpos (C_1)	If nullable (C_2) then lastpos (C_1) $\cup$ lastpos (C_2) else lastpos (C_2)
*	true	firstpos (C_1)	lastpos (C_1)



To the left we write firstpos and to the right we write lastpos. Now we will compute followpos.

Node	Followpos
1	{1, 2, 3}
2	{1, 2, 3}
3	{4}
4	-

Step 3 : The transition graph will be



Step 4 : Assume firstpos (root) = {1, 2, 3} as

State A

$$\begin{aligned} \text{Dtran [A, a]} &= \text{Followpos (1)} \cup \text{Followpos (3)} \\ &= \{1, 2, 3, 4\} \rightarrow \text{State B} \end{aligned}$$

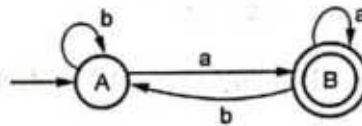
$$\begin{aligned} \text{Dtran [A, b]} &= \text{Followpos (2)} \\ &= \{1, 2, 3\} \text{ i.e. state A} \end{aligned}$$

$$\begin{aligned} \text{Dtran [B, a]} &= \text{Followpos (1)} \cup \text{Followpos (3)} \\ &= \{1, 2, 3, 4\} = \text{State B} \end{aligned}$$

$$\begin{aligned} \text{Dtran [B, b]} &= \text{Followpos (2)} \\ &= \{1, 2, 3\} \text{ i.e. state A} \end{aligned}$$

As no new state getting generated. DFA will be

The state B is a final state as it contains final state {4}.

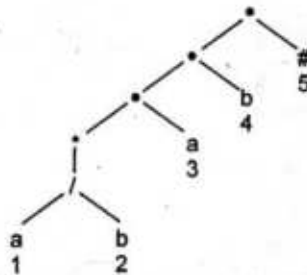


Q.34 Construct the NFA using Thompson's notation for following regular expression and then convert it to DFA.

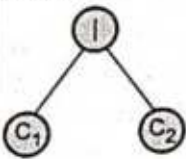
$(a/b)^*ab\#$

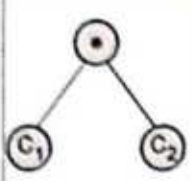
[JNTU : Part B, Marks 5]

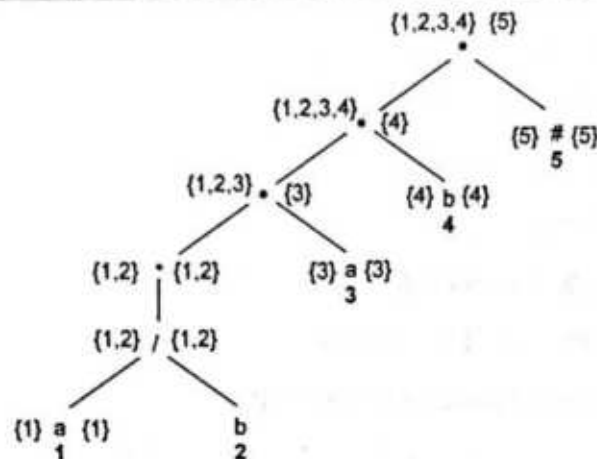
Ans. : Step 1 : We will construct syntax tree.



Step 2 : We will compute nullable, firstpos and lastpos using following table

Node n	nullable (n)	firstpos (n)	lastpos (n)
Leaf ϵ	true	ϕ	ϕ
Leaf i	false	{i}	{i}
	nullable (C_1) or nullable (C_2)	firstpos (C_1) or firstpos (C_2)	lastpos (C_1) or lastpos (C_2)

	nullable (C_1) and nullable (C_2)	If nullable (C_1) then firstpos (C_1) $\cup$ firstpos (C_2) else firstpos (C_1)	If nullable (C_2) then lastpos (C_1) $\cup$ lastpos (C_2) else lastpos (C_2)
*	true	firstpos (C_1)	lastpos (C_1)

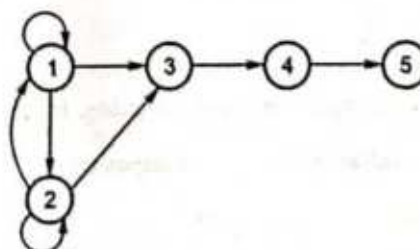


The left value written in curly bracket is firstpos and that written to the right is lastpos.

The firstpos is computed as follows.

Node	Followpos
1	{1, 2, 3}
2	{1, 2, 3}
3	{4}
4	{5}
5	-

Step 3 : The transition graph will be



Step 4 : Assume firstpos(root) = {1, 2, 3} = A

$$\begin{aligned}
 \text{Dtram [A, a]} &= \text{Followpos (1)} \cup \text{Followpos (3)} \\
 &= \{1, 2, 3\} \cup \{4\} \\
 &= \{1, 2, 3, 4\} \rightarrow \text{State B}
 \end{aligned}$$

Dtran [A, b] = Followpos (2)

= {1, 2, 3} → State A

Dtran [B, a] = B

Dtran [B, b] = Followpos (2) ∪ Followpos (4)

= {1, 2, 3} ∪ {5}

= {1, 2, 3, 4, 5} → State C

Dtran [C, a] = Followpos (3) ∪ Followpos (1)

= State B

Dtran [C, a] = B

Dtran [C, b] = Followpos (2)

Dtran [C, b] = A

The DFA will be shown in Fig. Q.34.1

The state C contains final state {5}. Hence C is a final state in DFA.

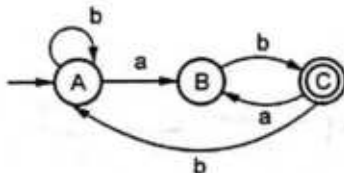


Fig. Q.34.1

1.8 Lexical Analyzer Generator

Q.35 What is LEX ? Explain it in detail.

[JNTU : Part B, Marks 5]

Ans. : • Basically LEX is a unix utility which generates the lexical analyzer.

• LEX scans the source program in order to get the stream of tokens and these tokens are related together so that various programming constructs such as expressions, block statements, procedures, control structures can be realised.

• The LEX specification file can be created using the extension .l (often pronounced as dot L). For example, the specification file can be x.l.

• This x.l file is then given to LEX compiler to produce lex.yy.c.

• This lex.yy.c is a C program which is actually a lexical analyzer program. The LEX specification file stores the regular expressions for the tokens and the lex.yy.c file consists of the tabular representation of the transition diagrams

constructed for the regular expression. The lexemes can be recognized with the help of this tabular representation of transition diagram.

- Finally the compiler compiles this generated lex.yy.c and produces an object program a.out. When some input stream is given to a.out then sequence of tokens get generated.

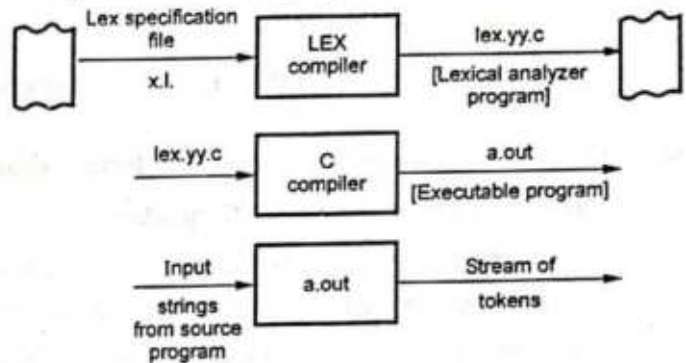


Fig. Q.35.1 Generation of lexical analyzer using LEX

Q.36 Give the structure of LEX.

[JNTU : Part B, Marks 5]

Ans. : The LEX program consists of three parts -

1. Declaration section
2. Rule section and
3. Procedure section.

```

%{
Declaration section
%}
%%
Rule section
%%
  
```

Auxiliary procedure section

- In the declaration section declaration of variable constants can be done. Some regular definitions can also be written in this section. The regular definitions are basically components of regular expressions appearing in the rule section.
- The rule section consists of regular expressions with associated actions. These translation rules can be given in the form as -

```

R1 {action1}
R2 {action2}
.
.
.
Rn {actionn}
  
```

Where each R_i is a regular expression and each action_i is a program fragment describing what action is to be taken for corresponding regular expression. These actions can be specified by piece of C code.

And third section is a auxiliary procedure section in which all the required procedures are defined.

Q.37 Define compiler. Explain in brief about the LEX compiler.

[JNTU : Part B, Nov 17, Marks 5]

Ans. : Compiler - Refer Q.3.

LEX Compiler - Refer Q.35.

1.9 Optimization of DFA based Pattern Matchers

Q.38 Explain the process of building the lexical analyzer generator.

[JNTU : Part B, Marks 5]

Ans. : Design of Lexical Analyzer Generator

- To design lexical analyzer generator, the pattern of regular expressions are designed first.
- These patterns are for recognizing various tokens from input string.
- From these patterns it is easy to design a Non-deterministic Finite Automata (NFA).
- But the simulation of DFA is easier by program. Hence we convert the NFA drawn from these patterns to DFA.

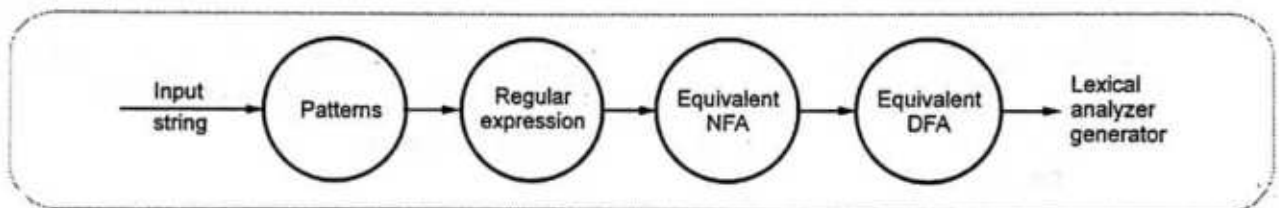


Fig. Q.38.1 Building of lexical analyzer generator

Q.39 A LEX program is given below -

Auxiliary Definitions

(none)

Translation rules

a

abb

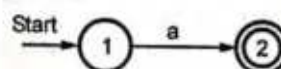
a^*b^+

Implement the LEX Program as DFA.

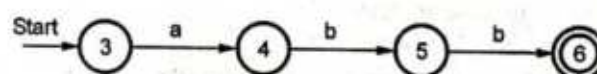
Ans. : We will implement the given LEX program as DFA in following steps -

Step 1 : For each regular expression first we will build the Finite Automata (FA).

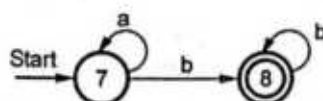
Pattern 1 :



Pattern 2 :



Pattern 3 :



Step 2 : Now we will combine all these patterns by designing NFA with ϵ .

Step 3 : The above drawn combined NFA can recognize the pattern in the given LEX Program.

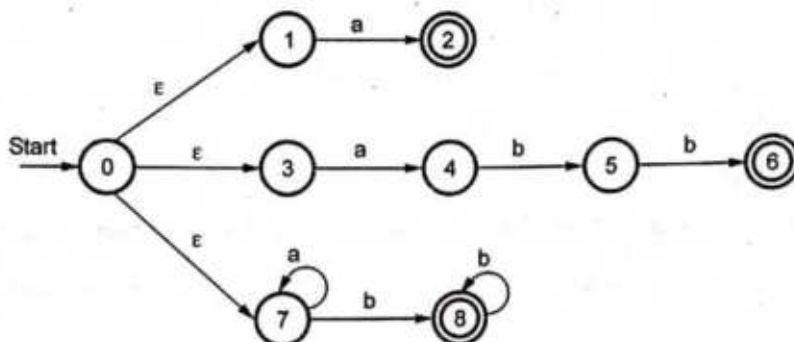
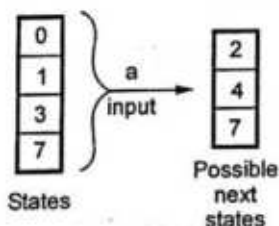


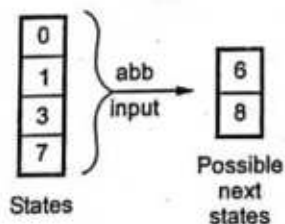
Fig. Q.39.1 Combined NFA

For instance

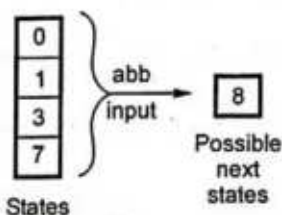
String **a** matches with pattern 1 .



String **abb** matches with pattern 2 .



String **aab** matches with pattern 3 .



Step 4 : Now we will convert the combined NFA to DFA.

First of all we will obtain

$$\epsilon\text{-closure} \{0\} = \{0, 1, 3, 7\} = [0137] \text{ new state.}$$

$$\begin{aligned} \delta([0137], a) &= (\delta(0, a) \cup \delta(1, a) \cup \delta(3, a) \cup \delta(7, a)) \\ &= (\phi \cup 2 \cup 4 \cup 7) \\ &= (2, 4, 7) \\ &= (2, 4, 7) \rightarrow \text{new state.} \end{aligned}$$

$$\begin{aligned} \delta([0137], b) &= (\delta(0, b) \cup \delta(1, b) \cup \delta(3, b) \cup \delta(7, b)) \\ &= (\phi \cup \phi \cup \phi \cup 8) \\ &= [8] \rightarrow \text{new state} \end{aligned}$$

The pattern announced for [0137] will be **none** because 0, 1, 3 or 7 all these states are non-final states.

As in above computation two new states i.e. [2, 4, 7] and [8] are formed, we will find input transitions for these states.

$$\begin{aligned} \delta([247], a) &= (\delta(2, a) \cup \delta(4, a) \cup \delta(7, a)) \\ &= \phi \cup \phi \cup 7 \\ &= [7] \rightarrow \text{new state} \end{aligned}$$

$$\begin{aligned} \delta([247], b) &= (\delta(2, b) \cup \delta(4, b) \cup \delta(7, b)) \\ &= (\phi \cup 5 \cup 8) \\ &= [58] \rightarrow \text{new state.} \end{aligned}$$

Thus we will obtain the input transitions for the new states that are getting generated. Finally the transition table can be

State \ Input	Input		Pattern recognised
	a	b	
[0137]	[247]	[8]	None
[247]	[7]	[58]	a
[8]	-	[8]	a*b ⁺
[7]	[7]	[8]	None
[58]	-	[68]	a*b ⁺
[68]	-	[8]	abb

Q.40 Write regular expressions and NFA for the following patterns. Use auxiliary definitions where convenient.

- The set of words having a,e,i,o,u appearing in that order, although not necessarily consecutively.
- Comments as in C.

[JNTU : May-09, 12, Marks 8]

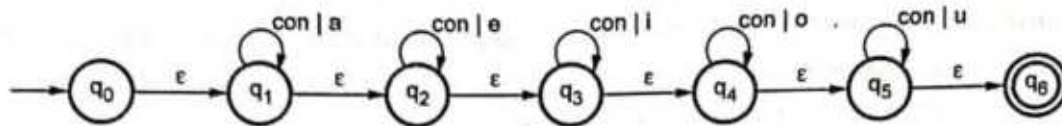
Ans. : i) The word may contain some consonant and vowels. The condition is that these vowels must appear in the order a,e,i,o,u. These vowels may be consecutive or not.

Auxiliary definition -

con [a e i o u]⁺

r.e = (con | a)^{*} (con | e)^{*} (con | i)^{*} (con | o)^{*} (con | u)^{*}

The NFA will be



ii) A single line comment is given by //.

Auxiliary definitions

letter [a - zA - Z]⁺

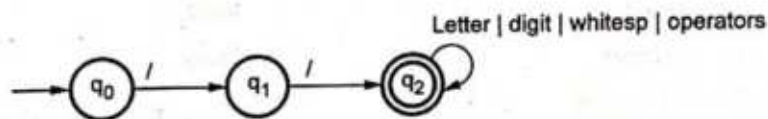
digit [0 - 9]⁺

Whitesp [\t]⁺

operators [<>=<=>=+- */]⁺

Regular expression = // (letter | digit | whitesp | operators)^{*}

The NFA will be



A multiline comment can be given by

/ *.....*/.

Auxiliary definitions

letter [a - zA - Z]⁺

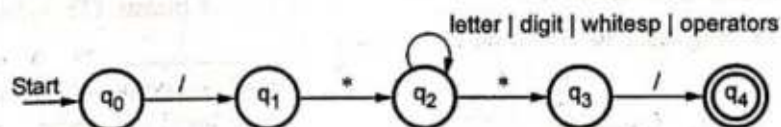
digit [0 - 9]⁺

Whitesp [\t]⁺

operators [<>=<=>=+- */]⁺

The regular expression = /*(letter | digit | whitesp | operators)^{*}*/

The NFA will be



2

Syntax Analysis

2.1 Introduction to Syntax Analysis

Q.1 Explain the term - syntax analysis.

[JNTU : Part A, Marks 2]

Ans. : Syntax analysis is a process which takes the input string w and produces a syntax tree. If there exists any error in the syntax of the programming construct then the syntax error messages are generated.

Q.2 Explain the reasons for separating lexical analysis phase from syntax analysis.

[JNTU : Part A, Marks 2]

Ans. : The lexical analyzer scans the input program and collects the tokens from it. On the other hand parser builds a parser tree using these tokens. These are two important activities and these activities are independently carried out by these two phases. Separating out these two phases has two advantages - Firstly it accelerates the process of compilation and secondly the errors in the source input can be identified precisely.

2.2 Context Free Grammar

Q.3 Define a context free grammar.

[JNTU : Part A, May-18, March-16, Marks 3]

Ans. : Definition :

The context free grammar can be formally defined as a set denoted by $G = (V, T, P, S)$ where V and T are set of non-terminals and terminals respectively. P is set of production rules, where each production rule is in the form of

non-terminal $\rightarrow$ non-terminals

or non-terminal $\rightarrow$ terminals

S is a start symbol.

For example,

$P = \{ S \rightarrow S + S$

$S \rightarrow S * S$

$S \rightarrow (S)$

$S \rightarrow 4 \}$

If the language is $4 + 4 * 4$ then we can use the production rules given by P . The start symbol is S . The number of non-terminals in the rules P is one and the only non terminal i.e. S . The terminals are $+$, $*$, $(,)$ and 4 .

We are using following conventions.

1. The capital letters are used to denote the non-terminals.
2. The lower case letters are used to denote the terminals.

Q.4 Explain concept of derivation tree.

Ans. : Let $G = (V, T, P, S)$ be a Context Free Grammar.

The derivation tree is a tree which can be constructed by following properties.

- i) The root has label S .
- ii) Every vertex can be derived from $(V \cup T \cup \epsilon)$.
- iii) If there exists a vertex A with children $R_1, R_2, \dots, R_n$ then there should be production $A \rightarrow R_1 R_2 \dots R_n$.
- iv) The leaf nodes are from set T and interior nodes are from set V .

Q.5 Explain in brief about left most and rightmost derivations.

[JNTU : Part A, Dec-17, Marks 3]

Ans. : If the leftmost non terminal in a sentential form is chosen for derivation then that derivation is called leftmost derivation.

If the rightmost non terminal in a sentential form is chosen for derivation then that derivation is called rightmost derivation.

For example - Refer Q.6.

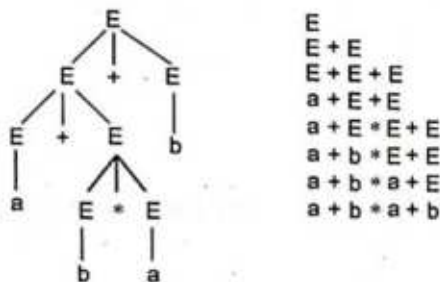
Q.6 Consider the grammar given below -

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid a \mid b$$

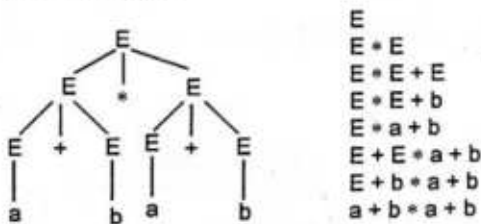
Obtain leftmost and rightmost derivation for the string $a + b * a + b$.

[JNTU : May-09, April-11, Dec.-11, Marks 8]

Ans. : Leftmost derivation



Rightmost derivation



Q.7 Consider

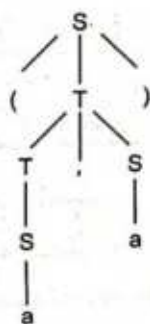
$$S \rightarrow a \mid (T)$$

$$T \rightarrow T, S \mid S$$

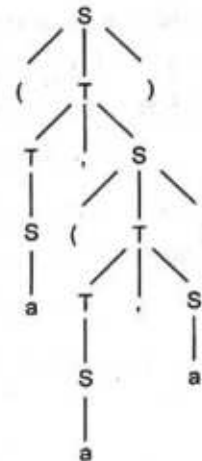
- Draw parse trees for (a,a) and $(a, (a, a))$
- Construct leftmost and rightmost derivation for (a, e) and $(a, (a, e))$
- What language does the grammar generate.

[JNTU : Dec.-11, Marks 16]

Ans. : a) i) Parse tree for (a, a) is



ii) Parse tree for $(a, (a, a))$ is



b) i) Leftmost and rightmost derivation for (a, e) we will put $e = a$.

S	S
(T)	(T)
(T, S)	(T, S)
(S, S)	(T, a)
(a, S)	(S, a)
(a, a)	(a, a)

ii) Leftmost and rightmost derivation for $(a, (a, e))$ We will put $e = a$.

S	S
(T)	(T)
(T, S)	(T, S)
(S, S)	(T, (T))
(S, (T))	(T, (T, S))
(a, (T))	(T, (T, a))
(a, (T, S))	(T, (S, a))
(a, (S, S))	(T, (a, a))
(a, (a, S))	(S, (a, a))
(a, (a, a))	(a, (a, a))

c) This grammar generates all strings with well formedness of parenthesis.

2.3 Writing a Grammar

Q.8 Write a CFG for the 'while' statement in 'C' language.

[JNTU : Jan.-10, Marks 8]

Ans. : The CFG $G = \{V, T, P, S\}$ where V is a set of nonterminals, T is a set of terminal symbols, P is a set of production rules and S is a start symbol. The set of production rules

$$\begin{aligned}
 P &= \{ S \rightarrow \text{while (condition) stmt} \\
 \text{condition} &\rightarrow \text{id relop id} \\
 \text{relop} &\rightarrow < | > | < = | > = | ! = | = = \\
 S &\rightarrow \text{while (condition) } \{ L \} \\
 L &\rightarrow \text{stmt } L | \text{stmt} \\
 &\} \\
 V &= (S, \text{condition}, \text{relop}, L); \\
 T &= (\text{while}, (,), <, >, < =, > =, ! =, = =, \text{id}, \text{stmt})
 \end{aligned}$$

Q.9 What is the string generated by the grammar $A \rightarrow (A)A$.

[JNTU : April-11, Marks 8]

Ans. : Let the production rule for the CFG is

$$A \rightarrow (A)A$$

as there is no terminal symbol generation by this grammar, we can not derive any string from it. Hence we will assume the rule to be

$$A \rightarrow (A)A$$

$$A \rightarrow () A$$

$$A \rightarrow () | \epsilon$$

The strings that will get generated will be $\{(), ()(), (()), ((()())), \dots\}$

Hence the language denoted by this grammar is for well formedness of parenthesis.

2.4 Parsing Techniques

Q.10 Explain various parsing techniques used in compiler.

[JNTU : Part B, Marks 5]

Ans. : When the parse tree can be constructed from root and expanded to leaves then such type of parser is called Top-down parser. The name itself tells us that the parse tree can be built from top to bottom.

When the parse tree can be constructed from leaves to root, then such type of parser is called as bottom-up parser. Thus the parse tree is built in bottom up manner.

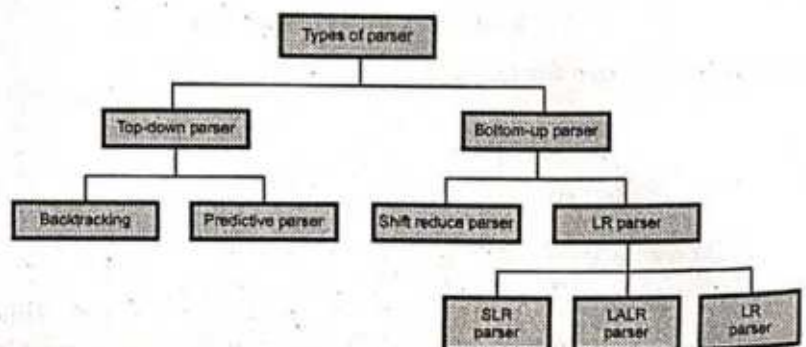


Fig. Q.10.1 Parsing techniques

Q.11 What is the difference between top down parser and bottom up parser ?

[JNTU : Part B, March-16, Marks 5]

Ans. :

Sr. No.	Top down parser	Bottom up parser
1.	Parse tree can be built from root to leaves.	Parse tree is built from leaves to root.
2.	This is simple to implement.	This complex to implement.
3.	This is less efficient parsing techniques. Various problems that occur during top down technique are ambiguity left recursion.	When the bottom up parser handles ambiguous grammar conflicts occur in parse table.
4.	It is applicable to small class of languages.	It is applicable to a broad class of languages.
5.	Various parsing techniques are 1) Recursive descent parser 2) Predictive parser.	Various parsing techniques are 1) Shift reduce 2) Operator precedence 3) LR parser.

2.5 Top Down Parsing

Q.12 What are problems in top down parsing ?

Ans. : There are certain problems in top-down parsing. In order to implement the parsing we need to eliminate these problems. These problems are -

- (1) Backtracking
- (2) Left Factoring
- (3) Left Recursion
- (4) Ambiguity.

Q.13 Write a rule of left factoring a grammar and give example.

[JNTU : Part B, Marks 5]

Ans. : In general if

$$A \rightarrow \alpha\beta_1 | \alpha\beta_2$$

is a production then it is not possible for us to take a decision whether to choose first rule or second. In such a situation the above grammar can be left factored as,

$$A \rightarrow \alpha A'$$

$$A \rightarrow \beta_1 | \beta_2$$

For example : Consider the following grammar.

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

The left factored grammar becomes,

$$S \rightarrow iEtSS' | a$$

$$S' \rightarrow eS | \epsilon$$

$$E \rightarrow b$$

Q.14 Define left recursive grammar.

[JNTU : Part A, Dec.-17, Marks 2]

Ans. : Left Recursion : The left recursive grammar is a grammar which is as given below.

$$A \xRightarrow{+} A \alpha$$

Here $\xRightarrow{+}$ means deriving the input in one or more steps.

To eliminate left recursion we need to modify the grammar. Let, G be a context free grammar having a production rule with left recursion.

$$\left. \begin{array}{l} A \rightarrow A\alpha \\ A \rightarrow \beta \end{array} \right\} \dots(1)$$

Then we eliminate left recursion by re-writing the production rule as :

$$\left. \begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \\ A' \rightarrow \epsilon \end{array} \right\} \dots(2)$$

Q.15 Eliminate left recursion for the following grammar

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

[JNTU : Part A, Nov.-15, Marks 3]

Ans. : $E \rightarrow E + T \mid T$

We can map this grammar with the rule $A \rightarrow A \alpha \mid \beta$. Then using equation (2) (in Q.14) we can say,

$$A = E$$

$$\alpha = + T$$

$$\beta = T$$

Then the rule becomes,

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

Similarly for the rule,

$$T \rightarrow T * F \mid F$$

We can eliminate left recursion as

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

The grammar for arithmetic expression can be equivalently written as -

$$E \rightarrow E + T \mid T \quad E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id \Rightarrow F \rightarrow (E) \mid id$$

Q.16 Discuss in brief about left recursion and left factoring with examples.

[JNTU : Part B, Nov.-17, Marks 5]

Ans. : Left Recursion : Refer Q.14.

Left Factoring : Refer Q.13.

Q.17 Define : Left recursive state the rule to remove left recursive from the grammar. Eliminate left recursive from following grammar.

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Ac \mid Sd \mid f$$

[JNTU : Part B, Marks 5]

Ans. : The left recursive grammar is a grammar which is as given below :

$$A \xrightarrow{+} A\alpha$$

To eliminate left recursion we apply following rule.

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2 \text{ is a rule then}$$

$$A \rightarrow \beta_1 A' \mid \beta_2 A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \epsilon$$

If we replace S by Aa|b then there exists only one rule

$$A \rightarrow Ac \mid Aad \mid bd \mid f$$

After eliminating left recursion the grammar becomes

$$A \rightarrow bdA' \mid fA'$$

$$A' \rightarrow cA' \mid adA' \mid \epsilon$$

To summarize

$$S \rightarrow Aa \mid b$$

$$A \rightarrow bdA' \mid fA'$$

$$A' \rightarrow cA' \mid adA' \mid \epsilon$$

Q.18 Check the following grammar is left recursive or not. Justify your answer. If left recursive then make grammar as non-left recursive.

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

[JNTU : Part B, Marks 5]

Ans. : The following rule is left recursive

$$L \rightarrow L, S \mid S$$

We will use following formula -

If $A \rightarrow A\alpha \mid \beta$ then $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$. Hence we can eliminate the left recursive rule using above formula

$$\begin{array}{ccc} L & \rightarrow & L, S \mid S \\ \uparrow & \uparrow & \uparrow \\ A & A & \alpha \quad \beta \end{array} \Rightarrow \begin{array}{ccc} L & \rightarrow & SL' \\ L' & \rightarrow & ,SL' \mid \epsilon \end{array}$$

Q.19 What is backtracking ? Explain it with suitable example.

[JNTU : Part B, Marks 5]

Ans. : Backtracking is a technique in which for expansion of non-terminal symbol we choose one alternative and if some mismatch occurs then we try another alternative if any.

For example :

$$S \rightarrow xPz$$

$$P \rightarrow yw \mid y$$

Then

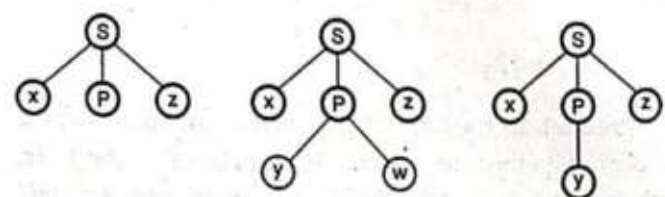


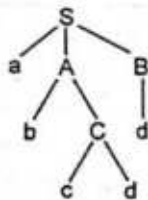
Fig. Q.19.1

If for a non-terminal there are multiple production rules beginning with the same input symbol then to get the correct derivation we need to try all these alternatives. Secondly, in backtracking we need to move some levels upward in order to check the possibilities. This increases lot of overhead in implementation of parsing. And hence it becomes necessary to eliminate the backtracking by modifying the grammar.

Q.20 Define ambiguous grammar. Check whether the grammar $S \rightarrow aAB$, $A \rightarrow bC|cd$, $C \rightarrow cd$, $B \rightarrow c/d$ is ambiguous or not. [JNTU : Part B, March-16, Marks 5]

Ans. : Definition : A grammar G is said to be ambiguous if it generates more than one parse trees for sentence of language $L(G)$.

Consider the string $abcd$. It can be derived as -

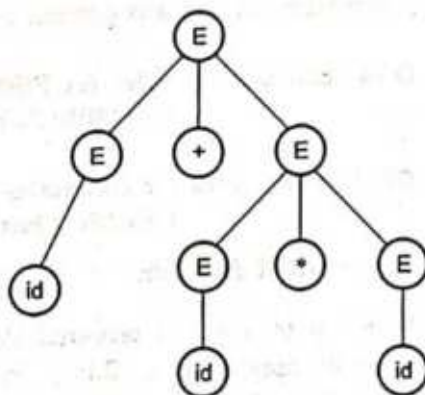


As there is only one possible parse tree that can be generated for deriving the given string, the given grammar is not ambiguous.

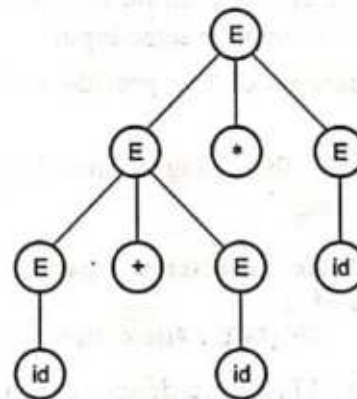
**Q.21 Consider the following grammar,
 $E \rightarrow E + E \mid E * E \mid (E) \mid id$
 Show that it is ambiguous. Eliminate the ambiguity from this grammar.**

[JNTU : May-08, 09, Aug.-08, April-09, Marks 8, Jan.-14, Marks 7]

Ans. : We will derive the string $id + id * id$.



Parse tree - 1



Parse tree - 2

Fig. Q.21.1 Parse Trees

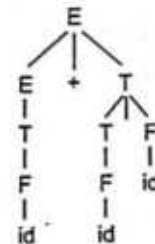
As two different parse trees can be drawn for deriving the same string. Hence given grammar is ambiguous. To eliminate ambiguity, observe that given grammar has left associative operators. Hence we will make use of left recursive production rules to eliminate ambiguity. The unambiguous grammar will be,

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

The parse tree for $id + id * id$ will be,



Q.22 Implement the following grammar using recursive descent parser.

$S \rightarrow Aa \mid bAc \mid bBa$, $A \rightarrow d$, $B \rightarrow d$

[JNTU : Part B, Marks 5]

Ans. :

Procedure $S()$

```

{
    if (lookahead = 'b')
    {
        B(); // or A() because both gives 'd'
        if (lookahead = 'c')
            match('c')
        else if (lookahead = 'a')
            match('a')
    }
}
  
```

```

    else if (lookahead = '$')
    {
        declare SUCCESS ;
    }
    else
    {
        A ( ) ;
        if (lookahead = 'a')
            match ('a') ;
    }
}
Procedure A ( )
{
    if (lookahead = 'd')
        match ('d')
    else
        error ( ) ;
}
Procedure B ( )
{
    if (lookahead = 'd')
        match ('d')
    else
        error ( ) ;
}

```

Advantages of Recursive descent parser

1. Recursive descent parsers are simple to build.
2. Recursive descent parsers can be constructed with the help of parse tree.

Limitations of Recursive descent parser

1. Recursive descent parsers are not very efficient as compared to other parsing techniques.
2. There are chances that the program for RD parser may enter in an infinite loop for some input.
3. Recursive descent parser can not provide good error messaging
4. It is difficult to parse the string if lookahead symbol is arbitrarily long.

Q.23 Explain non-recursive predictive parsers. Draw the block diagram of it.

[JNTU : Part B, Marks 5]

Ans. : • The predictive LL(1) top-down parsing algorithm is of non-recursive type.

- In this type of parsing a table is built.

- For LL(1) - the first L means the input is scanned from left to right. The second L means it uses **leftmost derivation** for input string. And the number 1 in the input symbol means it uses only one input symbol (lookahead) to predict the parsing process.
- The simple block diagram for LL(1) parser is as shown in Fig. Q.23.1.

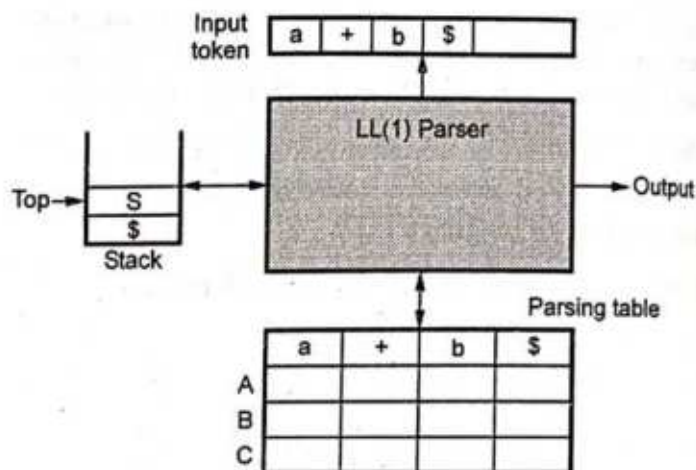


Fig. Q.23.1 Model for LL(1) parser

- The data structures used by LL(1) are i) input buffer ii) stack iii) parsing table.
- The LL(1) parser uses **input buffer** to store the input tokens. The **stack** is used to hold the left sentential form. The symbols in R.H.S. of rule are pushed into the stack in **reverse order** i.e. from right to left. Thus use of stack makes this algorithm non-recursive. The **table** is basically a two dimensional array. The table has row for non-terminal and column for terminals. The table can be represented as $M[A, a]$ where A is a non-terminal and a is current input symbol.

Q.24 List out the rules for FIRST and FOLLOW.

[JNTU : Part A, Dec.-17, Marks 3]

OR List the rules for computing FOLLOW set.

[JNTU : Part A, Nov.-15, Marks 2]

Ans. : FIRST function

$FIRST(\alpha)$ is a set of terminal symbols that are first symbols appearing at R.H.S. in derivation of α . If $\alpha \Rightarrow \epsilon$ then ϵ is also in $FIRST(\alpha)$.

Following are the rules used to compute the FIRST functions.

1. If the terminal symbol a the $\text{FIRST}(a) = \{a\}$.
2. If there is a rule $X \rightarrow \epsilon$ then $\text{FIRST}(X) = \{\epsilon\}$.
3. For the rule $A \rightarrow X_1 X_2 X_3 \dots X_k$ $\text{FIRST}(A) = (\text{FIRST}(X_1) \cup \text{FIRST}(X_2) \cup \text{FIRST}(X_3) \dots \cup \text{FIRST}(X_k))$.
Where $k \leq n$ such that $1 \leq j \leq k-1$.

FOLLOW function

$\text{FOLLOW}(A)$ is defined as the set of terminal symbols that appear immediately to the right of A . In other words

$\text{FOLLOW}(A) = \{ a \mid S \Rightarrow^* \alpha A a \beta \text{ where } \alpha \text{ and } \beta \text{ are some grammar symbols may be terminal or non-terminal.} \}$

The rules for computing FOLLOW function are as given below -

1. For the start symbol S place $\$$ in $\text{FOLLOW}(S)$.
2. If there is a production $A \rightarrow \alpha B \beta$ then everything in $\text{FIRST}(\beta)$ without ϵ is to be placed in $\text{FOLLOW}(B)$.
3. If there is a production $A \rightarrow \alpha B \beta$ or $A \rightarrow \alpha B$ and $\text{FIRST}(\beta) = \{\epsilon\}$ then $\text{FOLLOW}(A) = \text{FOLLOW}(B)$ or $\text{FOLLOW}(B) = \text{FOLLOW}(A)$. That means everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

Q.25 Construct FIRST and FOLLOW for the grammar.

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

[JNTU : Part B, Nov.-17, Marks 5]

Ans. :

$\text{FIRST}(E) = \{ (, \text{id} \}$

$\text{FIRST}(T) = \{ (, \text{id} \}$

$\text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FOLLOW}(E) = \{ \$,), + \}$

$\text{FOLLOW}(T) = \{ \$,), +, *, \}$

$\text{FOLLOW}(F) = \{ \$,), +, *, \}$

Q.26 Construct predictive parsing table for grammar.

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

[JNTU : Part B, May-16, Marks 5]

Ans. : The given grammar contains left recursion. We will eliminate left recursion first. Refer Q.15 for elimination of left recursion from given grammar. On eliminating left recursion we get -

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

First create the table as follows.

	id	+	*	(	)	\$
E						
E'						
T						
T'						
F						

Now we will fill up the entries in the table using the above given algorithm. For that consider each rule one by one.

$$E \rightarrow TE'$$

$$A \rightarrow \alpha$$

$$A = E, \alpha = TE'$$

$$\text{FIRST}(TE') \text{ if } E' = \epsilon \text{ then } \text{FIRST}(T) = \{(, id\}$$

$$M[E, (] = E \rightarrow TE'$$

$$M[E, id] = E \rightarrow TE'$$

$$E' \rightarrow +TE'$$

$$A \rightarrow \alpha$$

$$A = E', \alpha = +TE'$$

$$\text{FIRST}(+TE') = \{+\}$$

$$\text{Hence } M[E, +] = E' \rightarrow +TE'$$

$$E' \rightarrow \epsilon$$

$$A \rightarrow \alpha$$

$$A = E', \alpha = \epsilon \text{ then}$$

$$\text{FOLLOW}(E') = \{), \$\}$$

$$M[E',)] = E' \rightarrow \epsilon$$

$$M[E, \$] = E' \rightarrow \epsilon$$

$$T \rightarrow FT'$$

$$A \rightarrow \alpha$$

$$A = E', \alpha = FT'$$

$$\text{FIRST}(FT') = \text{FIRST}(F) = \{(, id\}$$

$$\text{Hence } M[F, (] = T \rightarrow FT'$$

$$\text{And } M[F, id] = T \rightarrow FT'$$

$$T \rightarrow *FT'$$

$$A \rightarrow \alpha$$

$$A = T, \alpha = *FT'$$

$$\text{FIRST}(*FT') = \{*\}$$

$$\text{Hence } M[T, *] = T \rightarrow *FT'$$

$$T' \rightarrow \epsilon$$

$$A \rightarrow \alpha$$

$$A = T', \alpha = \epsilon$$

$$\text{FOLLOW}(T') = \{+,), \$\}$$

$$\text{Hence } M[T', +] = T' \rightarrow \epsilon$$

$$M[T',)] = T' \rightarrow \epsilon$$

$$M[T', \$] = T' \rightarrow \epsilon$$

$$F \rightarrow (E)$$

$$A \rightarrow \alpha$$

$$A = F, \alpha = (E)$$

$$\text{FIRST}((E)) = \{(\}$$

$$\text{Hence } M[F, (] = F \rightarrow (E)$$

$F \rightarrow id$

$A \rightarrow \alpha$

$A = F, \alpha = id$

$FIRST(id) = \{ id \}$

Hence $M[F, id] = F \rightarrow id$

The complete table can be as shown below.

	id	+	*	(	)	\$
E	$E \rightarrow TE'$	Error	Error	$E \rightarrow TE'$	Error	Error
E'	Error	$E \rightarrow +TE'$	Error	Error	$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$E \rightarrow FT'$	Error	Error	$T' \rightarrow FT'$	Error	Error
T'	Error	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	Error	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$	Error	Error	$F \rightarrow (E)$	Error	Error

Now the input string $id + id * id \$$ can be parsed using above table. At the initial configuration the stack will contain start symbol E, in the input buffer input string is placed.

Stack	Input	Action
\$E	$id + id * id \$$	

Now symbol E is at top of the stack and input pointer is at first id, hence $M[E, id]$ is referred. This entry tells us $E \rightarrow TE'$, so we will push E' first then T.

Stack	Input	Action
$\$E' T$	$id + id * id \$$	$E \rightarrow TE'$
$\$E' T' F$	$id + id * id \$$	$E \rightarrow FT'$
$\$E' T' id$	$id + id * id \$$	$F \rightarrow id$
$\$E' T'$	$+ id * id \$$	
$\$E'$	$+ id * id \$$	$T' \rightarrow \epsilon$
$\$E' T+$	$+ id * id \$$	$E' \rightarrow +TE'$
$\$E' T$	$id * id \$$	
$\$E' T' F$	$id * id \$$	$T \rightarrow FT'$
$\$E' T' id$	$id * id \$$	$F \rightarrow id$
$\$E' T'$	$* id \$$	
$\$E' T' F*$	$* id \$$	$T \rightarrow *FT'$
$\$E' T' F$	$id \$$	
$\$E' T' id$	$id \$$	$F \rightarrow id$
$\$E' T'$	$\$$	
$\$E'$	$\$$	$T \rightarrow \epsilon$
$\$$	$\$$	$E' \rightarrow \epsilon$

Q.27 Construct predictive parsing table for the following grammar :

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S$

Check whether the (a,a) belong to that grammar or not.

[JNTU : Part B, Nov.-15, March-17, May-18, Marks 10]

Ans. : As the given grammar is left recursive because of

$$L \rightarrow L, S \mid S$$

We will first eliminate left recursion. As $A \rightarrow A \alpha \mid \beta$ can be converted as

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

We can write $L \rightarrow L, S \mid S$ as

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid \epsilon$$

Now the grammar taken for predictive parsing is -

$$S \rightarrow (L) \mid a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid \epsilon$$

Now we will compute FIRST and FOLLOW of non-terminals

$$\therefore \text{FIRST}(S) = \{ (, a \}$$

$$\text{FIRST}(L) = \{ (, a \}$$

$$\text{FIRST}(L') = \{ ', \epsilon \}$$

$$\text{FOLLOW}(S) = \{ ',), \$ \}$$

$$\text{FOLLOW}(L) = \text{FOLLOW}(L') = \{) \}$$

The predictive parsing table can be constructed as

	a	(	)	'	\$
S	$S \rightarrow a$	$S \rightarrow (L)$			
L	$L \rightarrow SL'$	$L \rightarrow SL'$			
L'			$L' \rightarrow \epsilon$	$L' \rightarrow , SL'$	

We will parse the string (a, a) using that table shown below.

State	Input	Actions
\$ S	(a, a) \$	
\$) L (	(a, a) \$	$S \rightarrow (L)$
\$) L	a, a) \$	$L \rightarrow SL'$
\$) L' S	a, a) \$	$S \rightarrow a$
\$) L' a	a, a) \$	
\$) L'	, a) \$	$L' \rightarrow , SL'$
\$) L' S,	, a) \$	
\$) L' S	a) \$	$S \rightarrow a$
\$) L' a	a) \$	
\$) L'	) \$	$L' \rightarrow \epsilon$
\$)	) \$	
\$	\$	Accept

Q.28 Show that following grammar :

$$S \rightarrow AaAb \mid BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

is LL (1).

[JNTU : Part B, May-13, Marks 8]

Ans. : Consider the grammar :

$$S \rightarrow AaAb$$

$$S \rightarrow BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

Now we will compute FIRST and FOLLOW functions.

$$\text{FIRST}(S) = \{ a, b \}$$

if we put

$$S \rightarrow AaAb$$

$$S \rightarrow aAb \quad \text{When } A \rightarrow \epsilon$$

Also $S \rightarrow BbBa$

$$S \rightarrow bBa \quad \text{When } B \rightarrow \epsilon$$

$$\text{FIRST}(A) = \text{FIRST}(B) = \{\epsilon\}$$

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \text{FOLLOW}(B) = \{a, b\}$$

The LL(1) parsing table is

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

Now consider the string "ba". For parsing -

Stack	Input	Action
\$\$	ba\$	$S \rightarrow BbBa$
\$aBbB	ba\$	$B \rightarrow \epsilon$
\$aBb	ba\$	
\$aB	a\$	$B \rightarrow \epsilon$
\$a	a\$	
\$	\$	Accept

This shows that the given grammar is LL(1)

Q.29 Consider the grammar

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

Whether it is LL(1) grammar. Give the explanation whether (i, t, e, b, a) are terminal symbols.

[JNTU : Part B, Dec.-11, Marks 16]

Ans. : Let the given grammar will be,

$$S \rightarrow iCtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

Now we will compute FIRST and FOLLOW for given nonterminals.

$$\text{FIRST}(S) = \{i, a\}$$

$$\text{FOLLOW}(S) = \{e, \$ \}$$

$$\text{FIRST}(S') = \{e, \epsilon\}$$

$$\text{FOLLOW}(S') = \{e, \$ \}$$

$$\text{FIRST}(E) = \{b\}$$

$$\text{FOLLOW}(E) = \{t\}$$

The predictive parsing table will be

	a	b	e	t	i	\$
S	$S \rightarrow a$				$S \rightarrow iEtSS'$	
S'			$S' \rightarrow eS$ $S' \rightarrow \epsilon$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

As we have got multiple entries in $M[A, e]$ given grammar is not LL(1) grammar.

The (i, t, e, a, b) are the terminal symbols because they do not derive any production rule.

Q.30 For the following grammar.

$$D \rightarrow TL;$$

$$L \rightarrow L, id \mid id$$

$$T \rightarrow int \mid float$$

1) Remove left recursion (if required)

2) Find first and follow for each non-terminal for resultant grammar

3) Construct LL (1) Parsing table

4) Parse the following String (Show Stack actions Clearly) and drawe parse Parse tree for the input :
int id, id; [JNTU : Part B, Marks 5]

Ans. : 1) Formula for removing the left recursion is as follows :

if $A \rightarrow A\alpha \mid \beta$ is the rule. Then it can be converted to

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

$D \rightarrow TL;$	No action as the rule is not left recursive.
$L \rightarrow L, id \mid id$	$L \rightarrow id L'$ $L' \rightarrow, id L'$ $L' \rightarrow \epsilon$
$T \rightarrow int \mid Float$	No action as the rule rule has no left recursion.

2) The first and follow functions :

FIRST (D) = {int, float}

FIRST (L) = {id}

FIRST (L') = {, , ε}

FIRST (T) = {int, float}

FOLLOW (D) = {\$}

FOLLOW (L) = {;}

FOLLOW (L') = {;}

FOLLOW (T) = {id}

3) Construction of Parsing table :

	id	,	int	float	;	\$
D			D → TL;	D → TL;		
L	L → idL'					
L'		L' → , idL'			L' → ε	
T			T → int	T → float		

4) Parsing of string int id, id;

Stack	Input	Action
SD	int id,id,\$	D → TL
S;LT	int id,id,\$	T → int
S;Lint	int id,id,\$	
S;L	id,id,\$	L → id L'
S;L'id	id,id,\$	
S;L'	, id,\$	L' → , id L'
S;L'id,	, id,\$	
S;L'id	id,\$	
S;L'	;\$	L' → ε
\$	;\$	
\$	\$	ACCEPT

Q.31 Verify the following grammar is LL(1) :

$S \rightarrow S \#$

$S \rightarrow A|B$

$A \rightarrow a|\epsilon$

$B \rightarrow b|\epsilon$

Give a trace of the parser for each of the following input strings

a) aab #

b) ccd #

Ans. : Let

$S \rightarrow S \# | A | B$

$A \rightarrow a | \epsilon$

$B \rightarrow b | \epsilon$

be the given grammar. We will eliminate left recursion.

$A \rightarrow A \alpha | B_1 | B_2, \dots \Rightarrow A \rightarrow \beta_1 A' | \beta_2 A' | \dots$

[JNTU : Part B, May-12, Marks 15]

$A \rightarrow \alpha A' | \epsilon$
 $\therefore S \rightarrow S \# | A | B$ becomes $S \rightarrow AS' | BS'$
 $S' \rightarrow \# S' | \epsilon$

Hence we get,

$S \rightarrow AS' | BS'$
 $S' \rightarrow \# S' | \epsilon$
 $A \rightarrow a | \epsilon$
 $B \rightarrow b | \epsilon$

We will now compute FIRST and FOLLOW for each non terminal symbol.

FIRST (S) = {a, b, #} FOLLOW (S) = {\$}
 FIRST (S') = {#, ε} FOLLOW (S') = {\$}
 FIRST (A) = {a, ε} FOLLOW (A) = {#, \$}
 FIRST (B) = {b, ε} FOLLOW (B) = {#, \$}

The parsing table will be

	a	b	#	\$
S	$S \rightarrow AS'$	$S \rightarrow BS'$		
S'			$S' \rightarrow \# S'$	$S' \rightarrow \epsilon$
A	$A \rightarrow a$			$A \rightarrow \epsilon$
B		$B \rightarrow b$		$B \rightarrow \epsilon$

a) Simulation of aab #

State	Input	Actions
\$ S	aab # \$	$S \rightarrow AS'$
\$ S'A	aab #	$A \rightarrow a$
\$ S'a	aab #	
\$ S'	ab #	No action defined

Hence aab # does not belong to this grammar.

b) As c, d are not generated by any of the non terminal symbols. This string can not be derived from given grammar.

Q.32 Construct LL (1) parse table for the following grammar.

$S \rightarrow Aa | bAc | Bc | bBa$

$A \rightarrow d$

$B \rightarrow d$ [JNTU : Part B : Dec.-12, Marks 8]

Ans. : The FIRST and FOLLOWS can be as follows

Non-terminal	FIRST	FOLLOWS
S	{b, d}	{}
A	{d}	{a, c}
B	{d}	{a, c}

The predictive parsing table will be,

	a	b	c	d	\$
S		$S \rightarrow b A c$ $S \rightarrow b B a$		$S \rightarrow A a$ $S \rightarrow B c$	
A				$A \rightarrow d$	
B				$B \rightarrow d$	

2.6 Bottom Up Parsing

Q.33 What are the actions performed by shift reduce parser ? [JNTU : Part A, March-16, Marks 2]

Ans. : Shift reduce parser attempts to construct parse tree from leaves to root. Thus it works on the same principle of bottom up parser. A shift reduce parser requires following data structures.

1. The input buffer storing the input string.
2. A stack for storing and accessing the L.H.S. and R.H.S. of rules.

The initial configuration of Shift reduce parser is as shown in Fig. Q.33.1

The parser performs following basic operations.

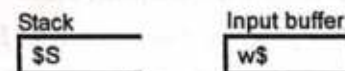


Fig. Q.33.1 Initial configuration

1. **Shift** : Moving of the symbols from input buffer onto the stack, this action is called shift.
2. **Reduce** : If the handle appears on the top of the stack then reduction of it by appropriate rule is done. That means R.H.S. of rule is popped of and L.H.S. is pushed in. This action is called Reduce action.
3. **Accept** : If the stack contains start symbol only and input buffer is empty at the same time then that action is called accept. When accept state is obtained in the process of parsing then it means a successful parsing is done.

4. Error : A situation in which parser cannot either shift or reduce the symbols, it cannot even perform the accept action is called as error.

Q.34 Consider the grammar

$E \rightarrow E - E$

$E \rightarrow E * E$

$E \rightarrow id$

Perform Shift-Reduce parsing of the input string "id1-id2*id3".

[JNTU : Part B, Marks 5]

Ans. :

Stack	Input buffer	Parsing action
\$	id1-id2*id3\$	Shift
\$id1	-id2*id3\$	Reduce by $E \rightarrow id$
\$E	-id2*id3\$	Shift
\$E-	id2*id3\$	Shift
\$E-id2	*id3\$	Reduce by $E \rightarrow id$
\$E-E	*id3\$	Shift
\$E-E*	id3\$	Shift
\$E-E*id3	\$	Reduce by $E \rightarrow id$
\$E-E*E	\$	Reduce by $E \rightarrow E*E$
\$E-E	\$	Reduce by $E \rightarrow E-E$
\$E	\$	Accept

Here we have followed two rules.

1. If the incoming operator has more priority than in stack operator then perform shift.
2. If in stack operator has same or less priority than the priority of incoming operator then perform reduce.

Q.35 Define handle and handle pruning. Explain the stack implementation of shift reduce parser with the help of example. [JNTU : Part B, Marks 5]

Ans. : Handle : Handle is a string of substring that matches the right side of the production and we can reduce such a string by non-terminal on left hand side production. Formally it can be defined as -

Defination of Handle : "Handle of right sentential form γ is a production $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right sentential form in rightmost derivation of γ ".

Handle Pruning : Handle pruning is a process of detecting the handles and using them in reduction.

For example :

Consider the grammar

$E \rightarrow E+E$

$E \rightarrow id$

Now consider the string $id + id + id$ and the rightmost derivation is

$E \Rightarrow E + E$
rm

$E \Rightarrow E + E + E$
rm

$E \Rightarrow E + \textcircled{E + id}$
rm

$E \Rightarrow E + id + \textcircled{id}$
rm

$E \Rightarrow id + id + id$
rm

The bold strings are called handles.

Right sentential form	Handle	Production
$id + id + id$	id	$E \rightarrow id$
$E + id + id$	id	$E \rightarrow id$
$E + E + id$	id	$E \rightarrow id$
$E + E + E$	$E + E$	$E \rightarrow E + E$
$E + E$	$E + E$	$E \rightarrow E + E$
E		

Shift Reduce Parsing - Refer Q.33.

Q.36 What is operator precedence grammar ? Give an example. [JNTU : Part A, March 17, Marks 2]

Ans. : A grammar G is said to be operator precedence if it posses following properties -

1. No production on the right side is ϵ .
2. There should not be any production rule possessing two adjacent non-terminals at the right hand side.

Consider the grammar for arithmetic expressions.

$E \rightarrow EAE \mid (E) \mid -E \mid id$

$A \rightarrow + \mid - \mid * \mid / \mid ^$

This grammar is not an operator precedent grammar as in the production rule.

$$E \rightarrow EAE$$

It contains two consecutive non-terminals. Hence first we will convert it into equivalent operator precedence grammar by removing A.

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid E \wedge E$$

$$E \rightarrow (E) \mid - E \mid id$$

In operator precedence parsing we will first define precedence relations $<\cdot$, $\cdot$ and $\cdot >$ between pair of terminals. The meaning of these relations is

$p <\cdot q$ p gives more precedence than q .

$p \cdot q$ p has same precedence as q .

$p \cdot > q$ p takes precedence over q .

These meanings appear similar to the less than, equal to and greater than operators. Now by considering the precedence relation between the arithmetic operators we will construct the operator precedence table. The operators precedences we have considered are $id, +, *, \$$.

	id	+	*	\$
id		$\cdot >$	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$< \cdot$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	

Fig. Q.36.1 Precedence relation table

Now consider the string.

$id + id * id$

We will insert \$ symbols at the start and end of the input string. We will also insert precedence operator by referring the precedence relation table.

$\$ <\cdot id \cdot > + <\cdot id \cdot > * <\cdot id \cdot > \$$

We will follow following steps to parse the given string -

- Scan the input from left to right until first $\cdot >$ is encountered.
- Scan backwards over $=$ until $<\cdot$ is encountered.
- The handle is a string between $<\cdot$ and $\cdot >$.

The parsing can be done as follows.

$\$ <\cdot id \cdot > + <\cdot id \cdot > * <\cdot id \cdot > \$$	Handle id is obtained between $<\cdot \cdot >$ Reduce this by $E \rightarrow id$
$E + <\cdot id \cdot > * <\cdot id \cdot > \$$	Handle id is obtained between $<\cdot \cdot >$ Reduce this by $E \rightarrow id$
$E + E * <\cdot id \cdot > \$$	Handle id is obtained between $<\cdot \cdot >$ Reduce this by $E \rightarrow id$

$E + E * E$	Remove all the non-terminals.
$+ *$	Insert \$ at the beginning at the end. Also Insert the precedence operators.
$\$ < \cdot + < \cdot * \cdot > \$$	The * operator is surrounded by $< \cdot \cdot >$. This indicates that * becomes handle. That means we have to reduce $E * E$ operation first.
$\$ < \cdot + \cdot > \$$	Now + becomes handle. Hence we evaluate $E + E$.
$\$ \$$	Parsing is done.

2.7 Introduction to LR Parsing

Q.37 What does the term LR stand for ?

[JNTU : Part A, Marks 2]

Ans. : • L stands for Left to Right scanning of input stream.

- The R stands for construction of rightmost derivation in reverse.
- The LR parsing is an efficient bottom-up parsing technique.

Q.38 List the properties of LR Parser.

[JNTU : Part A, May-18, Marks 2]

Ans. : LR parser is widely used for following reasons,

1. LR parsers can be constructed to recognize most of the programming languages for which context free grammar can be written.
2. The class of grammar that can be parsed by LR parser is a superset of class of grammars that can be parsed using predictive parsers.
3. LR parser works using non backtracking shift reduce technique yet it is efficient one.
4. LR Parsers detect syntactical errors very efficiently.

Q.39 What is significance of lookahead operator in LR parsing ?

[JNTU : Part A, March-17, Marks 3]

Ans. : • Lookahead is an extra token read from the current character user is reading. This helps to decide which grammar rule is applicable for deriving the string.

- It increases the efficiency of parsing.
- It avoids the problem of backtracking.

Q.40 Describe in brief about types of LR parsers.

[JNTU : Part A, March-16, Marks 3]

Ans. : The LR parsers are -

- 1) Simple LR Parser (SLR)
- 2) Canonical LR Parser (CLR)
- 3) Lookahead LR Parser (LALR)

Q.41 Explain the terms -

(1) LR(0) Items (2) Augmented Grammar (3) Kernel Items (4) Functions for canonical set of Items (5) Viable Prefix.

Ans. : 1) The LR(0) item for grammar G is production rule in which symbol • is inserted at some position in R.H.S. of the rule. For example

$S \rightarrow \bullet ABC$
 $S \rightarrow A \bullet BC$
 $S \rightarrow AB \bullet C$
 $S \rightarrow ABC \bullet$

The production $S \rightarrow \epsilon$ generates only one item $S \rightarrow \bullet$.

2) **Augmented grammar** : If a grammar G is having start symbol S then augmented grammar is a new grammar G' in which S' is a new start symbol such that $S' \rightarrow S$. The purpose of this grammar is to indicate the acceptance of input. That is when parser is about to reduce $S' \rightarrow S$ it reaches to acceptance state.

3) **Kernel items** : It is collection of items $S \rightarrow \bullet S$ and all the items whose dots are not at the leftmost end of R.H.S. of the rule.

Non-Kernel items : The collection of all the items in which • are at the left end of R.H.S. of the rule.

4) **Functions closure and goto** : These are two important functions required to create collection of canonical set of items.

5) **Viable prefix** : It is the set of prefixes in the right sentential form of production $A \rightarrow \alpha$. This set can appear on the stack during shift/reduce action.

Q.42 Define CLOSURE(I).

[JNTU : Part A, Nov.-15, Marks 3]

Ans. : For a context free grammar G, if I is the set of items then the function closure(I) can be constructed using following rules.

1. Consider I is a set of canonical items and initially every item I is added to closure(I).
2. If rule $A \rightarrow \alpha \bullet B \beta$ is a rule in closure(I) and there is another rule for B such as $B \rightarrow \gamma$ then

closure(I) : $A \rightarrow \alpha \bullet B \beta$
 $B \rightarrow \bullet \gamma$

This rule has to be applied until no more new items can be added to closure(I).

The meaning of rule $A \rightarrow \alpha \bullet B \beta$ is that during derivation of the input string at some point we may require strings derivable from $B \beta$ as input. A non-terminal immediately to the right of • indicates that it has to be expanded shortly.

Q.43 Explain GOTO function in LR parsing.

Ans. : The function goto can be defines as follows.

If there is a production $A \rightarrow \alpha \bullet B \beta$ then $\text{goto}(A \rightarrow \alpha \bullet B \beta, B) = A \rightarrow \alpha B \bullet \beta$. That means simply shifting of • one position ahead over the grammar symbol (may be terminal or non-terminal). The rule $A \rightarrow \alpha \bullet B \beta$ is in I then the same goto function can be written as $\text{goto}(I, B)$.

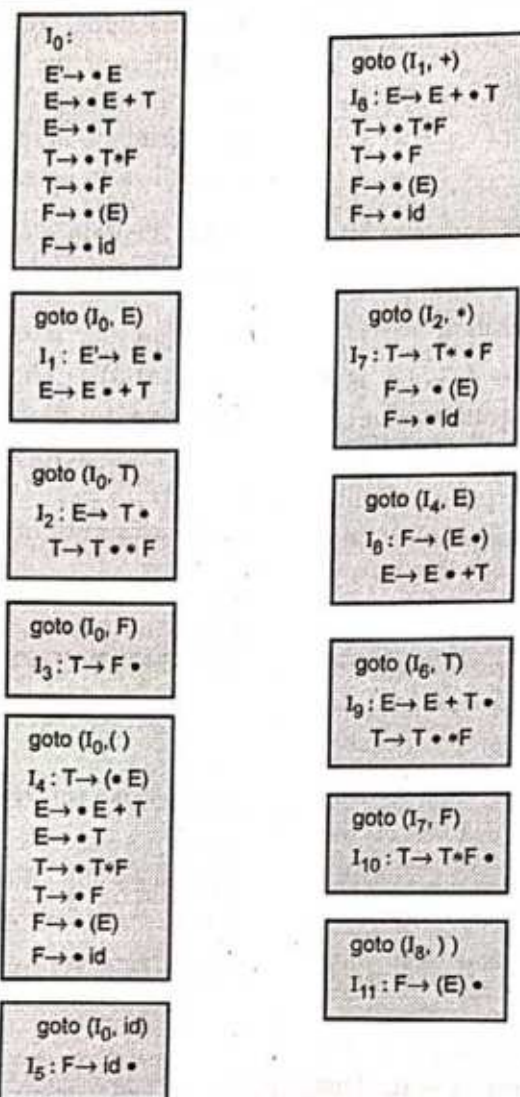
Q.44 Construct SLR parsing table for the following grammar

$E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow (E) \mid id$

[JNTU : Part B, March-16, Nov.-17, Marks 5, Nov.-16, Marks 10]

Ans. : We will first construct a collection of canonical set of items for the above grammar. The set of items generated by this method are also called SLR(0) items. As there is no lookahead symbol in this set of items, zero is put in the bracket.

State	Action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				Accept			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			



Consider the rules with numbering as :

- 1) $E \rightarrow E+T$
- 2) $E \rightarrow T$
- 3) $T \rightarrow T * F$
- 4) $T \rightarrow F$
- 5) $F \rightarrow (E)$
- 6) $F \rightarrow id$

Input string : $id * id + id$

We will consider two data structures while taking the parsing actions and those are - Stack and input buffer.

Stack	Input buffer	Action table	goto table	Parsing action
\$0	$id * id + id \$$	$[0, id] = s5$		Shift
$\$0id5$	$* id + id \$$	$[5, *] = r6$	$[0, F] = 3$	Reduce by $F \rightarrow id$

\$0F3	*id+id\$	[3,*]=r4	[0,T]=2	Reduce by $T \rightarrow F$
\$0T2	*id+id\$	[2,*]=s7		Shift
\$0T2*7	id+id\$	[7,id]=s5		Shift
\$0T2*7id5	+id\$	[5,+]=r6	[7,F]=10	Reduce by $F \rightarrow id$
\$0 T2*7F10	+id\$	[10,+]=r3	[0,T]=2	Reduce by $T \rightarrow T * F$
\$0T2	+id\$	[2,+]=r2	[0,E]=1	Reduce by $E \rightarrow T$
\$0E1	+id\$	[1,+]=s6		Shift
\$0E1+6	id\$	[6,id]=s5		Shift
\$0E1+6id5	\$	[5,\$]=r6	[6,F]=3	Reduce by $F \rightarrow id$
\$0E1+6F3	\$	[3,\$]=r4	[6,T]=9	Reduce by $T \rightarrow F$
\$0E1+6T9	\$	[9,\$]=r1	[0,E]=1	$E \rightarrow E + T$
\$0E1	\$	Accept		Accept

Q.45 Construct the LR(0) items for the dangling-else grammar.

[JNTU : Part A, Dec.-16, Marks 3]

Ans. : Step 1 : The dangling else grammar is as given below -

stmt $\rightarrow$ if expr then stmt

| if expr then stmt else stmt

| other

Step 2 : The simplified version of above grammar can be written as

$S \rightarrow iS | iSeS | o$

Step 3 : The canonical set of LR(0) items is as given below -

$I_0 :$ $S' \rightarrow \bullet S$ $S \rightarrow \bullet iS$ $S \rightarrow \bullet iSeS$ $S \rightarrow \bullet o$	$I_1 :$ $S' \rightarrow S \bullet$
$I_2 :$ $S \rightarrow i \bullet S$ $S \rightarrow i \bullet SeS$ $S \rightarrow i \bullet o$	$I_3 :$ $S \rightarrow o \bullet$
$I_4 :$ $S \rightarrow iS \bullet$ $S \rightarrow iS \bullet eS$	$I_5 :$ $S \rightarrow iSe \bullet S$ $S \rightarrow \bullet iS$ $S \rightarrow \bullet iSeS$ $S \rightarrow \bullet o$
$I_6 :$ $S \rightarrow iSeS \bullet$	

Q.46 Find the LR(0) set of items for the following grammar. Describe state diagram and construct parse table of that

$S \rightarrow CC$

$C \rightarrow cC \mid d$

[JNTU : Part B, March-17, Marks 10]

Ans. : We will construct canonical set of LR(0) items

$I_0 :$ $S' \rightarrow \bullet S$ $I_4 :$ $\text{goto}(I_0, d)$
 $S \rightarrow \bullet C C$ $C \rightarrow d \bullet$

$C \rightarrow \bullet c C$

$C \rightarrow \bullet d$

$I_1 :$ $\text{goto}(I_0, S)$ $I_5 :$ $\text{goto}(I_2, C)$
 $S' \rightarrow S \bullet$ $S \rightarrow C C \bullet$

$I_2 :$ $\text{goto}(I_0, C)$ $I_6 :$ $\text{goto}(I_3, C)$
 $S \rightarrow C \bullet C$ $C \rightarrow c, C \bullet$

$C \rightarrow \bullet c C$

$C \rightarrow \bullet d$

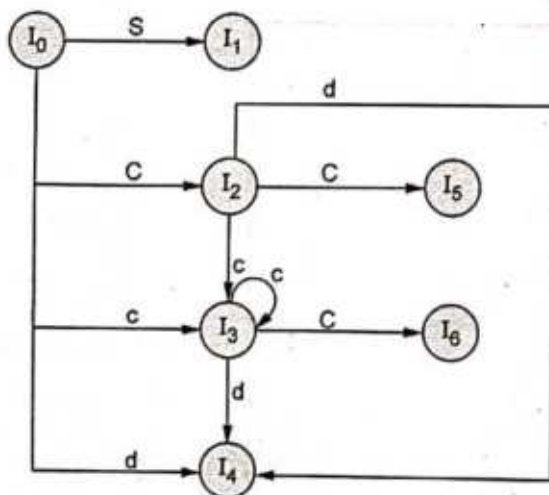
$I_3 :$ $\text{goto}(I_0, c)$

$C \rightarrow c \bullet C$

$C \rightarrow \bullet c C$

$C \rightarrow \bullet d$

The state diagram is,



The parsing table is,

State No.	Action			Goto	
	c	d	\$	S	C
0	s3	s4		1	2

1			ACCEPT		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

Q.47 Construct the collection of LR(0) item sets and draw the goto graph for the following grammar $S \rightarrow S S \mid a \mid \epsilon$.

Indicate the conflicts (if any) in the various states of the SLR parser. [JNTU : Part B, Marks 5]

Ans. : We will number the production rules in the grammar.

1) $S \rightarrow SS$

2) $S \rightarrow a$

3) $S \rightarrow \epsilon$

Now let us construct canonical set of items -

$I_0 :$ $S' \rightarrow \bullet S$ This is augmented grammar.

$S \rightarrow \bullet SS$

$S \rightarrow \bullet a$

$S \rightarrow \bullet$

$I_1 :$ $\text{goto}(I_0, S)$

$S' \rightarrow S \bullet$

$S \rightarrow S \bullet S$

$S \rightarrow \bullet SS$

$S \rightarrow \bullet a$

$S \rightarrow \bullet$

$I_2 :$ $\text{goto}(I_0, a)$

$S \rightarrow a \bullet$

$I_3 :$ $\text{goto}(I_1, S)$

$S \rightarrow SS \bullet$

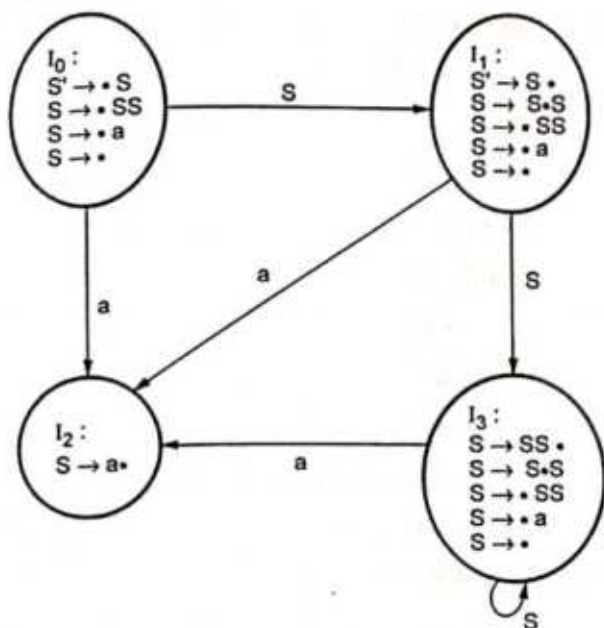
$S \rightarrow S \bullet S$

$S \rightarrow \bullet SS$

$S \rightarrow \bullet a$

$S \rightarrow \bullet$

The goto graph will be



$FOLLOW(S) = \{a, \$\}$

The construction of SLR(1) table will be.

In I_3 state there is a production rule $S \rightarrow \cdot a$. If we apply $goto(I_3, a)$ then we will get I_2 state. Hence $M[3, a] = S2$. In I_3 there is a production $S \rightarrow \cdot SS$ which matches with $A \rightarrow \alpha \cdot$. Rule $S \rightarrow SS$ is rule number 1. And $FOLLOW(S) = \{a, \$\}$
 $\therefore M[3, a] = M[3, \$] = r_1$

	Action		Goto
	a	\$	
0	S2		1
1	S2	Accept	3
2	r2	r2	
3	S2 r1	r1	

Conflict occurs. [shift-reduce conflict]

Hence conflict occurs in state I_3 .

Q.48 Construct SLR parsing table for the following grammar

$S \rightarrow AS|b$

$A \rightarrow SA|a$ [JNTU : Part B, April-11, Marks 10]

Ans. : We will construct a collection of canonical set of items for the given grammar.

$I_0 :$ $S' \rightarrow \cdot S$ $I_4 :$ $goto(I_0, a)$
 $S \rightarrow \cdot AS$ $A \rightarrow \cdot a$
 $S \rightarrow \cdot b$ $I_5 :$ $goto(I_1, A)$
 $A \rightarrow \cdot SA$ $A \rightarrow SA \cdot$
 $A \rightarrow \cdot a$ $A \rightarrow A \cdot S$
 $I_1 :$ $goto(I_0, S)$ $S \rightarrow \cdot AS$
 $S' \rightarrow S \cdot$ $S \rightarrow \cdot b$
 $A \rightarrow S \cdot A$ $A \rightarrow \cdot SA$
 $A \rightarrow \cdot SA$ $A \rightarrow \cdot a$
 $A \rightarrow \cdot a$ $I_6 :$ $goto(I_1, S)$
 $S \rightarrow \cdot AS$ $A \rightarrow S \cdot A$
 $S \rightarrow \cdot b$ $A \rightarrow \cdot SA$
 $A \rightarrow \cdot a$ $A \rightarrow \cdot a$
 $I_2 :$ $goto(I_0, A)$ $S \rightarrow \cdot AS$
 $S \rightarrow A \cdot S$ $S \rightarrow \cdot b$
 $S \rightarrow \cdot AS$ $I_7 :$ $goto(I_2, S)$
 $S \rightarrow \cdot b$ $S \rightarrow AS \cdot$
 $A \rightarrow \cdot SA$ $A \rightarrow S \cdot A$
 $A \rightarrow \cdot a$ $A \rightarrow \cdot SA$
 $A \rightarrow \cdot a$ $A \rightarrow \cdot a$
 $S \rightarrow \cdot AS$
 $S \rightarrow \cdot b$

We will list out the given rules

- $S \rightarrow AS$ $FOLLOW(S) = \{a, b, \$\}$
- $S \rightarrow b$ $FOLLOW(A) = \{a, b\}$
- $A \rightarrow SA$
- $A \rightarrow a$

The parsing table will be

	Action			goto	
	a	b	\$	S	A
0	S4	S3		1	2
1	S4	S3	Accept	6	5
2	S4	S3		7	2
3	r2	r2	r2		

4	r4	r4			
5	S4/r3	S3/r3		7	2
6	S4	S3		6	5
7	S4/r1	S3/r1	r1	6	5

Q.49 Construct the collection of non empty sets of LR(0) items for the following augmented grammar.

$S \rightarrow E$

$E \rightarrow TE \mid T$

[JNTU : Part B, Dec.-11, Marks 16]

Ans. :

$I_0 : S \rightarrow \bullet E$

$E \rightarrow \bullet TE$

$E \rightarrow \bullet T$

$I_1 : \text{goto}(I_0, E)$

$S \rightarrow E \bullet$

$I_2 : \text{goto}(I_0, T)$

$E \rightarrow T \bullet E$

$E \rightarrow T \bullet$

$E \rightarrow \bullet TE$

$E \rightarrow \bullet T$

$I_3 : \text{goto}(I_2, E)$

$E \rightarrow TE \bullet$

Q.50 Check whether the following grammar is SLR(1) or not. Explain your answer with reasons.

$S \rightarrow L = R$

$S \rightarrow R$

$L \rightarrow * R$

$L \rightarrow id$

$R \rightarrow L$

[JNTU : Part-B, Marks 5]

Ans. : We will first build a canonical set of items. Hence a collection of LR(0) items is as given below.

$I_0 : S' \rightarrow \bullet S$

$S \rightarrow \bullet L = R$

$S \rightarrow \bullet R$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet id$

$R \rightarrow \bullet L$

$I_1 : \text{goto}(I_0, S)$

$S' \rightarrow S \bullet$

$I_2 : \text{goto}(I_0, L)$

$S \rightarrow L \bullet = R$

$R \rightarrow L \bullet$

$I_3 : \text{goto}(I_0, R)$

$S \rightarrow R \bullet$

$I_4 : \text{goto}(I_0, *)$

$L \rightarrow * \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow * \bullet R$

$L \rightarrow * \bullet id$

$I_5 : \text{goto}(I_0, id)$

$L \rightarrow id \bullet$

$I_6 : \text{goto}(I_2, =)$

$S \rightarrow L = \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow * \bullet R$

$L \rightarrow * \bullet id$

$I_7 : \text{goto}(I_4, R)$

$L \rightarrow * R \bullet$

$I_8 : \text{goto}(I_4, L)$

$R \rightarrow L \bullet$

$I_9 : \text{goto}(I_6, R)$

$S \rightarrow L = R \bullet$

Now we will build the parsing table for the above set of items.

State	Action				Goto		
	=	*	id	\$	S	L	R
0							
1							
2	s6/r5						
3							
4		s4	s5				

5							
6		s4	s5				
7							
8							
9							

As we are getting multiple entries in Action [2, =] = s6 and r5. That means shift and reduce both a shift/reduce conflict occurs on input symbol =. Hence given grammar is not SLR(1) grammar.

Q.51 Construct a DFA whose states are the canonical collection of LR(1) items for the following augmented grammar,

$S \rightarrow A$
 $A \rightarrow BA \mid \epsilon$
 $B \rightarrow aB \mid b$ [JNTU : Part B, Jan.-10, Marks 16]

Ans. : Initially we will start with the rule

$$S \rightarrow \bullet A, \$$$

We will add the rules $A \rightarrow \bullet BA$ and $A \rightarrow \bullet$ to I_0 .

Now let us map the rule $[A \rightarrow \alpha \bullet X \beta, a]$ with $S \rightarrow \bullet A, \$$.

Such that $A = S, \alpha = \epsilon, X = A, \beta = \epsilon, a = \$$.

Then second component of $A \rightarrow \bullet BA$ and $A \rightarrow \bullet$

Will be = $\text{FIRST}(\beta a)$

= $\text{FIRST}(\epsilon, \$)$

= $\text{FIRST}(\$)$

= $\{\$ \}$

$\therefore S \rightarrow \bullet A, \$$

$A \rightarrow \bullet BA, \$$

$A \rightarrow \bullet, \$$

As there is a rule $A \rightarrow \bullet BA$ we must add the rules $B \rightarrow \bullet a B$ and $B \rightarrow \bullet b$ to I_0 . Now to compute second component of B 's rule we will use

$A \rightarrow \bullet BA, \$$ for mapping with $[A \rightarrow \alpha \bullet X \beta, a]$

$A = A, \alpha = \epsilon, X = B, \beta = A, a = \$$.

Then second component of $B \rightarrow \bullet a B$ and $B \rightarrow \bullet b$

will be = $\text{FIRST}(\beta a)$

= $\text{FIRST}(A, \$)$

= $\text{FIRST}(A)$

= $\{a, b, \epsilon\}$

= $\{a, b\}$

$\therefore B \rightarrow \bullet a B, a/b$

$B \rightarrow \bullet b, a/b$

Hence I_0 will be

$I_0 : S \rightarrow \bullet A, \$$

$A \rightarrow \bullet BA, \$$

$A \rightarrow \bullet, \$$

$B \rightarrow \bullet a B, a/b$

$B \rightarrow \bullet b, a/b$

$I_1 : \text{goto}(I_0, A)$

$S \rightarrow \bullet A, \$$

$I_2 : \text{goto}(I_0, B)$

$A \rightarrow B \bullet A, \$$

$A \rightarrow \bullet BA, \$$

$A \rightarrow \bullet, \$$

$B \rightarrow \bullet a B, a/b$

$B \rightarrow \bullet b, a/b$

$I_3 : \text{goto}(I_0, a)$

$B \rightarrow a \bullet B, a/b$

$B \rightarrow \bullet a B, a/b$

$B \rightarrow \bullet b, a/b$

$I_4 : \text{goto}(I_0, b)$

$B \rightarrow b \bullet, a/b$

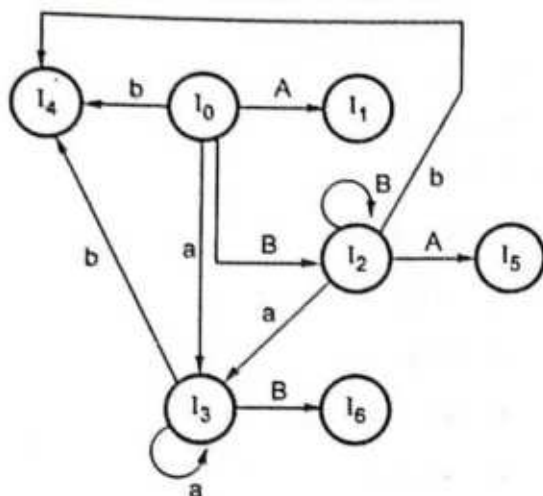
$I_5 : \text{goto}(I_2, A)$

$A \rightarrow BA \bullet, \$$

$I_6 : \text{goto}(I_2, B)$

$B \rightarrow a B \bullet, a/b$

The DFA can be constructed as follows -



Q.52 Construct the LR(1) parsing table for the following grammar.

- 1) $S \rightarrow CC$
- 2) $C \rightarrow cC$
- 3) $C \rightarrow d$

[JNTU : Part B, Marks 5]

I_0 :
 $S' \rightarrow \bullet S, \$$
 $S \rightarrow \bullet CC, \$$
 $C \rightarrow \bullet cC, c/d$
 $C \rightarrow \bullet d, c/d$

I_1 : goto (I_0, S)
 $S' \rightarrow S \bullet, \$$

I_2 : goto (I_0, C)
 $S \rightarrow C \bullet C, \$$
 $C \rightarrow \bullet cC, \$$
 $C \rightarrow \bullet d, \$$

I_3 : goto (I_0, c)
 $C \rightarrow c \bullet C, c/d$
 $C \rightarrow \bullet cC, c/d$
 $C \rightarrow \bullet d, c/d$

I_4 : goto (I_0, d)
 $C \rightarrow d \bullet, c/d$

I_5 : goto (I_2, C)
 $S \rightarrow CC \bullet, \$$

I_6 : goto (I_2, c)
 $C' \rightarrow c \bullet C, \$$
 $C \rightarrow \bullet cC, \$$
 $C \rightarrow \bullet d, \$$

I_7 : goto (I_2, d)
 $C \rightarrow d \bullet, \$$

I_8 : goto (I_3, C)
 $C \rightarrow cC \bullet, c/d$

I_9 : goto (I_6, C)
 $C \rightarrow cC \bullet, \$$

In I_0

$C \rightarrow \bullet d \quad c/d$

$A \rightarrow \alpha \bullet a \beta \quad b$

$A = C, \alpha = \epsilon, c = d, \beta = \epsilon, b = c/d$

goto(I_0, d) = I_4

hence action[0, d] = shift 4

For state I_4

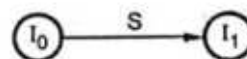
$C \rightarrow d \bullet, c/d$

$A \rightarrow \alpha \bullet, c$

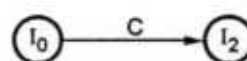
$A = C, \alpha = d, c = c/d$

action[4, c] = reduce by $C \rightarrow d$ i.e. rule 3

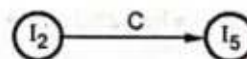
$S' \rightarrow S \bullet, \$$ in I_1



goto (I_0, S) = I_1



goto (I_0, C) = I_2



goto (I_2, C) = I_5

So we will create action[1, \$] = accept.

The goto table can be filled by using the goto functions.

For instance goto(I_0, S) = I_1 . Hence goto[0, S] = 1. Continuing in this fashion we can fill up the LR(1) parsing table as follows.

	Action			goto	
	a	d	\$	S	C
0	s3	s4		1	2
1			Accept		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		

Ans. :

First we will construct the set of LR(1) items.

Now consider I_0 in which there is a rule matching with $[A \rightarrow \alpha \bullet c\beta, b]$ as

$C \rightarrow \bullet cC, c/d$ and if the goto is applied on a then we get the state I_3 . Hence we will create entry action[0, a] = shift 3. Similarly,

6	s6	s7		9
7			r3	
8	r2	r2		
9			r2	

The remaining blank entries in the table are considered as syntactical error.

Parsing the Input using LR(1) parsing table

Using above parsing table we can parse the input string "aadd" as

Stack	Input buffer	Action table	goto table	Parsing action
\$0	aadd\$	action[0, a]=s3		
\$0a3	add\$	action[3, a]=s3		Shift
\$0a3a3	dd\$	action[3, d]=s4		Shift
\$0a3a3d4	d\$	action[4, d]=r3	[3, C]=8	Reduce by $C \rightarrow d$
\$0a3a3C8	d\$	action[8, d]=r2	[3, C]=8	Reduce by $C \rightarrow aC$
\$0a3C8	d\$	action[8, d]=r2	[0, C]=2	Reduce by $C \rightarrow aC$
\$0C2	d\$	action[2, d]=s7		Shift
\$0C2d7	\$	action[7, \$]=r3	[2, C]=5	Reduce by $C \rightarrow d$
\$0C2C5	\$	action[5, \$]=r1	[0, S]=1	Reduce by $S \rightarrow CC$
\$0S1	\$	accept		

Thus the given input string is successfully parsed using LR parser or canonical LR parser.

Q.53 Construct CLR parsing table for the following grammar

$E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$

[JNTU : Part B, Nov.-17, Marks 5]

Ans. : We will construct canonical set of LR(1) items.

$I_0 :$ $E' \rightarrow \bullet E, \$$ $E \rightarrow \bullet E + T, \$$ $E \rightarrow \bullet T, \$$ $T \rightarrow \bullet T * F, \$$ $T \rightarrow \bullet F, \$$ $\rightarrow \bullet (E), \$$ $F \rightarrow \bullet id, \$$	$I_1 : \text{goto } (I_0, E)$ $E' \rightarrow E \bullet, \$$ $E \rightarrow E \bullet + T, \$$
$I_2 : \text{goto } (I_0, T)$ $E \rightarrow T \bullet, \$$ $T E' \rightarrow T \bullet * F, \$$	$I_3 : \text{goto } (I_0, F)$ $T \rightarrow F \bullet, \$$

$I_4 : \text{goto } (I_0, 0)$ $F \rightarrow (\bullet E), \$$ $E \rightarrow \bullet E + T,)$ $E \rightarrow \bullet T,)$ $T \rightarrow \bullet T * F,)$ $T \rightarrow \bullet F,)$ $F \rightarrow \bullet (E),)$ $F \rightarrow \bullet \text{id},)$	$I_5 : \text{goto } (I_0, d)$ $F \rightarrow \text{id} \bullet, \$$
$I_6 : \text{goto } (I_1, +)$ $E \rightarrow E + \bullet T, \$$ $T \rightarrow \bullet T * F, \$$ $T \rightarrow \bullet F, \$$ $F \rightarrow \bullet (E), \$$ $F \rightarrow \bullet \text{id}, \$$	$I_7 : \text{goto } (I_2, *)$ $T \rightarrow T \bullet * F, \$$ $F \rightarrow \bullet (E), \$$ $F \rightarrow \bullet \text{id}, \$$
$I_8 : \text{goto } (I_4, E)$ $F \rightarrow (E \bullet), \$$ $E \rightarrow E \bullet + T,)$	$I_9 : \text{goto } (I_6, T)$ $E \rightarrow E + T \bullet, \$$ $T \rightarrow T \bullet * F, \$$
$I_{10} : \text{goto } (I_7, F)$ $T \rightarrow T * F \bullet, \$$	$I_{11} : \text{goto } (I_8,))$ $E \rightarrow (E) \bullet, \$$

Let us number out the original grammar rules as these rule numbers are used for shift and reduce operations in parsing table.

- i) $E \rightarrow E + T$
- 2) $E \rightarrow T$
- 3) $T \rightarrow T * F$
- 4) $T \rightarrow F$
- 5) $F \rightarrow (E)$
- 6) $F \rightarrow \text{id}$

The parsing table can be given below -

State	Action						Goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				ACCEPT			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4					
7	s5			s4				9	3

8		s6			s11				10
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Q.54 Construct LALR parser table for the following grammar for CLR parser.

$S \rightarrow L = R$, $S \rightarrow R$, $L \rightarrow *R$, $R \rightarrow L$.

[JNTU : Part B, May-18, Marks 10]

Ans. : To derive the given grammar using LALR method we have to build the canonical collection of items using LR(1) items.

The LR(1) items are

- | | |
|---|--|
| $I_0 : S' \rightarrow \cdot S, \$$ | $I_6 : \text{goto } (I_2, =)$ |
| $S \rightarrow \cdot L = R \$$ | $S \rightarrow L = \cdot R \$$ |
| $S \rightarrow \cdot R, \$$ | $R \rightarrow \cdot L, \$$ |
| $L \rightarrow \cdot * R, = \$$ | $L \rightarrow \cdot * R, \$$ |
| $L \rightarrow \cdot \text{id}, = \$$ | $L \rightarrow \cdot \text{id}, \$$ |
| $R \rightarrow \cdot L, \$$ | |
| $I_1 : \text{goto } (I_0, S)$ | $I_7 : \text{goto } (I_4, R)$ |
| $S' \rightarrow S \cdot, \$$ | $L \rightarrow * R \cdot = \$$ |
| $I_2 : \text{goto } (I_0, L)$ | $I_8 : \text{goto } (I_4, L)$ |
| $S \rightarrow L \cdot = R, \$$ | $R \rightarrow L \cdot, = \$$ |
| $R \rightarrow L \cdot, \$$ | |
| | $I_9 : \text{goto } (I_6, R)$ |
| | $S \rightarrow L = R \cdot, \$$ |
| $I_3 : \text{goto } (I_0, R)$ | |
| $S \rightarrow R \cdot, \$$ | $I_{10} : \text{goto } (I_6, L)$ |
| $I_4 : \text{goto } (I_0, *)$ | $R \rightarrow L \cdot, \$$ |
| $L \rightarrow * \cdot R, = \$$ | $I_{11} : \text{goto } (I_6, *)$ |
| $R \rightarrow \cdot L, = \$$ | $L \rightarrow * \cdot R, \$$ |
| $L \rightarrow \cdot * R, = \$$ | $R \rightarrow \cdot L, \$$ |
| $L \rightarrow \cdot \text{id}, = \$$ | $L \rightarrow \cdot * R, \$$ |
| | $L \rightarrow \cdot \text{id}, \$$ |
| | $I_{12} : \text{goto } (I_6, \text{id})$ |
| | $L \rightarrow \text{id} \cdot, \$$ |
| $I_5 : \text{goto } (I_0, \text{id})$ | $I_{13} : \text{goto } (I_{11}, R)$ |
| $L \rightarrow \text{id} \cdot, = \$$ | $L \rightarrow * R \cdot, \$$ |

From above set of items we have got

i) I_4 and I_{11} give same production but lookheads are different. Hence merge them to form I_{411}

ii) $I_5 = I_{12}$ Hence I_{512}

iii) $I_7 = I_{13}$ Hence I_{713}

iv) $I_8 = I_{10}$ Hence I_{810}

Therefore the set of items for LALR are

$I_0: S' \rightarrow \cdot S, \$$ $I_{713}: L \rightarrow \cdot R \cdot, = | \$$

$S \rightarrow \cdot L = R, \$$

$S \rightarrow \cdot R, \$$ $I_{810}: R \rightarrow L \cdot, = | \$$

$L \rightarrow \cdot \cdot R, = | \$$

$L \rightarrow \cdot id, = | \$$ $I_9: S \rightarrow L = R \cdot, \$$

$R \rightarrow \cdot L, \$$

$I_1: S' \rightarrow S \cdot, \$$

$I_2: S \rightarrow L \cdot, = R, \$$

$R \rightarrow L \cdot, \$$

$I_3: S \rightarrow R \cdot, \$$

$I_{411}: L \rightarrow \cdot \cdot R, = | \$$

$R \rightarrow L \cdot, = | \$$

$L \rightarrow \cdot \cdot R, = | \$$

$L \rightarrow \cdot Id, = | \$$

$I_{512}: L \rightarrow id \cdot, = | \$$

$I_6: S \rightarrow L = \cdot R, \$$

$R \rightarrow \cdot L, \$$

$L \rightarrow \cdot \cdot R, \$$

$L \rightarrow \cdot id, \$$

From above set of items we have got

The parsing table can be constructed as follows -

State	Action				goto		
	id	*	=	\$	S	L	R
0	S5	S4			1	2	3
1				acc			
2			S6	r5			
3				r2			
4	S5	S4				8	7
5			r4	r4			
6	S12	S11				10	9
7			r3	r3			
8			r5	r5			
9				r1			
10				r5			
11	S12	S11				10	13
12				r4			
13				r3			

The LALR parsing table is

State	Action				goto		
	id	*	=	\$	S	L	R
0	S512	S411			1	2	3
1				acc			
2			S6	r5			
3				r2			
411	S512	S411				810	713
512			r4	r4			
6	S512	S411				810	9
713			r3	r3			
810			r5	r5			
9				r1			

Q.55 Give CLR parsing table for the grammar
 $S \rightarrow L = R \mid R L \rightarrow \cdot R \mid id R \rightarrow \cdot L$

[JNTU : Part B, March-16, Marks 5]

Ans. : Refer Q.54.

Q.56 Write an algorithm for computing LR(k) items sets.
 [JNTU : Part B, Nov.-16, Marks 5]

Ans. :

1. For the grammar G initially add $S' \rightarrow \cdot S$ in the set of item C.

2. For each set of items I_i in C and for each grammar symbol X (may be terminal or non-terminal) add closure (I_i, X). This process should be repeated by applying $\text{goto}(I_i, X)$ for each X in I_i such that $\text{goto}(I_i, X)$ is not empty and not in C . The set of items has to be constructed until no more set of items can be added to C .
3. The closure function can be computed as follows.
For each item $A \rightarrow \alpha \bullet X \beta$ and rule $X \rightarrow \gamma$ and $b \in \text{FIRST}(\beta a)$
such that $X \rightarrow \bullet \gamma$ and b is not in I then add $X \rightarrow \bullet \gamma, b$ to I .
4. Similarly the goto function can be computed as :
For each item $[A \rightarrow \alpha \bullet X \beta, a]$ is in I and rule $[A \rightarrow \alpha X \bullet \beta, a]$ is not in goto items then add $[A \rightarrow \alpha X \bullet \beta, a]$ to goto items.

Q.57 Write a procedure to construct LALR parsing table. [JNTU : Part B, March-17, Marks 5]

Ans. : The algorithm for construction of LALR parsing table is as given below.

Step 1 : Construct the LR(1) set of items.

Step 2 : Merge the two states I_i and I_j if the first component (i.e. the production rules with dots) are matching and create a new state replacing one of the older state such as $I_{ij} = I_i \cup I_j$

- Step 3 :** The parsing actions are based on each item I_i . The actions are as given below.
- a) If $[A \rightarrow \alpha \bullet a \beta]$ is in I_i and $\text{goto}(I_i, a) = I_j$ then create an entry in the action table $\text{action}[I, a] = \text{shift } j$.
 - b) If there is a production $[A \rightarrow \alpha \bullet, a]$ in I_i then in the action table action $[I_i, a] = \text{reduce by } A \rightarrow \alpha$. Here A should not be S' .
 - c) If there is a production $S' \rightarrow S \bullet, \$$ in I_i then action $[i, \$] = \text{accept}$.

Step 4 : The goto part of the LR table can be filled as : The goto transitions for state i is considered for non-terminals only. If $\text{goto}(I_i, A) = I_j$ then $\text{goto}[I_i, A] = j$.

Step 5 : If the parsing action conflict then the algorithm fails to produce LALR parser and grammar is not LALR(1). All the entries not defined by rule 3 and 4 are considered to be "error".

Q.58 Give LALR parsing table for the following grammar

$E \rightarrow E+T \mid T \mid T \rightarrow T^*F \mid F \mid F \rightarrow (E) \mid \text{id}$

[JNTU : Part B, Nov.-15, Marks 10]

Ans. : Refer Q.53.

Q.59 Show that the following grammar.

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

is LR (1) but not LALR(1). [JNTU : Part B]

Ans. : We will number out the production rules in given grammar.

- 1) $S \rightarrow Aa$
- 2) $S \rightarrow bAc$
- 3) $S \rightarrow Bc$
- 4) $S \rightarrow bBa$
- 5) $A \rightarrow d$
- 6) $B \rightarrow d$

Now we will first construct canonical set of LR(1) items.

$S' \rightarrow \bullet S, \$$

Initially for the augmented grammar the second component is $\$$. As this rule $[S' \rightarrow \bullet S, \$]$ is matching with $[A \rightarrow \alpha \bullet X \beta, a]$ where α is ϵ , X is S . B is ϵ and a is $\$$. We will apply closure on S and add all the production rules deriving S . The second component in these production rules will be $\$$. This is because for $[S' \rightarrow \bullet S, \$]$ the $\beta = \epsilon$ and $a = \$ \therefore \text{FIRST}(\beta a) = \epsilon$, $\text{FIRST}(\epsilon \$) = \text{FIRST}(\$) = \$$. Hence we will get

$S' \rightarrow \bullet S, \$$

$S \rightarrow \bullet Aa, \$$

$S \rightarrow \bullet bAc, \$$

$S \rightarrow \bullet Bc, \$$

$S \rightarrow \bullet bBa, \$$

Now we have got one rule $[S \rightarrow \cdot Aa, \$]$ which is matching with $[A \rightarrow a \cdot X\beta, a]$ and $[X \rightarrow \gamma]$. Hence we will add the closure on A. Hence $[A \rightarrow \cdot d]$ will be added in this list. Now the second component of $A \rightarrow \cdot d$ will be decided. As $[S \rightarrow \cdot Aa, \$]$ is matching with $[A \rightarrow A \cdot X\beta, a]$ having X as A, β as a and a as \$.

Then $\text{FIRST}(\beta a) = \text{FIRST}(a\$) = \text{FIRST}(a) = a$. Hence rule $[A \rightarrow \cdot d, a]$ will be added.

Now,

$S' \rightarrow \cdot S, \$$

$S \rightarrow \cdot Aa, \$$

$S \rightarrow \cdot bAc, \$$

$S \rightarrow \cdot Bc, \$$

$S \rightarrow \cdot bBa, \$$

$A \rightarrow \cdot d, a$

Now notice the rule $S \rightarrow \cdot Bc, \$$. It suggest to apply closure on B (As after dot B comes immediately) This rule is matching with $[A \rightarrow \alpha \cdot X\beta, a]$ and $[X \rightarrow \gamma]$. Hence we will add rule deriving B. Hence $B \rightarrow \cdot d$ will be added in the above list. Now

$S \rightarrow \cdot Bc, \$$ can be mapped with $[A \rightarrow \alpha \cdot X\beta, a]$.

Where $\alpha = \epsilon$

$X = B$

$\beta = c$

$a = \$$

Hence $\text{FIRST}(\beta a) = \text{FIRST}(c\$) = \text{FIRST}(c) = c$.

Hence rule $[B \rightarrow \cdot d, c]$ will be added.

Hence finally I_0 will be -

$I_0 :$

- $S' \rightarrow \cdot S$
- $S \rightarrow \cdot Aa, \$$
- $S \rightarrow \cdot bAc, \$$
- $S \rightarrow \cdot Bc, \$$
- $S \rightarrow \cdot bBa, \$$

$A \rightarrow \cdot d, a$

$B \rightarrow \cdot d, c$

Continue with applying goto on each symbol.

$I_1 : \text{goto}(I_0, S)$

$S' \rightarrow S \cdot, \$$

There is no chance of applying closure or goto in this state. Hence it will have only one rule.

$I_2 : \text{goto}(I_0, A)$

$S' \rightarrow A \cdot a, \$$

Applied goto on A. But as after dot a terminal symbol comes we cannot apply any rule further. The second component is carried as it is.

$I_3 : \text{goto}(I_0, b)$

$S \rightarrow b \cdot Ac, \$$

$S \rightarrow b \cdot Ba, \$$

$A \rightarrow \cdot d, c$

$B \rightarrow \cdot d, a$

After dot A comes hence rule for $A \rightarrow \cdot d$ will be added. As $[S \rightarrow b \cdot Ac, \$]$ is matching with $[A \rightarrow \alpha \cdot X\beta, a]$ and $[X \rightarrow \gamma]$, $\text{FIRST}(\beta a) = \text{FIRST}(c\$) = \text{FIRST}(c) = c$. The second component of $A \rightarrow \cdot d$ is c.

$I_4 : \text{goto}(I_0, B)$

$S' \rightarrow B \cdot c, \$$

The goto on B is applied and second component is carried as it is.

$I_5 : \text{goto}(I_0, d)$

$A \rightarrow d \cdot, a$

$B \rightarrow d \cdot, c$

The goto on d in state I_0 is applied with corresponding second components as it is.

Continuing in this fashion we will get further states.

$I_6 : \text{goto}(I_2, a)$

$S \rightarrow Aa \cdot, \$$

$I_9 : \text{goto}(I_3, d)$

$A \rightarrow d \cdot, c$

$B \rightarrow d \cdot, a$

I_7 : goto (I_3 , A)
 $S \rightarrow bA \cdot c, \$$

I_{10} : goto (I_4 , c)
 $S \rightarrow Bc \cdot, \$$

I_8 : goto (I_3 , B)
 $S \rightarrow bB \cdot a, \$$

I_{11} : goto (I_7 , c)
 $S \rightarrow bAc \cdot, \$$

I_{12} : goto (I_8 , a)
 $S \rightarrow bBa \cdot, \$$

Now using the above set of LR(1) items we will construct LR(1) parsing table as follows.

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0		s3		s5		1	2	4
1					ACCEPT			
2	s6							
3				s9			7	8
4			s10					
5	r5		r6					
6					r1			
7			s11					
8	s12							
9	r6		r5					
10					r3			
11					r2			
12					r4			

We can parse the string "bda" using above constructed LR (1) parsing table as :

Stack	Input buffer	Action
\$0	bda\$	Shift 3
\$0b3	da\$	Shift 9

\$0b3d9	a\$	Reduce by $B \rightarrow d$
\$0b3B8	a\$	Shift 12
\$0b3B8a12	\$	Reduce by $S \rightarrow bBa$
\$0S1	\$	Accept

Now we will construct a set of LALR(1) items. In this construction we will simply merge the states deriving same production rules which differ in their second components only. In above set of LR(1) items state I_5 and I_9 are such states i.e.

I_5 : goto (I_0 , d) I_9 : goto (I_3 , d)
 $A \rightarrow d \cdot, a$ $A \rightarrow d \cdot, c$
 $B \rightarrow d \cdot, c$ $B \rightarrow d \cdot, a$

We will form only one state by merging state 5 and 9 as

I_{59} : goto (I_0 , d)
 $A \rightarrow d \cdot, a / c$
 $B \rightarrow d \cdot, a / c$

Hence the LALR(1) set of items are given as :

I_0 : $S' \rightarrow \cdot S, \$$ I_6 : goto (I_2 , a)
 $S \rightarrow \cdot Aa, \$$ $S \rightarrow Aa \cdot, \$$
 $S \rightarrow \cdot bAc, \$$ I_7 : goto (I_3 , A)
 $S \rightarrow \cdot Bc, \$$ $S \rightarrow bA \cdot c, \$$
 $A \rightarrow \cdot d, a$ I_8 : goto (I_3 , B)
 $B \rightarrow \cdot d, c$ $S \rightarrow bB \cdot a, \$$

I_1 : goto (I_0 , S) I_{10} : goto (I_4 , c)
 $S' \rightarrow S \cdot, \$$ $S \rightarrow Bc \cdot, \$$

I_2 : goto (I_0 , A) I_{11} : goto (I_7 , c)
 $S \rightarrow A \cdot a, \$$ $S \rightarrow bAc \cdot, \$$

I_3 : goto (I_0 , b) I_{12} : goto (I_8 , a)
 $S \rightarrow b \cdot Ac, \$$ $S \rightarrow bBa \cdot, \$$

$S \rightarrow b \cdot Ba, \$$

$A \rightarrow \cdot d, c$

$B \rightarrow \cdot d, a$

$I_4 : \text{goto } (I_0, B)$

$S \rightarrow B \cdot c, \$$

$I_{59} : \text{goto } (I_0, d)$

$A \rightarrow d \cdot, a / c$

$B \rightarrow d \cdot, a / c$

The LALR parsing table will be -

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0		s3		s5		1	2	4
1					ACCEPT			
2	s6							
3				s9			7	8
4			s10					
59	r5 r6		r5 r6					
6					r1			
7			s11					
8	s12							
10					r3			
11					r2			
12					r4			

The parsing table shows multiple entries in Action [59, a] and Action [59, c]. This is called **reduce/reduce conflict**. Because of this conflict we cannot parse input.

Thus it is shown that given grammar is LR(1) but not LALR(1)

Q.60 Design LALR (1) parser for following grammar

$S \rightarrow a A d \mid b B d \mid a B c \mid b A c$

$A \rightarrow e$

$B \rightarrow e$

[JNTU : Part B, June-14, Marks 8]

Ans. : We will first write an augmented grammar and number it out.

1. $S' \rightarrow S$

2. $S \rightarrow a A d$

3. $S \rightarrow b B d$

4. $S \rightarrow a B c$

5. $S \rightarrow b A c$

6. $A \rightarrow e$

7. $B \rightarrow e$

Now, let us generate LR (1) Canonical set of items.

$I_0 : S' \rightarrow \cdot S, \$$

$S \rightarrow \cdot a A d, \$$

$S \rightarrow \cdot b B d, \$$

$S \rightarrow \cdot a B c, \$$

$S \rightarrow \cdot b A c, \$$

$I_1 : \text{goto } (I_0, S) \rightarrow I_6 : \text{goto } (I_2, e)$

$S' \rightarrow S \cdot, \$$ $A \rightarrow e \cdot, d$

$B \rightarrow e \cdot, c$

$I_2 : \text{goto } (I_0, a) \quad I_7 : \text{goto } (I_3, B)$

$S \rightarrow a \cdot A d, \$$ $S \rightarrow b B \cdot d, \$$

$S \rightarrow a \cdot B c, \$$ $I_8 : \text{goto } (I_3, A)$

$A \rightarrow \cdot e, d$ $S \rightarrow b A \cdot c, \$$

$B \rightarrow \cdot e, c \rightarrow I_9 : \text{goto } (I_3, e)$

$I_3 : \text{goto } (I_0, b) \quad A \rightarrow e \cdot, c$

$S \rightarrow b \cdot B d, \$$ $B \rightarrow e \cdot, d$

$S \rightarrow b \cdot A c, \$$ $I_{10} : \text{goto } (I_4, d)$

$A \rightarrow \cdot e, c$ $S \rightarrow a A d \cdot, \$$

$B \rightarrow \cdot e, d$ $I_{11} : \text{goto } (I_5, c)$

$I_4 : \text{goto } (I_2, A) \quad S \rightarrow a B c \cdot, \$$

$S \rightarrow a A \cdot d, \$$ $I_{12} : \text{goto } (I_7, d)$

I_5 : $\text{goto } (I_2, B)$ $S \rightarrow bBd \cdot, \$$
 $S \rightarrow aB \cdot c, \$$ I_{13} : $\text{goto } (I_8, c)$
 $S \rightarrow bAc \cdot, \$$

From above construction we can combine I_6 and I_9 to form I_{69} state as,

I_{69} : $A \rightarrow e \cdot, c/d$ and $B \rightarrow \cdot e, c/d$

The parsing table can be constructed as follows :

	Action						goto		
	a	b	c	d	e	\$	S	A	B
0	S2	S3					1		
1						Accept			
2					S6			4	5
3					S9			8	7
4				S10					
5			S11						
69			r6/r7	r6/r7					
7				S12					
8			S13						
10						r2			
11						r4			
12						r3			
13						r5			

As we get Table [69] as reduce/reduce conflict given grammar is not LALR.

Q.61 Compare and contrast between SLR, LALR and LR parsers [JNTU : Part A, May-18, Marks 10]

Ans. :

Sr. No.	SLR parser	LALR parser	Canonical LR parser
1.	SLR parser is smallest in size.	The LALR and SLR have the same size.	LR parser or canonical LR parser is largest in size.

2.	It is an easiest method based on FOLLOW function.	This method is applicable to wider class than SLR.	This method is most powerful than SLR and LALR.
3.	This method exposes less syntactic features than that of LR parsers.	Most of the syntactic features of a language are expressed in LALR.	This method exposes less syntactic features than that of LR parsers.
4.	Error detection is not immediate in SLR.	Error detection is not immediate in LALR.	Immediate error detection is done by LR parser.
5.	It requires less time and space complexity.	The time and space complexity is more in LALR but efficient methods exist for constructing LALR parsers directly.	The time and space complexity is more for canonical LR parser.

2.8 Using Ambiguous Grammar

Q.62 Find the SLR parsing table for the given grammar.

$E \rightarrow E+E \mid E \cdot E \mid (E) \mid id.$

And parse the sentence $(a+b) \cdot c$

[JNTU : Part B, Nov.-15, Marks 10]

Ans. :

$I_0:$ $E' \rightarrow \bullet E$ $E \rightarrow \bullet E + E$ $E \rightarrow \bullet E * E$ $E \rightarrow \bullet (E)$ $E \rightarrow \bullet id$	$I_5: goto(I_1, *)$ $E \rightarrow E \bullet \bullet E$ $E \rightarrow \bullet E + E$ $E \rightarrow \bullet E * E$ $E \rightarrow \bullet (E)$ $E \rightarrow \bullet id$
$I_1: goto(I_0, E)$ $E' \rightarrow E \bullet$ $E \rightarrow E \bullet + E$ $E \rightarrow E \bullet * E$	$I_6: goto(I_2, E)$ $E \rightarrow (E \bullet)$ $E \rightarrow E \bullet + E$ $E \rightarrow E \bullet * E$
$I_2: goto(I_0, ($ $E' \rightarrow (\bullet E)$ $E \rightarrow \bullet E + E$ $E \rightarrow \bullet E * E$ $E \rightarrow \bullet (E)$ $E \rightarrow \bullet id$	$I_7: goto(I_4, E)$ $E \rightarrow E + E \bullet$ $E \rightarrow E \bullet + E$ $E \rightarrow E \bullet * E$
$I_3: goto(I_0, id)$ $E \rightarrow \bullet id$	$I_8: goto(I_5, E)$ $E \rightarrow E \bullet E \bullet$ $E \rightarrow E \bullet + E$ $E \rightarrow E \bullet * E$
$I_4: goto(I_1, +)$ $E \rightarrow E + \bullet E$ $E \rightarrow \bullet E + E$ $E \rightarrow \bullet E * E$ $E \rightarrow \bullet (E)$ $E \rightarrow \bullet id$	$I_9: goto(I_6,))$ $E \rightarrow (E) \bullet$

Here FOLLOW(E) = {+, *,), \$}.

We have computed FOLLOW (E) as it will be required while processing.

Now using the same rules of building SLR(0) parsing table we will generate the parsing table for above set of items.

State	Action						Goto
	id	+	*	(	)	\$	
0	s3			s2			E
1		s4	s5			Accept	1
2	s3			s2			6
3		r4	r4		r4	r4	
4	s3			s2			7
5	s3			s2			8
6		s4	s5		s9		
7		s4 or r1	s5 or r1		r1	r1	
8		s4 or r2	s5 or r2		r2	r2	
9			r3		r3	r3	

Simulation of $(a + b) * c$

Stack	Input	Actions with conflict resolution
\$0	$(a + b) * c$	Shift
$(a + b) * c$	$a + b) * c$	Shift
$(a + b) * c$	$+ b) * c$	Reduce by $E \rightarrow id$
$(a + b) * c$	$+ b) * c$	Shift
$(a + b) * c$	$b) * c$	Shift
$(a + b) * c$	$) * c$	Reduce by $E \rightarrow id$
$(a + b) * c$	$) * c$	Reduce by $E \rightarrow E + E$
$(a + b) * c$	$) * c$	Shift
$(a + b) * c$	$* c$	Reduce by $E \rightarrow (E)$
$(a + b) * c$	$* c$	Shift
$(a + b) * c$	c	Shift
$(a + b) * c$	c	Reduce by $E \rightarrow id$
$(a + b) * c$	c	Reduce by $E \rightarrow E * E$
$(a + b) * c$	c	Accept

2.9 Parser Generators

Q.63 Give the specification of the YACC Parser generator [JNTU : Part A, Dec.-16, Marks 2]

Ans. : The YACC specification file consists of three parts declaration section, translation rule section and supporting C functions.

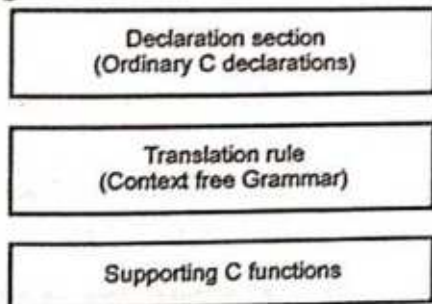


Fig. Q.63.1 Parts of YACC specification

The specification file with these sections can be written as

```

%{
/* declaration section */
}%
%%
/* Translation rule section */
%%
/* Required C functions */
  
```

1. **Declaration part :** In this section ordinary C declarations can be put. Not only this we can also declare grammar tokens in this section. The declaration of tokens should be within `%{` and `%}`.

2. **The translation rule section :** It consists of all the production rules of context free grammar with corresponding actions. For instance

```

rule 1      action 1
rule 2      action 2
.
.
.
rule n      action n
  
```

If there are more than one alternatives to a single rule then those alternatives should be separated by `|` character. The actions are typical C statements. If CFG is

$LHS \rightarrow \text{alternative 1} | \text{alternative 2} | \dots | \text{alternative n}$

Then

```

LHS : alternative 1  { action 1 }
      | alternative 2  { action 2 }
      .
      .
      .
      alternative n  { action n }
      ;
  
```

3. **C functions section :** This section consists of one main function in which the routine `yyparse()` will be called. And it also consists of required C functions.

Q.64 Discuss in brief about YACC.

[JNTU : Part B, Nov.-17, March-17, Marks 5]

OR Write short notes on - YACC.

[JNTU : Part A, May-18, Marks 3]

Ans. : • YACC is one such automatic tool for generating the parser program.

- YACC stands for Yet Another Compiler Compiler which is basically the utility available from UNIX.
- Basically YACC is LALR parser generator.
- The YACC can report conflicts or ambiguities (if at all) in the form of error messages.

- The LEX tool is for lexical analyzer. LEX and YACC work together to analyse the program syntactically.
- The typical YACC translator can be represented as shown in Fig. Q.64.1.

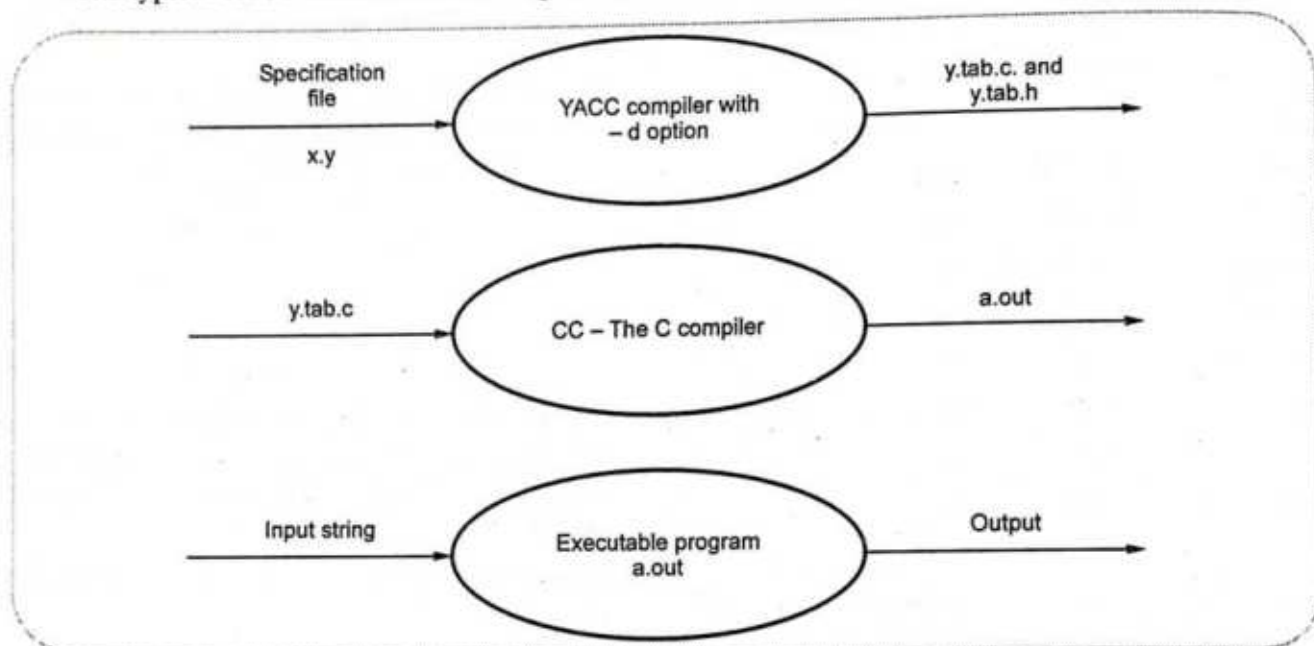


Fig. Q.64.1 YACC : Parser generator model

- First we write a YACC specification file; let us name it as x.y. This file is given to the YACC compiler by UNIX command

```
yacc x.y
```

- Then it will generate a parser program using your YACC specification file. This parser program has a standard name as y.tab.c. This is basically parser program in C generated automatically. You can also give the command with - d option

```
yacc - d x.y
```

By - d option two files will get generated one is y.tab.c and other is y.tab.h. The header file y.tab.h will store all the tokens and so you need not have to create y.tab.h explicitly.

The generated y.tab.c program will then be compiled by C compiler and generates the executable a.out file. Then you can test your YACC program with the help of some valid and invalid strings.

Q.65 Write a YACC program that will take regular expression as input and produce its parse tree as output.

[JNTU : Part B, March-16, Nov.-16, Marks 5]

Ans. :

```

/*Program Name :calci.y */
%{
    double memvar;
}%

/*To define possible symbol types*/
%union
{
    double dval;
}

```



```

/*Tokens used which are returned by lexer*/
%token <dval> NUMBER
%token <dval> MEM
%token LOG SINE nLOG COS TAN

/*Defining the precedence and associativity*/
%left '-' '+' /*Lowest precedence*/
%left '*' '/'
%right '^'
%left LOG SINE nLOG COS TAN /*Highest precedence*/

/*No associativity*/
%nonassoc UMINUS /*Unary Minus*/

/*Sets the type for non-terminal*/
%type <dval> expression

%%
/*Start state*/
start: statement '\n'
      | start statement '\n'
      ;

/*For storing the answer(memory)*/
statement: MEM '=' expression {memvar = $3;}
          | expression {printf("Answer = %g\n", $1);}
          ; /*For printing the answer*/

/*For binary arithmetic operators*/
expression: expression '+' expression {$$ = $1 + $3;}
          | expression '-' expression {$$ = $1 - $3;}
          | expression '*' expression {$$ = $1 * $3;}
          | expression '/' expression
          { /*To handle divide by zero case*/
            if($3 == 0)
              yyerror("divide by zero");
            else
              $$ = $1 / $3;
          }
          | expression '^' expression {$$ = pow($1, $3);}
          ;

/*For unary operators*/
expression: '-' expression %prec UMINUS {$$ = -$2;}
          /*%prec UMINUS signifies that unary minus should have
            the highest precedence*/
          | '(' expression ')' {$$ = $2;}
          | LOG expression {$$ = log($2)/log(10);}
          | nLOG expression {$$ = log($2);}
          /*Trigonometric functions*/
          | SINE expression {$$ = sin($2 * 3.141592654 / 180);}
          | COS expression {$$ = cos($2 * 3.141592654 / 180);}
          | TAN expression {$$ = tan($2 * 3.141592654 / 180);}
          | NUMBER {$$ = $1;}

```

```

| MEM      {$$ = memvar;}
; /*Retrieving the memory contents*/

%%
main()
{
    printf("Enter the expression: ");
    yyparse();
}
int yyerror(char *error)
{
    fprintf(stderr, "%s\n", error);
}

```

Multiple Choice Questions for Mid Term Exam

- Q.1 ____ parser use the principle of bottom up parsing. [JNTU : Aug.-16]
- ☐ a Recursive decent ☐ b Shift-reduce
☐ c Predictive, ☐ d None
- Q.2 A top - down parser generates _____. [JNTU : Aug.-16]
- ☐ a left most derivation
☐ b right-most derivation
☐ c right most derivation in reverse
☐ d all of these
- Q.3 Recursive descent parser is _____. [JNTU : Aug.-16]
- ☐ a bottom up parser
☐ b top down parser
☐ c all of these
☐ d none of these
- Q.4 A ____ is a data structure containing a record for each identifier, with fields for the attributes of the identifier. [JNTU : Aug.-16]
- ☐ a syntax tree ☐ b heap
☐ c symbol table ☐ d array
- Q.5 YACC source program contains _____. [JNTU : Aug.-16]
- ☐ a declarations,
☐ b translation Rules,
☐ c auxiliary Procedures,
☐ d all the above
- Q.6 ____ for a grammar G is a rule of G with a dot inserted at some position in the right hand side of the rule. [JNTU : Aug.-16]
- ☐ a LR(K) items ☐ b LR (0)
☐ c LR (1) items ☐ d None
- Q.7 Which of these is true about LR PARSING ?
- ☐ a Is most general non - backtracking shift-reduce parsing,
☐ b It is still efficient
☐ c Both a and b
☐ d None of these
- Q.8 The grammar $S \rightarrow aSa|bSc$ is _____. [JNTU : Aug.-16]
- ☐ a LL(1) but not LR(1),
☐ b LR(1) but not LR(1)
☐ c Both LL(1) and LR(1),
☐ d Neither LL(1) nor LR(1)
- Q.9 The construction of the canonical collection of the sets of LR (1) items are similar to the construction of the canonical collection of the sets of LR (0) items, except that _____. [JNTU : Aug.-16]
- ☐ a closure and go to operations work a little bit different.
☐ b closure and go to operations work similarly.
☐ c closure and additive operations work a little bit different.
☐ d closure and associatively operations work a little bit different.
- Q.10 A grammar that produces more than one parse tree for some sentence is called as _____. [JNTU : Feb.-17]
- ☐ a ambiguous ☐ b unambiguous
☐ c regular ☐ d all of these
- Q.11 What is the name of the process that determining whether a string of tokens can be generated by a grammar ? [JNTU : Feb.-17]
- ☐ a Analyzing ☐ b Recognizing
☐ c Translating ☐ d Parsing

3

Syntax Directed Translation and Intermediate Code Generation

Part I : Syntax Directed Translation

3.1 : Syntax Directed Definition

Q.1 What does a semantic analysis do ?

[JNTU : Part A, Nov.-15, Marks 3]

- Ans. : (1) It checks whether all the used identifiers are declared or not.
 (2) It checks whether the identifiers are declared only once.
 (3) It checks the data types and its type compatibility with identifiers and operations.
 (4) It checks the scope of identifiers and functions.
 (5) It checks whether the procedures and functions defined only once.
 (6) It checks whether the procedures and functions are defined with right number and type of arguments.

Q.2 Explain the concept of Syntax Directed Definition (SDD) in detail. [JNTU : Part B, Marks 5]

Ans. : • The syntax-directed definition is a kind of abstract specification. The conceptual view of syntax-directed translation can be as shown in Fig. Q.2.1.

- Firstly we parse the input token stream and a syntax tree is generated. Then the tree is being traversed for evaluating the semantic rules at the parse tree nodes.

- The implementation need not have to follow all the steps given in Fig. Q.2.1. In single pass implementation semantic rules can be evaluated during parsing without explicitly constructing a parse tree, or dependency graph. In such semantic evaluation, at the nodes of the syntax tree, values of the attribute are defined for the given input string. Such a parse tree containing the values of attributes at each node is called an **annotated** or **decorated** the parse tree.

Definition : Syntax-directed definition is a generalization of context free grammar in which each grammar production $X \rightarrow \alpha$ is associated with it a set of semantic rules of the form $a := f(b_1, b_2, \dots, b_k)$, where a is an attribute obtained from the function f .

Q.3 What is attributed Grammar ? Define and explain the synthesized attributes and inherited attributes.

Ans. : An attributed grammar is a context free grammar that has been extended to provide context sensitive information by appending attributes to its non-terminals.

Formal Definition - An attributed grammar is a context free grammar with attributes, semantic rules and condition. Let G be an attribute grammar denoted by $G = (N, \Sigma, P, S)$ where P is a set of production rules in the form $A \rightarrow \alpha$. These rules are associated with set of semantic rules of the form

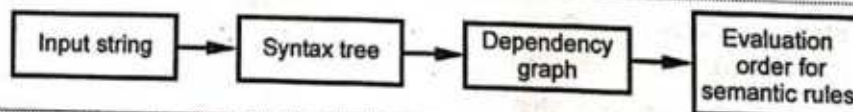


Fig. Q.2.1 Syntax-directed translation

$b : = f(a_1, a_2, a_3, \dots, a_n)$ where f denotes a functionality.

- Each attribute has a domain of possible values.
- An attribute may be assigned values from its domain during parsing.
- The attributes can be evaluated in assignments or conditions.

There are two types of attributes

1) Synthesized attributes 2) Inherited attribute

The semantic analysis phase is responsible for performing syntax directed translation.

1. **Synthesized Attributes** : The attribute 'a' is called synthesized attribute of X and $b_1, b_2, \dots, b_k$ are attributes belonging to the production symbols.

The value of synthesized attribute at a node is computed from the values of attributes at the children of that node in the parse tree.

Example :

Consider the context free grammar as

$S \rightarrow EN$
 $E \rightarrow E + T$
 $E \rightarrow E - T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow T / F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow \text{digit}$
 $N \rightarrow ;$

The syntax-directed definition can be written for the above grammar by using semantic actions for each production.

Production rule	Semantic actions
$S \rightarrow EN$	$\text{Print}(E.\text{val})$
$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$

$E \rightarrow E_1 - T$	$E.\text{val} = E_1.\text{val} - T.\text{val}$
$E \rightarrow T$	$E.\text{val} = T.\text{val}$
$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} \times F.\text{val}$
$T \rightarrow T_1 / F$	$T.\text{val} = T_1.\text{val} / F.\text{val}$
$T \rightarrow F$	$T.\text{val} = F.\text{val}$
$F \rightarrow (E)$	$F.\text{val} = E.\text{val}$
$F \rightarrow \text{digit}$	$F.\text{val} = \text{digit.lexval}$
$N \rightarrow ;$	Can be ignored by lexical analyzer as ; is terminating symbol.

2. **Inherited Attributes** : The attribute 'a' is called inherited attribute of one of the grammar symbol on the right side of the production (i.e. α) and $b_1, b_2, \dots, b_k$ are belonging to either X or α .
The inherited attributes can be computed from the values of the attributes at the siblings and parent of that node.

Consider an example and let us compute the inherited attributes.

Annotate the parse tree for the computation of inherited attributes for the given string : int a, b, c; the grammar is as given below.

$S \rightarrow T L$
 $T \rightarrow \text{int}$
 $T \rightarrow \text{float}$
 $T \rightarrow \text{char}$
 $T \rightarrow \text{double}$
 $L \rightarrow L_1, \text{id}$
 $L \rightarrow \text{pid.}$

For the string int a, b, c we have to distribute the data type int to all the identifiers a, b and c; such that a becomes integer, b becomes integer and c becomes integer. Following steps are to be followed :

1. Construct the syntax-directed definition using semantic action.
2. Annotate the parse tree with inherited attributes by processing in top-down fashion.

The syntax-directed definition for the above given grammar is

Production rule	Semantic actions
$S \rightarrow T L$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{float}$	$T.type := \text{float}$
$T \rightarrow \text{char}$	$T.type := \text{char}$
$T \rightarrow \text{double}$	$T.type := \text{double}$
$L \rightarrow L_1, \text{id}$	$L_1.in := L.in$ $\text{Enter_type}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{Enter_type}(\text{id.entry}, L.in)$

The annotated parse tree is as shown in Fig. Q.3.1.

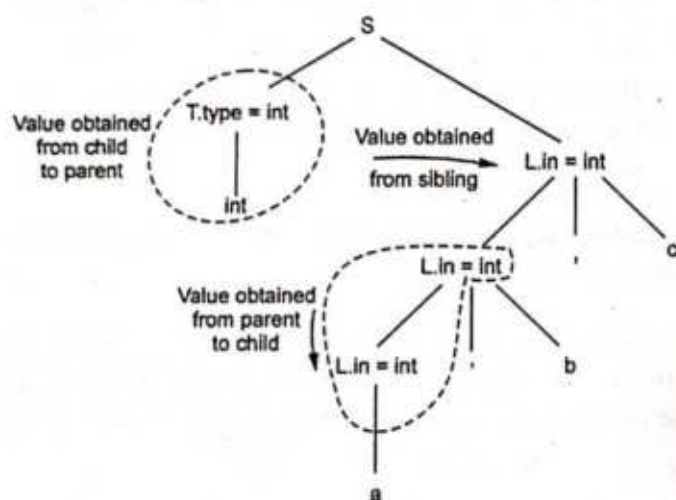


Fig. Q.3.1 Annotated parse tree

Q.4 Define and write the difference between synthesized attributes and inherited attributes.

[JNTU : Part A, Dec.-16, Marks 3]

Ans. :

Sr. No.	Synthesized translation	Inherited translation
1.	The attribute 'a' is called synthesized attribute of X and $b_1, b_2, \dots, b_k$ are attributes belonging to the production symbols. When $x \rightarrow \alpha$ is a rule associated with $a := f(b_1, b_2, \dots, b_k)$	The attribute 'a' is called inherited attribute of one of the grammar symbol on the right side of the production and $b_1, b_2, \dots, b_k$ are belonging to either X or production α . When $x \rightarrow \alpha$ is a rule associated with $a := f(b_1, b_2, \dots, b_k)$

2.	In synthesized translation the synthesized attributes are computed.	In inherited translation, the inherited attributes are computed.
3.	The synthesized attributes are computed using values of children.	The inherited attributes are computed using values of parent and sibling nodes.
4.	Information is passed up in syntax tree.	Information is passed down in syntax tree.
5.	Computing synthesized attributes is simple.	Computing inherited attributes is complex.
6.	Example :	Example :

Q.5 Develop a syntax directed definition for following grammar

$E \rightarrow TE'$
 $E \rightarrow +TE' | \epsilon$
 $T \rightarrow (E)$
 $T \rightarrow \text{id}$

[JNTU : Part B, Marks 5]

Ans. : The syntax directed definition is as follows.

Production rule	Semantic rule
$E \rightarrow TE'$	$E'.inh = T.val$ $E'.val = E'.syn$
$E' \rightarrow +TE'$	$E'_1.inh = E'.inh + T.val$ $E'.syn = E'_1.syn$
$E' \rightarrow \epsilon$	$E'.syn = E'.inh$
$T \rightarrow (E)$	$T.val = E.val$
$T \rightarrow \text{id}$	$T.val = \text{id}.lexval$

3.2 : Evaluation Orders for SDD

Q.6 Write short note on - dependency graph.

[JNTU : Part B, Jan.-10, Marks 5]

Ans. : • The directed graph that represents the interdependencies between synthesized and inherited attributes at nodes in the parse tree is called dependency graph.

• For the rule $X \rightarrow YZ$ the semantic action is given by $X.x := f(Y.y, Z.z)$ then

• Synthesized attribute is $X.x$ and $X.x$ depends upon attributes $Y.y$ and $Z.z$.

For example - Consider following grammar for representing its dependency graph

$$E \rightarrow E_1 + E_2$$

$$E \rightarrow E_1 * E_2$$

The semantic rules for the above grammar is as given below.

Production rule	Semantic rule
$E \rightarrow E_1 + E_2$	$E.val := E_1.val + E_2.val$
$E \rightarrow E_1 * E_2$	$E.val := E_1.val \times E_2.val$

The dependency graph is as shown in Fig. Q.6.1.

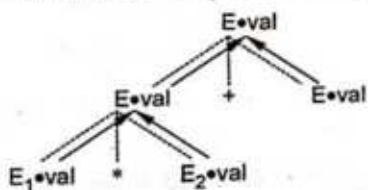


Fig. Q.6.1 Dependency graph

The synthesized attributes can be represented by $\cdot val$. Hence the synthesized attributes are given by $E.val$, $E_1.val$ and $E_2.val$. The dependencies among the nodes is given by solid arrows. The arrows from E_1 and E_2 show that value of E depends upon E_1 and E_2 . We have represented parse tree using dotted lines.

Q.7 Design the dependency graph for the following grammar.

$S \rightarrow T \text{ List}$
 $T \rightarrow \text{int}$
 $T \rightarrow \text{float}$
 $T \rightarrow \text{char}$
 $T \rightarrow \text{double}$
 $\text{List} \rightarrow \text{List}_1, \text{id}$
 $\text{List} \rightarrow \text{id}$

Ans. : The dotted line is for representing the parse tree.

The semantic rules for the above grammar is as given below.

Production rule	Semantic actions
$S \rightarrow T \text{ List}$	$\text{List.in} := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{float}$	$T.type := \text{float}$
$T \rightarrow \text{char}$	$T.type := \text{char}$
$T \rightarrow \text{double}$	$T.type := \text{double}$
$\text{List} \rightarrow \text{List}_1, \text{id}$	$\text{List}_1.in := \text{List.in}$ $\text{Enter_type}(\text{id.entry}, \text{List.in})$
$\text{List} \rightarrow \text{id}$	$\text{Enter_type}(\text{id.entry}, \text{List.in})$

The dependency graph is as shown in Fig. Q.7.1.

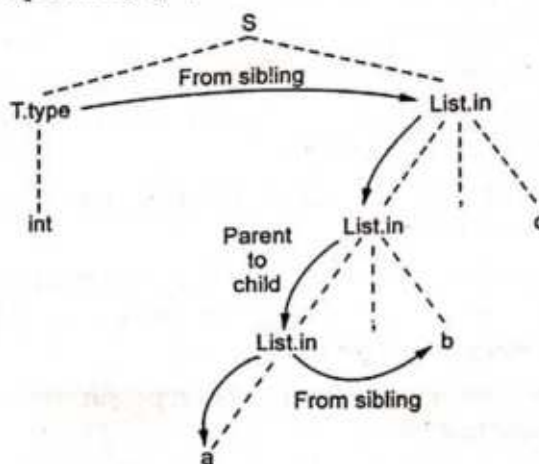


Fig. Q.7.1 Dependency graph

The dependencies among the nodes can be shown by solid arrows. In the above drawn dependency graph how the values can be inherited from the parent or sibling node is shown clearly. Hence the name for the attributes is inherited attributes.

Q.8 Explain the evaluation order for SDD.

Ans. : • The topological sort of the dependency graph decides the evaluation order in a parse tree.

• In deciding evaluation order the semantic rules in the syntax-directed definitions are used. Thus the translation is specified by syntax-directed definitions. Therefore the precise definition of syntax-directed definition is required.

Q.13 Write a context free grammar for the arithmetic expressions. Develop a syntax directed definition for the grammar. Draw an annotated parse tree for the input expression : $(3*2+2) * 4$.

[JNTU : Part B, Marks 5]

Ans. : The annotated parse tree can be obtained from the syntax directed definition given in Table Q.13.1.

Syntax directed definition

Production Rule	Semantic Actions
$S \rightarrow E$	Print ($E \cdot \text{val}$)
$S \rightarrow E_1 + T$	$E \cdot \text{val} := E_1 \cdot \text{Val} + T \cdot \text{Val}$
$E \rightarrow T$	$E \cdot \text{val} := T \cdot \text{val}$
$T \rightarrow T_1 * F$	$T \cdot \text{val} := T_1 \cdot \text{Val} \times F \cdot \text{Val}$
$T \rightarrow F$	$T \cdot \text{val} := F \cdot \text{Val}$
$F \rightarrow (E)$	$F \cdot \text{Val} := E \cdot \text{val}$
$F \rightarrow \text{digit}$	$F \cdot \text{Val} := \text{digit} \cdot \text{lexval}$

Table Q.13.1

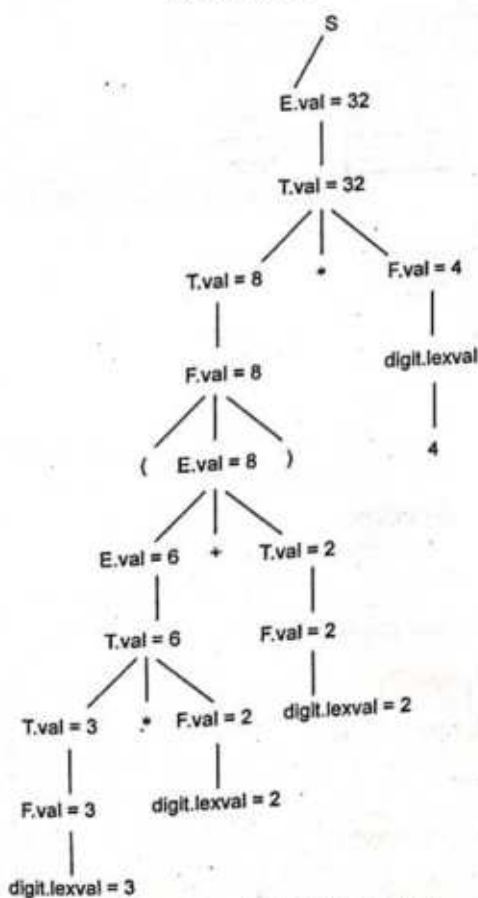


Fig. Q.13.1 Annotated parse tree

3.3 : Applications of Syntax Directed Translation

Q.14 Explain the construction of syntax trees for expression used for syntax directed translation.

Ans. : • The syntax tree is an abstract representation of the language constructs. The syntax trees are used to write the translation routines using syntax-directed definitions.

• The grammar considered for the expression is

$$E \rightarrow E_1 + T$$

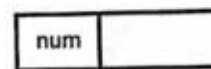
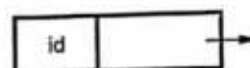
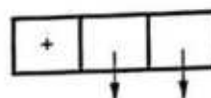
$$E \rightarrow E_1 - T$$

$$E \rightarrow E_1 * T$$

$$E \rightarrow T$$

$$T \rightarrow \text{id}$$

$$T \rightarrow \text{num}$$



- Constructing syntax tree for an expression means translation of expression into postfix form.
- The nodes for each operator and operand is created.
- Each node can be implemented as a record with multiple fields.
- Following are the functions used in syntax tree for expression.

1. **mknode(op,left,right)** : This function creates a node with the field operator having operator as label, and the two pointers to left and right.
2. **mkleaf(id,entry)** : This function creates an identifier node with label id and a pointer to symbol table is given by 'entry'.
3. **mkleaf(num,val)** : This function creates node for number with label num and val is for value of that number.

• For example - Consider, expression $x*y-5+z$.

Step 1 : Convert the expression from infix to postfix $xy*5-z+$.

Step 2 : Make use of the functions `mknnode()`, `mkleaf(id,ptr)` and `mkleaf(num,val)`.

Step 3 : The sequence of function calls is given.

Postfix expression $xy*5 - z+$

Symbol	Operation
x	$p_1 = \text{mkleaf}(\text{id}, \text{ptr to entry x})$
y	$p_2 = \text{mkleaf}(\text{id}, \text{ptr to entry y})$
*	$p_3 = \text{mknnode}(*, p_1, p_2)$
5	$p_4 = \text{mkleaf}(\text{num}, 5)$
-	$p_5 = \text{mknnode}(-, p_3, p_4)$
z	$p_6 = \text{mkleaf}(\text{id}, \text{ptr to entry z})$
+	$p_7 = \text{mknnode}(+, p_5, p_6)$

Consider the string $x*y-5+z$ and let us draw the syntax tree.

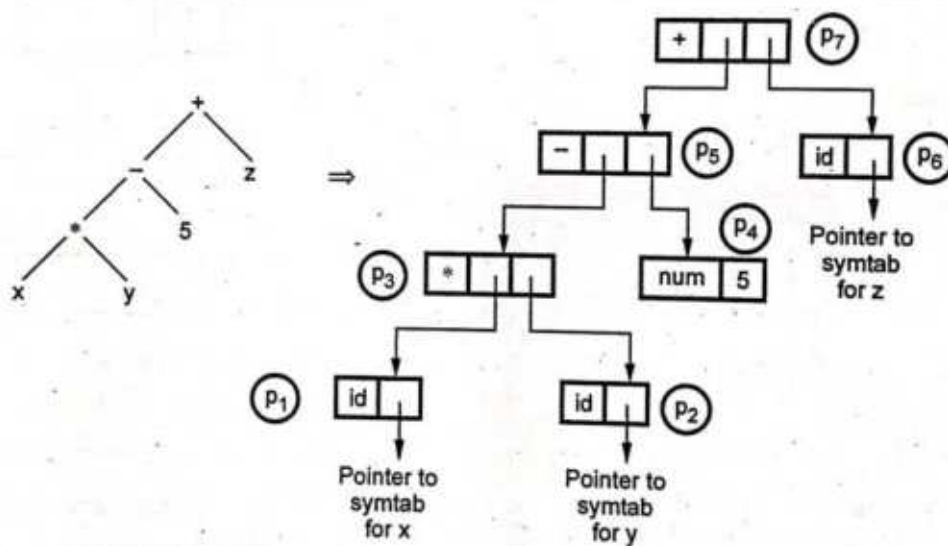


Fig. Q.14.1 Syntax tree

The syntax-directed definition for the above grammar is as given below.

Production rule	Semantic operation
$E \rightarrow E_1 + T$	$E.nptr := \text{mknnode}('+', E_1.nptr, T.nptr)$
$E \rightarrow E_1 - T$	$E.nptr := \text{mknnode}('-', E_1.nptr, T.nptr)$
$E \rightarrow E_1 * T$	$E.nptr := \text{mknnode}('*', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr := T.nptr$
$T \rightarrow \text{id}$	$E.nptr := \text{mkleaf}(\text{id}, \text{id}, \text{ptr_entry})$
$T \rightarrow \text{num}$	$T.nptr := \text{mkleaf}(\text{num}, \text{num}, \text{val})$

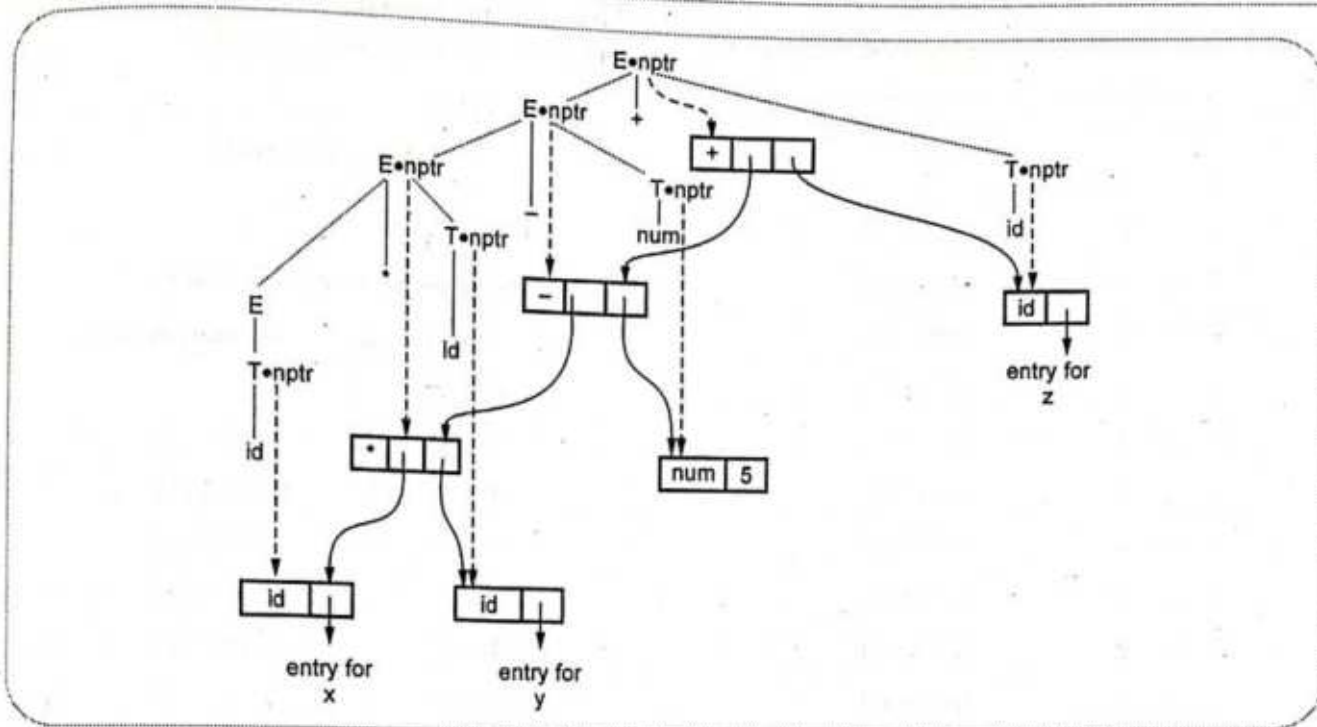


Fig. Q.14.1 (a) Constructed syntax tree

Q.15 Define syntax tree. What is s-attributed definition? Explain construction of syntax tree for the expression $a - 4 + c$ using SDD.

[JNTU : Part B, Marks 5]

Ans. : Syntax tree : It is hierarchical arrangement of tokens generated by lexical analysis.

S-attributed definition : Refer Q.20.

The syntax tree is :

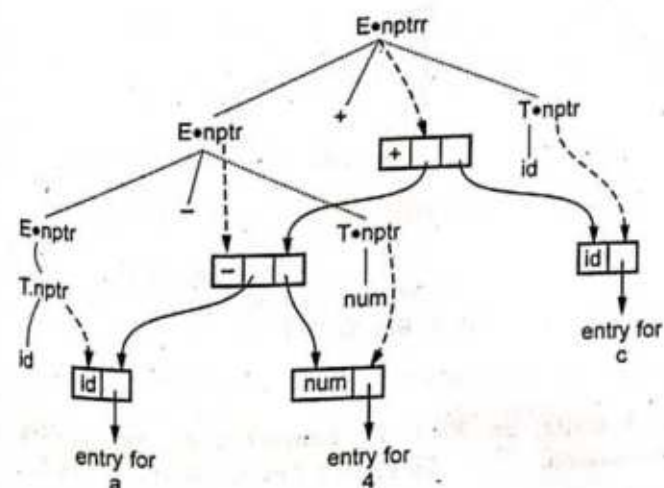


Fig. Q.15.1 Syntax tree

3.4 : Syntax Directed Translation Schemes

Q.16 Give the translation scheme that converts infix to postfix form for the following grammar. Also generate the annotated parse tree for input string $2+6+1$.

$E \rightarrow E+T$ {print('+')}

$E \rightarrow T$

$T \rightarrow 0$ {print('0')}

$T \rightarrow 1$ {print('1')}

$T \rightarrow 2$ {print('2')}

$T \rightarrow 3$ {print('3')}

$T \rightarrow 4$ {print('4')}

$T \rightarrow 5$ {print('5')}

$T \rightarrow 6$ {print('6')}

$T \rightarrow 7$ {print('7')}

$T \rightarrow 8$ {print('8')}

$T \rightarrow 9$ {print('9')}

[JNTU : Part B, Marks 5]

Ans. : Let us convert this grammar into non left recursive form

$E \rightarrow TP$

$T \rightarrow 0|1|2|3|4|5|6|7|8|9$

$P \rightarrow +TP \mid \epsilon$

The translation scheme for this grammar is as given below.

Productions	Semantic actions
$E \rightarrow TP$	
$T \rightarrow 0$	{print('0')}
$T \rightarrow 1$	{print('1')}
$T \rightarrow 2$	{print('2')}
$T \rightarrow 3$	{print('3')}
$T \rightarrow 4$	{print('4')}
$T \rightarrow 5$	{print('5')}
$T \rightarrow 6$	{print('6')}
$T \rightarrow 7$	{print('7')}
$T \rightarrow 8$	{print('8')}
$T \rightarrow 9$	{print('9')}
$P \rightarrow +TP \mid \epsilon$	{print('+')} P ϵ

The annotated parse tree for the string 2+6+1 is as given in Fig. Q.16.1.

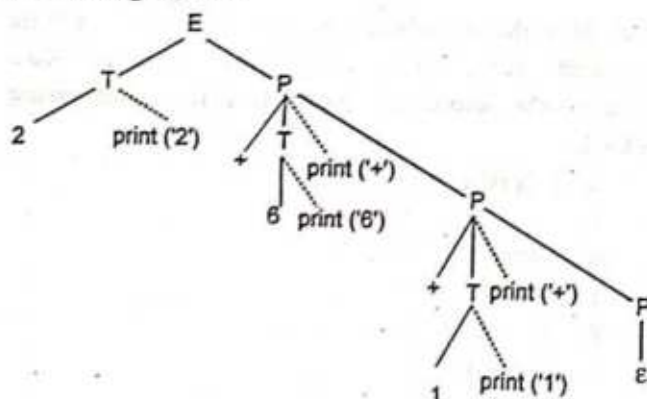


Fig. Q.16.1 Annotated parse tree

The translation scheme proceeds using depth first traversal. The annotated parse tree shows the semantic actions associated appropriate node. Thus this tree shows the translation scheme for mapping of infix form to postfix form. The associated semantic actions can be shown using dotted lines.

Q.17 Give the translation scheme that converts infix to postfix expression for the following grammar and also generate the annotated parse tree for input string 7+3+2 $E \rightarrow E+T$, $E \rightarrow T$, $T \rightarrow 0|1|2|3|4|5|6|7|8|9$

[JNTU : Part B, Marks 5]

Ans. : Let us convert this grammar into a form in which the left recursion is removed

$$E \rightarrow TP$$

$$T \rightarrow 0|1|2|3|4|5|6|7|8|9$$

$$P \rightarrow +TP \mid \epsilon$$

The translation scheme will be

Production	Semantic Actions
$E \rightarrow TP$	
$T \rightarrow 0$	{ Print ('0') }
$T \rightarrow 1$	{ Print ('1') }
$T \rightarrow 2$	{ Print ('2') }
$T \rightarrow 3$	{ Print ('3') }
$T \rightarrow 4$	{ Print ('4') }
$T \rightarrow 5$	{ Print ('5') }
$T \rightarrow 6$	{ Print ('6') }
$T \rightarrow 7$	{ Print ('7') }
$T \rightarrow 8$	{ Print ('8') }
$T \rightarrow 9$	{ Print ('9') }
$P \rightarrow +TP \mid \epsilon$	{ Print ('+') } P ϵ

The annotated parse tree for the string 7+3+2 is as given below

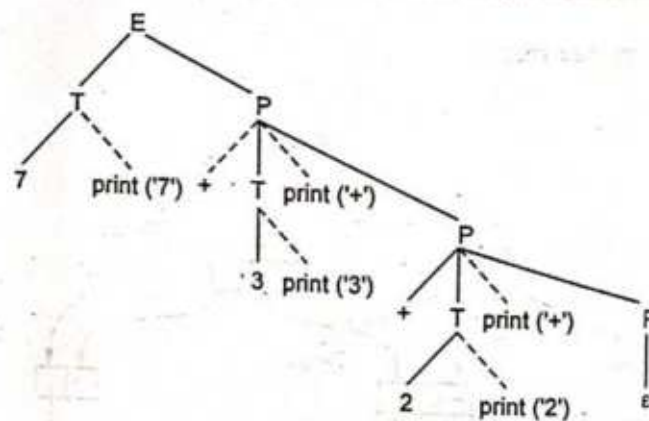


Fig. Q.17.1

Q.18 Write an SDT to convert infix to postfix expression. [JNTU : Part B, Nov.-16, Marks 5]

Ans. : Refer Q.16.

Q.19 Construct an annotated parse tree for $9-5+2$. [JNTU : Part B, March-16, Marks 5]

Ans. : We will first draw the simple parse tree for given expression.

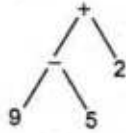


Fig. Q.19.1

Now let us write syntax directed definition for given expression evaluation.

Production	Semantic rules
$\text{expr} \rightarrow \text{expr}_1 + \text{term}$	$\text{expr.t} = \text{expr}_1.t \parallel \text{term.t} \parallel '+'$
$\text{expr} \rightarrow \text{expr}_1 - \text{term}$	$\text{expr.t} = \text{expr}_1.t \parallel \text{term.t} \parallel '-'$
$\text{expr} \rightarrow \text{term}$	$\text{expr.t} = \text{term.t}$
$\text{term} \rightarrow 0$	$\text{term.t} = '0'$
$\text{term} \rightarrow 1$	$\text{term.t} = '1'$
...	...
$\text{term} \rightarrow 9$	$\text{term.t} = 9$

The annotated parse tree is

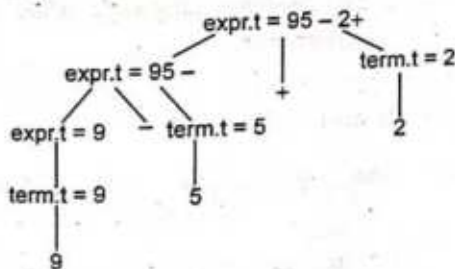


Fig. Q.19.2 Evaluation of $9-5+2$ into $95-2+$

3.5 : Implementing L Attributed SDD

Q.20 What is S and L attributes ?

[JNTU : Part A, March-17, Marks 2]

Ans. : S-Attributes : The grammar that contains all the syntactic rules along with the semantic rules having synthesized attributes only, is called s-attributed definitions.

L-Attributes : The syntax-directed definition can be defined as the L-attributed for the production rule

$A \rightarrow X_1 X_2 \dots X_n$ where the inherited attribute X_k is such that $1 \leq k \leq n$. The production $A \rightarrow X_1 X_2 \dots X_n$ is such that

- 1) It depends upon the attributes of the symbol $X_1, X_2, \dots, X_{j-1}$ to the left of X_j .
- 2) It also depends upon the inherited attribute A.

Q.21 Explain why every S-attributed definition is L-attributed. [JNTU : Part A, Marks 3]

Ans. : Following are the restrictions that can be applied on inherited attributes of X_k where $1 \leq k \leq n$ and $A \rightarrow X_1 X_2 \dots X_n$.

- i) Suppose $A \rightarrow X_1 X_2 \dots X_{j-1} X_j \dots X_n$ is a production then attribute of X_j should depend upon attributes of $X_1 X_2 \dots X_{j-1}$.
- ii) The attributes of X_k can be dependant upon inherited attributes of A.

Because of these restrictions on inherited attributes every S-attributed definition is L-attributed.

Q.22 Differentiate between S-attributed grammar and L-attributed grammar. [JNTU : Part B, Marks 4]

Ans. :

Sr. No.	S-attributed grammar	L-attributed grammar
1.	This is a class of attributed grammar having no inherited attributes but only synthesized attributes.	This is the class of grammar in which the attributes to be evaluated in one left-to-right traversal of the abstract syntax tree. This is a class of attributed grammar in which inherited attributes can be evaluated.
2.	The S-attributed grammar can be incorporated in both top down parsing and bottom up parsing.	The L-attributed grammar is incorporated in using top down parsing.
3.	The YACC family can be broadly considered as S-attributed grammar.	Many programming languages are L-attributed. Special types of compilers, the narrow compilers, are based on some form of L-attributed grammar.

4.	S-attributed grammar can be L-attributed grammar.	L-attributed grammar can not be S-attributed.
----	---	---

Q.23 Write S - attributed grammar to convert the given grammar with infix operators to prefix operators.

$L \rightarrow E, E \rightarrow E + T, E \rightarrow E - T, E \rightarrow T,$

$T \rightarrow T * F, T \rightarrow T / F, T \rightarrow F$

$F \rightarrow F \uparrow P, F \rightarrow P, P \rightarrow (E), P \rightarrow id$

[JNTU : Part B, Dec.-05, Marks 8]

Ans. : The grammar that contains all the syntactic rules along with the semantic rules having synthesized attributes only, is called S - attributed grammar. Such a grammar for converting infix operators to prefix is given by using the 'val' as S - attribute.

Production Rule	Semantic Rule
$L \rightarrow E$	$L \cdot val := E \cdot val$
$E \rightarrow E + T$	$E \cdot val := '+' E \cdot val T \cdot val$
$E \rightarrow E - T$	$E \cdot val := '-' E \cdot val T \cdot val$
$E \rightarrow T$	$E \cdot val := T \cdot val$
$T \rightarrow T * F$	$T \cdot val := '*' T \cdot val F \cdot val$
$T \rightarrow T / F$	$T \cdot val := '/' T \cdot val F \cdot val$
$T \rightarrow F$	$T \cdot val := F \cdot val$
$F \rightarrow F \uparrow P$	$F \cdot val := '\uparrow' F \cdot val P \cdot val$
$F \rightarrow P$	$F \cdot val := P \cdot val$
$P \rightarrow (E)$	$P \cdot val := E \cdot val$
$P \rightarrow id$	$P \cdot val := id \cdot lexval$

Q.24 Consider the following grammar G.

$E^1 \rightarrow E$

$E \rightarrow E + n | n$

Give the parsing action of a bottom up parser for the string $n + n$. [JNTU : Part B, May-12, Marks 8]

Ans. : The syntax directed translation for given CFG will be as follows -

Production rule	Code fragment
$E' \rightarrow E$	Print (value [top])

$E \rightarrow E + n$	value [top] := value [top - 2] + value [top]
$E \rightarrow n$	

The sequence of moves in bottom up parsing of the string $n + n$ is

Input string	State	Value	Production rule used
$n + n$	-	-	
$+ n$	n	n	
$+ n$	E	n	$E \rightarrow n$
n	E+	n -	
	E + n	n - n	
	E	value of $n + n$	$E \rightarrow E + n$

Part II : Intermediate Code Generation

3.6 : Concept of Intermediate Code

Q.25 What is intermediate code ?

Ans. : • During the translation of a source program into the object code for a target machine, a compiler may generate a middle-level language code, which is known as intermediate code.

• The complexity of this code lies between the source language code and the object code.

Q.26 Write the properties of intermediate language. [JNTU : Part A, Marks 4]

Ans. : Following are the properties of intermediate languages :

1. The intermediate language is an easy form of source language which can be generated efficiently by the compiler.
2. The generation of intermediate language should lead to efficient code generation.
3. The intermediate language should act as effective mediator between front and back end.
4. The intermediate language should be flexible enough so that optimized code can be generated.

Q.27 List out the benefits of using machine independent intermediate forms.

[JNTU : Part A, Marks 3]

Ans. : There are certain benefits of generating machine independent intermediate code :

1. A compiler for different machines can be created by attaching different backend to the existing front ends of each machine.
2. A compiler for different source languages (on the same machine) can be created by providing different front ends for corresponding source languages to existing back end.
3. A machine independent code optimizer can be applied to intermediate code in order to optimize the code generation.

3.7 : Variants of Syntax Trees

Q.28 Explain different intermediate forms.

[JNTU : Part B, Marks 5]

Ans. : 1. Abstract syntax tree

The natural hierarchical structure is represented by syntax trees. Directed Acyclic Graph or DAG is very much similar to syntax trees but they are in more compact form.

The code being generated as intermediate should be such that the remaining processing of the subsequent phases should be easy.

Consider the input string $x := -a*b + -a*b$ for the syntax tree and DAG representation.

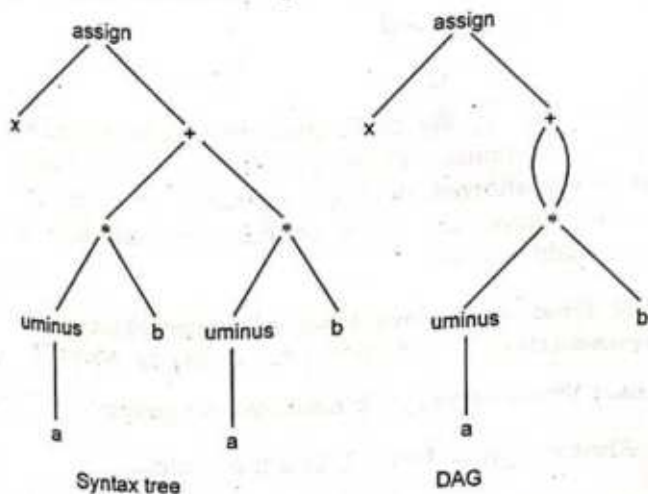


Fig. Q.28.1 Syntax tree and DAG

2. Polish notation

Basically, the linearization of syntax trees is polish notation. In this representation, the operator can be easily associated with the corresponding operands. This is the most natural way of representation in expression evaluation.

The polish notation is also called as **Prefix notation** in which the operator occurs first and then operands are arranged.

For example :

$$(a + b) * (c - d)$$

can be written as

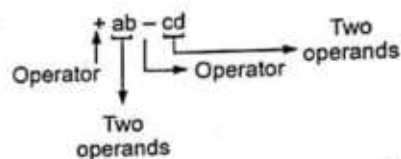


Fig. Q.28.2 Example of polish notation

3. Three address code

In three address code form at the most three addresses are used to represent any statement. The general form of three address code representation is -

$$a := b \text{ op } c$$

where a, b or c are the operands that can be names, constants, compiler generated temporaries and op represents the operator. The operators can be fixed or floating point arithmetic operator or logical operators on Boolean valued data. Only single operation at right side of the expression is allowed at a time.

For the expression like $a = b + c + d$ the three address code will be

$$t_1 := b + c$$

$$t_2 := t_1 + d$$

$$a = t_2$$

Here t_1 and t_2 are the temporary names generated by the compiler.

Q.29 Draw syntax tree for the arithmetic expressions $a * (b + c) - d/2$

Also write the given expression in postfix notation.

[JNTU : Part B, June-14, Marks 5]

Ans. :

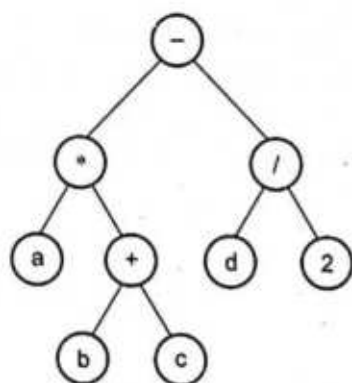


Fig. Q.29.1 Syntax tree

The postfix for is

$$a * (b + c) - d / 2$$

$$a * T_1 - d / 2 \quad T_1 = bc +$$

$$T_2 - d / 2 \quad T_2 = aT_1 *$$

$$T_2 - T_3 \quad T_3 = d2 /$$

$$T_4 \quad T_4 = T_2 T_3 -$$

By backward substitution we get,

$$T_4$$

$$T_2 T_3 -$$

$$aT_1 * T_3 -$$

$$a bc + T_3 -$$

$$abc + * d2 / - \text{ is postfix expression}$$

Q.30 What is the difference between parse tree and syntax tree ? Write appropriate grammar and draw parse as well as syntax tree for $a*(a-a^{\wedge}a)$.

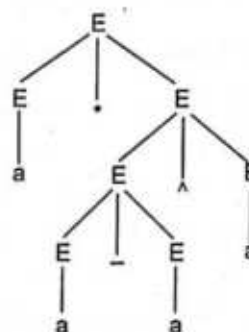
[JNTU : Part B, Marks 5]

Ans. : Parse tree represents concrete syntax of the grammar whereas syntax tree represents the abstract syntax of the grammar.

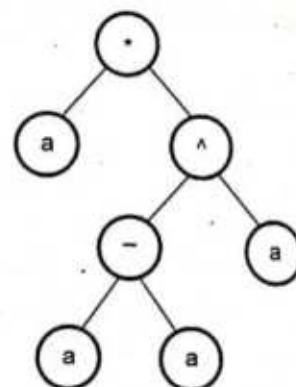
Parse trees have interior nodes as non-terminals and leaf nodes as terminals.

In syntax tree interior nodes are operators and leaves are operands.

For example :



Parse tree



Syntax tree

Fig. Q.30.1

3.8 : Three Address Code

Q.31 What is three address code ?

Ans. : In three address code form at the most three addresses are used to represent any statement. The general form of three address code representation is -

$$a := b \text{ op } c$$

where a, b or c are the operands that can be names, constants, compiler generated temporaries and op represents the operator. The operators can be fixed or floating point arithmetic operator or logical operators on Boolean valued data. Only single operation at right side of the expression is allowed at a time.

For the expression like $a = b+c+d$ the three address code will be

$$t_1 := b+c$$

$$t_2 := t_1 + d$$

$$a = t_2$$

Here t_1 and t_2 are the temporary names generated by the compiler. There are at the most three addresses allowed (two for operands and one for result). Hence, the name of this representation is three-address code.

Q.32 What are various types of intermediate code representation ? [JNTU : Part A, May-18, Marks 2]

Ans. : Various types of intermediate languages are

1. Abstract Syntax Tree
2. Polish Notations
3. Three Address Code.

Q.33 Explain quadruple, triple and indirect triple with suitable example.

[JNTU : Part B, Marks 5]

Ans. : Three address code is an abstract form of intermediate code that can be implemented as a record with the address fields. There are three representations used for three address code such as quadruples, triples and indirect triples.

Quadruple representation

The quadruple is a structure with at the most four fields such as op, arg1, arg2, result. The op field is used to represent the internal code for operator, the arg1 and arg2 represent the two operands used and result field is used to store the result of an expression.

For example : Consider the input statement $x := -a * b + -a * b$

Quadruple

The three address code is

$t_1 := \text{uminus } a$
 $t_2 := t_1 * b$
 $t_3 := -a$
 $t_4 := t_3 * b$
 $t_5 := t_2 + t_4$
 $x := t_5$

	Op	Arg1	Arg2	result
(0)	uminus	a		t_1
(1)	*	t_1	b	t_2
(2)	uminus	a		t_3
(3)	*	t_3	b	t_4
(4)	+	t_2	t_4	t_5
(5)	:=	t_5		x

Triples

In the triple representation the use of temporary variables is avoided by referring the pointers in the symbol table.

For the expression $x := -a * b + -a * b$ the triple representation is as given below

Number	Op	Arg1	Arg2
(0)	uminus	a	
(1)	*	(0)	b
(2)	uminus	a	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	:=	x	(4)

Indirect triples

In the indirect triple representation the listing of triples is been done. And listing pointers are used instead of using statements.

Number	Op	Arg1	Arg2
(0)	uminus	a	
(1)	*	(11)	b
(2)	uminus	a	
(3)	*	(13)	b
(4)	+	(12)	(14)
(5)	:=	x	(15)

	Statement
(0)	(11)
(1)	(12)
(2)	(13)
(3)	(14)
(4)	(15)
(5)	(16)

Q.34 Write the quadruple, triple, indirect triple for the statement
 $a := b * -c + b * -c$

[JNTU : Part B, May-08, Marks 8]

Ans. : The three address code is

$t_1 := \text{uminus } c$

$t_2 := b * t_1$

$t_3 := \text{uminus } c$

$t_4 := b * t_3$

$t_5 := t_2 + t_4$

$a := t_5$

From above three address code we can obtain quadruple, triple as :

The quadruple representation is

	Operator	Operand 1	Operand 2	Result
(0)	uminus	c		t_1
(1)	*	b	t_1	t_2
(2)	uminus	c		t_3
(3)	*	b	t_3	t_4
(4)	+	t_2	t_4	t_5
(5)	:=	t_5		a

The triple representation is

	op	operand 1	operand 2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	:=	a	(4)

The indirect triple is

	op	operand 1	operand 2
(0)	uminus	c	
(1)	*	b	(10)
(2)	uminus	c	
(3)	*	b	(12)
(4)	+	(1)	(13)
(5)	:=	a	(14)

	Statement
(0)	(10)
(1)	(11)
(2)	(12)
(3)	(13)
(4)	(14)
(5)	(15)

Q.35 Write quadruple, triple for the following expression.

$$(x + y) * (y + z) + (x + y + z)$$

[JNTU : Part B, June-14, Marks 5]

Ans. : Three address code

$$t_1 := x + y$$

$$t_2 := y + z$$

$$t_3 := t_1 * t_2 / (x + y) * (y + z) /$$

$$t_4 := x + t_2 / (x + y + z) /$$

The Quadruple representation is -

	Operator	Operand1	Operand2	Result
(0)	+	x	y	t_1
(1)	+	y	z	t_2
(2)	*	t_1	t_2	t_3
(3)	+	x	t_2	t_4
(4)	+	t_3	t_4	t_5

The triple representation is -

	Operator	Operand1	Operand2
(0)	+	x	y
(1)	+	y	z
(2)	*	(0)	(1)
(3)	+	x	(1)
(4)	+	(2)	(3)

Q.36 Translate the following expression :

$$(a+b)*(c+d)+(a+b+c)$$

(a) Quadruple (b) Triples (c) Indirect Triple

[JNTU : Part B, March-17, Marks (3+3+4)]

Ans. : The Three address code for given expression is as follows -

$$t_1 := a + b$$

$$t_2 := c + d$$

$$t_3 := t_1 * t_2$$

$$t_4 := t_3 + c$$

$$t_5 := t_3 + t_4$$

(a) Quadruple

	Operator	Operand1	Operand2	Result
(0)	+	a	b	t_1
(1)	+	c	d	t_2
(2)	*	t_1	t_2	t_3
(3)	+	t_1	c	t_4
(4)	+	t_3	t_4	t_5

(b) Triple

	Operator	Operand1	Operand2
(0)	+	a	b
(1)	+	c	d
(2)	*	(0)	(1)
(3)	+	(0)	c
(4)	+	(2)	(3)

(c) Indirect Triple

	Operator	Operand 1	Operand 2	Statement
(0)	+	a	b	(0)
(1)	+	c	d	(1)
(2)	*	(10)	(11)	(2)
(3)	+	(10)	c	(3)
(4)	+	(12)	(13)	(4)

Q.37 Convert the following statement into triple, indirect triple and quadruple forms. $A = (B + C) * E + (B + C) * F$

[JNTU : PART B, Marks 5]

Ans. : Quadruple

Location	Operator	Operand 1	Operand 2	result
(1)	+	B	C	t_1
(2)	*	t_1	E	t_2
(3)	*	t_1	F	t_3
(4)	+	t_2	t_3	t_4
(5)	=	t_4		A

Triple

Location	Operator	Operand 1	Operand 2
(1)	+	B	C
(2)	\$	(1)	E
(3)	*	(1)	F
(4)	+	(2)	(3)
(5)	=	(4)	A

Indirect Triple

Location	Operator	Operand 1	Operand 2	Statement
(11)	+	B	C	(11)
(12)	\$	(11)	E	(12)
(13)	*	(11)	F	(13)
(14)	+	(12)	(13)	(14)
(15)	=	(14)	A	(15)

3.9 : Types and Declarations**Q.38 What is type system ?**

[JNTU : Part A, Marks 2]

Ans. : • A type system is a collection of rules for assigning type expression to the language constructs.

- The type system is implemented by **type checker**. The type checker finds whether the **types are compatible** to each other or not by finding the corresponding **type expression**.

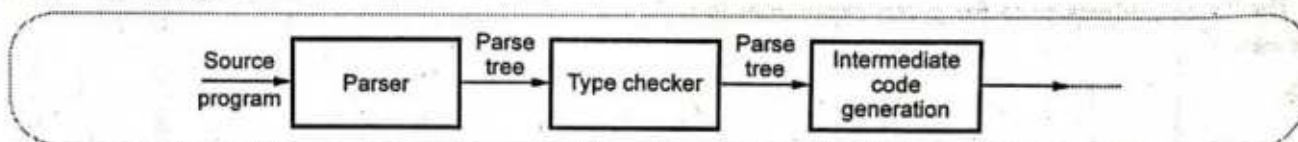
Q.39 Explain in brief the role of type checker in compiler.

[JNTU : Part A, Marks 3]

Ans. : The type analysis and type checking is an important activity done in the semantic analysis phase. The need for type checking is -

- To detect the **errors** arising in the expression due to incompatible operand.
- To generate **intermediate code** for expressions and statements. Typically language supports two types of data types - basic and constructed.

The basic data types are - integer, character, real, Boolean, enumerated data type. And arrays, record(structure), set and pointer are the constructed types. The constructed data types are build using basic data types.

**Fig. Q.39.1 Role of Type checker****Q.40 What is type expression ?**

[JNTU : Part A, March-16, Marks 2]

Ans. : The systematic way of expressing type of language construct is called type expression. The basic type such as int, float, double, char is also a type expression.

Q.41 Define Type Equivalence.

[JNTU : Part A, Dec 17, Marks 2, March 16, Marks 3]

Ans. : Type equivalence means that two types are equivalent if an operand of one type in an expression is substituted for one of the other type, without type conversions.

Q.42 How to check structural equivalence of the two type expressions. [JNTU : Part A, Dec.-16, Marks 2]

Ans. : • When two expressions are the same basic type or formed by applying the same constructor to structurally equivalent types then those expressions are called structurally equivalent.

For example :

S ₁	S ₂	Equivalence	Reason
char	char	S ₁ is equivalent to S ₂	Similar basic types
Pointer (char)	Pointer (char)	S ₁ is equivalent to S ₂	Similar constructor pointer to the char type.

The structural equivalence of two type expressions can be checked using following algorithm.

Q.43 Explain the name equivalence of type expression. [JNTU : Part B, Marks 5]

Ans. : • In the name equivalence the type expressions are given the names. Hence two type expressions are said to be name equivalent if and only if they are identical.

• For example : Consider a code segment as follows-

```
typedef struct Node
{
    int x;
}node;
node *first,*second;
struct Node *last1,*last2;
```

• In the above code the variables first and second are name equivalent similarly variables last1 and last2 are name equivalent. But first and last1 are not name equivalent as the names for these two type expressions are different. Although all the four variables first, second, last1, last2 are similar under structural equivalence since they are representing the same type pointer('struct Node').

Q.44 Explain brief about equivalence of type expressions. [JNTU : Part B, March-16, Marks 5]

Ans. : Refer Q.42 and Q.43

3.10 : Type Checking

Q.45 Explain how the types and relative addresses of declared names are computed and how scope information is dealt with.

[JNTU : Part B, May-18, Marks 10]

Ans. : • The type checker is a translation scheme in which the type of each expression from the types of subexpressions is obtained. The type checker can decide the types for arrays, pointers, statements and functions.

Let us consider a grammar for source language.

$S \rightarrow D;E$

$D \rightarrow D;D \mid T \text{ LIST}$

$T \rightarrow \text{char} \mid \text{int} \mid \text{float}$

$\text{LIST} \rightarrow L_1 [\text{num}] \mid *L_1 \mid \text{id}$

$E \rightarrow \text{literal} \mid \text{num} \mid \text{id} \mid E \text{ mod } E \mid E[E] \mid *E \mid E \text{ op } E \mid E(E)$

The type checker ensures following things -

- Each identifier must be declared before the use.
- The use of identifier must be within the scope.
- An identifier must not have multiple definitions at a time within the same scope.

The given grammar has basic data types as char, int, float and for reporting the errors type_error.

We assume that the index of the array start at 0. For example if

```
int arr[100];
```

leads to type expression as array(0,1...99,int) similarly

```
int *ptr;
```

The type expression for the above statement can be pointer(int)

The constructor pointer is applied to the type integer.

Let us write the translation scheme for the above grammar.

Production rule	Type expression
$S \rightarrow D;E$	This means all declarations before expressions
$D \rightarrow T \text{ LIST}$	$\text{LIST.type} := T.\text{type}$
$\text{LIST} \rightarrow \text{id}$	$\{\text{addtype}(\text{id.entry}, \text{LIST.type})\}$
$T \rightarrow \text{char}$	$\{T.\text{type} := \text{char}\}$
$T \rightarrow \text{int}$	$\{T.\text{type} := \text{int}\}$
$T \rightarrow \text{float}$	$\{T.\text{type} := \text{float}\}$
$\text{LIST} \rightarrow *L_1$	$\{L_1.\text{type} := \text{pointer}(\text{LIST.type})\}$
$\text{LIST} \rightarrow L_1[\text{num}]$	$\{L_1.\text{type} := \text{array}(0 \dots \text{num.val}-1, \text{LIST.type})\}$

Type checker

This translation scheme is given for declaration of identifiers.

Q.46 Explain in brief about type checking and Type Conversion. [JNTU : Part B, Nov.-17, Marks 5]

Ans. : Type Checking - Refer Q.45

Type Conversion - Type conversion is a process of converting one type to another.

There are two types of conversions : implicit conversion and explicit conversion.

If the conversion is done automatically by the compiler then it is called **implicit conversion**. The implicit conversions are also called as **coercions**. In this type of conversion there is no loss of information for example integer can be converted to float but not vice versa. That means in conversion from integer to float there is no loss of information but for converting the information from float to integer there is a loss.

The conversion is said to be **explicit** if the programmer specifically writes something for converting one type to another.

For example -

```
int xyz,p;
p=(float)xyz;
```

The identifier xyz is type-casted and this is how explicit conversion from int to float takes place.

Q.47 Explain briefly about polymorphic functions.

[JNTU : Part B, Nov.-16, March-16, Marks 5]

Ans. : • **Definition :** The functions having same name but different types of arguments are called polymorphic functions.

- Polymorphism is in short the ability to call different functions by just using one type of function call.

- The primary use of polymorphism is **code reuse**.

- An adhoc polymorphism is also called as **overloading**.

- **Definition of overloaded symbol :** An overloaded symbol is a symbol that has different meaning which is dependent upon its context.

For example - Overloading multiplication operator * means $a * b$ where a and b can be integer, real, complex numbers or can be matrices.

- The overloading needs to be resolved by the compiler. The overloading gets **resolved** when unique meaning for corresponding overloaded symbol is determined. The resolution of overloading is called **operator identification**.

- It is not always possible to resolve overloading by looking only at the argument of function - because there may be some subexpressions involved and those subexpressions may have a set of possible types.

For example : Consider an overloaded function, in the language Ada.

The declaration of function * will be -

```
function "*" (m, n : integer) return complex;
function "*" (p, q : complex) return complex;
```

- The arguments of this function can be either integer or complex. Then the possible types for the arguments are treated as subexpressions. The possible types of subexpressions are -

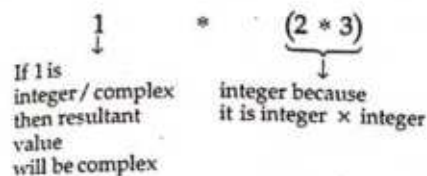
integer * integer $\rightarrow$ integer

integer $\times$ integer $\rightarrow$ complex

complex $\times$ complex $\rightarrow$ complex

• For the integers 1, 2, 3 with above declaration the subexpression $(2 * 3)$ can be integer or complex.

Consider an expression



• But if $(1*2)*3$ is complex then the subexpression $(1*2)$ must be complex.

3.11 : Control Flow

Q.48 Obtain the translation scheme for obtaining the tree address code for the grammar -

$S \rightarrow id := E$
 $E \rightarrow E_1 + E_2$
 $E \rightarrow E_1 * E_2$
 $E \rightarrow - E_1$
 $E \rightarrow (E_1)$
 $E \rightarrow id$

OR

Give the translator scheme to convert the expression grammar into three-address code.

[JNTU : Aug.-07, May-06, 07, Nov.-03, Marks 8]

Ans. : Here we will use the translation scheme which in turn will generate the three address code.

Production Rule	Semantic actions
$S \rightarrow id := E$	<pre> { id_entry := look_up(id.name); if id_entry $\neq$ nil then append(id_entry, ' := ' E.place) else error; /* id not declared */ }</pre>
$E \rightarrow E_1 + E_2$	<pre> { E.place := newtemp(); append(E.place, ' := ' E₁.place '+' E₂.place) }</pre>

$E \rightarrow E_1 * E_2$	<pre> { E.place := newtemp(); append(E.place, ' := ' E₁.place '*' E₂.place) }</pre>
$E \rightarrow - E_1$	<pre> { E.place := newtemp(); append(E.place, ' := ' 'uminus' E₁.place) }</pre>
$E \rightarrow (E_1)$	<pre> { E.place := E₁.place }</pre>
$E \rightarrow id$	<pre> { id_entry := look_up(id.name); if id_entry $\neq$ nil then append(id_entry, ' := ' E.place) else error; /* id not declared */ }</pre>

- The look_up returns the entry for id.name in the symbol table if it exists there.
- The function append is for appending the three address code to the output file. Otherwise an error will be reported.
- newtemp() is the function for generating new temporary variables.
- E.place is used to hold the value of E.

Consider the assignment statement $x := (a+b) * (c+d)$

We will assume all these identifiers are of the same type. Let us have Syntax Directed Definition and annotated parse tree as -

Production Rule	Semantic action for Attribute evaluation	Output
$E \rightarrow id$	$E.place := a$	
$E \rightarrow id$	$E.place := b$	
$E \rightarrow E_1 + E_2$	$E.place := t_1$	$t_1 := a+b$
$E \rightarrow id$	$E.place := c$	
$E \rightarrow id$	$E.place := d$	
$E \rightarrow E_1 + E_2$	$E.place := t_2$	$t_2 := c+d$
$E \rightarrow E_1 * E_2$	$E.place := t_3$	$t_3 := (a+b)*(c+d)$
$S \rightarrow id := E$		$x := t_3$

The annotated parse tree can be drawn as follows -

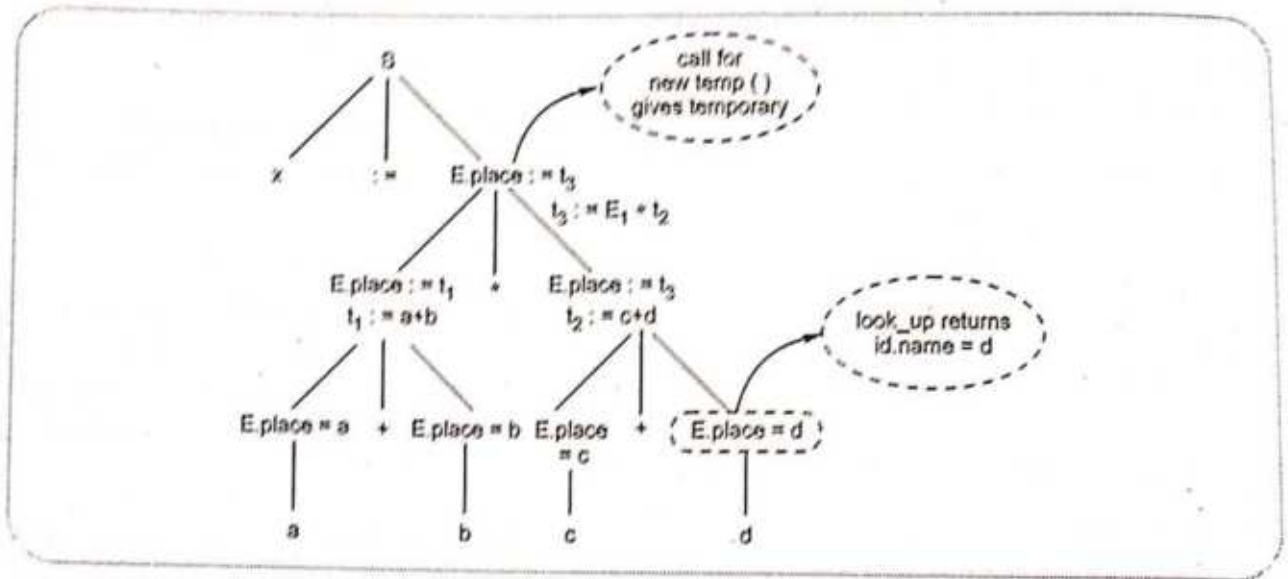


Fig. Q.48.1 Annotated parse tree for generation of three address code

Q.49 How would you generate the intermediate code for the flow of control statements ? Explain with examples.

[JNTU : Part B, May-18, Marks 10]

Ans. : The control statements are if-then-else and while-do. The grammar for such statements is as shown below

$S \rightarrow \text{if } E \text{ then } S_1$
 $\quad \mid \text{if } E \text{ then } S_1 \text{ else } S_2$
 $\quad \mid \text{while } E \text{ do } S_1$

While generating three address code -

- To generate new symbolic label the function new_label() is used.
- With the expression $E.true$ and $E.false$ are the labels associated.
- $S.code$ and $E.code$ is for generating three address code.

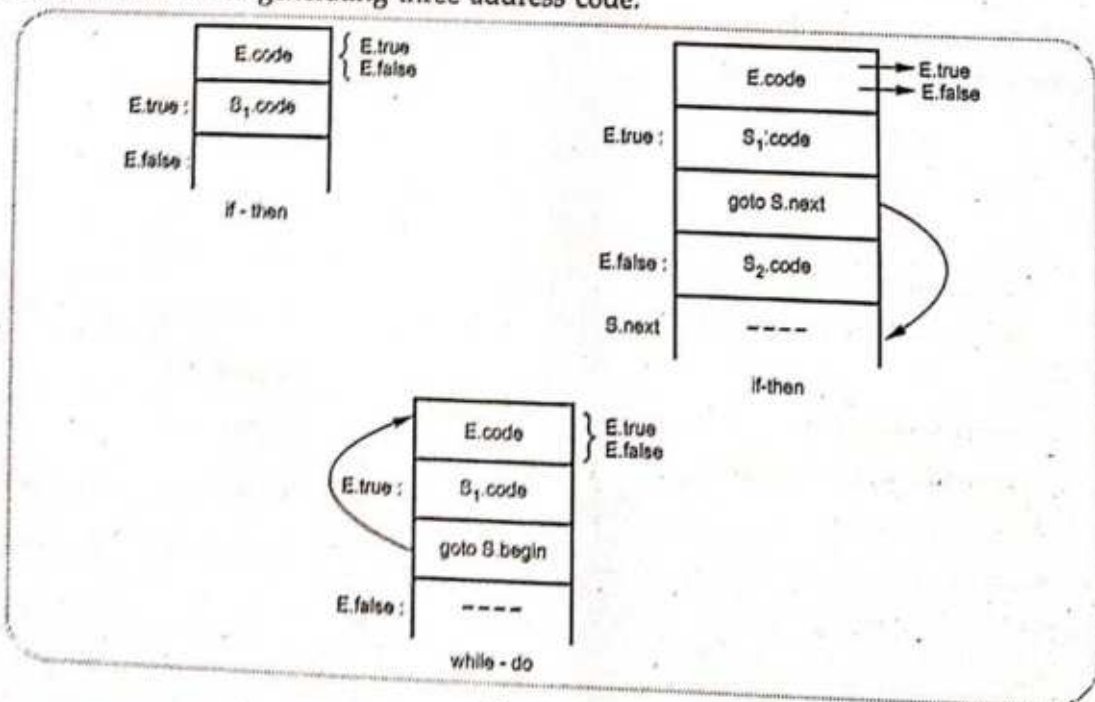


Fig. Q.49.1

$S \rightarrow \text{if } E \text{ then } S_1$



$E.\text{true} := \text{new_label}()$

$E.\text{false} := S.\text{next}$

$S_1.\text{next} := S.\text{next}$

$S.\text{code} := E.\text{code} \parallel \text{gen_code}(E.\text{true} ':') \parallel S_1.\text{code}$

In the above translation scheme $\parallel$ is used to concatenate the strings. The function gen_code is used to evaluate the non quoted arguments passed to it and to concatenate complete string. The $S.\text{code}$ is the important rule which ultimately generates the three address code.

Consider the statement is if $a < b$ then $a = a + 5$ else $a = a + 7$

The three address code for if-then is -

100 : if $a < b$ goto L1

101 : goto 103

102 : L1: $a = a + 5$ /*E.true*/

103: $a = a + 7$

Here $E.\text{code}$ is "if $a < b$ " L1 denotes $E.\text{true}$ and $E.\text{false}$ is shown by jumping to line 103(i.e. $S.\text{next}$)

Similarly,

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$



$E.\text{true} := \text{new_label}()$

$E.\text{false} := \text{new_label}()$

$S_1.\text{next} := S.\text{next}$

$S_2.\text{next} := S.\text{next}$

$S.\text{code} := E.\text{code} \parallel \text{gen_code}(E.\text{true} ':') \parallel$

$S_1.\text{code} \parallel \text{gen_code}(\text{'goto'}, S.\text{next}) \parallel$

$\text{gen_code}(E.\text{false} ':') \parallel S_2.\text{code}$

$S.\text{begin} := \text{new_label}()$

$E.\text{true} := \text{new_label}()$

$E.\text{false} := S.\text{next}$

$S_1.\text{next} := S.\text{begin}$

$S.\text{code} := \text{gen_code}(S.\text{begin} ':') \parallel E.\text{code} \parallel$

$\text{gen_code}(E.\text{true} ':') \parallel S_1.\text{code} \parallel \text{gen_code}(\text{'goto'}, S.\text{begin})$

$S \rightarrow \text{while } E \text{ do } S_1$

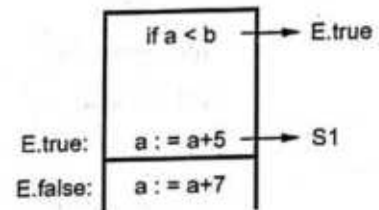


Fig. Q.49.2

Q.50 Generate intermediate code for the following code segment along with the required syntax directed translation scheme:

If($a > b$)

$x = a + b$

else

$x = a - b$

Where a and x are real and b is on int data type.

[JNTU : Part B, Nov.-15, Marks 10]

Ans. : Three address code

Quadruple :

```

100 : if a > b then
      goto 102
101 : goto 105
102 : t1:=a+b
103 : x:= t1
104 : goto 108
105 : t1:=a-b
106 : x:= t1
107 : goto 108
108 : .
    
```

	Operator	Operand 1	Operand 2	Result
100	>	a	b	goto 102
101		goto 105		
102	+	a	b	t1
103	:=	t1		x
104		goto 108		
105	-	a	b	t1
106	:=	t1		x
107		goto 108		

Q.51 Translate the following C fragment into three address code.

```

int i
int a [10] [10]
i = 0
while (i < 10)
{
    a [i] [i] = 1;
    i++;
}
    
```

[JNTU : PART B, Marks 5]

Ans. : We consider the values stored in 2-D array is in row major form. Assuming 4 byte allocation per word a [i] [j] will be at location $[addr(a) + (10*i+j)*4]$

$$\therefore a[i][i] = addr(a) + 44 * i$$

The 3 address code then will be -

```

i = 0
L1: if i < 10 goto L2
      t1 = 44*i
      a[t1] = 1
      t2 = i+1
      i = t2
      goto t1
L2: .
    
```

Q.52 Write a three address code for following C statement

```

for (i = 1 ; i <= 10 ; i++)
{
    a [i] = x * 5 ;
}
    
```

Ans. : The 3 address code is

```

Loop:  i := 1
        t1 := x * 5
        t2 := &a
        t3 := sizeof(int)
        t4 := t3 * i
        t5 := t2 + t4
        *t5 := t1
        i := i + 1
        if i <= 10 goto Loop

```

3.12 : Backpatching

Q.53 What is backpatching ? What are the functions used to implement backpatching ?

[JNTU : Part A, Marks 3]

Ans. : **Definition :** Backpatching is the activity of filling up unspecified information of labels using appropriate semantic actions in during the code generation process.

- The backpatching is commonly used for -
 - (i) Boolean expression (ii) Flow of control statements
- To generate code using backpatching. In the semantic actions following functions are used :
 - **mklist(i)** creates the new list. The index i is passed as an argument to this function where i is an index to the array of quadruple.
 - **merge_list (p1, p2)** this function concatenates two lists pointed by p1 and p2. It returns the pointer to the concatenated list.
 - **backpatch (p, i)** inserts i as target label for the statement pointed by pointer p.

Q.54 Using Backpatching, generate an intermediate code for following expression.

$A < B \text{ OR } C < D \text{ AND } P < Q$

[JNTU : PART B, Marks 10]

Ans. : For the given expression ($A < B \text{ OR } C < D \text{ AND } P < Q$) We will scan it from left to right.

$A < B$ matches with the rule $E \rightarrow id_1 \text{ relop } id_2$. In response to this statement the three address code will be generated as,

```

100 if A < B goto _
101 goto _

```

} as semantic actions are
 append ('if id₁ place relop id₂ place goto _')
 append ('goto _')

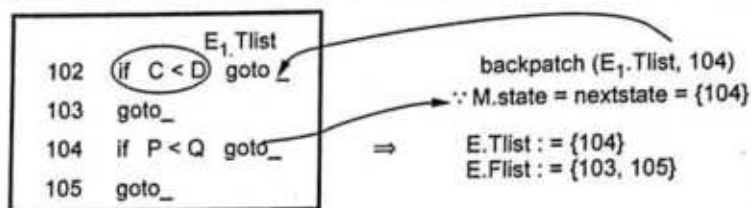
Similarly, for the remaining part of expression the three address code will be -

```

102 if C < D goto _
103 goto _
104 if P < Q goto _
105 goto _

```

Now for the expression if $A < B \text{ OR } C < D \text{ AND } P < Q$. We will solve AND operation first then OR, as AND has higher precedence over OR.



Hence

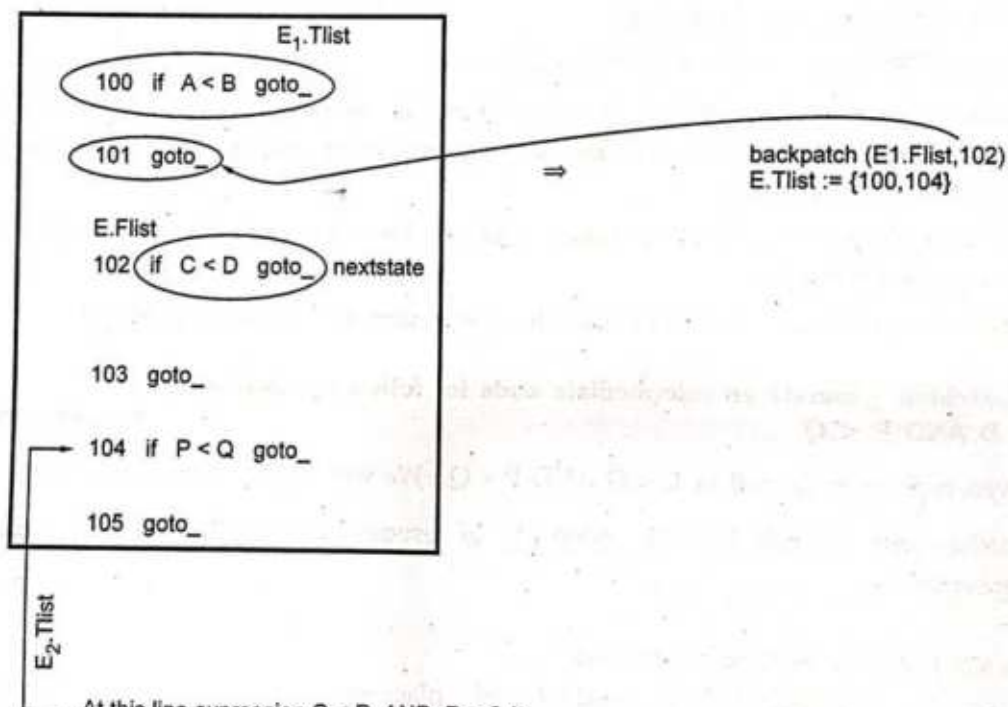
```
102 if C < D goto 104
103 goto _
104 if P < Q goto _
105 goto _
```

Now consider,

A < B

OR

C < D AND P < Q



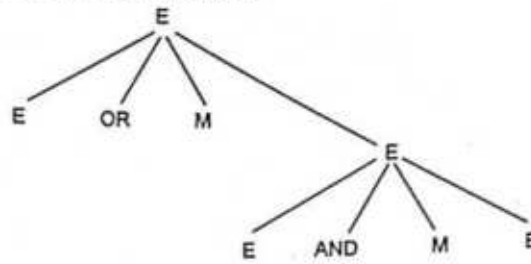
At this line expression C < D AND P < Q is decided to be true. That means if C < D is true and if P < Q is also true then C < D AND P < Q = TRUE. Hence $E_2.Tlist = \{104\}$

Finally, the 3-address code with backpatching will be :

```
100 if A < B goto _
101 goto 102
102 if C < D goto 104
103 goto _
104 if P < Q goto _
105 goto _
```

The annotated part tree can be constructed as follows :

Step 1:

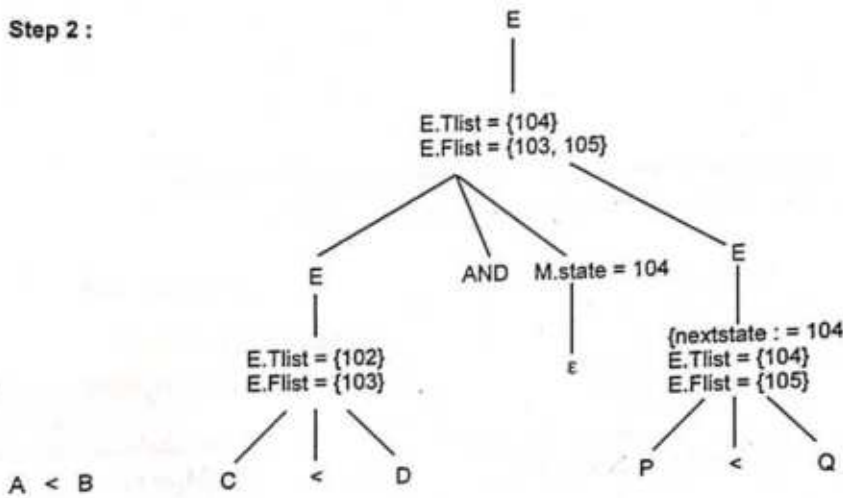


A < B

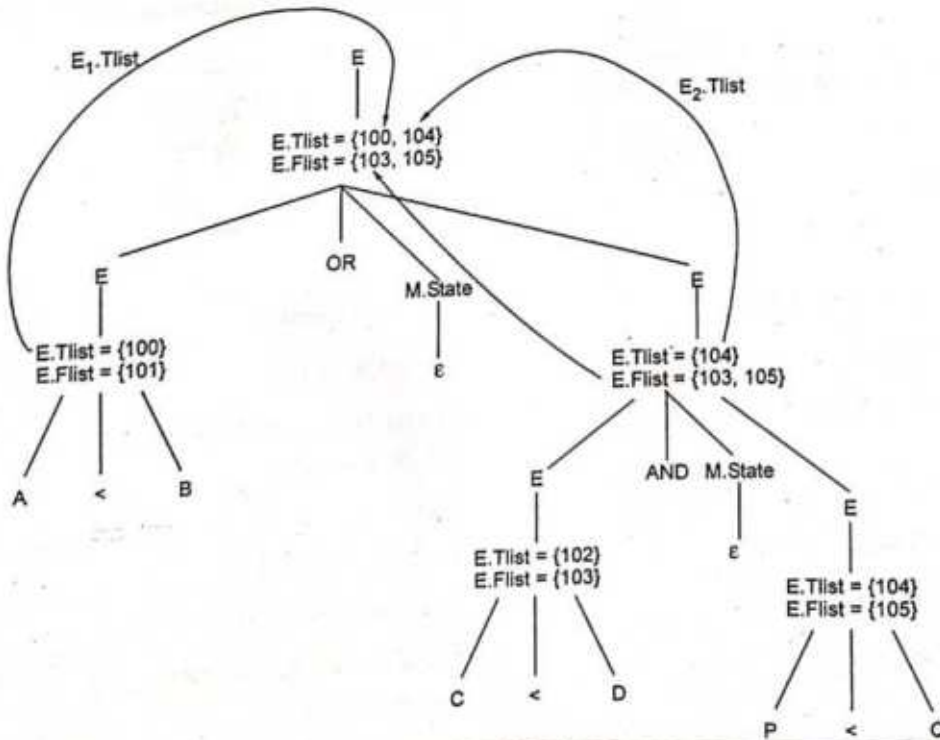
C < D

P < Q

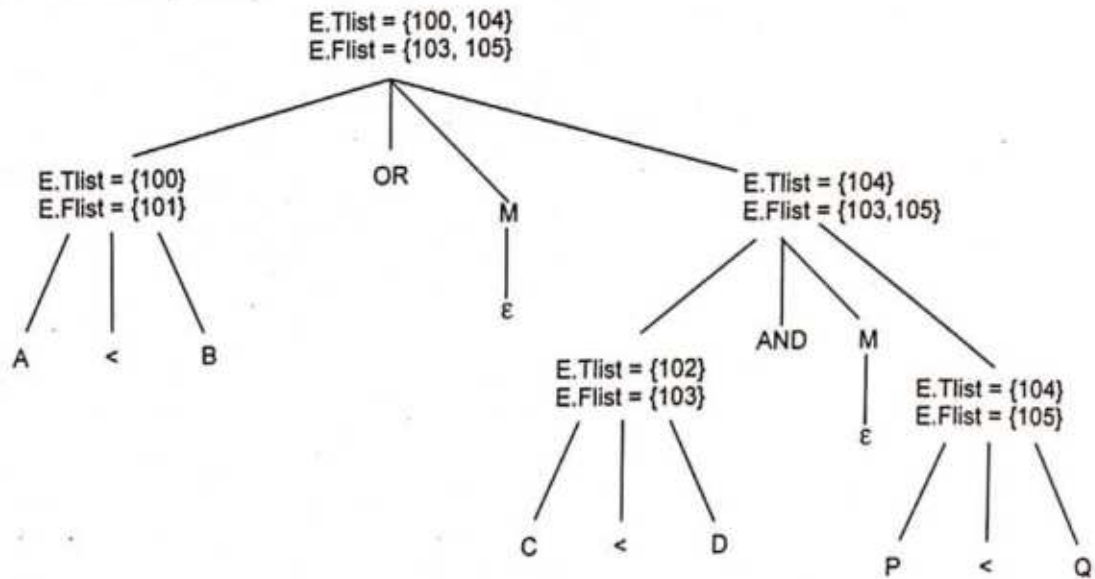
Step 2:



Step 3:



Finally the annotated parse tree will be :



Q.55 Give translation scheme for control flow statement using backtracking.

Ans. : The translation scheme for the above grammar is as given below

Production rule	Semantic action
$S \rightarrow \text{if } E \text{ then } M S_1$	{ backpatch(E.Tlist,M.state); S.next := merge(E.Flist,S ₁ .next) }
$S \rightarrow \text{if } E \text{ then } M_1 S_1 \text{ N else } M_2 S_2$	{ backpatch(E.Tlist,M ₁ .state); backpatch(E.Flist,M ₂ .state); S.next := merge(S ₁ .next,merge(N.next,S ₂ .next)) }
$S \rightarrow \text{while } M_1 E \text{ then do } M_2 S$	{ backpatch(S ₁ .next,M ₁ .state); backpatch (E.Tlist,M ₂ .state); S.next=E.Flist; append('goto' M ₁ .state); }
$S \rightarrow \text{begin } L \text{ end}$	{ S.next := L.next }
$S \rightarrow A$	{ S.next := NULL;}
$L \rightarrow L_1; M S$	{ backpatch(L ₁ .next,M.state); L.next := S.next; }
$L \rightarrow S$	{ L.next := S.next ;}
$M \rightarrow \epsilon$	{ M.state := nextstate;}
$N \rightarrow \epsilon$	{ N.next := mklist(nextstate); append('goto _');}

- The synthesized attributes Tlist and Flist are used to generate the jumping code for Boolean expressions. For the true statement E.Tlist will be generated and for the false statement E.Flist is generated.
- The attribute 'state' will be associated with the M and that is used to record the number(address) of the statement. It is denoted as M.state. The 'nextstate' will point to the next statement.
- The attribute next is used to indicate the pointer to the next list. The S.next and L.next records the pointer to the next list of S and L respectively. The S.next := NULL initializes S.next to an empty list.

Thus backpatching can be used to translate the flow of control statement in one pass.

Q.56 Translate (a or b) and (c < d) and (d < e) into three address statements using backpatching.

[JNTU : Part B, Marks 5]

Ans : We will write the three address code for (a or b) and (c < d) and (d < e) as follows -

100 := if a or b goto -

101 := if c < d goto -

102 := goto -

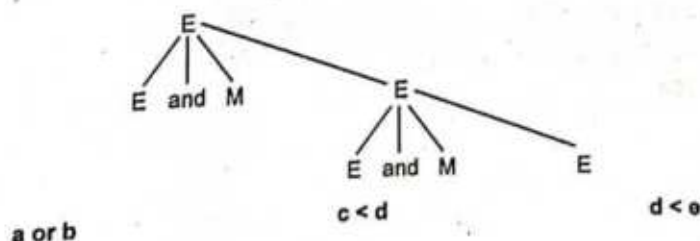
103 := if d < e goto -

104 := goto -

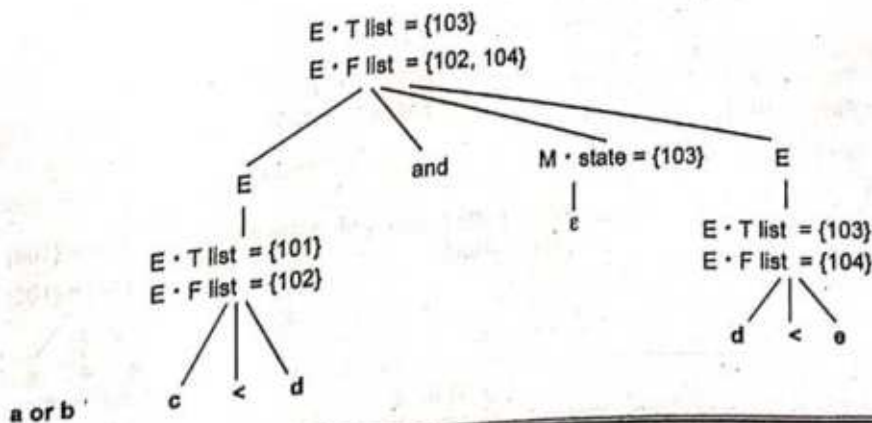
Now we will fill up the blank entries in above code using backpatching.

The annotated parse tree can be constructed as follows -

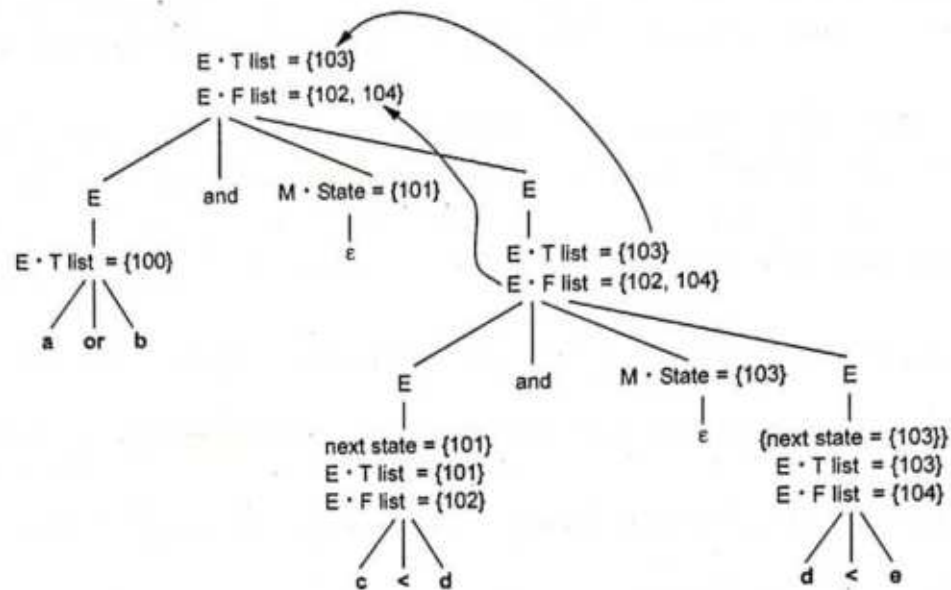
Step 1 :



Step 2 :



Step 3 :



Hence the three address code after backpatching will be

```

100 := if a or b goto 101
101 := if c < d goto 103
102 := goto -
103 := if d < e goto -
104 := goto -
    
```

The branch 103 is true exit point and branch 102 and 104 are false exit points.

Q.57 Generate the three address code and syntax tree for the following expression
 $a < b \text{ or } c < d \text{ and } e < f$.

[JNTU : Part B, Marks 5]

Ans. : The three address code will be -

```

100 : if a < b goto -
101 : goto 102
102 : if c < d goto 104
103 : goto -
104 : if e < f goto -
105 : goto -
    
```

The syntax tree will be

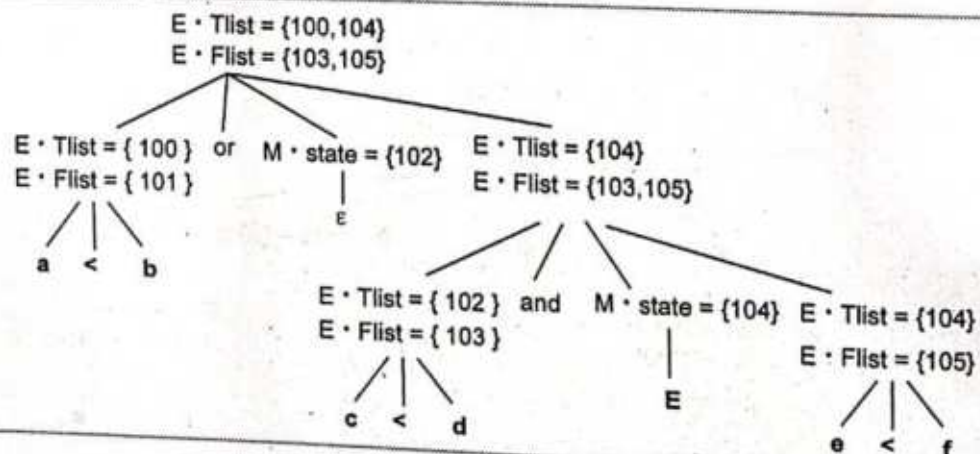


Fig. Q.57.2

Q.58 List out the motivations for back patching.

[JNTU : Part A, Marks 2]

Ans. : i) In a Boolean expression suppose there are two expression E1 && E2. If the expression E1 is false then there is no need to compute rest of the expression. But if we generate the code for the expression E1 we must know where to jump on encountering the false E1 expression. But this can be accomplished by the backpatching technique.

ii) For the flow of control statements(such as if, else, while), the jumping location can be identified using the backpatching.

3.13 : Switch Statement

Q.59 Write the semantic actions to generate the three address code for case statement of any language ?

[JNTU : Part B, May-12, Marks 7]

Ans. : The syntax for switch-case statement can be as shown below :

switch expression

```
{
case value: statement
case value: statement
...
case value: statement
default : statement
}
```

The translation scheme for case statement :

Production Rule	Semantic Action
switch E	Evaluate E into t such that t=E
{	goto check
case v ₁ : s ₁	L ₁ : code for s ₁
case v ₂ : s ₂	goto last
...	L ₂ : code for s ₂
case v _{n-1} : s _{n-1}	goto last
default: s _n	L _n : code for s _n
}	goto last
	check: if t=v ₁ goto L ₁
	if t=v ₂ goto L ₂
	...
	if t=v _{n-1} goto L _{n-1}
	goto L _n
	last:

Q.60 Generate the three address code for, switch(ch)

```
{
case 1:c=a+b;
break;
case 2:c=a-b;
break;
}
```

Ans. : The three address code can be

```
100 :      if ch = 1 goto L1
101 :      if ch = 2 goto L2
102 :      L1: t1:=a+b
103 :      c= t1
104 :      goto Last
105 :      L2: t1:=a-b
106 :      c= t1
107 :      goto last
108 :      Last :
```

3.14 : Intermediate Code for Procedures

Q.61 Give the semantic rules for declaration in a procedure?

[JNTU : Part B, April-11, Jan-12, Marks 10]

Ans. : • Procedure or function is an important programming construct which is used to obtain the modularity in the user program.

• Consider the grammar for a simple procedure call

S → call id (L)

L → L, E

L → E

Here the non terminal S denotes the statement and non terminal.

L denotes the list of parameters.

And E denotes the expression it could be id as well.

- The translation scheme can be as given below -

Production Rule	Semantic Action
$S \rightarrow \text{call id (L)}$	{ for each item p in queue do append('param' p); append('call' id.place) }
$L \rightarrow L, E$	{ insert E.place in the queue }
$L \rightarrow E$	{ initialize queue and insert E.place in the queue }

- The data structure queue is used to hold the various parameters of the procedure. The keyword param is used to denote the list of parameters passed to the procedure. The call to the procedure is given by 'call id' where id denotes the name of the procedure.
- E.place gives the value of the parameter which is inserted in the queue.
- For $L \rightarrow E$ the queue gets empty and a single pointer to the symbol table is obtained. This pointer denotes the value of E.

Multiple Choice Questions for Mid Term Exam

Q.1 In semantic analysis following task is done

- ☐ a separation of tokens from the source program
- ☐ b building of syntax tree
- ☒ c type checking
- ☐ d all of the above

Q.2 Coercion is

- ☒ a implicit type conversion
- ☐ b explicit type conversion
- ☐ c implicit type conversion depending upon the expression
- ☐ d none of the above

Q.3 Checking the scope of the variable is done in

- ☐ a syntax analysis
- ☒ b semantic Analysis
- ☐ c lexical analysis
- ☐ d during the execution of the program

Q.4 What can't be done in the semantic interpretation? [JNTU : Aug 16]

- ☐ a Logical term,
- ☐ b Complete logical sentence,
- ☒ c Both a and b,
- ☐ d None of the mentioned

Q.5 In a bottom up evaluation of a syntax direction definition, inherited attributes can [JNTU : Feb.-17]

- ☐ a always be evaluated
- ☒ b be evaluated only if the definition is L-attributed
- ☐ c be evaluated only if the definition has synthesized attributes
- ☐ d none

Q.6 Generation of intermediate code based on a abstract machine model is useful in compilers because - [JNTU : Feb 17]

- ☒ a It makes implementation of lexical analysis and syntax analysis easier
- ☐ b Syntax directed translation can be written for intermediate code generation.
- ☐ c It enhances the probability of the front end of the compiler
- ☐ d It is not possible to generate code for real machines directly from high level language programs.

Q.7 Semantic errors can be detected at

- ☐ a compile time
- ☐ b run time
- ☒ c both at compile time and run time
- ☐ d none of the above

4

Run-Time Environment
and Code Generation

Part I : Run-Time Environment

4.1 : Storage Organization

Q.1 Explain the run time storage organization of a program. [JNTU : Part B, Jan.-12, Marks 8]

Ans. : • The compiler demands for a block of memory to operating system. The compiler utilizes this block of memory for running (executing) the compiled program. This block of memory is called run time storage.

• The run time storage is subdivided to hold code and data such as :

- i) The generated target code
- ii) Data objects
- iii) Information which keeps track of procedure activations.

• The size of generated code is fixed. Hence the target code occupies the statically determined area of the memory. Compiler places the target code at the lower end of the memory.

• The amount of memory required by the data objects is known at the compiled time and hence data objects also can be placed at the statically determined area of the memory. Compiler prefers to place the data objects in the statically determined area because these data objects then can be compiled into target code. For example in FORTRAN all the data objects are allocated statically. Hence the static data area is on the top of code area. The subdivision of run time memory as shown by following Fig. Q.1.1.

• The counterpart of control stack is used to manage the active procedures. Managing of active procedures means that when a call occurs then execution of activation is interrupted and information about status of the stack is saved on

the stack. When the control returns from the call this suspended activation is resumed after storing the values of relevant registers. Also the program counter is set to the point immediately after the call. This information is stored in the stack area of run time storage. Some data objects which are contained in this activation can be allocated on the stack along with the relevant information.

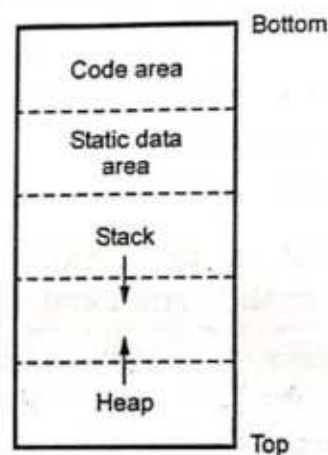


Fig. Q.1.1 Run time storage organization

- The heap area is the area of run time storage in which the other information is stored. For example memory for some data items is allocated under the program control. Memory required for these data items is obtained from this heap area. Memory for some activation is also allocated from heap area.
- The size of stack and heap is not fixed it may grow or shrink interchangeably during the program execution.
- Pascal and C need the run time stack.

Q.2 Explain various storage allocation strategies with its merits and demerits.

[JNTU : Part B, Nov.-16, Marks 10]

OR Compare different storage allocation strategies.

[JNTU : Part B, Marks 5]

Ans. :

Sr. No.	Static allocation	Stack allocation	Heap allocation
1.	Static allocation is done for all data objects at compile time.	In stack allocation, stack is used to manage runtime storage.	In heap allocation, heap is used to manage dynamic memory allocation.
2.	Data structures can not be created dynamically because in static allocation compiler can determine the amount of storage required by each data object.	Data structures and data objects can be created dynamically.	Data structures and data objects can be created dynamically.
3.	Memory allocation : The names of data objects are bound to storage at compile time.	Memory allocation : Using Last In First Out (LIFO) activation records and data objects are pushed onto the stack. The memory addressing can be done using index and registers.	Memory allocation : A contiguous block of memory from heap is allocated for activation record or data object. A linked list is maintained for free blocks.
4.	Merits and limitations : This allocation strategy is simple to implement but supports static allocation only. Similarly recursive procedures are not supported by static allocation strategy.	Merits and limitations : It supports dynamic memory allocation but it is slower than static allocation strategy. Supports recursive procedures but references to non local variables after activation record can not be retained.	Merits and limitations : Efficient memory management is done using linked list. The deallocated space can be reused. But since memory block is allocated using best fit, holes may get introduced in the memory.

Q.3 Explain in brief about heap storage allocation Strategy. [JNTU : Part B, Mar.-16, Marks 5]

Ans. :

- If the values of non local variables must be retained even after the activation record then such a retaining is not possible by stack allocation. This limitation of stack allocation is because of its Last In First Out nature. For retaining of such local variables heap allocation strategy is used.
- The heap allocation allocates the continuous block of memory when required for storage of activation records or other data object. This allocated memory can be deallocated when activation ends. This deallocated (free) space can be further reused by heap manager.
- The efficient heap management can be done by
 - Creating a linked list for the free blocks and when any memory is deallocated that block of memory is appended in the linked list.
 - Allocate the most suitable block of memory from the linked list. i.e. use best fit technique for allocation of block.

Q.4 What are the advantages of heap storage allocation ? [JNTU : Part A, Dec.-17, Marks 2]

Ans. : The advantages of heap storage allocation are as follows -

- There is no limit on memory size. One can use as much memory as required.
- Any object created in the heap space has global access and can be referenced from anywhere of the application.
- It is a flexible storage allocation strategy.
- Automatic garbage management technique can be used for heap storage allocation.

Q.5 Explain in brief about stack storage allocation strategy. [JNTU : Part B, Nov.-17, Marks 5]

Ans. : Refer Q.2.

4.2 : Stack Allocation of Space

Q.6 Define activation record. Explain in brief about the fields in activation record.

[JNTU : Part A, Mar.-17, Marks 2
Part B, Nov 17, Marks 5]

DECODE

Ans. : • The activation record is a block of memory used for managing information needed by a single execution of a procedure.

• FORTRAN uses the static data area to store the activation record where as in PASCAL and C the activation record is situated in stack area. The contents of activation record are as shown in the following Fig. Q.6.1.

Return value
Actual parameters
Control link (Dynamic link)
Access link (Static link)
Saved machine status
Local variables
Temporaries

Fig. Q.6.1 Model of activation record

• Various fields of activation record are as follows -

1. **Temporary values** - The temporary variables are needed during the evaluation of expressions. Such variables are stored in the temporary field of activation record.
2. **Local variables** - The local data is a data that is local to the execution of procedure is stored in this field of activation record.
3. **Saved machine registers** - This field holds the information regarding the status of machine just before the procedure is called. This field contains the machine registers and program counter.
4. **Control link** - This field is optional. It points to the activation record of the calling procedure. This link is also called dynamic link.
5. **Access link** - This field is also optional. It refers to the non local data in other activation record. This field is also called static link field.
6. **Actual parameters** - This field holds the information about the actual parameters. These actual parameters are passed to the called procedure.
7. **Return values** - This field is used to store the result of a function call.

• The size of each field of activation record is determined at the time when a procedure is called.

Q.7 What is activation tree ?

[JNTU : Part A, Marks 2]

Ans. : Activation tree is a data structure used for specifying the flow of control among the procedures.

In an activation tree -

- i) Each node shows an activation of procedure.
- ii) For showing activation of main program root is used.
- iii) If lifetime of 'x' occurs before the lifetime of 'y' then x becomes left child of y.
- iv) If control flows from x to y then 'x' is parent node of 'y'.

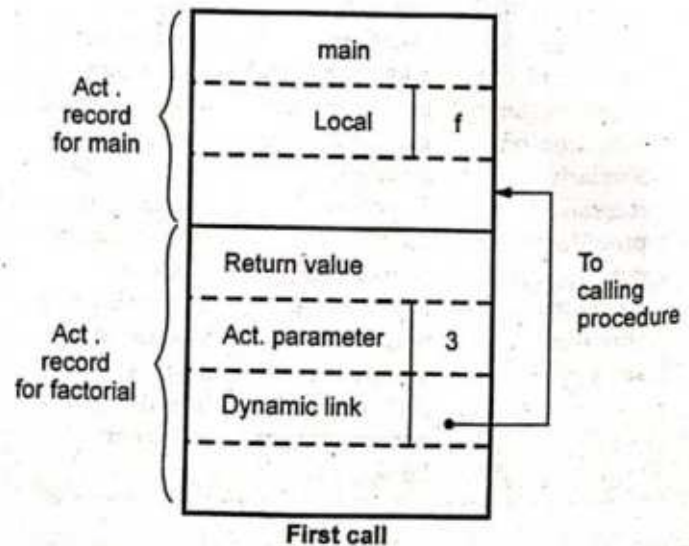
Q.8 By taking the example of factorial program explain how activation record will look like for every recursive call in case of factorial (3).

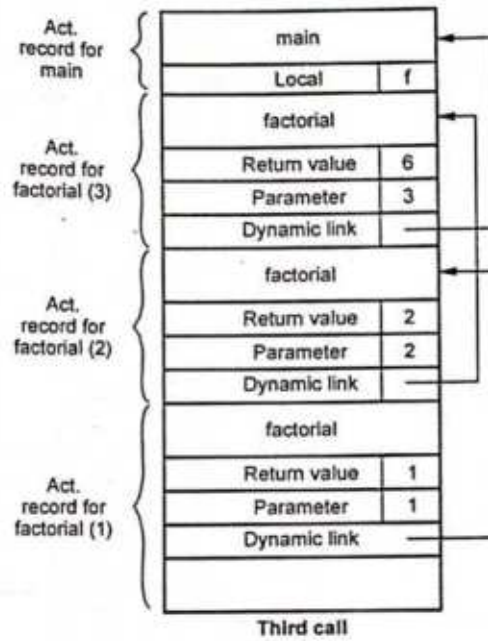
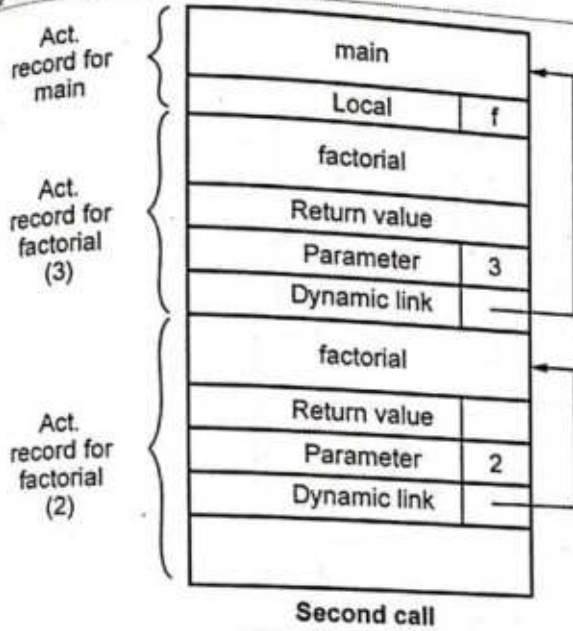
[Part B, Marks 5]

Ans. :

```
main()
{
    int f;
    f = factorial (3);
}

int factorial (int n)
{
    if (n==1)
        return 1;
    use
        return (n*factorial (n-1));
}
```





Q.9 Explain what information are stored in stack during function calls. Consider the following code :

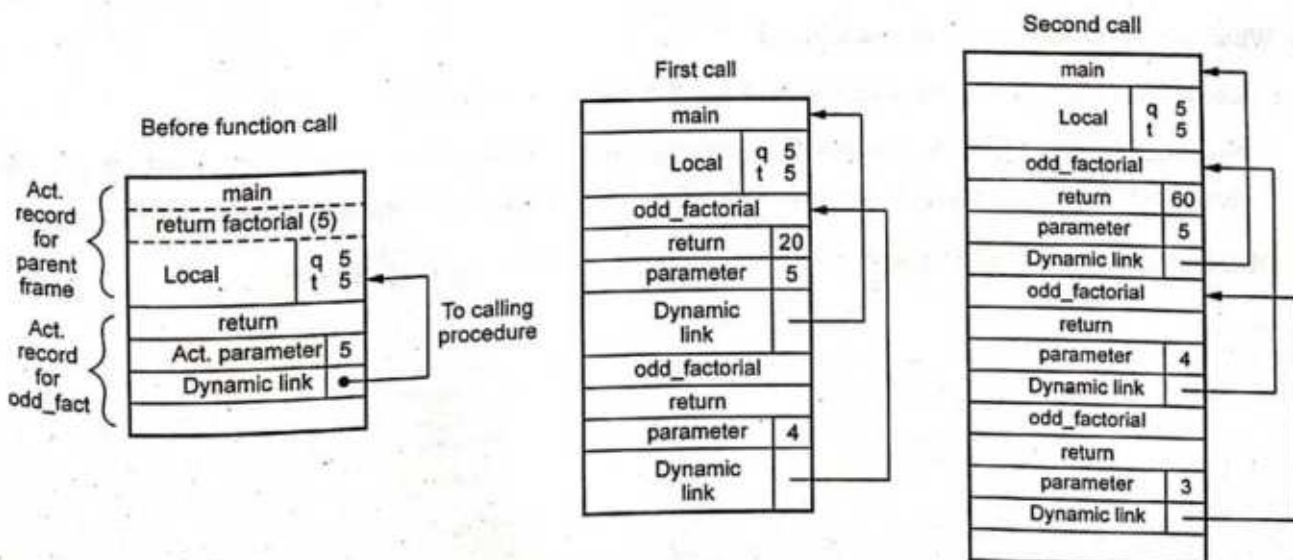
```

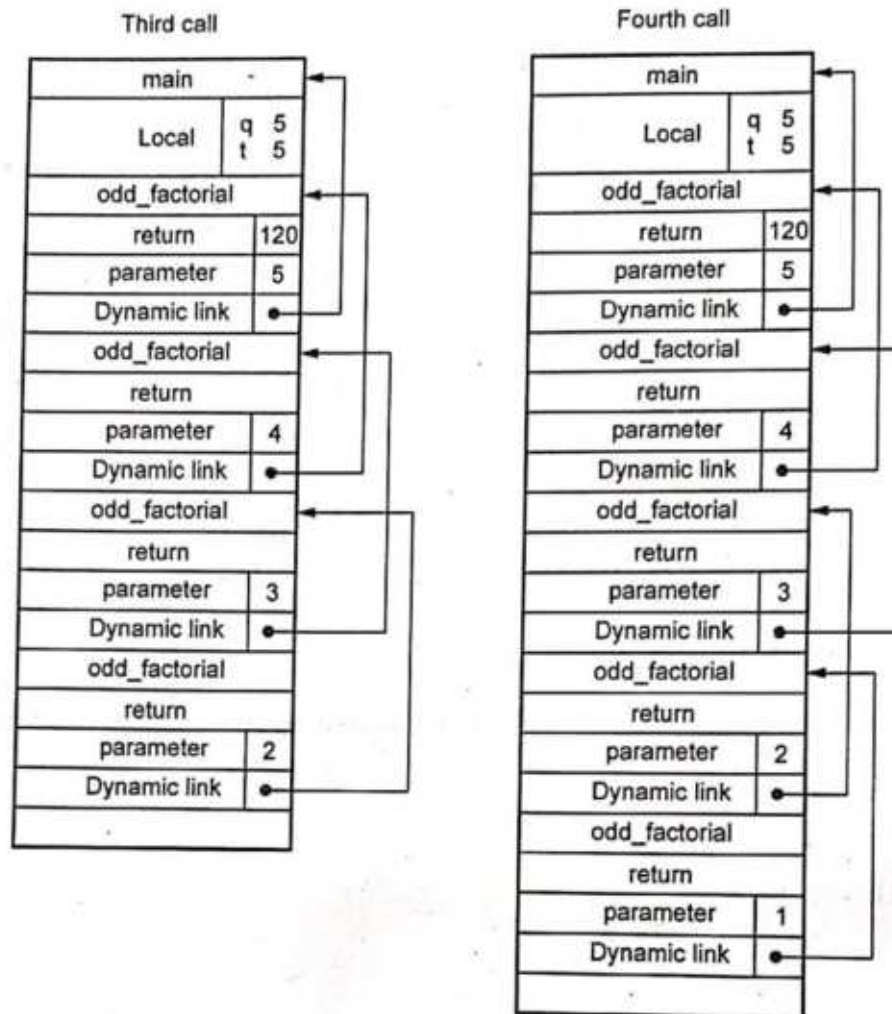
Line No's
1. int odd_factorial (int q) {
2. int factorial (int n) {
3. if (n == 1) return q;
4. int t = factorial (n-1);
5. return n * t;
6. }
7. if (q % 2 == 0)
8. return q;
9. return factorial (t);
10. }
    
```

Consider calling the function `odd_factorial (5)`. Show the state of the stack, including parent frame pointers, during the execution of the return statement in line 3.

[JNTU : Part B, May-12, Marks 15]

Ans. :





Finally 120 will be returned to main.

4.3 : Access to Non Local Data on the Stack

Q.10 What are local and non local variables ?

Ans. : **Local variable** : A variable declared in block B is called local variable of block B.

Non Local variable : A variable of enclosing block which is accessible within block B is called non-local variable of block B. To understand the local and nonlocal variables let us see one example.

Q.11 Obtain the static scope of the declarations made in the following piece of code.

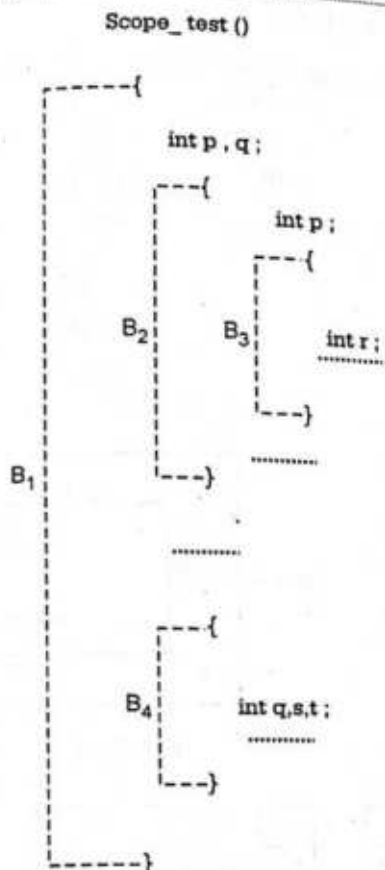


Fig. Q.11.1

Ans.: To denote the scope of the variable we will use name of the variable followed by the block name.

Block	Local variable	Non Local variable
B ₁	pB ₁ , qB ₁	-
B ₂	pB ₂	pB ₁ , qB ₁
B ₃	rB ₃	pB ₂ , qB ₁
B ₄	qB ₄ , sB ₄ , tB ₄	pB ₁ , qB ₁

Q.12 What is static and dynamic scope rule ?

Ans.: 1. **Static scope rule** : The static scope rule is also called as lexical scope. In this type the scope is determined by examining the program text. PASCAL, C and ADA are the languages that use the static scope rule. These languages are also called as block structured languages.

2. **Dynamic scope rule** : For non block structured languages this dynamic scope allocation rules are used.

The dynamic scope rule determines the scope of declaration of the names at run time by considering the current activation.

LISP and SNOBOL are the languages which use dynamic scope rule.

Q.13 Obtain the static scope of the declarations made in the following piece of code. [JNTU : Part B]



Fig. Q.13.1

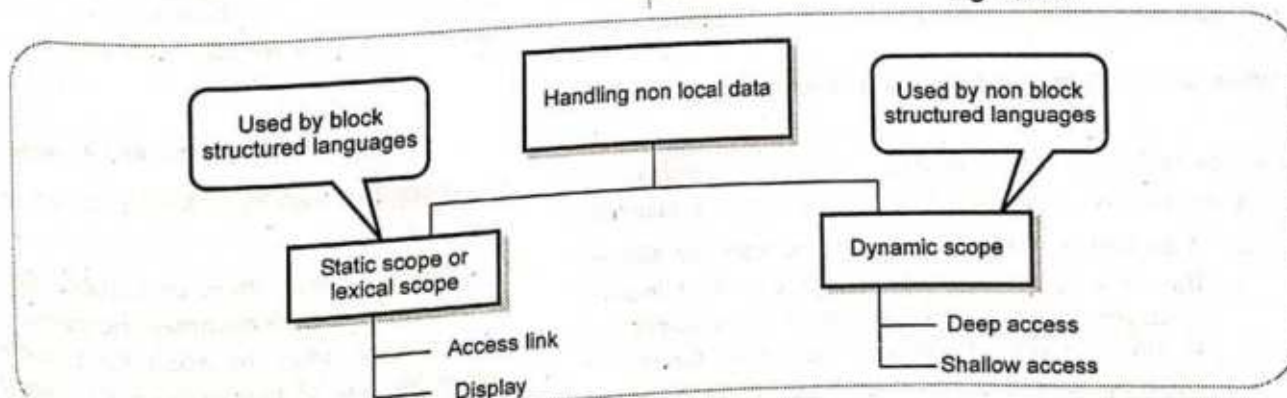
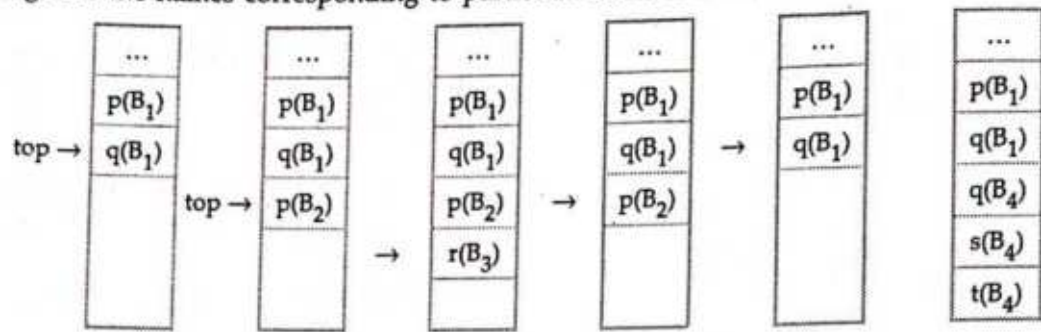


Fig. Q.11.2 Access to non local data

Ans. : The storage can be allocated for a complete procedure body at one time.

The storage for the names corresponding to particular block can be as shown below.



Q.14 Explain the behaviour of stack, used for allocation of storage, for the execution of each statement of the block structured program :

[JNTU : Part B]

Ans. : The block structure storage allocation which can be done using stack is as given below.

Thus block structure storage allocation can be done by stack.

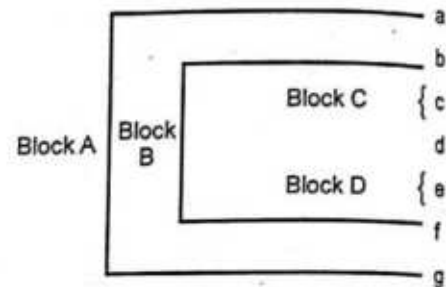


Fig. Q.14.1

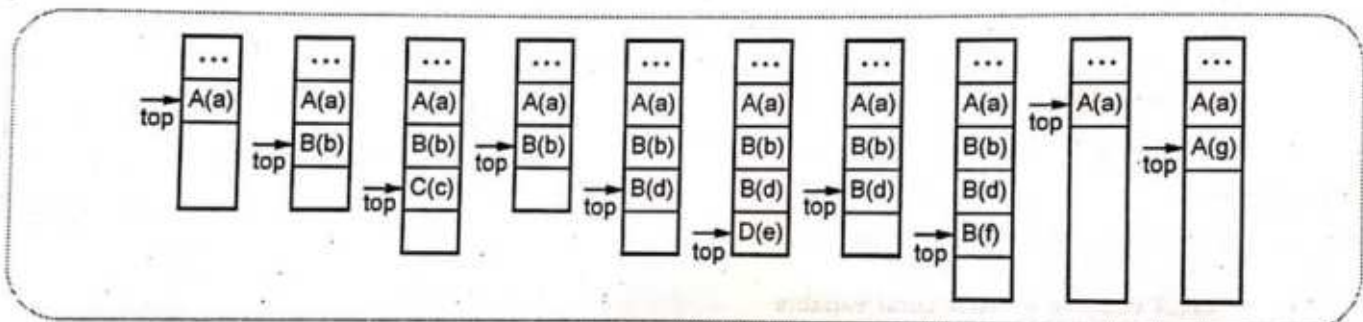


Fig. Q.14.1 (a)

4.4 : Heap Management

Q.15 What is heap management ?

[JNTU : Part A, Marks 2]

Ans. : Heap is a portion of memory that can remain allocated for indefinite time. The memory created using dynamic allocation technique makes use of heap memory. For example in C++ or Java we can make use.

Q.16 What are the tasks of memory manager ?

[JNTU : Part A, Marks 3]

Ans. : • The task of memory manager is to keep track of all the free space that can be available in the heap. Following are the two important functions of memory manager -

1. **Allocation :** When a program requests for allocation of memory for some object or variable then the memory manager allocates a chunk of memory to the object/variable. Sometimes the memory manager makes use of the free space available in the heap memory in order to satisfy the request. If still the required free space is not available then it increases the size of heap storage by getting some bytes of memory from virtual memory of operating system.

2. **Deallocation** : The memory manager returns the deallocated memory to the pool of free storage maintained in the heap memory. This free space is used by the memory manager to satisfy the request for memory allocation.

Q.17 What are the desired properties of memory manager ? [JNTU : Part A, Marks 3]

Ans. : The desired properties of memory manager are :

1. **Space efficiency** : The memory manager must minimize the requirement of total heap space required by the program. For that matter, the larger programs are allowed to run in fixed virtual address space. The reduction in fragmentation also helps in achieving the space efficiency.
2. **Program efficiency** : By using the heap memory appropriately the program efficiency can be achieved. The object that are required more frequently must be placed in the accessible region, this will help to run the program faster. The principle of locality is used to achieve the program efficiency.
3. **Low overhead** : Memory allocation and deallocation are the most frequently performed operations in the program. If these operations are executed efficiently then it reduces the overhead in program execution.

Q.18 Explain memory hierarchy in detail.

[JNTU : Part B, Marks 5]

Ans. : • Compiler optimization is very much dependent upon the memory management technique.

- The efficiency of program is determined by - 1) Number of instructions getting executed and 2) The amount time required by each instruction for execution. The time taken by the instruction to execute greatly depends upon how much time it takes to access different parts of the memory.
- The significant difference in memory access time is due to **limitations of hardware technologies**. Quite often there are small storage with fast access and large storage with slow access. To make use of the efficient memory access all the modern computers arrange in memory in **hierarchy**. Fig. Q.18.1 represents the memory hierarchy.

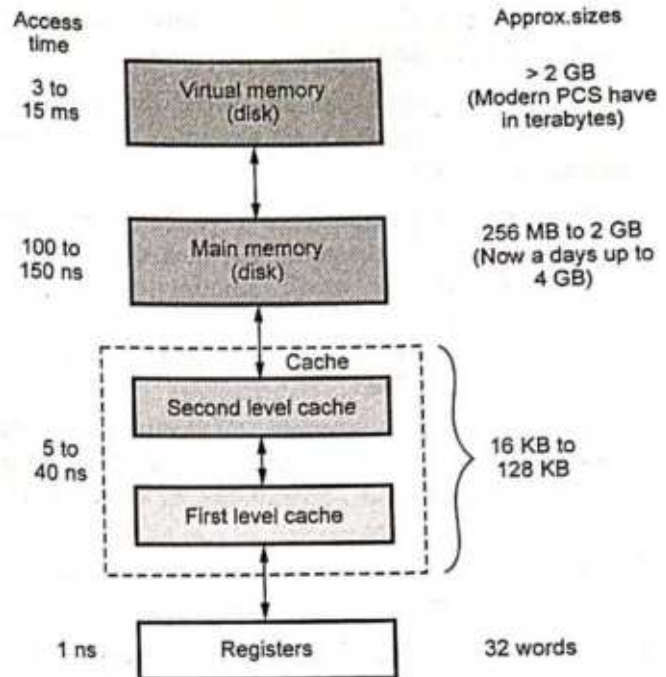


Fig. Q.18.1 Memory hierarchy

- The processors has several registers which can be controlled by the software. Then the levels of cache memory come. It is there in kilobytes to several megabytes. The next level has the physical main memory which is made up of hundreds of megabytes to gigabytes of dynamic RAM. Finally the virtual memory which has several gigabytes of memory.
- During the memory access, the data is looked in the lower level of memory and if it is not found there then the next level is accessed and so on. This simplifies the programming task and the same program can work effectively on various machines with different memory configurations.

Q.19 Write short note on - Locality in programs.

[JNTU : Part B, Marks 5]

Ans. : • Many programs spent lot of time in executing small amount of code by accessing very little amount of memory. Thus exhibits the high degree of locality.

- Programs has two types of localities - **temporal locality** and **spatial locality**.
- The program has **temporal locality** if the locations it access are likely to accessed again within in short period of time. The program has the **spatial locality**

if the locations close to the locations it access are likely to be accessed within a short time.

- Following are some reasons why the programs spent most of its time in executing very small amount of code -

1. Most of the programs contain the instructions that are never executed. Programs make use of libraries that use small amount of provided functionalities. Many times there are changes in the requirements of the systems or the legacy systems get evolved.
 2. Typically, small amount of code is often executed. For example the code for performing input and output operations or exceptional cases.
 3. Many programs spend a lot of time in executing the inner loops or the recursive calls.
- The locality makes use of **memory hierarchy** for increasing the efficiency in program execution. For instance by placing the most common data in fast but small storage and less frequently needed data in slow and large storage the memory access time can be reduced.
 - We can improve the temporal and special locality of data access in the program by changing the order of computation. For example we can bring the data from slow level(main memory) to fast level(disk) and perform the computation on this data. This makes the execution faster.

Q.20 What is fragmentation ? Explain.

[JNTU : Part A, Marks 3]

Ans. : • Due to frequent memory allocation and deallocation the heap memory becomes fragmented. That is the memory contains many free slots as well as many chunks of allocated memory. The free slots in the memory are called **holes**.

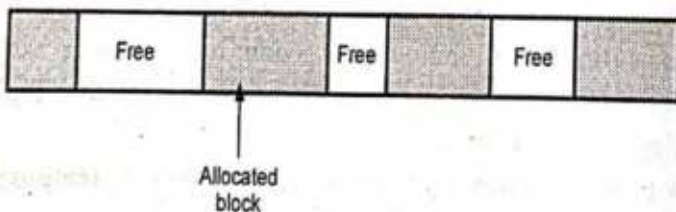


Fig. Q.20.1 Heap memory (Fragmented)

- When a memory allocation request comes, the memory manager places the request into large enough hole. This will satisfy the request but at the same time it creates one more small hole.
- With the deallocation request, the freed space is returned to the pool of free space. If memory allocation or deallocation is not done carefully many small noncontiguous holes in the memory may get created. This leads to fragmentation.

Q.21 Explain in detail, the strategy for reducing fragmentation in heap memory.

[JNTU : Part B, Marks 5]

Ans. : Two strategies are used for reducing fragmentation :

1) Best and next fit

• Best fit :

- In the best fit strategy, the smallest hole which is large enough for the object that demands for allocation of memory is used.
- In this method the memory manager searches the entire list of free slots and finds the hole which is the best possible that can be used for allocating the memory for the target object.
- Although this strategy produces best allocation space, this method is less efficient as it requires more time in searching the appropriate hole and create many small holes.

• Next fit :

- In this strategy, the free list of holes is simply scanned to find the large enough hole to hold the object.
- This strategy is more efficient than the best fit. However due to this strategy the small holes tend to accumulate at the beginning of the free list.
- The problem of small holes accumulating is solved with next fit strategy which starts each search where the last one left off. It wraps around to the beginning when the end of the list is reached.

2) Managing and coalescing free space

- Due allocation of memory randomly many holes are created in the memory.

- Coalescing is the technique by which many free spaces are combined together to form a large chunk of free space.
- The advantage of this strategy is that the fragmentation gets reduced and a large free chunk is available for allocating the space for the object of large size (since many small chunks can not hold the large object).

4.5 : Introduction to Garbage Collection

Q.22 Explain the term garbage collection

[JNTU : Part A, Marks 2]

Ans. : • Garbage is a substantial amount of memory which gets allocated dynamically but is unreachable memory. If this memory is unreachable then it simply becomes the wastage of memory. Hence a new technique called garbage collector is introduced.

- Garbage collection is a technique in which the chunks memory holding unreachable objects are reclaimed.

Q.23 What is mutator in garbage collection ?

Ans. : Mutator is a user program which creates the objects using the heap memory. These mutators may refer or drop the references to another objects. The objects become garbage when the mutator programs can not reach them. Then the job of garbage collector is to reclaim such objects and handing them to memory manager.

Q.24 Explain the concept reachable objects in fragmentation.

Ans. : • The data that can be accessed by the program directly without using the reference to pointer is called root set.

- In Java, the root set consists of all the static field members and all the variables on its stack. Thus the program can access the members of root set any time. The objects that are references to these data members are called reachable objects.

Q.25 What are the design goals of garbage collection ?

[JNTU : Part B, Marks 5]

Ans. : 1) Type safety

- The automatic garbage collection is not possible with all the languages. There are some languages

the type of data components can be determined. Such languages are said to be type safe. For example ML.

- There are some languages whose type cannot be determined at the compile time but can be determined at run time. Such languages are called dynamically typed languages. For example Java.
- If the languages are not statically or dynamically type safe then such languages are called unsafe. For example C, C++.
- The unsafe languages are bad candidates of garbage collection mechanism. In unsafe languages the memory addresses are manipulated arbitrarily and no storage can be reclaimed safely.

2) Performance metrics

- Following are some performance metrics that must be considered while designing the garbage collector

i) **Space usage** : The garbage collection must avoid fragmentation and should make best use of memory.

ii) **Overall execution time** : The garbage collection is slow. It should not increase total run time of application. Its performance is determined by how it accesses the memory subsystem.

iii) **Program locality** : Simply the running time can not determine the speed of the garbage collection. The garbage collection controls the placement of data. Due to this locality of data gets affected. The locality of data plays an important role in determining the total required running time of the program.

iv) **Pause time** : Garbage collectors cause the programs (mutators) to pause suddenly for arbitrarily long time. Hence instead of minimizing overall execution time it is essential to minimize the pause time. In real time applications certain computations must be completed in some specific time. In this case the maximum pause time must be reduced. In fact, Garbage collection is occasionally used in case of real time applications.

Q.26 Explain the four operations that can be performed to change the set of reachable objects.

[JNTU : Part A, Marks 4]

Compiler Design

Ans. : Following are four basic operations that a program(mutator) can perform to change the set of reachable objects -

1. **Object allocation** : Memory manager allocates the chunk of memory for the objects. Due to this new objects are getting added in the list of reachable objects.
2. **Parameter passing and return** : Reference of objects can be passed as parameter and some results are returned back. The objects that are denoted by the references remain reachable.
3. **Reference assignments** : When we write the assignment statement $a=b$ where a and b are both references then the object referred by b is also referred by a . Another consequence of this is that the original reference of a gets lost and now it points to the object pointed by b . If the object to which the b is referring becomes unreachable then all the objects that are reachable through b also becomes unreachable.
4. **Procedure returns** : The local variables in a the procedure have the limited scope. If this procedure holds the reachable references to the objects then those objects become unreachable when the procedure returns back to the caller.

Q.27 What is reference counter used in garbage collection ? [JNTU : Part A, Marks 2]

Ans. : Reference count is a special counter used during implicit memory allocation. With reference counting mechanism in garbage collector every object must have a field for the reference count.

4.6 : Introduction to Trace Based Collection

Q.28 What are different states that may occur during trace based algorithms ?

Ans. : Following are the states in trace based algorithm -

- 1) **Free** : A chunk is in free state if it is ready to get allocated. A free chunk does not hold a reachable object.
- 2) **Unreached** : Initially chunks are supposed to be unreachable. Whenever a chunk is allocated by memory manager its state is set to unreachable state. After the round of garbage collection, the

state of reachable object is reset to unreached to get ready for next round.

- 3) **Unscanned** : The reachable chunks can be either in unscanned or scanned states. If the chunk is a reachable chunk and but its pointers have not been scanned, then that chunk is in unscanned state.
- 4) **Scanned** : Every unscanned object will lead to a scanned state. To scan an object, we examine each of the pointers within it and follow those pointers to the objects to which they refer. If a reference is to an unreached object, then that object is put in the unscanned state. When the scan of an object is completed, that object is placed in the Scanned state. A scanned object can only contain references to other scanned or unscanned objects and never to unreached objects.

Q.29 Explain basic marks and sweep algorithm.

[JNTU : Part B, Marks 5]

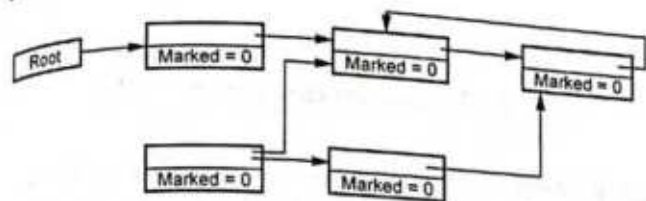
Ans. : The mark and sweep algorithm is a garbage collection algorithm in which two major operations are performed - marks and sweep.

Mark phase : Before entering the object into the mark phase, there is a mark bit associated with each object which is set to false(0). In the mark stage, start from the root node in your application and set the marked bit for all the reachable items to 1(true). Now to perform this operation we simply need to do a graph traversal, a depth first search. In this traversing, each object is considered as a node. Then all the nodes that are reachable from this node or root note are considered as reachable nodes and set their mark bit to true.

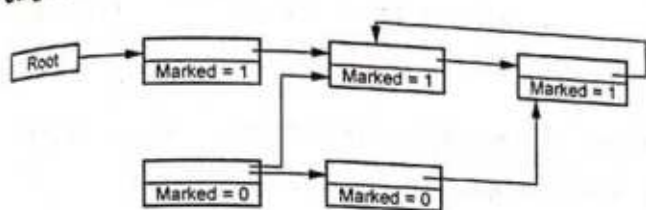
Sweep phase : In this phase, all the unreachable objects are cleared. That means it clears the heap memory for all the unreachable objects. All those objects whose marked value is set to false are cleared from the heap memory. The unreachable objects are thus reclaimed.

Now the mark value for all the reachable objects is set to false again since we will run the algorithm and again we will go through the mark phase to mark all the reachable objects.

This algorithm is also called tracing algorithm.
Before garbage collection



Stage : After mark phase



Stage : After sweep phase

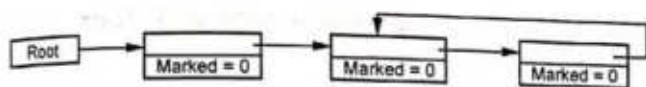


Fig. Q.29.1 Stages in mark-sweep algorithm

Part II : Code Generation

4.7 : Issues in Design of Code Generator

Q.30 Mention the properties that the code generator should possess.

[JNTU : Part A, May-18, Marks 2, Nov.-15, Marks 3]

Ans. : Various properties desired by an object code generation phase are

- **Correctness** - It should produce a correct code and do not alter the purpose of source code.
- **High quality** - It should produce a high quality object code.

- **Efficient use of resources of the target machine** -

While generating the code it is necessary to know the target machine on which it is going to get generated. By this the code generation phase can make an efficient use of resources of the target machine. For instance memory utilization while allocating the registers or utilization of arithmetic logic unit while performing the arithmetic operations.

- **Quick code generation** - This is a most desired feature of code generation phase. It is necessary that the code generation phase should produce the code quickly after compiling the source program.

Q.31 Describe code generator design issues.

[JNTU : Part B, Marks 5]

Ans. : Input to the code generator : The code generation phase takes intermediate code as input. This intermediate code along with the symbol table information is used to determine the runtime addresses of the data objects. The code generation phase requires the complete error free intermediate code as input for generating the target code.

Target programs : The output of code generator is target code. Typically, the target code comes in three forms such as : Absolute machine language, relocatable machine language and assembly language. The advantage of producing target code in absolute machine form is that it can be placed directly at the fixed memory location and then can be executed immediately.

Memory management : Both the front end and code generator performs the task of mapping the names in the source program to addresses to the data objects in run time memory. The names used in the three address code refer to the entries in the symbol table. The type in a declaration statement determines the amount of storage(memory) needed to store the declared identifier. Thus using the symbol table information about memory requirements code generator determines the addresses in the target code.

Register allocation : If the instruction contains register operands then such a use becomes shorter and faster than that of using operands in the memory. Hence while generating a good code efficient utilization of register is an important factor.

Choice of evaluation order : The evaluation order is an important factor in generating an efficient target code. Some orders require less number of registers to hold the intermediate results than the others. Picking up the best order is one of the difficulty in code generation.

Q.32 Discuss about instruction selection and register allocation. [JNTU : Part B, Nov.-17, Marks 5]

Ans. : Refer Q.31.

4.8 : The Target Language

Q.33 What are the forms of a target program ?

[JNTU : Part A, Dec.-17, Marks 2]

OR What are various possible outputs of the code generator. [JNTU : Part A, Dec.-16, Marks 2]

Ans. : • The output of code generation is an object code or machine code. This code normally comes in following forms

1. Absolute code
2. Relocatable machine code
3. Assembler code

• Let us discuss each in detail.

1. **Absolute code** - Absolute code is a machine code that contains reference to actual addresses within program's address space. The generated code can be placed directly in the memory and execution starts immediately. Generally small programs can be compiled and executed quickly because of absolute code generation. A number of "student-job" compilers such as WATFIV and PL/C produce absolute code.

2. **Relocatable code** - Producing a relocatable machine language program as output allows subprograms to be compiled separately. A set of relocatable object modules can be linked together and loaded for execution with the help of a *linking loader*. The advantage of generating relocatable machine language code is that we gain a great deal of flexibility in being able to compile subroutines separately and to call other previously compiled programs from an object module.

If the target machine does not handle relocation automatically, the compiler must provide explicit relocation information to the loader to link the separately compiled program segments.

3. **Assembler code** - Producing an assembly language program as output makes the process of code generation somewhat easier. We can generate symbolic instructions and use the macro facilities of the assembler to help in generation of code. But generating assembler code as an output makes code generation process slower. (because assembling, linking and loading is required.)

Q.34 What are the object code forms ? Explain the issues in code generation.

[JNTU : Part B, Mar.-16, Marks 5]

Ans. : Refer Q.33 and Q.31.

4.9 : Addresses in the Target Code

Q.35 Explain the form of target machine code in detail. [JNTU : Part B, Marks 5]

Ans. : • We will assume that in the target computer addresses are given in bytes and four bytes form a word.

- There are n general purpose registers $R_0, R_1, \dots, R_{n-1}$.
- The two address instruction is of the form.

op source destination

where *op* is an opcode and source and destination are data fields.

For instance :

MOV - moves from source to destination.

ADD - add source to destination.

SUB - subtracts source from destination.

The source and destination are specified by registers and memory locations

- The addressing modes used are as follows

Addressing mode	Form	Address	Added cost
absolute	M	M	1
register	R	R	0
indexed	c(R)	c + contents(R)	1
indirect register	*R	contents(R)	0
indirect indexed	*c(R)	contents(c+contents(R))	1
literal	#c	c	1

- If we have absolute or register addressing mode we can use M or register for source or destination.
- For example, the instruction MOV R_1, M stores the contents of register R_1 into memory location M.

• For the indexed addressing mode the address offset from the value of register R0 can be written as

MOV 7(R1), M

• Means it stores the value contents (7+ contents(R1)) to the memory location M.

The last two addressing modes represent the indirect addresses indicated by *.

• For the instruction MOV *7(R0),M

It stores the value contents(contents (7+ contents(R0))) to the memory location M.

• In the literal addressing mode the source becomes constant.

• For example MOV #5,R0

By this instruction we can store the constant 5 into register R0.

4.10 : Basic Blocks and Flow Graphs

Q.36 Define basic blocks.

[JNTU : Part A, March-16, Marks 3, Nov.-15, Marks 2]

Ans.: The basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching.

An example of the basic block is as shown below.

```
t1 := a * 5
t2 := t1 + 7
t3 := t2 - 5
t4 := t1 + t3
t5 := t2 + b
```

Q.37 List the terminologies used in basic blocks.

[JNTU : Part A, May-18, Marks 2]

Ans.: • Define and use - The three address statement $a := b+c$ is said to define a and to use b and c.

• Live and dead - The name in the basic block is said to be live at a given point if its value is used after that point in the program. And the name (variable) in the basic block is said to be dead at a given point if its value is never used after that point in the program.

Q.38 How can you identify the leader in a basic block?

[JNTU : Part A, Mar.-16, Marks 3]

Ans.: Following rules are used to identify leader in a basic block -

- The first statement is a leader.
- Any target statement of conditional or unconditional goto is a leader.
- Any statement that immediately follow a goto or unconditional goto is a leader.

Q.39 Write procedure to identify basic blocks.

[JNTU : Part B, March-17, Marks 5]

OR Write an algorithm for constructing basic block.

[JNTU : Part A, Dec.-16, Marks 3]

Ans.: Any given program can be partitioned into basic blocks by using following algorithm. We assume that an intermediate code is already generated for the given program.

- First determine the leaders by using following rules.
 - The first statement is a leader.
 - Any target statement of conditional or unconditional goto is a leader.
 - Any statement that immediately follow a goto or unconditional goto is a leader.
- The basic block is formed starting at the leader statement and ending just before the next leader statement appearing.

Q.40 Consider the following program code for computing dot product of two vectors a and b of length 10 and partition it into basic blocks.

prod = 0;

i = 1;

do

{

prod = prod + a[i] * b[i];

i=i+1;

}while(i<=10); [JNTU : Part B, May-09, Marks 16]

Ans.: First we will write the equivalent three address code for the above program.

- prod := 0
- i := 1
- t₁ := 4 * i
- t₂ := a[t₁] /* computation of a[i] */

5. $t_3 := 4 * i$
6. $t_4 := b[t_3]$ /* computation of $b[i]$ */
7. $t_5 := t_2 * t_4$
8. $t_6 := \text{prod} + t_5$
9. $\text{prod} := t_6$
10. $t_7 := i + 1$
11. $i := t_7$
12. if $i \leq 10$ goto (3)

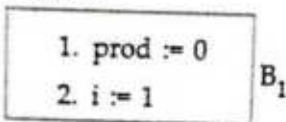
According to the algorithm

Statement 1 is a leader by rule 1(a).

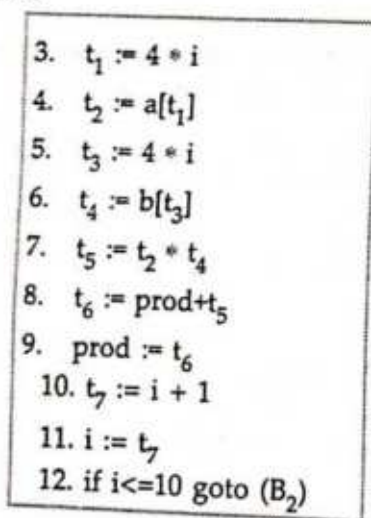
Statement 3 is also leader by rule 1(b).

Hence, statement 1 and 2 form the basic block. Similarly statement 3 to 12 form another basic block.

Block 1



Block 2



Q.41 What is a flow graph ?

☞ [JNTU : Part A, May-18, Dec.-17, Marks 3, Dec.-16, Marks 2]

Ans. : A flow graph is a directed graph in which the flow control information is added to the basic blocks.

For example, consider the three address code as

1. $\text{prod} := 0$
2. $i := 1$
3. $t_1 := 4 * i$
4. $t_2 := a[t_1]$ /* computation of $a[i]$ */

5. $t_3 := 4 * i$
6. $t_4 := b[t_3]$ /* computation of $b[i]$ */
7. $t_5 := t_2 * t_4$
8. $t_6 := \text{prod} + t_5$
9. $\text{prod} := t_6$
10. $t_7 := i + 1$
11. $i := t_7$
12. if $i \leq 10$ goto (3)

The flow graph for the above code can be drawn as follows.

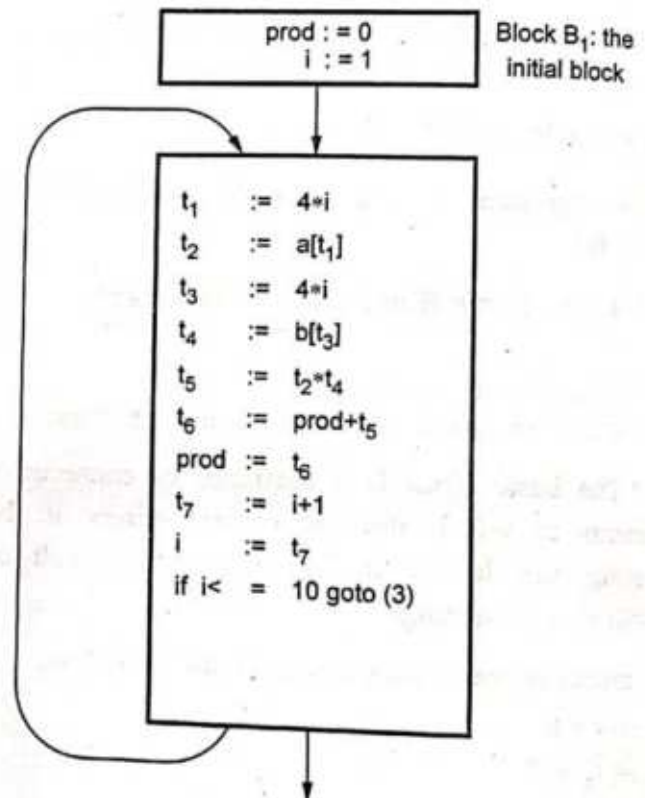


Fig. Q.41.1 Flow graph

Q.42 Define flow graph. Explain how a given program can be converted to a flow graph ?

☞ [JNTU : Part B, Mar.-16, Marks 5]

Ans. : Refer Q.41.

4.11 : Optimization of Basic Blocks

Q.43 What is DAG ? Mention its applications.

☞ [JNTU : Part A, May-18, Nov.-15, Marks 3]

Ans. : DAG : DAG stands for directed acyclic graph. It is a graph that is directed and without cycles connecting the other edges.

For example -

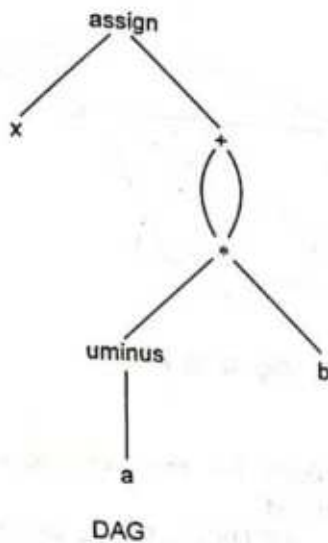


Fig. Q.43.1 DAG

Applications : The DAG is used in

1. Determining the common sub-expressions (expressions computed more than once).
2. Determining which names are used inside the block and computed outside the block.
3. Determining which statements of the block could have their computed value outside the block.
4. Simplifying the list of quadruples by eliminating the common sub-expressions and not performing the assignment of the form $x := y$ unless and until it is a must.

Q.44 Write an algorithm for construction of DAG

[JNTU : Part B, Marks 5]

Ans. : We assume the three address statement could of following types

Case (i) $x := y \text{ op } z$

Case (ii) $x := \text{op } y$

Case (iii) $x := y$

With the help of following steps the DAG can be constructed

Step 1 : If y is undefined then create node(y). Similarly if z is undefined create a node(z).

Step 2 : For the case(i) create a node(op) whose left child is node(y) and node(z) will be the right child. Also check for any common subexpressions. For the

case(ii) determine whether is a node labeled op , such node will have a child node(y). In case(iii) node n will be node(y).

Step 3 : Delete x from list of identifiers for node(x). Append x to the list of attached identifiers for node n found in 2.

Q.45 For the following statements draw the DAG.

$t_1 = a + b;$

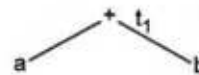
$t_2 = a * b;$

$t_3 = t_1 * b;$

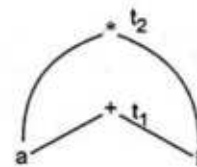
$c = t_3 + t_2;$

[JNTU : Part B, Jan.-12, Marks 5]

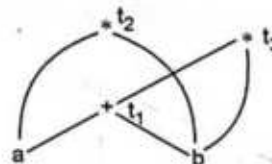
Ans. : Step 1 :



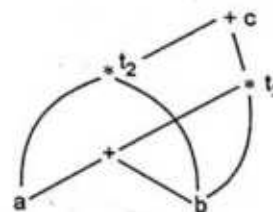
Step 2 :



Step 3 :



Step 4 :



Q.46 For the following statements draw the DAG.

$t_8 = a * b$

$t_6 = t_8 + b$

$t_5 = a - b$

$t_4 = t_6 * t_5$

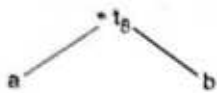
$t_3 = t_4 - e$

$t_2 = t_8 * t_4$

$t_1 = t_2 * t_3$

[JNTU : Part B, Jan.-12, Marks 5]

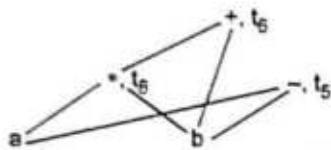
Ans. : Step 1 :



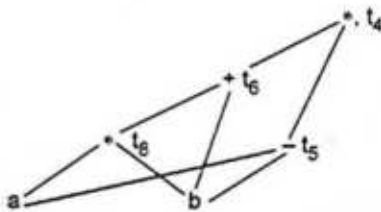
Step 2 :



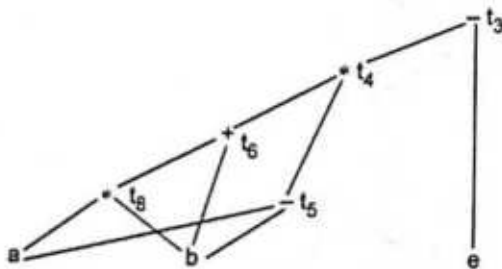
Step 3 :



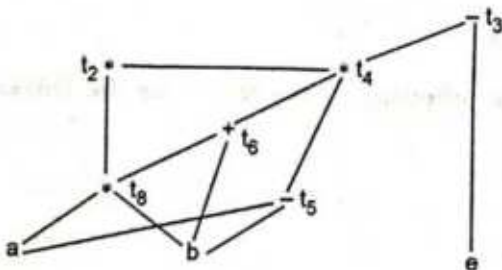
Step 4 :



Step 5 :



Step 6 :



Step 7 :

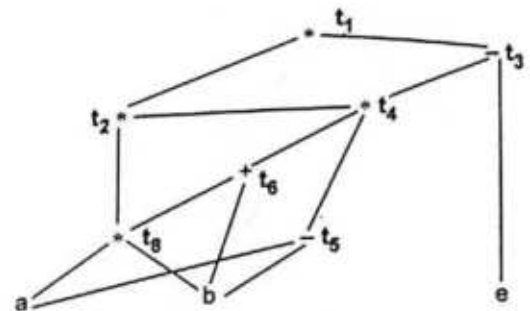
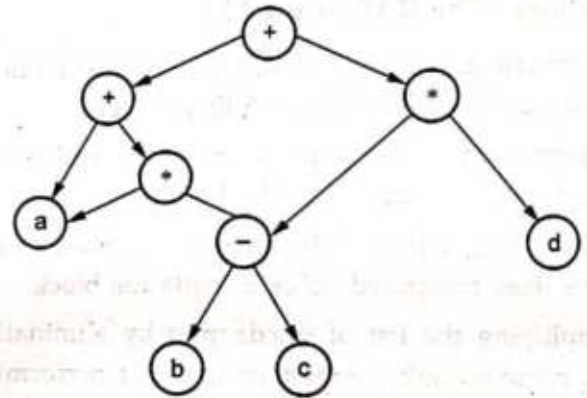


Fig. Q.46.1 DAG

Q.47 Construct DAG for the expression $a + a * (b - c) + (b - c) * d$

[JNTU : Part B, Dec.-12, Marks 5]

Ans. : The DAG will be



Q.48 Construct DAG for the following basic block :

$T1 = A + B$

$T2 = C + D$

$T2 = E - T2$

$T4 = T1 - T3$

[JNTU : Part A, Dec.-16, Marks 3]

Ans. : We will construct a DAG for

$$t_1 = a + b$$

$$t_2 = c + d$$

$$t_3 = e - t_2$$

$$t_4 = t_1 - t_3$$

For the above order of evaluation we get the code as
[Assuming two registers R_0 and R_1 are available]

MOV a, R_0

ADD b, R_0

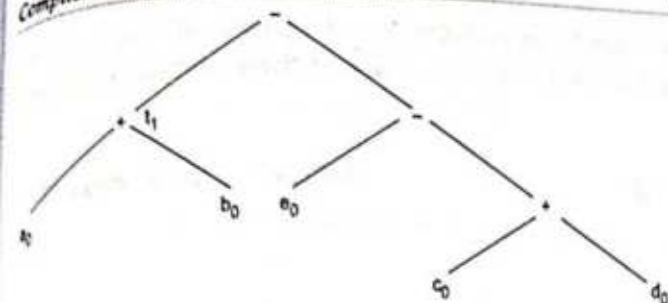


Fig. Q.48.1

MOV c, R₁

ADD d, R₁

MOV R₀, t₁

MOV e, R₀

SUB R₁, R₀

MOV t₁, R₁

SUB R₀, R₁

MOV R₁, t₄

If we change the sequence of computation as :

$$t_2 = c + d$$

$$t_3 = e - t_2$$

$$t_1 = a + b$$

$$t_4 = t_1 - t_3$$

Then the generated code will be

MOV c, R₀

ADD d, R₀

MOV e, R₁

SUB R₀, R₁

MOV a, R₀

ADD b, R₀

SUB R₁, R₀

MOV R₀, t₄

Thus by rearranging the order the code can be generated.

Q.49 What is DAG ? Construct DAG for the following basic block ?

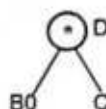
D=B*C; E=A+B; B=B*C; A=E-D;

[JNTU : Part B, Nov.-17, Mar.-16, Marks 10]

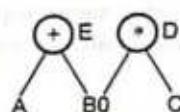
Ans. : Definition of DAG : Refer Q.43.

DAG for basic block :

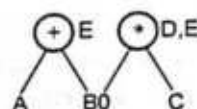
Step 1 :



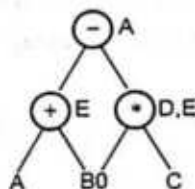
Step 2 :



Step 3 :



Step 4 : Finally we get the following DAG



Q.50 Construct DAG for $a + a * (b - c) + (b - c) * d$. Also generate three address code for same.

[JNTU : Part B, Marks 5]

Ans. : The three address code is

$$t_1 = b - c$$

$$t_2 = a * t_1$$

$$t_3 = a + t_2$$

$$t_4 = t_1 * d$$

$$t_5 = t_3 + t_4$$

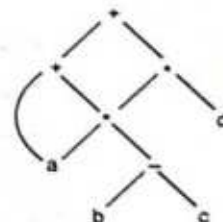


Fig. Q.50.1 DAG for $a + a * (b - c) + (b - c) * d$

Q.51 Describe how DAG can be used in register allocation process ? Give examples.

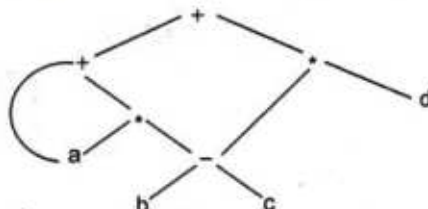
[JNTU : Part B, Nov.-15, May-18, Marks 10]

Ans. : The DAG can be used for code generation. In this process the three address code is generated and DAG is constructed for this three address code. Sometimes we may need to rearrange the order of three address code for efficient usage of register.

Q.52 Draw a DAG for expression $a + a * (b - c) + (b - c) * d$

[JNTU : Part B, Marks 5]

Ans. :



Q.53 Explain how type checking and error reporting is performed in a compiler. Draw syntax tree and DAG for following statement. Write three address codes from both. $a = (a + b * c) \wedge (b * c) + b * c$.

[JNTU : Part B, Marks 5]

Ans. : The type checker is a translation scheme in which the type of each expression from the types of subexpressions is obtained. The type checker can decide the types for arrays, pointers, statements and functions.

The type checker ensures following things -

- Each identifier must be declared before the use.
- The use of identifier must be within the scope.
- An identifier must not have multiple definitions at a time within the same scope.

The given grammar has basic data types as char, int, float and for reporting the errors type_error.

We assume that the index of the array start at 0. For example if

`int arr[100];`

leads to type expression as `array(0,1...99,int)` similarly

`int *ptr;`

The type expression for the above statement can be `pointer(int)`

Let us write the translation scheme for the above grammar.

Production rule	Type expression
$S \rightarrow D;E$	This means all declarations before expressions
$D \rightarrow T \text{ LIST}$	<code>LIST.type:=T.type</code>
$\text{LIST} \rightarrow \text{id}$	<code>{addtype(id.entry,LIST.type)}</code>
$T \rightarrow \text{char}$	<code>{T.type:=char}</code>
$T \rightarrow \text{int}$	<code>{T.type:=int}</code>
$T \rightarrow \text{float}$	<code>{T.type:=float}</code>
$\text{LIST} \rightarrow *L_1$	<code>{L_1.type := pointer(LIST.type)}</code>
$\text{LIST} \rightarrow L_1[\text{num}]$	<code>{L_1.type:=array(0...num.val-1,LIST.type)}</code>

Type checker

After obtaining the type expression it becomes convenient to write the semantic rules. These are the semantic rules for type checking of expression.

For example : while performing binary operation the data types are decided as follows

$E \rightarrow E_1 \text{ op } E_2$ {E.type:= if E_1 .type:= int and E_2 .type:= int

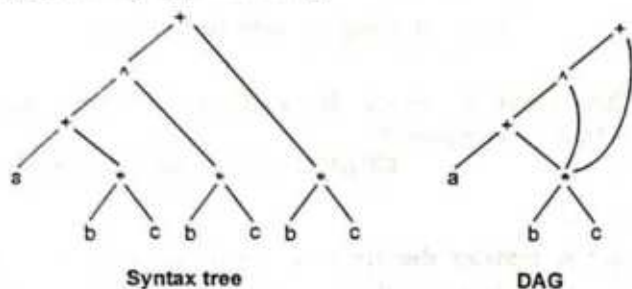
then int

else if E_1 .type:= float and E_2 .type:= float then float

else if E_1 .type:= char and E_2 .type:= char then char

else type_error

The type_error routine is used to report the error that occur during type checking.



Three address code for syntax tree	Three address code for DAG
$t_1 := b * c$	$t_1 := b * c$
$t_2 := a + t_1$	$t_2 := a + t_1$
$t_3 := b * c$	$t_3 := t_2 \wedge t_1$
$t_4 := t_2 \wedge t_3$	$t_4 := t_3 + t_1$
$t_5 := b * c$	
$t_6 := t_4 + t_5$	

Q.54 Construct DAG of basic blocks after converting the code in 3-address representation :

$i = 1$;
 $j = 2$;
 repeat
 $A[i] = j$
 $j = j * 2$;
 $i = i + 1$;
 until $(i > 10)$.

[JNTU : Part B, Marks 5]

Ans. : 1) $i := 1$

2) $j := 2$

3) $t1 := 4 * i$ /* 4 is a width */

4) $t2 := \text{Addr}(A) - 4$

5) $t2[t1] := j$

6) $j := j * 2$

7) $i := i + 1$

8) if $i < 10$ goto (3)

DAG can be

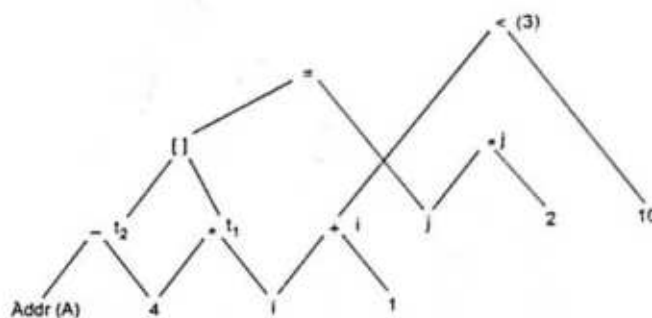
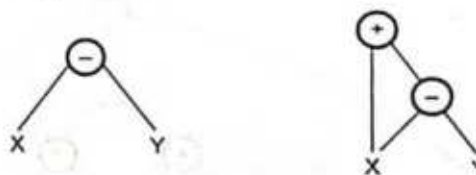


Fig. Q.54.1

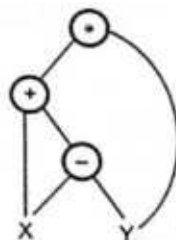
Q.55 Show the DAG for statement $Z = X - Y + X * Y * U - V/W + X + V$. [JNTU : Part B, Marks 5]

Ans. : Step 1

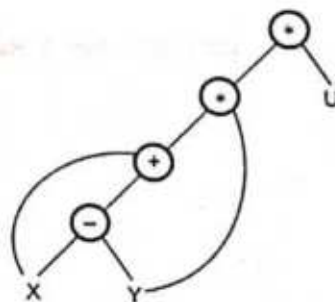
Step 2 : $X - Y + X$



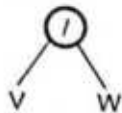
Step 3 : $X - Y + X * Y$



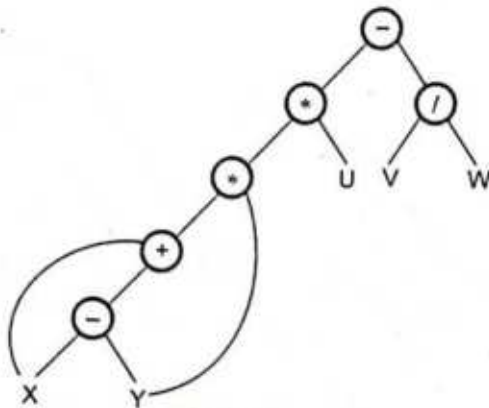
Step 4 : $X - Y + X * Y * U$



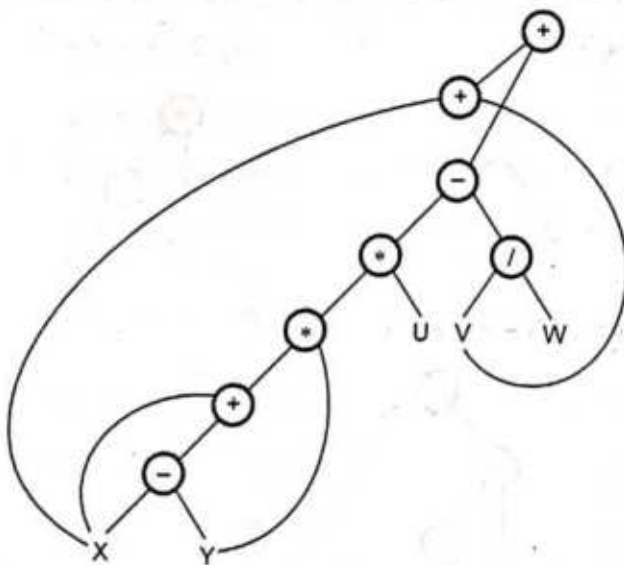
Step 5 : V / W



Step 6 : $X - Y + X * Y * U - V / W$



Step 7 : $X - Y + X * Y * U - V / W + X + V$



Q.56 Draw DAG for given block :

$a = b + c$

$b = a - d$

$c = b + c$

$d = a - d$

[JNTU : Part B, Marks 5]

Ans. :

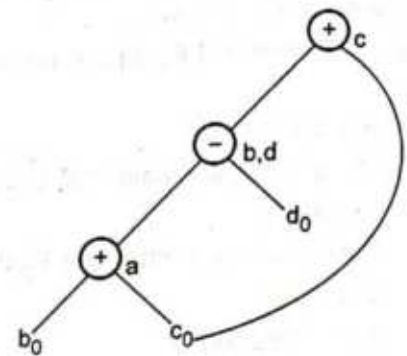


Fig: Q.56.1

4.12 : A Simple Code Generator

Q.57 What is meant by register descriptor and address descriptor ?

[JNTU : Part A, Mar.-17, Marks 2]

Ans. :

1. A register descriptor is used to keep track of what is currently in each register. The register descriptors show that initially all the registers are empty. As the code generation for the block progresses the registers will hold the values of computations.

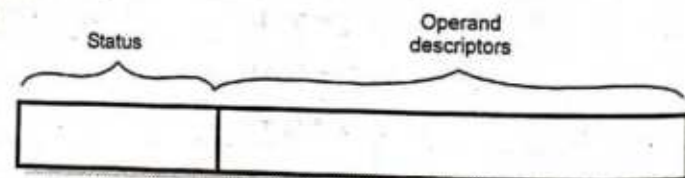


Fig. Q.57.1 Register descriptor

2. The address descriptor stores the location where the current value of the name can be found at run time. The information about locations can be stored in the symbol table and is used to access the variables.

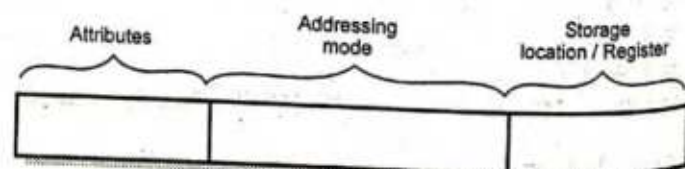


Fig. Q.57.2 Address descriptor

- The address descriptor has following fields

- The attributes mean type of the operand. It generally refers to the name of temporary variables.
- The addressing mode indicates whether the addresses are of type 'S', 'R', 'IS', 'IR'.
S is used to indicate value of operand in storage.
R is used to indicate value of operand in register.
IS indicates that the address of operand is stored in storage i.e. indirect accessing.
IR indicates that the address of operand is stored in register i.e. indirect accessing.
- The third field is location field which indicates whether the address is in storage location or in register.

Q.58 Generate the code for the following expression : $x = (a+b) - ((c+d) - e)$. Also compute its cost. [JNTU : Part B, Mar.-17, Marks 10]

Ans. : We will write the three address code for the given expression

$T1 = a + b$

$T2 = c + d$

$T3 = T2 - e$

$T4 = T1 - T3$

The code can be generated as follows

MOV a, R0

ADD b, R0 /* $R0 = a+b$ */

MOV d, R1

ADD c, R1 /* $R1 = c+d$ */

SUB e, R1

SUB R1, R0

MOV R0, x

The cost of above instructions can be computed as -

Instruction	Cost
MOV a, R0	$1 + 1 + 0 = 2$
ADD b, R0	$1 + 1 + 0 = 2$
MOV d, R1	$1 + 1 + 0 = 2$
ADD c, R1	$1 + 1 + 0 = 2$
SUB e, R1	$1 + 1 + 0 = 2$
SUB R1, R0	$1 + 0 + 0 = 1$
MOV R0, x	$1 + 0 + 1 = 2$
Total Cost	13

Q.59 Consider the following code sequence

(i) MOV B, R0 (ii) MOV B, A
ADD C, R0 ADD C, A
MOV R0, A

Calculate cost of above instructions.

[JNTU : Part B, Aug.-07, Marks 10]

Ans. : MOV B, R0

ADD C, R0

MOV R0, A

For instruction

MOV B, R0
↓ ↓ ↓
1 1 0
i.e. Cost of instruction Cost of memory location cost of register

∴ MOV B, R0 → cost = $1 + 1 + 0 = 2$

ADD C, R0 → cost = $1 + 1 + 0 = 2$

MOV R0, A → cost = $1 + 0 + 1 = 2$

Total cost = 6

ii)

MOV B, A
↓ ↓ ↓
[Cost of instruction] [Cost of memory location] [Cost of memory location]
↓ ↓ ↓
1 1 1

∴ MOV B, A → cost = 1 + 1 + 1 = 3
ADD C, A → cost = 1 + 1 + 1 = 3

Total cost = 6

Q.60 Write an algorithm for code generation using GETREG. [JNTU : Part B, Marks 5]

Ans. : Read the expression in the form of Operator, operand1 and operand2 and generate code using following algorithm -

```
Gen_Code(operator,operand1,operand2)
{
    if (operand1.addressmode = 'R')
    {
        if (operator = '+')
            Generate('ADD operand2,R0');
        else if(operator = '-')
            Generate('SUB operand2,R0');
        else if(operator = '*')
            Generate('MUL operand2,R0');
        else if(operator = '/')
            Generate('DIV operand2,R0');
    }
    else if (operand2.addressmode = 'R')
    {
        if (operator = '+')
            Generate('ADD operand1,R0');
        else if(operator = '-')
            Generate('SUB operand1,R0');
        else if(operator = '*')
            Generate('MUL operand1,R0');
        else if(operator = '/')
            Generate('DIV operand1,R0');
    }
    else
    {
        Generate('MOV operand2,R0');
        if (operator = '+')
            Generate('ADD operand2,R0');
        else if(operator = '-')
            Generate('SUB operand2,R0');
        else if(operator = '*')
            Generate('MUL operand2,R0');
        else if(operator = '/')
            Generate('DIV operand2,R0');
    }
}
```

Q.61 Generate the code sequence using code generation algorithm for the following expression
 $W := (A - B) + (A - C) + (A - C).$

[JNTU : Part B, Aug.-08, Marks 8]

Ans. : We will write the 3 address code for given expression

$t_1 := A - B$

$t_2 := A - C$

$t_3 := t_1 + t_2$

$t_4 := t_3 + t_2$

$W := t_4$

Three address code	Target code sequence	Register descriptor	Operand/ address descriptor
$t_1 := A - B$	MOV A, R ₀ SUB B, R ₀	Empty R ₀ contains t ₁	t ₁ is in R ₀
$t_2 := A - C$	MOV A, R ₁ SUB C, R ₁	R ₁ contains A R ₁ contains t ₂	t ₂ is in R ₁
$t_3 := t_1 + t_2$	ADD R ₁ , R ₀	R ₀ contains t ₃	t ₃ in R ₀
$t_4 := t_3 + t_2$ $W := t_4$	ADD R ₁ , R ₀ MOV R ₀ , W	R ₀ contains t ₄ R ₀ contains W	W is in R ₀

Q.62 Generate code for the following C program using any code generation algorithm.

main()

```
{
    int i;
    int a[10];
    while(i<=10)
        a[i]=0;
}
```

[JNTU : Part B, May-08, Marks 10]

Ans. : First of all we will generate three address codes for the given C code.

Label 1 : if i<=10 goto Label 2

goto Label 3

Label 2 : $t_1 := i * 4$

$t_2 := \text{addr}(a)$

$t_2[t_1] := 0$

goto Label1

Label 3 : stop

The code generation using simple code generation algorithm for the corresponding code is

MOV I,R0

Label 1 : CMP R0,#10

JLE Label2

JMP Label3

Label 2 : MOV #0,a(R0)

JMP Label1

Label 3 : EXIT

Q.63 Generate code for following c statements

- (i) $x = f(a) + f(a) + f(a)$ (ii) $x = f(a) / g(b, c)$
 (iii) $x = f(f(a))$ (iv) $x = ++f(a)$

[JNTU : Part B, May-08, Marks 8]

Ans. : i) $f(a)$ denotes a call to the function. This call is made thrice. And then addition of their return values is done. The result is finally stored in variable x .

We will first write a three address code for given statement.

$t_1 := 0$

$t_2 := 1$

$t_3 := 3$

L_1 : if $t_2 \leq t_3$ goto Loop

goto End

Loop : parama

call $f,1$

return t_4

$t_1 := t_1 + t_4$

$t_2 := t_2 + 1$

goto L_1

End :

The code will be

- | | |
|--|--|
| (10) MOV #0, R ₀ | (11) MOV #1, R ₁ |
| (12) MOV #3, R ₂ | (13) CMP R ₁ , R ₂ |
| (14) CJ <= (16) | (15) HALT |
| (16) MOV a, R ₃ /* parameter a is in register R3 */ | |
| (17) GOTO 100 /* At location 100 procedure f is defined */. | |
| (18) MOV AH, R ₄ /* return val from function f is in AH */ | |
| (19) ADD R ₄ , R ₀ /* it is stored in R4 then added to R0 */ | |
| (20) MOV R ₀ , x /* R0 value is stored in x */ | |

```

(21) INC R1
(22) Loop (13)
:
(100) /* procedure for f */

```

ii) $x = f(a) / g(b, c)$

Param a

call f, 1

return t₁

Param b

Param c

call g, 2 /* 2 parameter to g */

return t₂

$x = t_1 / t_2$.

The code will be

```

(1) MOV a, R0
(2) GOTO 100
(3) MOV AX, R1 /* return value of f(a) in R1 */
(4) MOV b, R2
(5) MOV c, R3
(6) GOTO 200
(7) MOV AX, R4 /* return value of g(b, c) in R4 */
(8) DIV R1, R4 /* f(a)/g(b, c) */
(9) MOV R4, x. /* store result in x */
:
(100) /* procedure for f */
:
(200) /* procedure for g */
:

```

iii) Three address code for $x = f(f(a))$

param a

call f, 1

return t₁

param t₁

call f, 1

return t₂

$x := t_2$

The code will be

```

(1) MOV a, R0 /* parameter a in R0 */
(2) GOTO 100 /* at location 100 f(a) is defined */
(3) MOV AX, R1 /* return address of AX is stored in R */
(4) MOV R1, R2 /* Take parameter R1 in register R2 */
(5) GOTO 200 /* at 200 location procedure f(f(a)) */
(6) MOV AX, R3 /* return value in R3 */
(7) MOV R3, x /* finally in x */
:
(100) /* procedure for f(a) */
:
(200) /* f(f(a)) */
:

```

iv) $x = ++f(a)$

param a

call f, 1

return t₁

$t_1 = t_1 + 1$

$x := t_1$

The code will be

- (1) MOV a, R₀ /* parameter a is in R₀ */
- (2) GOTO 100 /* At location 100 f(a) is defined */
- (3) MOV AX, R₁ /* MOV return value in R₁ */
- (4) ADD # 1, R₁
- (5) MOV R₁, x /* Final result in variable x */
- ⋮
- (100) /* procedure for f(a) */

Q.64 Generate code for following statements for the target machine (target machine is a byte addressable machine with 4 bytes to a word and N general purpose registers). Assuming all variables are static. Assume 3 registers are available.

- (a) $x = a[I + 1]$
 - (b) $a[I] = b[c[I]]$
 - (c) $a[I][J] = b[I][k] * c[k][J]$
 - (d) $a[I] = a[I] + b[J]$
- [JNTU : Part B, May-05, Marks 8]

Ans. :

- (a) $x = a[I] + 1$

```

MOV I, R0
MOV a(R0), R1
ADD #1, R1
MOV R1, x.

```
 - (b) $a[I] = b[c[I]]$

```

MOV I, R0
MOV c(R0), R1 /* c[I] */
MOV b(R1), R2 /* b[c[I]] */
MOV R2, a(R0)

```
 - (c) $a[I][J] = b[I][k] * c[k][J]$
- The two dimensional array can be represented by one dimensional array.

For instance : Consider elements 10, 20, 30, 40, 50, 60, 70, 80

	1	2	3	4
1	10	20	30	40
2	50	60	70	80

Two dimensional array

1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80

↑
In 2D array $i = 2, j = 3$

In 2D array $i = 2, j = 3$

Hence in row major representation

$$A[i][j] = \text{Base address} + (j * \text{row-size} + i) * \text{Element size}$$

Element size = 4 bytes.

Row-size = 2

Then $A[i][j] = \text{Base address} + (2 * j + i) * 4$.

The code will be

```

MOV J, R0
MUL #2, R0
ADD k, R0
MUL #4, R0
MOV c(R0), R1 /* c[k][J] */
MOV k, R0
MUL #2, R0
ADD I, R0
MUL #4, R0
MUL b(R0), R1 /* R1 = b[I][k] * c[k][J] */
MOV J, R0
MUL #2, R0
ADD I, R0
MUL #4, R0

```

DECODE

MOV R₁, a(R₀)

(d) $a[I] = a[I] + b[J]$

MOV I, R₀

MOV a(R₀), R₁

MOV J, R₀

ADD b(R₀), R₁

MOV I, R₀

MOV a(R₀), R₂

MOV R₁, R₂

Q.65 Generate the code for the following C statements using its equivalent three address code.

a) $a = b + c$

b) $x = a/(b + c) \cdot d \cdot (e + f)$

c) $*A = p$

d) $A = B + C$ [JNTU : April-11, Marks 16]

Ans. : (a) $a = b + c$

MOV b, R0

ADD c, R0

MOV R0, a

(b) $x = a/(b + c) + d \cdot (e + f)$

MOV b, R0

ADD c, R0

ADD d, R0

MOV e, R1

ADD f, R1

MUL R1, R0

MOV a, R1

DIV R1, R0

MOV R0, x

(c) $*A = p$

MOV P, R0

MOV R0, *A

(d) MOV C R0

ADD B, R0

MOV R0, A

Q.66 Generate object code for the following statements $a = b + c$; $d = a + e$;

[JNTU : Part A, Nov.-15, Marks 2]

Ans. : The object code for $a = b + c$ is

MOV b, R0

ADD c, R0

MOV R0, a

The object code for $d = a + e$ is

MOV a, R0

ADD e, R0

MOV R0, d

4.13 : Peephole Optimization

Q.67 Explain various method to handle peephole optimization. [JNTU : Part B, Mar.-17, Marks 10]

Ans. : Definition

Peephole optimization is a simple and effective technique for locally improving target code. This technique is applied to improve the performance of the target program by examining the short sequence of target instructions and replacing these instructions by shorter or faster sequence.

The peephole optimization can be applied on the target code using following characteristic.

1. Redundant instruction elimination

- Especially the redundant loads and stores can be eliminated in this type of transformations.

For example :

MOV R0, x

MOV x, R0

We can eliminate the second instruction since x is already in R0. But if (MOV x, R0) is a label statement then we cannot remove it.

• We can eliminate the unreachable instructions. For example, following is a piece of C code.

```
sum=0
if(sum)
printf("%d",sum);
```

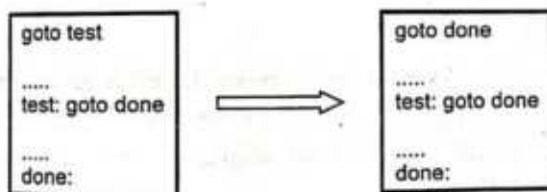
Now this if statement will never get executed hence we can eliminate such a unreachable code. Similarly

```
int fun(int a,int b)
{
c=a+b;
return c;
printf("%d",c); /* unreachable code and hence can be
eliminated */
}
```

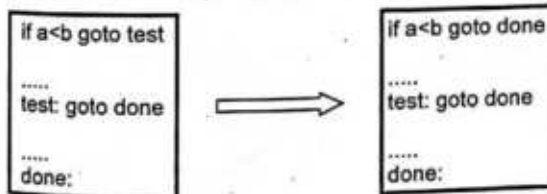
2. Flow of control optimization

Using peephole optimization unnecessary jumps on jumps can be eliminated.

For example,



Thus we reduce one jump by this transformation.



Another example could be

3. Algebraic simplification

Peephole optimization is an effective technique for algebraic simplification.

The statements such as

```
x := x + 0
```

or

```
x := x * 1
```

can be eliminated by peephole optimization.

4. Reduction in strength

Certain machine instructions are cheaper than the other. In order to improve the performance of the

intermediate code we can replace these instructions by equivalent cheaper instruction. For example, x^2 is cheaper than $x * x$. Similarly addition and subtraction is cheaper than multiplication and division. So we can effectively use equivalent addition and subtraction for multiplication and division.

5. Machine Idioms

The target instructions have equivalent machine instructions for performing some operations. Hence we can replace these target instructions by equivalent machine instructions in order to improve the efficiency. For example, some machines have auto-increment or auto-decrement addressing modes that are used to perform add or subtract operations.

Q.68 What is algebraic transformation ?

[JNTU : Part A, Dec.-17, Marks 2]

Ans. : • Algebraic transformations are kind of transformations that can be applied for algebraic expressions. This transformation is basically obtained for code optimization.

• The typical transformations that come under category of algebraic transformation are -

1) Removing of operations like -

- i) Multiplication by 1
- ii) Multiplication by 0
- iii) Addition by 0

2) Reorder instructions based on commutative properties of operators. For example $x + y$ is same $y + x$.

Q.69 Explain the following peephole optimization technique :

- a) Elimination of redundant code.
- b) Elimination of unreachable code.

[JNTU : Part B, Nov.-16, Marks 10]

Ans. : Refer Q.67.

4.14 : Register Allocation and Assignment

Q.70 How to allocate registers to instruction?

[JNTU : Part A, March-17, Marks 3]

OR What is meant register allocation and assignment ?

[JNTU : Part A, Mar.-16, Marks 3]

Ans. : • The most commonly used strategy to register allocation and assignment is to assign specific values to specific registers. For instance for base addresses separate set of registers can be used. Similarly for storing the stack pointers again a separate set of registers can be assigned; arithmetic computations can be done using separate set of registers. Remaining set of registers are used by the compiler for suitable purposes.

- The advantage of this approach is that the design process of compiler for code generation becomes simplified.
- The disadvantage of this method is that the design of compiler becomes complicated because of restrictive use of registers. At the same time certain set of registers remain totally unused over substantial portions of code and some set of registers get overloaded. But this disadvantage can be tolerable by most of the compiler and this approach can be adopted in most of the computing environment.
- Various strategies used in register allocation and assignment
 1. Global register allocation.
 2. Usage count.
 3. Register assignment for outer loop.
 4. Graph coloring for register assignment.

4.15 : Dynamic Programming Method of Code Generation

Q.71 Explain the working principle of dynamic programming of code generation with suitable example [JNTU : Part B, Marks 10]

Ans. : The dynamic programming works on following principle -

Suppose that an expression E is given. Then by dynamic programming algorithm the expression E is broken into subexpressions. The optimal code for these subexpressions is obtained. Then using the given operator of E the subexpressions are evaluated on "contiguously".

To evaluate a tree T the T_1 is evaluated first, then T_2 gets evaluated and then we proceed for root or it will be: evaluation of T_2 then T_1 and finally root.

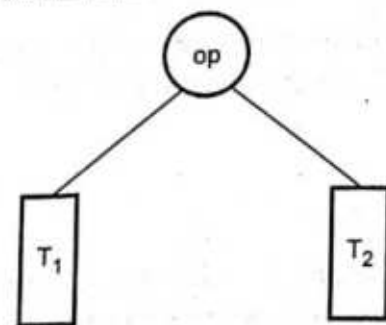


Fig. Q.71.1 Tree T

If T_1 gets evaluated completely then the computation is stored in some register. Thereafter T_2 is evaluated. Thus the order of evaluation is in contiguous fashion.

Example : By considering following instruction set each of unit cost and two registers R_0 and R_1 are available; obtain the cost vector for the expression.

$(a + b) * (c / d)$

$R_1 := M$

$R_1 := R_1 \text{ op } R_j$

$R_1 := R_1 \text{ op } M$

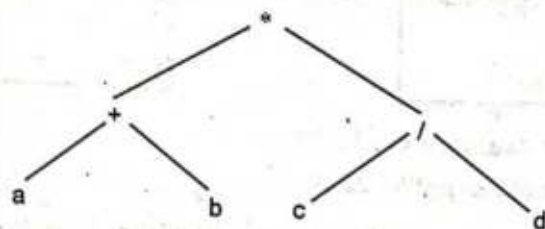
$R_1 := R_j$

$M := R_1$

Sol. : Here M refer to memory location and R_1 and R_j refers to R_0 or R_1 . For the expression $(a + b) * (c / d)$ first we will construct an expression tree or syntax tree as follows.

Now we will compute cost vectors in bottom-up fashion.

For node labelled as 'a'



$c[0] = 0$ This is the cost of computing a in memory. As a is already in memory location the cost turns out to be 0.

$c[1] = 1$ This is the cost of computing a in register. As we can load a into register i.e. $R_0 := a$ hence the cost is 1.

$c[2] = 1$ The cost of computing a into two available registers. We can utilize only one, once the cost of

two registers available will be same as cost of one register available.

Hence the cost vector of leaf node a is (0, 1, 1). In the same manner cost vector for node b is (0, 1, 1).

We will now compute the cost vector for right subtree (Note : The computation of cost vector is to be done in postorder manner).

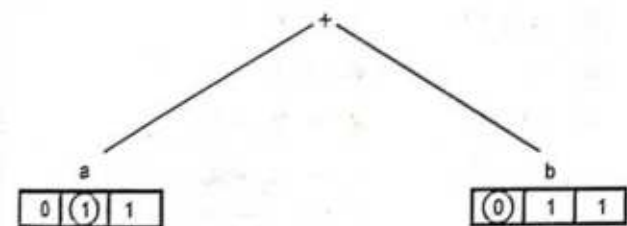
The cost vector for c is (0, 1, 1).

The cost vector for d is (0, 1, 1).

Consider node +

$c[0] = 3$ This is the cost of computing + into memory.

We will match $R_0 := R_0 + M$. The cost of one register available from left subtree and cost of computing right subtree from memory location.



As the node is operator we can add 1 to the total cost. Hence

$R_0 = 1 + 0 + 1 = 2$. But this is the cost computing available in

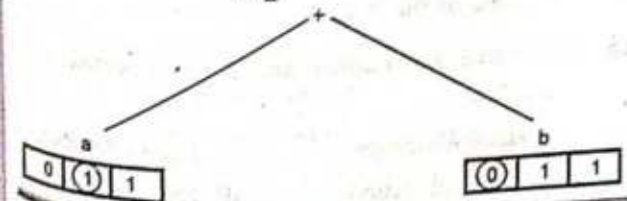
$R_0 = [as R_0 := R_0 + M]$ We can store R_0 at memory because we are computing $c[0]$ i.e. for memory location. Storing R_0 to memory again costs 1.

$\therefore$ Total cost $+ 1 = 2 + 1 = 3$

Hence $c[0] = 3$.

$c[1] = 2$ The cost of computing + in one register available.

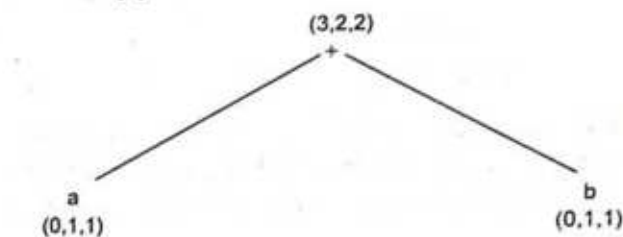
We will match $R_0 := R_0 + M$
plus 1 for cost of instruction
 $= 1 + 0 + 1$
 $= 2$



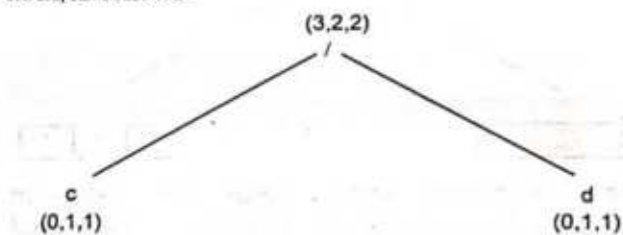
$c[2] = 2$ The cost of computing + in two registers available.

For this computation we can match it with two instructions $R_0 := R_0 + R_1$ and $R_0 := R_0 + M$. If $R_0 := R_0 + R_1$ is matched then we get $1 + 1 + 1 = 3$. But as both the left and right nodes are operands we can keep one operand in memory and one in register and there by obtain the optimum cost. Hence we will match $R_0 := R_0 + M$ even if cost is computed for two register available.

Hence, $c[2] = 1 + 0 + 1 = 2$.



Continuing in this fashion we can obtain the cost for node/as follows.

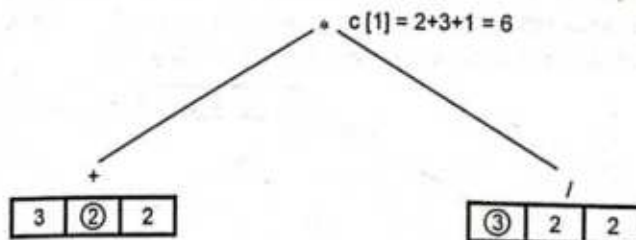


For the root node *

We will first determine the cost of computing root with one register available.

$c[1] = 6$

As we can match $R_0 := R_0 * M$. That means minimum cost of evaluating left subtree into one register, plus minimum cost of evaluating right subtree into memory, plus 1 for cost of instruction.

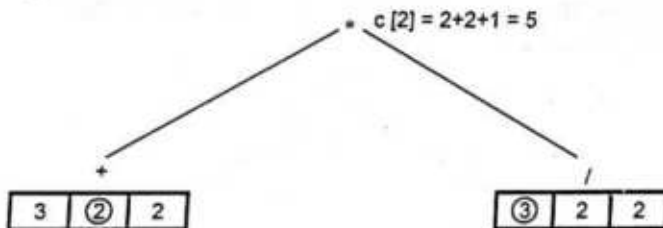


Now let us compute the minimum cost for * node with two registers available. There are 3 cases to compute the root node. These cases are also useful to decide the order of evaluation of subtrees.

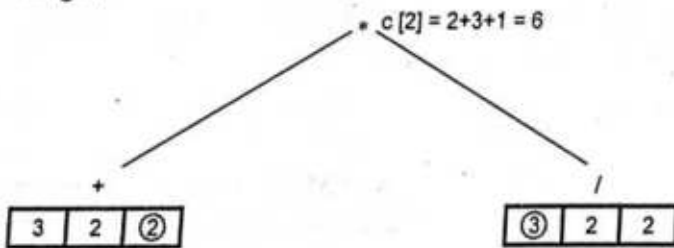
Case 1 : Compute using instruction $R_0 := R_0 + R_1$. The left subtree can be computed with two registers available and right subtree can be computed with one register. Therefore cost can be

$$\begin{aligned} c[2] &= \text{Cost of left subtree with 2 register} \\ &+ \text{Cost of right subtree with 1 register} \\ &+ 1 \text{ for instruction} \\ &= 2 + 2 + 1 = 5 \end{aligned}$$

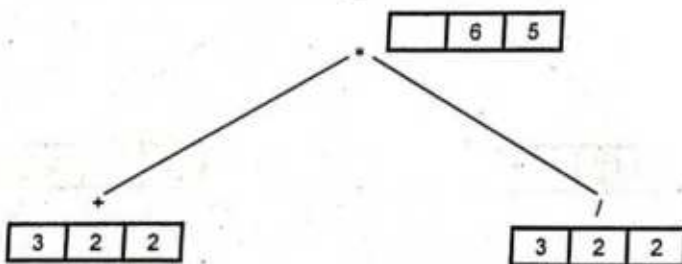
Case 2 : Compute using match for instruction $R_0 := R_0 + R_1$. By considering left subtree with one register in R_0 and right subtree with two registers in R_1 we get,



Case 3 : Compute using match for instruction $R_0 := R_0 + M$. Assume right subtree in memory and left subtree with two registers plus one for instruction we get,

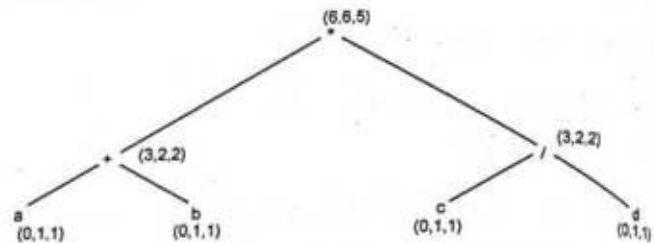


From above three cases, case 1 and case 2 gives optimum cost for $c[2]$. That means we can consider the evaluation order as mentioned in case 1 or from case 2. And cost of two registers available is 5.



Let us compute $c[0]$.

The optimum cost with all registers available is 5. [As we have calculated in case 1 or case2]. We want $c[0]$ means minimum cost of computing root into memory. We will then use $M = R_1$. Hence to store the value into memory location from register we will add 1 to 5. Hence $c[0] = 6$. Finally the syntax tree with cost vector defined will be,



By traversing this tree we can obtain the code as,

$R_0 := a$ or	$R_0 := c$
$R_0 := R_0 + b$	$R_0 := R_0 / d$
$R_1 := c$	$R_1 := a$
$R_1 := R_1 / d$	$R_1 := R_1 + b$
$R_1 := R_1 * R_0$	$R_0 := R_1 + R_0$

Multiple Choice Questions for Mid Term Exam

- Q.1** Compiler places target code at ____.
- ☒ a lower end of memory
 - ☐ b upper end of memory
 - ☐ c anywhere in the memory depending upon the code
 - ☐ d none of the above
- Q.2** Control stack in run time environment is used to manage ____.
- ☐ a data object
 - ☒ b active procedures
 - ☐ c target code
 - ☐ d none of the above
- Q.3** Recursive procedures are not supported by ____.
- ☐ a stack allocation
 - ☐ b heap allocation
 - ☒ c static allocation
 - ☐ d code area

5

Machine Independent Optimization

5.1 : Principal Sources of Optimization

Q.1 What is machine independent code Optimization? [JNTU : Part A, Dec.-17, Marks 3]

Ans. : The machine independent optimization is based on the characteristics of the programming languages for appropriate programming structure and usage of efficient arithmetic properties in order to reduce the execution time.

Q.2 What are criteria used for achieving machine independent optimization ?

[JNTU : Part A, Marks 4]

Ans. : The machine independent optimization can be achieved using following criteria -

1. The code should be analyzed completely and use alternative equivalent sequence of source code that will produce a minimum amount of target code.
2. Use appropriate program structure in order to improve the efficiency of target code.
3. From the source program eliminate the unreachable code.
4. Move two or more identical computations at one place and make use of the result instead of each time computing the expressions.

Q.3 What are the advantages of code optimization ? [JNTU : Part A, Marks 2]

Ans. :

1. The intermediate code can be optimized to produce efficient object code.
2. The code optimization allows faster execution of source program by making minor modifications.

Q.4 Explain various code optimization techniques.

OR Explain any three methods for code optimization

Ans. : Various code optimization techniques are as follows :

1) Common Sub-expression Elimination :

The common sub expression is an expression appearing repeatedly in the program which is computed previously. Then if the operands of this sub expression do not get changed at all then result of such sub expression is used instead of recomputing it each time.

For example :

$t_1 := 4 * i$

$t_2 := a[t_1]$

$t_3 := 4 * j$

$t_4 := 4 * i$

$t_5 := n;$

$t_6 := b[t_4] + t_5$

The above code can be optimized using the common sub expression elimination as,

$t_1 := 4 * i$

$t_2 := a[t_1]$

$t_3 := 4 * j$

$t_5 := n;$

$t_6 := b[t_1] + t_5$

The common sub expression $t_4 := 4 * i$ is eliminated as its computation is already in t_1 and value of i is not been changed from definition to use.

2) Variable Propagation :

Variable propagation means use of one variable instead of another.

For example :

```
x = pi;
...
area = x * r * r;
```

The optimization using variable propagation can be done as follows,

```
area = pi * r * r;
```

Here the variable x is eliminated. Here the necessary condition is that a variable must be assigned to another variable or some constant.

3) Code Motion :

There are two basic goals of code movement -

1. To reduce the size of the code i.e. to obtain the space complexity.
2. To reduce the frequency of execution of code i.e. to obtain the time complexity.

For example :

```
for(i=0;i<=10;i++)
{
  x=y*5;
  ...
  k=(y*5) + 50;
}
```

This can be optimized as

```
temp = y*z;
for(i=0;i<=10;i++)
{
  x=z;
  ...
  k=z + 50;
}
```

The code for $y * 5$ will be generated only once. And simply the result of that computation is used wherever necessary.

4) Strength Reduction :

The strength of certain operators is higher than others. For instance strength of $*$ is higher than $+$. In strength reduction technique the higher strength operators can be replaced by lower strength operators.

For example :

```
for(i=1;i<=50;i++)
{
  ...
  count = i * 7;
  ...
}
```

Here we get the count values as 7, 14, 21 and so on upto less than 50.

This code can be replaced by using strength reduction as follows -

```
temp=7
for(i=1;i<=50;i++)
{
  ...
  count = temp;
  temp = temp+7;
}
```

Thus we get the value of count as 7, 14, 21 and so on upto less than 50.

The induction variable is integer scalar identifier used in the form of

$v = v \pm \text{constant}$, Here v is a induction variable.

5) Dead Code Elimination :

A variable is said to be live in a program if the value contained into it is used subsequently. On the other hand, the variable is said to be dead at a point in a program if the value contained into it is never been used. The code containing such a variable supposed to be a dead code. And an optimization can be performed by eliminating such a dead code.

For example :

```
i=j;
...
x=i+10;
...
```

The optimization can be performed by eliminating the assignment statement $i = j$. This assignment statement is called dead assignment.

Q.5 What is dead code elimination and reduction in strength? [JNTU : Part A, March-17, Marks 2]

Ans. : Refer Q.4.

Q.6 Explain principle sources of optimization in detail [JNTU : Part B, March-16, May-18, Marks 10]

OR Discuss various techniques of function preserving transformations for code optimization.

[JNTU : Part B, Nov.-16, Marks 10]

Ans. : Refer Q.4.

Q.7 Explain three loop optimization technique with example. [JNTU : Part B, Marks 6]

Ans. :

1. Loop invariant method

In this optimization technique the computation inside the loop is avoided and thereby the computation overhead on compiler is avoided. This ultimately optimizes code generation.

For example :

```
for i:=0 to 10 do begin
  k:=i+a/b;
  ...
  ...
end;
```

can be written as

```
t:=a/b;
for i:=0 to 10 do begin
  k:=i+t;
  ...
  ...
end;
```

end;

2. Loop unrolling

In this method the number of jumps and tests can be reduced by writing the code two times.

For example :

```
int i=1;
while(i<=100)
{
  a[i]=b[i];
  i++;
}
```

Can be written as

```
int i=1;
while(i<=100)
{
  a[i]=b[i];
  i++;
  a[i]=b[i];
  i++;
}
```

3. Loop fusion

In loop fusion method several loops are merged to one loop

For example :

```
for i:=1 to n do
for j:=1 to m do
  a[i,j]:=10
```

Can be written as

```
for i:=1 to n
  *m do
  a[i]:=10
```

Q.8 Define loop optimization? Describe in brief about the transformation on basic blocks.

[JNTU : Part B, March-17, Nov.-17, Marks 10]

Ans. : **Loop optimization** : The code optimization can be significantly done in loops of the program. Specially inner loop is a place where program spends large amount of time. Hence if number of instructions are less in inner loop then the running time of the program will get decreased to a large extent. Hence loop optimization is a technique in which code optimization performed on inner loops.

Q.9 Define loop unrolling. Give an example

[JNTU : Part A, March-17, Marks 3]

Ans. : Refer Q.7.

Q.10 Construct basic blocks and data flow graph and identify loop invariant statements :

```

{
  j = 1
  while (j ≤ n)
  {
    A = B * C/D
    j = j + 1;
  }
}

```

[JNTU : Part B, Marks 5]

Ans. :

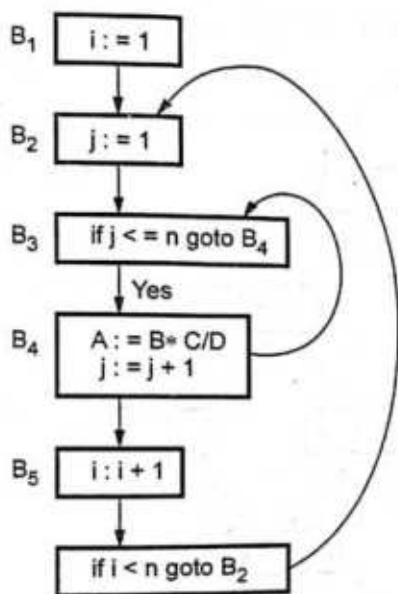


Fig. Q.10.1

The loop invariant statement is

while (j <= n)

{

A = B * C/D

...

...

Optimized
as

t := C/D

while (j <= n)

{

A = B * t

...

...

Q.11 Explain the following with an example :

a) Redundant sub expression elimination

b) Frequency reduction

c) Copy Propagation

[JNTU : Part B, Nov.-15, (3+3+4) Marks]

Ans. : Refer Q.4.

5.2 : Introduction to Data Flow Analysis

Q.12 Write about Data flow analysis of Structural programs [JNTU : Part B, May-18, Marks 10]

Ans. : • In data flow analysis the analysis is made on the data flow. That is the data flow analysis determines the information regarding the definition and use of the data in the program. Using this kind of analysis optimization can be done.

• The data flow analysis is basically a process in which the values are computed using data flow properties.

• Basic Terminologies Used in Data Flow Analysis :

- A program point containing the definition is called **definition point**.
- A program point at which a reference to a data item is made is called **reference point**.
- A program point at which some evaluating expression is given is called **evaluation point**.

For example :

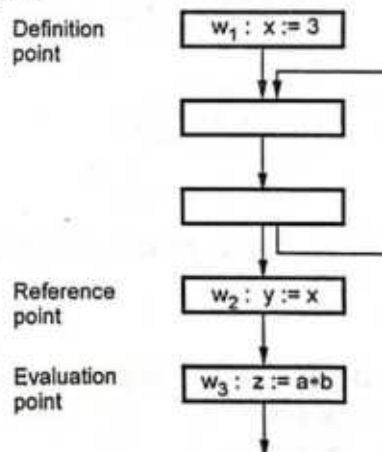


Fig. Q.12.1 (a) Program points

1. Available expression

An expression $x+y$ is available at a program point w if and only if along all paths are reaching to w .

- The expression $x+y$ is said to be available at its evaluation point.
- The expression $x+y$ is said to be available if no definition of any operand of expression (here either x or y) follows its last evaluation along the path. In other word, if neither of the two operands get modified before their use.

For example :

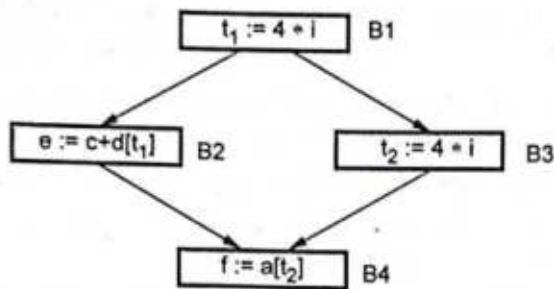


Fig. Q.12.1 (b)

The expression $4 * i$ is the available expression for B_2 , B_3 and B_4 because this expression is not been changed by any of the block before appearing in B_4 .

Advantage of available expression

- The use of available expression is to eliminate common sub expressions.

2. Reaching definitions

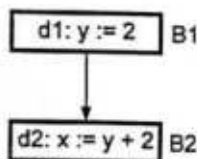


Fig. Q.12.2 (a)

A definition D reaches at point p if there is a path from D to p if there is a path from D to p along which D is not killed.

A definition D of variable x is killed when there is a redefinition of x .

For example :

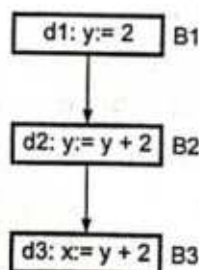


Fig. Q.12.2 (b)

The definition $d1$ is said to a **reaching definition** for block $B2$. But the definition $d1$ is not a reaching definition in block $B3$, because it is killed by definition $d2$ in block $B2$.

Advantage of reaching definition

- Reaching definitions are used in constant and variable propagation.

3. Live variable

A variable x is live at some point p if there is a path from p to the exit, along which the value of x is used before it is redefined. Otherwise the variable is said to be dead at that point.

For example :

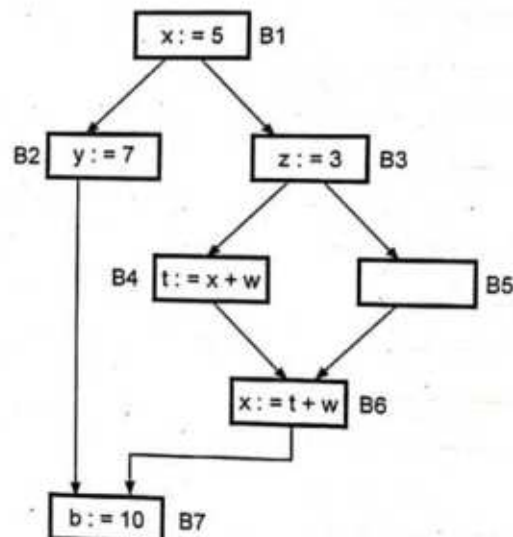


Fig. Q.12.3 Live variables

Advantages of live variables

- Live variables are useful in register allocation.
- Live variables are useful for dead code elimination.

4. Busy expression

An expression e is said to be a busy expression along some path $p_1 \dots p_j$ if and only if an evaluation of e exists along some path $p_1 \dots p_j$ and no definition of any operand exists before its evaluation along the path.

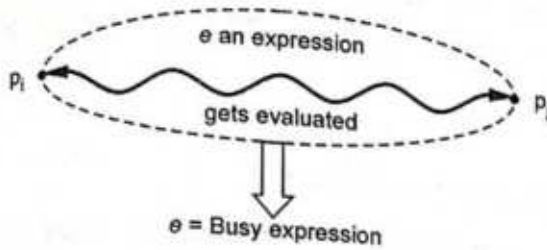


Fig. Q.12.4

Advantage of busy expression

- Busy expressions are useful in performing code movement optimization.

Q.13 What is liveness? Explain liveness with suitable example.

[JNTU : Part B, March-17, Marks 5]

Ans. : Refer Q.12 (3).

5.3 : Foundation Data Flow Analysis

Q.14 Explain how data flow equations are set up and solved for improving code.

[JNTU : Part B, Nov.-16, Marks 10, Part A, Marks 2]

Ans. : **Code Optimization** : Code optimization is a process of producing efficient target code by applying function preserving transformations on the set of code.

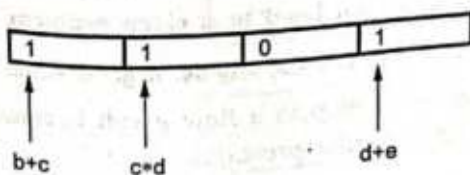
Data Flow Equation :

The data flow information can be represented by a either a set of properties or using bit vector with each bit representing a property.

For example :

$a := b+c$
 $d := c*d$
 $e := a-c$
 $f := d+e$

The set of property for theses statements can be,

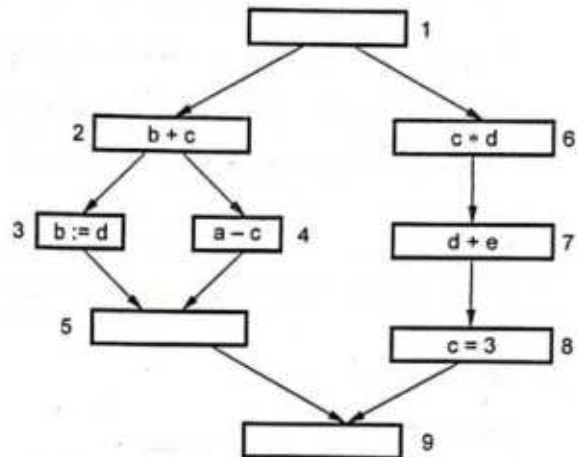


$(b+c, c \times d, d+e)$

The bit vector can be represented as

The bit vector representation is more compact form of representing the data flow information.

Consider the flow graph for the expressions $(b+c, c*d, a-c, d+e)$



Node	At entry	At exit
2	0000	$\{b+c\}=1000$
3	$\{b+c\}=1000$	0000
4	$\{b+c\}=1000$	$\{a-c\}=0010$
5	$\{a-c\}=0010$	$\{a-c\}=0010$
6	0000	$\{a-c\}=0010$
7	$\{c*d\}=0100$	$\{c*d\}=0101$ and $\{d+e\} = 0001$ i.e. $0101 \&\& 0001$
8	0101	$\{d+e\}=0001$
9	$\{a-c\}$ and $\{d+e\}=0011$	0011

5.4 : Constant Propagation

Q.15 Explain how copy propagation can be done using data flow equation ?

[JNTU : Part B, March-16, Marks 5, Nov.-17, Marks 10]

Ans. : The assignment in the form $a:=b$ is called copy statement. The idea behind the copy propagation transformation is to use b for a wherever possible after copy statement $a := b$. Let us see the algorithm for copy propagation.

Algorithm : Copy propagation

Input : A flow graph containing used-definition i.e. ud chains reaching to block B.

The flow graph should also consists of a set of copies $x := y$ that reach block B along every path and there should not be any change to either x or y along that path. We also need du chains (i.e. definition and used chain) so that use of every definition can be obtained.

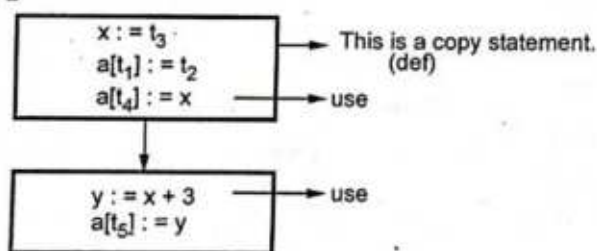
Output - A graph after applying copy propagation transformation.

Method - For each statement $x := y$ which is denoted by s perform following steps

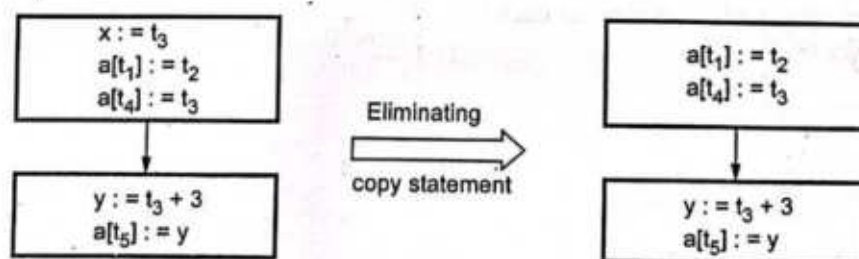
- 1) Determine the statements in which x is used. These statements should be reachable from definition of x .
- 2) There should not be any definition of x or y occur prior to use of x in that block containing s .
- 3) If s satisfies the condition mentioned in step (2) then remove s and replace all uses of x found in (1) by y .

For example :

Step 1 and 2



Since value of t_3 and x is not altered along the path from its definition we will replace x by t_3 and then eliminate the copy statement.



5.5 : Partial Redundancy Elimination

Q.16 Explain how redundant sub expression eliminates can be done at global level in a given problem.

[JNTU : Part B, April-03, May-04, Aug.-08, Marks 16]

Ans. : The *available expressions* allow us to determine if an expression at point p in a flow graph is common subexpression. Using following algorithm we can eliminate common subexpressions.

Algorithm - Algorithm for global common expression elimination.

Input - A flow graph with available expression.

Output - A flow graph after eliminating common subexpression.

Method - For every statement s of the form $a := b + c$ such that $b + c$ is available expression (i.e. $b + c$ is available at the beginning of block containing s and neither b nor c is defined prior to statement s) then perform following steps -

1. Discover the evaluations of $b + c$ that reach in the block containing statement s .
2. Create a new variable m .
3. Replace each statement $d := b + c$ which is obtained in step 1. by

$m := b + c$

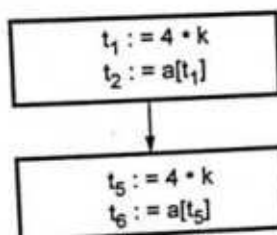
$d := m$

4. Replace statement s by $a := m$.

Let us apply this algorithm and perform global common subexpression elimination.

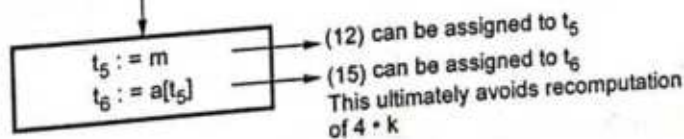
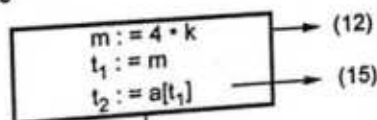
Example : Consider a flow graph as given below.

Step 1



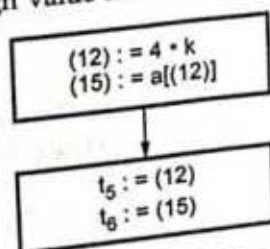
We will discover evaluation of $4 * k$, the $4 * k$ is available expression

Step 2 and 3



Step 4

Now if we assign value numbers to common subexpression then,



DECODE

5.6 : Loops in Flow Graphs

Q.17 Enlist the terminologies used for loops in flow graph [JNTU : Part A, Marks 2]

Ans. : Various terminologies used for loops in flow graph are -

1. Dominators
2. Natural Loops
3. Inner Loops
4. Pre-header
5. Reducible Flow graphs
6. Non Reducible flow graph

Q.18 Explain the term - Dominator.

[JNTU : Part A, Marks 2]

Ans. : In a flow graph, a node d dominates n if every path to node n from initial node goes through d only. This can be denoted as ' $d \text{ dom } n$ '. Every initial node dominates all the remaining nodes in the flow graph. Similarly every node dominates itself.

For example :

In flow graph as in Fig. Q.18.1,

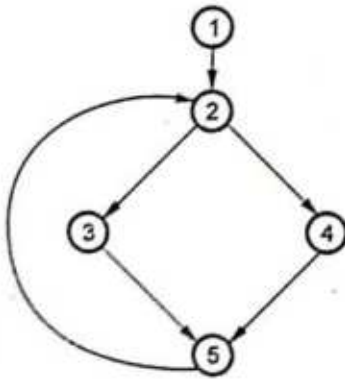


Fig. Q.18.1 Flow graph

Node 1 is initial node and it dominates every node as it is a initial node.

Node 2 dominates 3, 4 and 5 as there exists only one path from initial node to node 2 which is going through 2 (it is 1-2-3). Similarly for node 4 the only path exists is 1-2-4 and for node 5 the only path existing is 1-2-4-5 or 1-2-3-5 which is going through node 2.

Node 3 dominates itself similarly node 4 dominates itself. The reason is that for going to remaining node 5 we have another path 1-4-5 which is not through 3 (hence 3 dominates itself). Similarly for going to node 5 there is another path 1-3-5 which is not through 4 (hence 4 dominates itself).

Node 5 dominates no node.

Q.19 Explain the term - natural loops

[JNTU : Part A, Marks 2]

Ans. : Loop in a flow graph can be denoted by $n \rightarrow d$ such that $d \text{ dom } n$. These edges are called back edges and for a loop there can be more than one back edges. If there is $p \rightarrow q$ then q is a head and p is a tail. And head dominates tail.

For example :

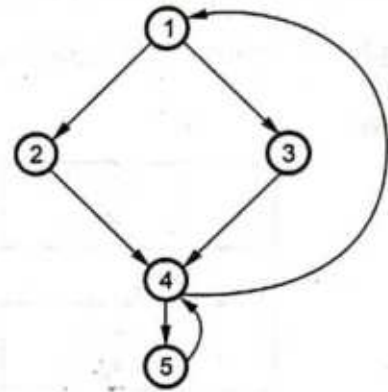


Fig. Q.19.1 Flow graph with loops

The loops in the above graph can be denoted by $4 \rightarrow 1$ i.e. $1 \text{ dom } 4$. Similarly $5 \rightarrow 4$ i.e. $4 \text{ dom } 5$.

The natural loop can be defined by a back edge $n \rightarrow d$ such that there exists a collection of all the nodes that can reach to n without going through d and at the same time d also can be added to this collection.

For example :

$6 \rightarrow 1$ is a natural loop because we can reach to all the remaining nodes from 6.

By observing all the predecessors of node 6 we can obtain a natural loop. Hence 2-3-4-5-6-1 is a collection in natural loop.

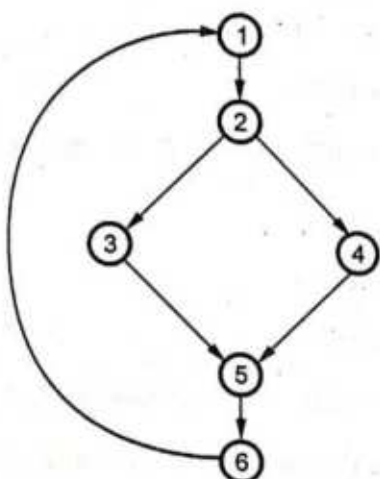


Fig. Q.19.2 Flow graph of natural loop

Q.20 Explain the term - Inner loops.

[JNTU : Part A, Marks 2]

Ans.: The inner loop is a loop that contains no other loop.

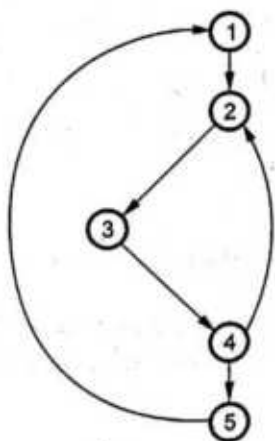


Fig. Q.20.1 Flow graph for Inner loop

Here the inner loop is $4 \rightarrow 2$ that means edge given by 2-3-4.

Q.21 Explain the term - Pre-header

[JNTU : Part A, Marks 2]

Ans.: The pre-header is a new block created such that successor of this block is the header block. All the computations that can be made before the header block can be made before the pre-header block.

The pre-header can be as shown in above Fig. Q.21.1.

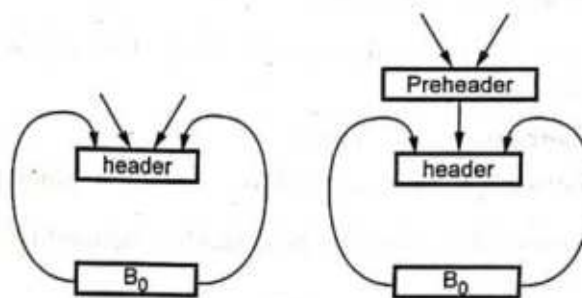


Fig. Q.21.1 Pre-header

Q.22 Explain reducible and non reducible flow graph with examples.

[JNTU : Part B, Nov.-17, Marks 5]

Ans.: Reducible flow graphs

The reducible graph is a flow graph in which there are two types of edges forward edges and backward edges. These edges have following properties,

- The forward edge form an acyclic graph.
- The back edges are such edges whose head dominates their tail.

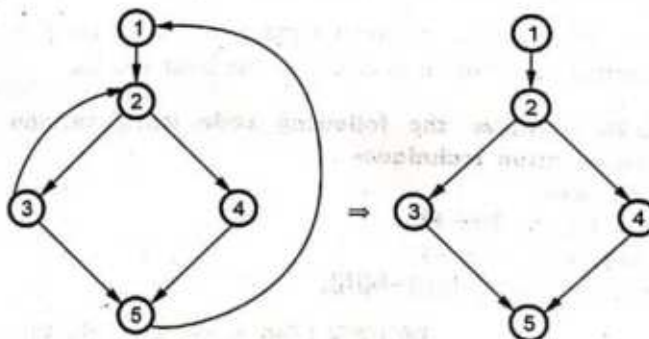


Fig. Q.22.1 Flow graph

The above flow graph is reducible. We can reduce this graph by removing the back edge from 3 to 2 edge. Similarly by removing the backedge from 5 to 1 we can reduce the above flow graph. And the resultant graph is a cyclic graph.

The program structure in which there is exclusive use of if-then, while-do or goto statements generates a flow graph which is always reducible.

Compiler Design

Non-reducible flow Graph

A non reducible flow graph is a flow graph in which -

- There are no back edges
- Forward edges may produce cycle in the graph.

For example - Following flow graph is non-reducible.

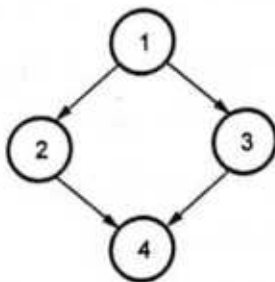


Fig. Q.22.2 Non-reducible Graph

Q.23 Which graph is used for identifying common sub expression in an expression?

[JNTU : Part A, March-16, Marks 2]

Ans.: The flow graph is used for identifying common sub expression in an expression.

A flow graph is a directed graph in which the flow control information is added to the basic blocks.

Q.24 Optimize the following code using various optimization techniques :

```

i=1; s=0;
for(i=1;i<=3;i++)
for(j=1;j<=3;j++)
c[i][j]=c[i][j]+a[i][j]+b[i][j]
  
```

[JNTU : Part B, Nov.-15, Marks 10]

Ans.: In the given code, there exists a two dimensional arrays. Hence we assume array of order 10×20 . That is $n1 = 10$ and $n2 = 20$.

Before optimizing this code we need to generate 3 address code for corresponding code fragment.

```

i := 1
s := 0
i := 1
L2: j := 1
L1: t1 = 4 * i
  
```

```

t1 := t1 + j
t2 := t1 * 4
t3 := a[t2]      /* t3 = a[i][j] */
t4 := i * 20
t4 := t4 + j
t5 := t4 * 4
t6 := b[t5]      /* t6 = b[i][j] */
t7 := t3 + t6    /* t7 = a[i][j] + b[i][j] */
t8 := i * 20
t8 := t8 + j
t9 := t8 * 4
t10 := c[t9]     /* t10 = c[i][j] */
t11 := t7 + t10
c[i][j] := t11
j := j + 1
if j <= 3 goto L1
i := i + 1
if i <= 3 goto L2
  
```

Now using common subexpression elimination we can optimize above code. After optimization

we will get -

```

s := 0
i := 1
L2: j := 1
L1: t1 := 4 * i
t1 := t1 + j
t2 := t1 * 4
t3 := a[t2]
t4 := b[t2]
t5 := t3 + t4
  
```

```

t6 := c[t2]
t7 := t5 + t6
c[i][j] := t7
j := j + 1
if j <= 3 goto L1
i := i + 1
if i <= 3 goto L2
    
```

Q.25 Consider the following flow graph and find
a) Dominators for each basic block.
b) Detect all the loops in the graph. For each loop find the back edge and header block information.

[JNTU : Part B, Marks 5]

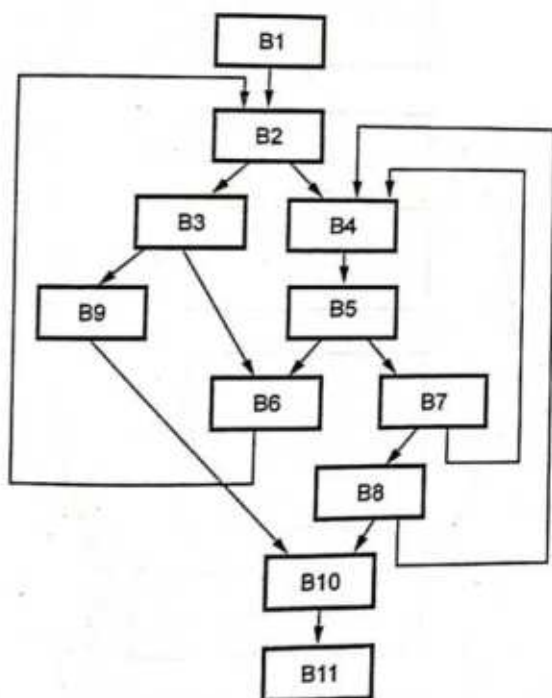


Fig. Q.25.1

Ans. : In a flow graph node 'd' dominates n if every path to node 'n' from initial node goes through 'd' node (Block) only.

Every node (Block) dominates itself.

B1 = As it is an initial node it dominates all the nodes.

B2 = B3, B4, B5, B6, B7, B8, B9, B10, B11.

B3 = B9

B4 = B5, B7, B8

B5 = B7, B8

B6 = Dominates itself because there exist another path also to go to other remaining nodes.

B7 = B8

B8 = Dominates itself

B9 = Dominates itself

B10 = Dominates node B11 because to reach to B11 we have to go through B10.

B11 = Dominates no node.

Loop 1

Back edge : B6 - B2

Header : B2

Loop 2

Back edge : B7 - B4

Header : B4

Loop 3

Back edge : B8 - B4

Header : B4

Q.26 Consider the following program which counts the primes from 2 to n using the Sieve method on a suitably large array

```

begin
  read n
  for i := 2 to n do
    a[i] := true /* Initialize */
  count := 0;
  for i := 2 to n ** .5 do
    if a[i] then /* i is a prime */
      begin
        count := count + 1
        for j := 2 * i to n by i do
          a[j] := false
          /* j is divisible by i */
        end ;
      end ;
    end ;
  end ;
    
```


print count

end (a) Translate the above program into three address statements assuming a 1s al-located static storage.

(b) Construct a flow graph from the three address statements and indicate the back edges and their natural loops

[JNTU : Part B, Jan.-10, Marks 16]

Ans. : (a) $i := 2$

$L_1: t_1 := i * 4$

$t_2 := \text{addr}_a$

$t_3 := t_2[t_1]$

$i := i + 1$

if $i < n$ goto L_1

count := 0

$i := 2$

$L_4: t_4 := i * 4$

$t_5 := \text{addr}_a$

$t_6 := t_5[t_4]$

: if $t_6 = 1$ then goto L_2

goto L_5

$L_2: \text{count} := \text{count} + 1$

$j := 2 * i$

$L_3: t_7 := 4 * j$

$t_8 := \text{addr}_a$

$t_9 := t_8[t_7]$

$t_9 := 0$

$j = j + 1$

if $j < n/i$ goto L_3

$i := i + 1$

if $i < n ** 0.5$ then goto L_4

$L_5: \text{print count}$

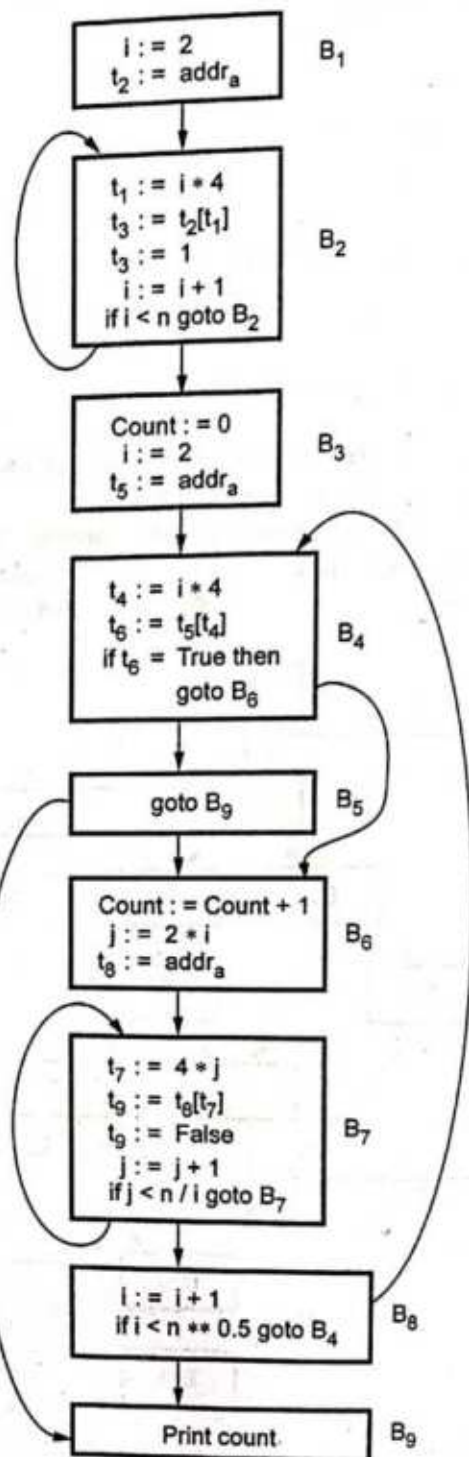


Fig. Q.26.1

Q.27 Consider the following CFG as shown in Fig. Q.27.1.

- Draw the dominator tree of the CFG.
- Identify the natural loops.

[JNTU : Part B, May-12, Marks 15]

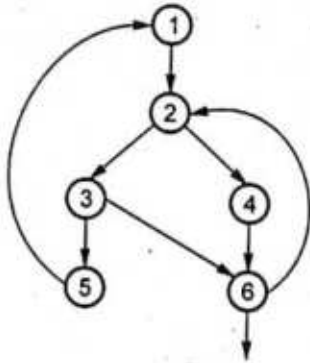
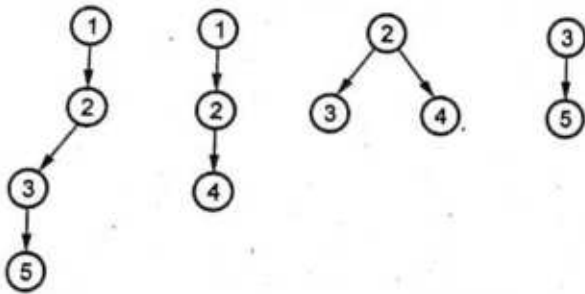
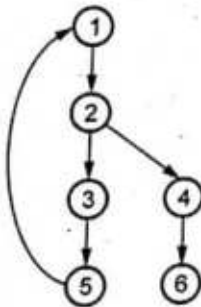


Fig. Q.27.1

Ans. : (a)



(b) Natural loop is 2-3-5-1 because we can reach to all the remaining nodes from 5.



Multiple Choice Questions for Mid Term Exam

Q1 Following techniques are based on loop optimization _____.

- ☐ a common sub-expression elimination
- ☐ b code motion
- ☐ c copy propagation
- ☐ d dead code elimination.

Q.2 Reduction in strength means _____.

- ☐ a removing loop invariant computations
- ☐ b replacing run time computation by compile time computation
- ☐ c removing common sub-expression elimination
- ☒ d replacing costly operation by cheaper one.

Q.3 Basic block can be analyzed by _____.

- ☒ a DAG
- ☐ b flow graph
- ☐ c graph that involves cycle
- ☐ d none of these.

Q.4 The loops can be identified using _____.

- ☐ a dominators
- ☐ b reducible graphs
- ☐ c depth first ordering
- ☐ d all of the above.

Q.5 Peephole optimization is _____.

- ☐ a applied to a small part of the code
- ☐ b can be applied to the portion of the code that is not contiguous
- ☐ c it can be applied repeatedly to get the optimized code
- ☒ d all of the above.

Q.6 Data flow equations can be computed using _____.

- ☐ a available expression
- ☐ b reaching definition
- ☐ c live variable analysis
- ☒ d all of the above.

Fill in the Blanks for Mid Term Exam

- Q.1 DAG stands for Directed Acyclic graph.
- Q.2 Replacing the multiplication by addition operation means _____

NOVEMBER - 2015

Compiler Design [R13]

Solved Paper
III - I (B.Tech)
(Computer Science and Engineering)

Time : 3 Hours]

[Maximum Marks : 75

Note : This question paper contains two parts A and B.
Part A is compulsory which carries 25 marks. Answer all questions in Part A. Part B consists of 5 units . Answer any one full question from each unit. Each question carries 10 marks and may have a, b, c as sub questions.

Part - A (25 Marks)

- Q.1** a) Define cross compiler. (Refer Q.10 of Chapter - 1) [2]
b) Eliminate immediate left recursion for the following grammar : [3]
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$ (Refer Q.15 of Chapter - 2)
c) List the rules for computing FOLLOW SET. (Refer Q.24 of Chapter - 2) [2]
d) Define CLOSURE (I). (Refer Q.42 of Chapter - 2) [3]
e) What is a symbol table ? (Not in New Syllabus) [2]
f) What does a semantic analysis do ? (Refer Q.1 of Chapter - 3) [3]
g) Define basic block in a flow graph. (Refer Q.36 of Chapter - 4) [2]
h) What is a DAG ? Mention its applications. (Refer Q.43 of Chapter - 4) [3]
i) Generate a object code for following statements $a = b + c$; $d = a + e$ (Refer Q.66 of Chapter - 4) [2]
j) Mention the properties that a code generator should possess. (Refer Q.30 of Chapter - 4) [3]

Part - B (50 Marks)

OR

- Q.2** What are the various phases of the compiler ? Explain each phase in detail. (Refer Q.5 of Chapter - 1) [10]

OR

- Q.3** Construct the predictive parser for the following grammar : $S \rightarrow (L) \mid a$ $L \rightarrow L, S \mid S$. (Refer Q.27 of Chapter - 2) [10]

- Q.4** Find the SLR parsing table for the given grammar : [10]
 $E \rightarrow E + E \mid E * E \mid (E) \mid id$ And parse the sentence $(a+b)*c$. (Refer Q.62 of Chapter - 2)

OR

- Q.5** Construct an LALR Parsing table for the following grammar [10]
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$ (Refer Q.58 of Chapter - 2)

Q.6 Generate intermediate code for the following code segment along with the required syntax directed translation scheme :

if ($a > b$)

$x = a + b$

else

$x = a - b$

Where a and x are of real and b of int type data. (Refer Q.50 of Chapter - 3)

[10]

OR

Q.7 Give syntax directed translation scheme for simple desk calculator. (Refer Q.11 of Chapter - 3)

[10]

Q.8 Explain the following with an example :

a) Redundant sub expression elimination

b) Frequency reduction

c) Copy propagation. (Refer Q.11 of Chapter - 5)

[3+3+4]

OR

Q.9 Optimize the following code using various optimization techniques :

$i = 1 ; s = 0 ;$

for ($i = 1 ; i \leq 3 ; i++$)

for ($j = 1 ; j \leq 3 ; j++$)

$c[i][j] = c[i][j] + a[i][j] + b[i][j]$ (Refer Q.24 of Chapter - 5)

[10]

Q.10 Explain in detail about machine dependent code optimization techniques. (Not in new syllabus)

[10]

OR

Q.11 Give an example to show how DAG is used for register allocation. (Refer Q.51 of Chapter - 4)

[10]

MARCH - 2016

Compiler Design [R13]

Solved Paper

III - I (B.Tech)

(Computer Science and Engineering)

Time : 3 Hours]

[Maximum Marks : 75]

Part - A (25 Marks)

- Q.1** a) Define bootstrapping. (Refer Q.8 of Chapter - 1) [2]
 b) What is context-free grammar ? (Refer Q.3 of Chapter - 2) [3]
 c) What are the actions performed by shift reduce parser ? (Refer Q.33 of Chapter - 2) [2]
 d) Describe in brief about types of LR parsers ? (Refer Q.40 of Chapter - 2) [3]
 e) What is type expression ? (Refer Q.40 of Chapter - 3) [2]
 f) Define type equivalence. (Refer Q.41 of Chapter - 3) [3]
 g) Define basic block ? (Refer Q.36 of Chapter - 4) [2]
 h) How can you identify the leader in a basic block ? (Refer Q.38 of Chapter - 4) [3]

- i) Which graph is used for identifying the common sub expression in an expression ? (Refer Q.23 of Chapter - 5) [2]
- j) What is meant register allocation and assignment ? (Refer Q.70 of Chapter - 4) [3]

Part-B (50 Marks)

- Q.2 a) Define regular expression ? Explain about the properties of regular expressions. (Refer Q.27 of Chapter - 1)
- b) Differentiate between top down and bottom up parsing techniques. (Refer Q.11 of Chapter - 2) [5+5]

OR

- Q.3 a) Define compiler ? Explain in brief about the syntax and semantic analysis of a compiler with an example ? (Refer Q.6 of Chapter - 1)
- b) Construct a predictive parsing table for the Grammar
 $E \rightarrow E+T/T, T \rightarrow T^*F/F, F \rightarrow (E)/id$. (Refer Q.26 of Chapter - 2) [5+5]

- Q.4 a) Construct SLR parsing table for the following grammar
 $E \rightarrow E+T/T, T \rightarrow T^*F/F, F \rightarrow (E)/id$. (Refer Q.44 of Chapter - 2)
- b) Discuss in brief about Yacc. (Refer Q.64 of Chapter - 2) [5+5]

OR

- Q.5 a) Construct CLR Parsing table for the grammar $S \rightarrow L=R/R, L \rightarrow ^*R/id, R \rightarrow L$ (Refer Q.55 of Chapter - 2)
- b) Define ambiguous grammar ? Check whether the grammar $S \rightarrow aAB$
 $A \rightarrow bC/cd, C \rightarrow cd, B \rightarrow c/d$ is ambiguous or not. (Refer Q.20 of Chapter - 2) [5+5]
- Q.6 a) Explain in detail about polymorphism. (Refer Q.47 of Chapter - 3)
- b) Explain in brief about heap storage allocation strategy. (Refer Q.3 of Chapter - 4) [5+5]

OR

- Q.7 a) Construct an annotated parse tree for $9-5+2$. (Refer Q.19 of Chapter - 3)
- b) Explain in brief about equivalence of type expressions. (Refer Q.44 of Chapter - 3) [5+5]
- Q.8 a) Explain in brief about different principal sources of optimization techniques with suitable examples. (Refer Q.6 of Chapter - 5)
- b) Define flow graph? Explain how a given program can be converted in to flow graph. (Refer Q.42 of Chapter - 4) [5+5]

OR

- Q.9 a) What is DAG ? Construct DAG for the following Basic block:
 $D := B^*C; E := A+B; B := B+C; A := E-D$; (Refer Q.49 of Chapter - 4)
- b) Explain how copy propagation can be done using data flow equation. (Refer Q.15 of Chapter - 5) [5+5]
- Q.10 Explain in detail the procedure that eliminates global common sub expression. (Not in new syllabus) [10]

OR

- Q.11 a) What are the object code forms ? Explain the issues in code generation. (Refer Q.34 of Chapter - 4)
- b) Explain about machine dependent code optimization. (Not in new syllabus) [5+5]

NOVEMBER - 2016**Compiler Design [R13]****Solved Paper**
III - I (B.Tech)
(Computer Science and Engineering)**Time : 3 Hours]****[Maximum Marks : 75]****Part - A (25 Marks)**

- Q.1**
- Write a brief note on bootstrap process. (Refer Q.8 of Chapter - 1) [2]
 - What are the differences between a compiler and an interpreter ? (Not in new syllabus) [3]
 - Give the specification of the YACC parser generator. (Refer Q.63 of Chapter - 2) [2]
 - Construct the LR(0) items for the "dangling-else" grammar. (Refer Q.45 of Chapter - 2) [3]
 - How to check structural equivalence of two type expressions ? (Refer Q.42 of Chapter - 3) [2]
 - Define and write the differences between synthesized attributes and inherited attributes. (Refer Q.4 of Chapter - 3) [3]
 - Write a short note on flow graph. (Refer Q.41 of Chapter - 4) [2]
 - Write an algorithm for constructing a basic block. (Refer Q.39 of Chapter - 4) [3]
 - Define various possible outputs of the code generator. (Refer Q.33 of Chapter - 4) [2]
 - Construct DAG for the following basic block : [3]

 $T1 = A+B$ $T2 = C+D$ $T3 = E - T2$ $T4 = T1 - T3$ (Refer Q.48 of Chapter - 4)**PART - B (50 Marks)**

- Q.2**
- Explain various error recovery strategies in lexical analysis. (Not in new syllabus)
 - Construct a finite automata and scanning algorithm for recognizing identifiers, numerical constants in C language. (Refer Q.29 of Chapter - 1) [5+5]

OR

- Q.3** Explain the various phases of a compiler with an illustrative example. (Refer Q.5 of Chapter - 1) [10]

- Q.4** Construct the LR parsing table for the following grammar : [10]

 $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid id$ (Refer Q.44 of Chapter - 2)**OR**

- Q.5**
- Write a YACC program that will take regular expression as input and produce its parse tree as output. (Refer Q.65 of Chapter - 2)

- Write an algorithm for computing LR(K) item sets. (Refer Q.56 of Chapter - 2) [5+5]

- Q.6 a) Write an SDT to convert infix to postfix expression. (Refer Q.18 of Chapter - 3)
 b) Explain briefly about polymorphic functions. (Refer Q.47 of Chapter - 3) [5+5]

OR

- Q.7 Explain various storage allocation strategies with its merits and demerits. (Refer Q.2 of Chapter - 4) [10]
 Q.8 Discuss various techniques of function preserving transformations for code optimization. (Refer Q.4 of Chapter - 5) [10]

OR

- Q.9 Explain how data flow equations are set up and solved for improving code. (Refer Q.14 of Chapter - 5) [10]
 Q.10 Explain the following peephole optimization techniques : [5+5]
 a) Elimination of redundant code
 b) Elimination of unreachable code. (Refer Q.69 of Chapter - 4)

OR

- Q.11 Explain in detail about machine dependent code optimization techniques with their drawbacks. (Not in new syllabus) [10]

MARCH - 2017
Compiler Design [R13]

Solved Paper
III - I (B.Tech)
(Computer Science and Engineering)

Time : 3 Hours]

[Maximum Marks : 75

Part - A (25 Marks)

- Q.1 a) Write regular expression over alphabet {a, b, c} containing at least one 'a' and at least one 'b' (Refer Q.23 of Chapter - 1) [2]
 b) What is input buffering? How is input buffering implemented ? (Refer Q.21 of Chapter - 1) [3]
 c) What is operator precedence grammar? Give an example. (Refer Q.36 of Chapter - 2) [2]
 d) What is significance of lookahead operator in LR parsing ? (Refer Q.39 of Chapter - 2) [3]
 e) What is the s-attributes and l-attributes ? (Refer Q.20 of Chapter - 3) [2]
 f) What is activation record ? (Refer Q.6 of Chapter - 4) [3]
 g) What is dead code elimination and reduction in strength ? (Refer Q.5 of Chapter - 5) [2]
 h) Define loop unrolling. Give an example. (Refer Q.9 of Chapter - 5) [3]
 i) What is meant by register descriptor and address descriptor ? (Refer Q.57 of Chapter - 4) [2]
 j) How to allocate registers to instruction ? (Refer Q.70 of Chapter - 4) [3]

PART - B (50 Marks)

- Q.2 a) Explain the concept of bootstrapping with example. (Refer Q.9 of Chapter - 1)

- b) Consider the following Conditional statement :

if ($x > 3$) then $y = 5$ else $y = 10$;

How does lexical analyzer help the above statement in process of compilation ?

(Refer Q.20 of Chapter - 1)

[4+6]

OR

- Q.3 Construct predictive parsing table for the following grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S$ (Refer Q.27 of Chapter - 2)

[10]

- Q.4 Find the LR (0) set of items for the following grammar. Describe state diagram and construct parse table of that

$S \rightarrow CC$

$C \rightarrow cC \mid d$ (Refer Q.46 of Chapter - 2)

[10]

OR

- Q.5 a) Write a procedure to construct LALR parsing table. (Refer Q.57 of Chapter - 2)

- b) Write short notes on YACC. (Refer Q.64 of Chapter - 2)

[5+5]

- Q.6 What is symbol table? Discuss various ways to organizing symbol table. (Not in new syllabus)

[10]

OR

- Q.7 Translate the following expression :

$(a + b) * (c + d) + (a + b + c)$ into

a) Quadruples b) Triples c) Indirect triples (Refer Q.36 of Chapter - 3)

[3+3+4]

- Q.8 a) What is liveness ? Explain liveness with suitable example. (Refer Q.13 of Chapter - 5)

- b) Write a procedure to identify basic blocks. (Refer Q.39 of Chapter - 4)

[5+5]

OR

- Q.9 Illustrate loop optimization with suitable example. (Refer Q.7 of Chapter - 5)

[10]

- Q.10 Explain various method to handle peephole optimization. (Refer Q.67 of Chapter - 4)

[10]

OR

- Q.11 Generate the code for the following expression: $x = (a + b) - ((c + d) - e)$. Also Compute its cost. (Refer Q.58 of Chapter - 4)

[10]

DECEMBER - 2017**Compiler Design [R15]****Solved Paper****III - I (B.Tech)****(Computer Science and Engineering)****Time : 3 Hours]****[Maximum Marks : 75****Note :** This question paper contain two parts A and B.

Part A is compulsory which carries 25 marks. Answer all questions in Part A. Part B consists of 5 units . Answer any one full question from each unit. Each question carries 10 marks and may have a, b, c as sub questions.

Part - A (25 Marks)

- Q.1 a) What are the features of a Lexical analyser ? (Refer Q.17 of Chapter - 1) [2]
 b) Explain in brief about left most and right most derivations. (Refer Q.5 of Chapter - 2) [3]
 c) Define left recursive grammar. (Refer Q.14 of Chapter - 2) [2]
 d) List out the rules for FIRST and FOLLOW. (Refer Q.24 of Chapter - 2) [3]
 e) What are the advantages of heap storage allocation ? (Refer Q.4 of Chapter - 4) [2]
 f) Define Type Equivalence. (Refer Q.41 of Chapter - 3) [3]
 g) What is algebraic transformation ? (Refer Q.68 of Chapter - 4) [2]
 h) Write in brief about flow graphs. (Refer Q.41 of Chapter - 4) [3]
 i) What are the forms of a target program ? (Refer Q.33 of Chapter - 4) [2]
 j) What is machine independent code optimization ? (Refer Q.1 of Chapter - 5) [3]

PART - B (50 Marks)

- Q.2 a) Define regular expression. Write about the identity rules for regular expressions. (Refer Q.22 of Chapter - 1)
 b) Discuss in brief about left recursion and left factoring with examples. (Refer Q.16 of Chapter - 2) [5+5]

OR

- Q.3 a) Define compiler. Explain in brief about the LEX compiler. (Refer Q.37 of Chapter - 1)
 b) Construct FIRST and FOLLOW for the Grammar :
 $E \rightarrow E+T/T, T \rightarrow T^*F/F, F \rightarrow (E)/id$ (Refer Q.25 of Chapter - 2) [5+5]
- Q.4 a) Construct SLR parsing table for the following grammar.
 $E \rightarrow E+T/T, T \rightarrow T^*F/F, F \rightarrow (E)/id$ (Refer Q.44 of Chapter - 2)
 b) Discuss in brief about YACC. (Refer Q.64 of Chapter - 2) [5+5]

OR

- Q.5 a) Construct CLR parsing table for the following grammar.
 $E \rightarrow E+T/T, T \rightarrow T^*F/F, F \rightarrow (E)/id$. (Refer Q.53 of Chapter - 2)
 b) Discuss in brief about error recovery strategies in Parsing. (Not in new syllabus) [5+5]
- Q.6 a) Explain in brief about Type checking and Type Conversion. (Refer Q.46 of Chapter - 3)

- b) Define Symbol table. Explain about the data structures used for Symbol table. (Not in new syllabus) [5+5]

OR

- Q.7 a) Explain in brief about Stack Storage allocation strategy ? (Refer Q.5 of Chapter - 4)
 b) Define activation record ? Explain in brief about the fields in activation record. (Refer Q.6 of Chapter - 4) [5+5]
- Q.8 What is DAG? Construct DAG for the following Basic block ?
 $D := B * C$; $E := A + B$; $B := B + C$; $A := E - D$; (Refer Q.49 of Chapter - 4) [10]

OR

- Q.9 Explain how copy propagation can be done using data flow equation ? (Refer Q.15 of Chapter - 5) [10]
- Q.10 Define loop optimization? Describe in brief about the transformation on basic blocks. (Refer Q.8 of Chapter - 5) [10]

OR

- Q.11 a) Explain reducible and non reducible flow graphs with examples. (Refer Q.22 of Chapter - 5)
 b) Discuss about Instruction Selection and Register allocation. (Refer Q.32 of Chapter - 4) [5+5]

MAY - 2018
Compiler Design [R15]

Solved Paper
III - I (B.Tech)
(Computer Science and Engineering)

Time : 3 Hours]

[Maximum Marks : 75

Part - A (25 Marks)

- Q.1 a) What are the two parts of a compilation ? Explain briefly. (Refer Q.4 of Chapter - 1) [2]
 b) Define a context free grammar. (Refer Q.3 of Chapter - 2) [3]
 c) List the properties of LR parser. (Refer Q.38 of Chapter - 2) [2]
 d) Write short notes on YACC. (Refer Q.64 of Chapter - 2) [3]
 e) What are the various types of intermediate code representation ? (Refer Q.32 of Chapter - 3) [2]
 f) Give the format of symbol table (Not in new syllabus) [3]
 g) List the terminologies used in basic blocks. (Refer Q.37 of Chapter - 4) [2]
 h) What is a flow graph ? (Refer Q.41 of Chapter - 4) [3]
 i) Mention the properties that a code generator should possess. (Refer Q.30 of Chapter - 4) [2]
 j) What is a DAG ? Mention its applications. (Refer Q.43 of Chapter - 4) [3]

PART - B (50 Marks)

- Q.2 Explain in detail about the role of lexical analyzer with the possible error recovery actions. (Refer Q.18 of Chapter - 1) [10]

OR

- Q.3 Construct Predictive parsing table for the following grammar : the necessary algorithm.
 $S \rightarrow (L) / a$
 $L \rightarrow L, S / S$ and check whether the (a, a) belong to that grammar or not. (Refer Q.27 of Chapter - 2) [10]
- Q.4 Give the LALR parsing table for the grammar.
 $S \rightarrow L = R / R \quad L \rightarrow * R / id \quad R \rightarrow L$ (Refer Q.54 of Chapter - 2) [10]

OR

- Q.5 Compare and contrast between SLR, LALR and LR parses. (Refer Q.61 of Chapter - 2) [10]
- Q.6 How would you generate the intermediate code for the flow of control statements ? Explain with examples. (Refer Q.49 of Chapter - 3) [10]

OR

- Q.7 Explain how the types and relative addresses of declared names are computed and how scope information is dealt with. (Refer Q.45 of Chapter - 3) [10]
- Q.8 Write about Data flow analysis of structural programs. (Refer Q.12 of Chapter - 5) [10]

OR

- Q.9 Explain the principle sources of optimization in detail. (Refer Q.6 of Chapter - 5) [10]
- Q.10 Give an example to explain in detail about machine dependent code optimization. (Not in new syllabus) [10]

OR

- Q.11 Describe how DAG can be used in register allocation process? Give examples. (Refer Q.51 of Chapter - 4) [10]

END...✍

SOLVED MODEL QUESTION PAPER

Compiler Design

Solved Paper
B.Tech, III-II (CSE/IT)

Time : 3 hours]

[Max. Marks : 75

Note : This question paper contains two parts A and B.

Part A is compulsory which carries 25 marks. Answer all questions in Part A. Part B consists of 5 Units. Answer any one full question from each unit. Each question carries 10 marks and may have a, b, c as sub questions.

PART - A (25 Marks)

- Q.1 a) What is compiler? (Refer Q.3 of Chapter 1) [2]
b) What is input buffering? How it is implemented. (Refer Q.21 of Chapter 1) [3]
c) Define left recursive grammar. (Refer Q.14 of Chapter 2) [2]
d) Give specification of the YACC parser generator. (Refer Q.63 of Chapter 2) [3]
e) What is attributed grammar? (Refer Q.3 of Chapter 3) [2]
f) What is heap management? (Refer Q.15 of Chapter 4) [3]
g) What is run time storage? (Refer Q.1 of Chapter 4) [2]
h) What is machine independent code optimization. (Refer Q.1 of Chapter 5) [3]
i) What are advantages of code optimization ? (Refer Q.3 of Chapter 5) [2]
j) What is algebraic transformation. (Refer Q.68 of Chapter 4) [3]

PART - B (50 Marks)

- Q.2 a) What is pass of a compiler? How compiler works in passes ? (Refer Q.12 of Chapter 1) [5]
b) Construct the NFA for following Regular Expression using Thompson's construction. Apply subset construction method to convert into DFA. $(a+b)^*abb$. (Refer Q.32 of Chapter 1) [5]

OR

- Q.3 Verify the following grammar is LL(1) :

$S \rightarrow S\#$

$S \rightarrow A|B$

$A \rightarrow a|e$

$B \rightarrow b|e$

Give a trace of the parser for each of the following input strings : a) $aab\#$ b) $ccd\#$
(Refer Q.31 of Chapter 2)

[10]

- Q.4 Show that the following grammar

$S \rightarrow Aa|bAc|Bc|bBa$

$A \rightarrow d$

$B \rightarrow d$

Is LR(1) but not LALR(1) (Refer Q.59 of Chapter 2)

[10]

OR

- Q.5 Compare and contrast SLR, LALR and LR parsers. (Refer Q.61 of Chapter 2) [10]
- Q.6 Give syntax directed translation scheme for simple desk calculator. (Refer Q.11 of Chapter 3) [10]

OR

- Q.7 a) Explain memory hierarchy in detail. (Refer Q.18 of Chapter 4) [5]
- b) Explain in detail, the various strategies for reducing fragmentation in heap memory. (Refer Q.21 of Chapter 4) [10]
- Q.8 Generate the code sequence using code generation algorithm for following expression $W = (A-B)+(A-C)+(A-C)$ (Refer Q.61 of Chapter 4) [10]

OR

- Q.9 a) Show the DAG for statement $Z = X-Y+X*Y*U-V/W+X+V$. (Refer Q.55 of Chapter 4) [5]
- b) Explain peephole optimization technique. (Refer Q.67 of Chapter 4) [5]
- Q.10 Explain various code optimization techniques. (Refer Q.4 of Chapter 5) [10]

OR

- Q.11 Write about data flow analysis of structural programs. (Refer Q.12 of Chapter 5) [10]

END...✍

MAY - 2019
Compiler Design [R16] - 136AQ

Solved Paper
(B.Tech) III - II
(CSE / IT)

[Maximum Marks : 75]

Time : 3 Hours]

Note : This question paper contains two parts A and B. Part A is compulsory which carries 25 marks. Answer all questions in Part A. Part B consists of 5 units. Answer any one full question from each unit. Each question carries 10 marks and may have a, b, c as sub questions.

Part - A

(25 Marks)

- Q.1**
- a) Define regular expression. (Refer Q.22 of Chapter - 1) [2]
 - b) Define linker and loader and explain briefly. (Refer Q.2 of Chapter - 1) [3]
 - c) Define ambiguous grammar. (Refer Q.20 of Chapter - 2) [2]
 - d) Compare SLR, CLR and LACR. (Refer Q.61 of Chapter - 2) [3]
 - e) What is coercion ? (Refer Q.46 of Chapter - 3) [2]
 - f) How to find evaluation order for SDD's ? (Refer Q.8 of Chapter - 3) [3]
 - g) What are the limitations of static allocation ? (Refer Q.2 of Chapter - 4) [2]
 - h) Write the fields and uses of symbol table. [3]

Ans. : Symbol table consists of entries for various entities such as variable name, function name, class name and so on. Various fields of symbol table are

- | | |
|-------------------|-----------------------|
| 1) Name of entity | 2) Type of entity |
| 3) Scope | 4) Location in memory |

- i) What is common sub-expression elimination ? Explain. (Refer Q.4 of Chapter - 5) [2]
- j) What are induction variables ? What is induction variable elimination ? [3]

Ans. : Induction variable is a variable which is increased or decreased by fixed amount on every iteration of a loop.

It is in the form

$$i = i \pm c$$

Induction variable can be eliminated by either moving some constant expressions outside the loop or by using strength reduction.

Part - B

(50 Marks)

- Q.2**
- a) Explain the procedure to convert regular expression to finite automata. (Refer Q.30 of Chapter - 1)
 - b) Explain various phases in the construction of compiler with a neat sketch, (Refer Q.5 of Chapter - 1) [5+5]

OR

- Q.3**
- a) What is the functionality of preprocessing and input buffering ?

Ans. : Functionality of Preprocessing : The tasks performed by the preprocessors are given as below

- (1) Preprocessors allow user to use macros in the program. Macro means some set of instructions which can be used repeatedly in the program. Thus macro preprocessing task is to be done by preprocessors.
- (2) Preprocessor also allows user to include the header files which may be required by the program.

For example :

```
#include <stdio.h>
```

By this statement the header file `stdio.h` can be included and user can make use of the functions defined in this header file. This task of preprocessor is called file inclusion.

Input Buffering: Refer Q.21 of chapter 1

b) Explain compiler construction tools.

[5 + 5]

Ans. : There are some specialized tools for helping in implementation of various phases of compilers. These tools are called **compiler construction tools**. These tools are also called as compiler-compiler, compiler-generators, or translator writing system. Various compiler construction tools are given as below -

1. **Scanner generator** - These generators generate lexical analyzers. The specification given to these generators are in the form of regular expressions.
The UNIX has utility for a scanner generator called LEX. The specification given to the LEX consists of regular expressions for representing various tokens.
2. **Parser generators** - These produce the syntax analyzer. The specification given to these generators is given in the form of context free grammar. Typically UNIX has a tool called YACC which is a parser generator.
3. **Syntax-directed translation engines** - In this tool the parse tree is scanned completely to generate an intermediate code. The translation is done for each node of the tree.
4. **Automatic code generator** - These generators take an intermediate code as input and converts each rule of intermediate language into equivalent machine language. The template matching technique is used. The intermediate code statements are replaced templates that represent the corresponding sequence of machine instructions.
5. **Data flow engines** - The data flow analysis is required to perform good code optimization. The data flow engines are basically useful in code optimization.

Q.4 a) What is left recursion ? Describe the algorithm used for eliminating left recursion ?

(Refer Q.14 of Chapter - 2)

b) Eliminate left recursion in the following :

$E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$ (Refer Q.15 of Chapter - 2)

[5 + 5]

OR

Q.5 a) What is ambiguous grammar ? Show that following grammar is ambiguous or not.

$A \rightarrow A + A \mid A - A \mid A * A \mid a$ (Refer Q.21 of Chapter - 2)

b) Verify whether the following grammar is LL(1) or not ?

$E \rightarrow E + T \mid T$

$$T \rightarrow T * F / F$$

$$F \rightarrow (F) \mid a \mid b \text{ (Refer Q.26 of Chapter - 2)}$$

[5 + 5]

Q.6 a) What are three address codes? Explain different types of representations of three address code.
(Refer Q.31 and Q.33 of Chapter - 3)

b) Write three codes for $x := A[y, z]$

[5 + 5]

Ans. : The formula for computing address of $A[y, z] = 4 * (20y + z) + (\text{base}_A - 84)$. We assume array size is 10×20 . The three address code for given expression $x := A[y, z]$ is,

$$t_1 := y * 20$$

$$t_1 := t_1 + z$$

$$t_1 := c / * c = \text{base}_A - 84 */$$

$$t_3 := 4 * t_1$$

$$t_4 := t_2[t_3]$$

$$x := t_4$$

OR

Q.7 a) What is type checker? Explain the specification of a simple type checker.
(Refer Q.38 and Q.39 of Chapter - 3)

b) Explain translation schema for array elements.

[5 + 5]

Ans. : The translation scheme for generating three address code is given by using appropriate semantic actions.

Rule No.	Production rule	Semantic rule
1)	$S \rightarrow L := E$	{ if L.offset := NULL /* L is id only */ then append(L.place ':=' E.place) else append(L.place '[' L.offset ']':='E.place) }
2)	$E \rightarrow E_1 + E_2$	{ E.place := newtemp(); append(E.place ':=' E ₁ .place '+' E ₂ .place) }
3)	$E \rightarrow (E_1)$	{ E.place := E ₁ .place }
4)	$E \rightarrow L$	{ if L.offset = NULL then E.place := L.place else { E.place := newtemp() append(E.place ':=' L.place '[' L.offset ']') } }



5)	$L \rightarrow \text{List }]$	{ L.place := newtemp() L.offset := newtemp() append(L.place ':=' c(List.array)); append(L.offset ':=' List.place '*' Size(List.array)) }
6)	$L \rightarrow \text{id}$	{ L.place := id.place; L.offset := NULL }
7)	$\text{List} \rightarrow \text{List}_1, E$	{ t := newtemp() dim := List ₁ .ndim + 1; append(t ':=' List ₁ .place '*' Limit(List ₁ .array, dim)); append(t ':=' t '+' E.place) List.array := List ₁ .array; List.place := t; List.ndim := dim; }
8)	$\text{List} \rightarrow \text{id } [E$	{ List.array := id.place List.place := E.place List.ndim := 1 }

Q.8 a) Explain about heap management. (Refer Q.15 and Q.16 of Chapter - 4)

b) Define reference counting. What is the role of reference counting in garbage collection ?

[5 + 5]

Ans. : Refer Q.27 of chapter - 4.

- The reference counting uses a simple algorithm to keep track of each object for total number of references.
- There exists a special field for each object called a reference count which holds the reference count of this object.
- For example

```
/* step 1 */
Objectobj1 = newInteger (10);
Objectobj2 = newInteger (20);
/* step 2 */
obj1 = obj2;
```

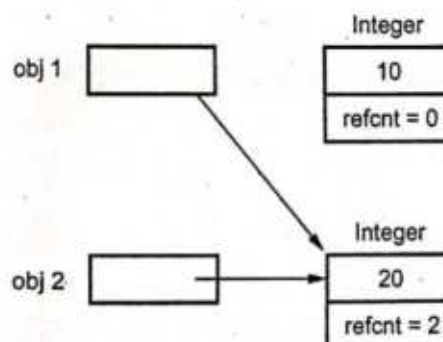
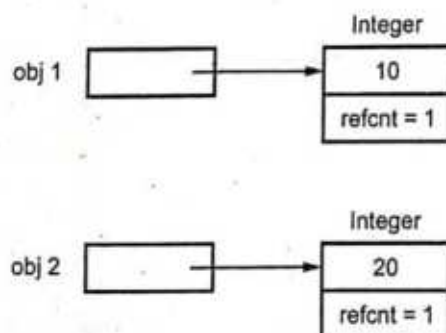


Fig. 1

In step 2 both obj1 and obj2 refers to same object. The reference count should be two.

- Note that the memory release method will be invoked immediately when reference count of an object goes to zero, i.e. when it becomes garbage. In this way garbage may be collected incrementally as it is created.
- Thus reference counting method for garbage collection is relatively simple. Garbage is easily identified if reference count is zero, the object is considered to be garbage.

OR

Q.9 a) Give the detailed description on DAG. (Refer Q.43 and Q.45 of Chapter - 4)

b) Explain different methods for register allocation and assignment. (Refer Q.70 of Chapter - 4)

[5 + 5]

Q.10 a) Explain redundancy elimination techniques. (Refer Q.67 of Chapter - 4)

b) Write the principal sources of optimization. (Refer Q.4 of Chapter - 5)

[5 + 5]

OR

Q.11 a) Explain loop optimization technique with example. (Refer Q.7 of Chapter - 5)

b) Explain constant propagation with example. (Refer Q.4 of Chapter - 5)

[5 + 5]

END.../

DECEMBER - 2019
Compiler Design [R16] - 136AQ

Solved Paper
(B.Tech) III - II
(CSE / IT)

Time : 3 Hours]

[Maximum Marks : 75

Note : This question paper contains two parts A and B.
Part A is compulsory which carries 25 marks. Answer all questions in Part A. Part B consists of 5 units . Answer any one full question from each unit. Each question carries 10 marks and may have a, b, c as sub questions.

Part - A

(25 Marks)

Q.1 a) What is language processor ?

[2]

Ans. : Language processor is basically type of translator that converts high level source program into machine level object program.

b) Differentiate between pass one and pass two of a compiler. (Refer Q.14 of Chapter - 1)

[3]

c) Define context free grammar. (Refer Q.3 of Chapter - 2)

[2]

d) What do you mean by ambiguous grammar ? Give example. (Refer Q.20, Q.21 of Chapter - 2)

[3]

e) Convert the expression $a = b^* - c + b^* - c$ into three address statements. (Refer Q.34 of Chapter - 3)

[2]

f) What is an attribute ? Explain different types of attribute ? (Refer Q.3 of Chapter - 3)

[3]

g) List the different storage allocation strategies. (Refer Q.2 of Chapter - 4)

[2]

h) What are the sub problems in register allocation strategies ?

[3]

Ans. : There are two subproblems associated with register allocation strategies -

(1) Register Allocation : It is process of deciding which values to be kept in registers.

(2) Register Assignment : It is an activity of choosing specific registers for storing the values.

i) Give the applications of DAG. (Refer Q.43 of Chapter - 4)

[2]

j) What is the role of flow graph optimizing a compiler ?

[3]

Ans. :

- The flow graph contains the control information for the set of basic blocks. The flow graph basically represents how the program control is getting parsed among the blocks.
- It is useful in loop optimization.

Part - B

(50 Marks)

Q.2 With a neat diagram explain the phases of a compiler in detail. (Refer Q.5 of Chapter - 1)

[10]

OR

Q.3 Give an example to design an automaton from a given regular expression. (Refer Q.30 of Chapter - 1)

[10]

Q.4 Test whether the grammar is LL(1) or not and construct a predictive parsing table for it :

$S \rightarrow iEtS \mid iEtSeS \mid a \quad E \rightarrow b.$ (Refer Q.29 of Chapter - 2)

[10]

- Q.5** Construct an LALR parsing table for the following grammar : [10]
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$ (Refer Q.53 of Chapter - 2)
- Q.6** Write syntax directed definition for flow of control statements. (Refer Q.49 of Chapter - 3) [10]
- OR
- Q.7** Write the three address code sequence for the $d := (a-b) + (a-c) + (a-c)$. (Refer Q.61 of Chapter - 4) [10]
- Q.8** Give example to explain peephole optimization technique in detail. (Refer Q.67 of Chapter - 4) [10]
- OR
- Q.9** Write in detail about the issues in the design of code generator. (Refer Q.31 of Chapter - 4) [10]
- Q.10** With an example explain common sub expression and dead code elimination methods in detail. (Refer Q.4 of Chapter - 5) [10]
- OR
- Q.11** What is DAG and flowgraph ? Explain their role in compilation process (Refer Q.41 and Q.43 of Chapter - 4) [10]

DECEMBER - 2020

Compiler Design [R16] - 136AQ

Solved Paper
(B.Tech) III - II
(CSE / IT)

Time : 2 Hours]

[Maximum Marks : 75

Answer any five questions

All questions carry equal marks

- Q.1 a)** What are functions of lexical analyzer ? (Refer Q.16 of Chapter - 1)
- b)** Construct DFA for the regular expression $(a+b)^* aa + aba^*$. [7+8]

Ans. : $RE = (a+b)^* aa + aba^*$

We will first construct NFA for above RE using direct method.

Step 1 : $R.E. = r_1 + r_2 = (a+b)^* aa + aba^*$

The FA for r_1 is as follows -

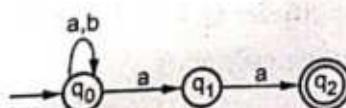


Fig. 1

The FA for r_2 is as follows -

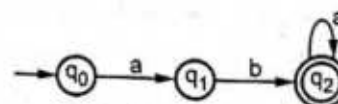


Fig. 2

Step 2 : We will combine both the above designed FAs.

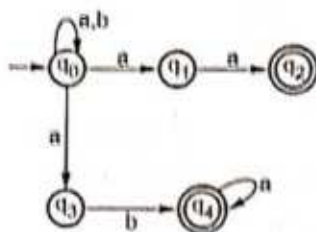


Fig. 3

Step 4 : We will now convert above NFA to DFA. For that purpose, we will first draw the transition table. It is as follows -

States \ I/p	a	b
→ [q ₀]	{q ₀ , q ₁ , q ₃ }	{q ₀ }
[q ₁]	{q ₂ }	φ
* [q ₂]	φ	φ
[q ₃]	φ	{q ₄ }
* [q ₄]	{q ₄ }	φ

Here → denotes start state and * denotes final state.

Step 5 :

$$\delta(q_0, a) = \{q_0, q_1, q_3\} = [q_0, q_1, q_3] \rightarrow \text{New state}$$

$$\delta(q_0, b) = [q_0]$$

$$\delta([q_0, q_1, q_3], a) = \{q_0, q_1, q_2, q_3\} = [q_0, q_1, q_2, q_3] \rightarrow \text{New state}$$

$$\delta([q_0, q_1, q_3], b) = \{q_0, q_4\} = [q_0, q_4] \rightarrow \text{New state}$$

$$\delta([q_0, q_1, q_2, q_3], a) = [q_0, q_1, q_2, q_3]$$

$$\delta([q_0, q_1, q_2, q_3], b) = [q_0, q_4]$$

$$\delta([q_0, q_4], a) = \{q_0, q_1, q_3, q_4\} = [q_0, q_1, q_3, q_4] \rightarrow \text{New state}$$

$$\delta([q_0, q_4], b) = [q_0]$$

$$\delta([q_0, q_1, q_3, q_4], a) = \{q_0, q_1, q_2, q_3, q_4\}$$

$$= [q_0, q_1, q_2, q_3, q_4] \rightarrow \text{New state}$$

$$\delta([q_0, q_1, q_3, q_4], b) = [q_0, q_4]$$

$$\delta([q_0, q_1, q_2, q_3, q_4], a) = [q_0, q_1, q_2, q_3, q_4]$$

$$\delta([q_0, q_1, q_2, q_3, q_4], b) = [q_0, q_4]$$

Step 6 : Now we will construct transition table and transition diagram from above computations

States \ i/p		
	a	b
→ [q ₀]	[q ₀ , q ₁ , q ₃]	[q ₀]
[q ₀ , q ₁ , q ₃]	[q ₀ , q ₁ , q ₂ , q ₃]	[q ₀ , q ₄]
* [q ₀ , q ₁ , q ₂ , q ₃]	[q ₀ , q ₁ , q ₂ , q ₃]	[q ₀ , q ₄]
* [q ₀ , q ₄]	[q ₀ , q ₁ , q ₃ , q ₄]	[q ₀]
* [q ₀ , q ₁ , q ₃ , q ₄]	[q ₀ , q ₁ , q ₂ , q ₃ , q ₄]	[q ₀ , q ₄]
* [q ₀ , q ₁ , q ₂ , q ₃ , q ₄]	[q ₀ , q ₁ , q ₂ , q ₃ , q ₄]	[q ₀ , q ₄]

Now we rename the states as follows

$$[q_0] = A, [q_0, q_1, q_3] = B, [q_0, q_1, q_2, q_3] = C$$

$$[q_0, q_4] = D, [q_0, q_1, q_3, q_4] = E, [q_0, q_1, q_2, q_3, q_4] = F$$

By observing above table, clearly we can merge some equivalent states. For instance -

$$[q_0, q_1, q_3, q_4] = [q_0, q_1, q_2, q_3, q_4] \text{ i.e. } E = F$$

The minimized DFA is -

	a	b
→ A	B	A
B	C	D
* C	C	D
* D	F	A
* F	F	D

The DFA is -

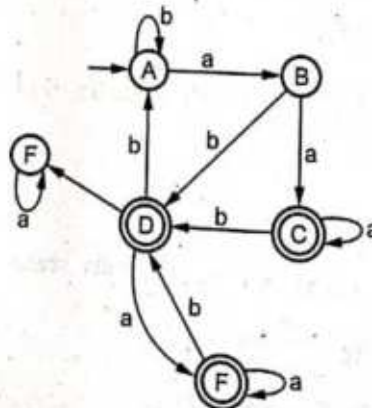


Fig. 4

- Q.2 a) Explain the concept of input buffering. (Refer Q.21 of Chapter - 1)
 b) Construct a finite automata for $(00)^*(11)^*$.

[7+8]

Ans. :

Step 1 : $r = (00)^*(11)^*$
 Let $r = r_1 r_2$
 where $r_1 = (00)^*$
 $r_2 = (11)^*$

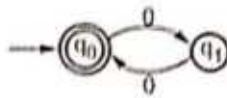
The FA for r_1 

Fig. 5

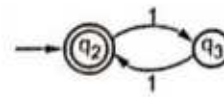
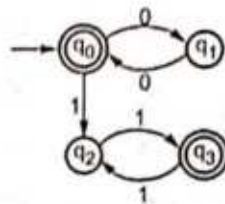
The FA for r_2 

Fig. 6

Step 2 : We will combine both FA's in a single FA



This is the required FA

Fig. 7

- Q.3 Construct LL(1) parsing table for the following grammar
 $S \rightarrow CC$
 $C \rightarrow cC \mid d$, where S is start variable, $V = \{S, C\}$, $T = \{c, d\}$.
 Write and apply LL(1) parsing algorithm to parse the string $cdcd$.

[15]

Ans. : Let the grammar be -

 $S \rightarrow CC$
 $C \rightarrow cC \mid d$

We will first compute FIRST and FOLLOW of non terminals

FIRST(S) = {c,d}

FOLLOW(S) = {\$}

FIRST(C) = {c,d}

FOLLOW(C) = {c,d}

The predictive parsing table can be constructed as follows -

	c	d	\$
S	$S \rightarrow CC$	$S \rightarrow CC$	
C	$C \rightarrow cC$	$C \rightarrow d$	

The input can be processed with the help of above parsing table. This processing is as shown below -
Consider input "cdcd"

State	Input	Actions
\$S	cdcd\$	
\$CC	cdcd\$	$S \rightarrow CC$
\$CCc	cdcd\$	$C \rightarrow cC$
\$CC	dcd\$	
\$Cd	dcd\$	$C \rightarrow d$
\$C	cd\$	
\$Cc	cd\$	$C \rightarrow cC$
\$C	d\$	$C \rightarrow d$
\$d	d\$	
\$	\$	ACCEPT

Q.4 What is shift - reduce conflict ? Identify different conflicts in the SLR parsing table of the grammar after constructing parsing table, where S is start variable, $V = \{S, A, B\}$, $T = \{a, b, c, d\}$.

$S \rightarrow Aa|bAc|Bc|bBa$

$A \rightarrow d$

$B \rightarrow d$

Ans. : Shift Reduce Conflict : Refer Q.33 of Chapter 2

[15]

Let the grammar be -

$S \rightarrow Aa|bAc|Bc|bBa$

$A \rightarrow d$

$A \rightarrow d$

Step 1 : Now we will first construct the canonical set of items for above grammar. This set is as given below -

$I_0 :$ $S' \rightarrow \bullet S$ $S \rightarrow \bullet Aa$ $S \rightarrow \bullet bAc$ $S \rightarrow \bullet Bc$ $S \rightarrow \bullet bBa$ $A \rightarrow \bullet d$ $B \rightarrow \bullet d$	$I_6 : \text{goto}(I_2, a)$ $S \rightarrow Aa \bullet$ $I_7 : \text{goto}(I_3, A)$ $S \rightarrow bA \bullet c$ $I_8 : \text{goto}(I_3, B)$ $S \rightarrow bB \bullet a$ $I_9 : \text{goto}(I_3, d)$ $A \rightarrow d \bullet$ $B \rightarrow d \bullet$
$I_1 : \text{goto}(I_0, S)$ $S' \rightarrow \bullet S$ $I_2 : \text{goto}(I_0, A)$ $S \rightarrow A \bullet a$ $I_3 : \text{goto}(I_0, b)$ $S \rightarrow b \bullet Ac$ $S \rightarrow b \bullet Ba$ $A \rightarrow \bullet d$ $B \rightarrow \bullet d$ $I_4 : \text{goto}(I_0, B)$ $S \rightarrow B \bullet c$ $I_5 : \text{goto}(I_0, d)$ $A \rightarrow d \bullet$ $B \rightarrow d \bullet$	$I_9 : \text{goto}(I_4, c)$ $S \rightarrow Bc \bullet$ $I_{10} : \text{goto}(I_7, c)$ $S \rightarrow bAc \bullet$ $I_{11} : \text{goto}(I_8, a)$ $S \rightarrow bBa \bullet$

This is same as I_9 .
So we will not consider this state.

Step 2 : We will now find the FOLLOW values for all the non terminals -

FOLLOW(S) = {\$}

FOLLOW(A) = {a, c}

FOLLOW(B) = {a, c}

Step 3 : Let us rewrite the grammar rules by numbering them out.

(1) $S \rightarrow Aa$ (2) $S \rightarrow bAc$ (3) $S \rightarrow Bc$ (4) $S \rightarrow bBa$ (5) $A \rightarrow d$ (6) $A \rightarrow d$

Step 4 : Now we will construct the SLR Parsing table using above set of items. The table is as follows -

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0		S3		S5		1	2	4
1					Accept			
2	S6							
3				S5			7	8
4			S9					
5	R5,R6		R5,R6					
6					R1			
7			S10					
8	S11							
9					R3			
10					R2			
11					R4			

From, above table clearly there is reduce-reduce conflict for [15,a] and [15,c] states.

Q.5 a) Briefly write about abstract syntax tree. (Refer Q.28 of Chapter - 3)

b) Perform quadruple, tripple, indirect triple implementation for the infix expression $(c+d)*(c-d)+(c+d)$. [7+8]

Ans. : Let the given expression is $(c+d)*(c-d)+(c+d)$

The Three Address Code is

$t1 = c+d$

$t2 = -d$

$t3 = c+t2$

$t4 = t1*t3$

$t5 = t4+t1$

Quadruple

	Operator	Operand1	Operand2	Result
(0)	+	c	d	t1
(1)	-	d		t2
(2)	+	c	t2	t3
(3)	*	t1	t3	t4
(4)	+	t4	t1	t5

Triple :

	Operator	Operand1	Operand2
(0)	+	c	d
(1)	-	d	
(2)	+	c	(1)
(3)	*	(0)	(2)
(4)	+	(3)	(0)

Indirect Triple :

	Operator	Operand1	Operand2
(0)	+	c	d
(1)	-	d	
(2)	+	c	(11)
(3)	*	(10)	(12)
(4)	+	(13)	(10)

	Statement
(0)	(10)
(1)	(11)
(2)	(12)
(3)	(13)
(4)	(14)

Q.6 a) Convert the following arithmetic expression into syntax tree and three address code $(a+b)*3*(a+b)$.

Ans. : Let the arithmetic expression is

$$(a+b)*3*(a+b)$$

The syntax tree is shown in Fig. 8.

The Three Address Code is

$$t1 = a+b$$

$$t2 = t1*3$$

$$t3 = a+b$$

$$t4 = t2*t3$$

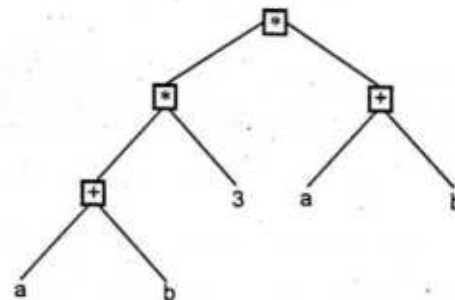


Fig. 8

b) Explain about inherited attributes and synthesized attributes. (Refer Q.3 of Chapter - 3)

[8+7]

Q.7 a) Explain the stack allocation strategy with example.

(Refer Q.6 and Q.8 of Chapter - 4)

b) Briefly explain different techniques for optimization of basic blocks.

(Refer Q.44 of Chapter - 4)

[7+8]

Q.8 Explain different principal sources of code optimization.

(Refer Q.4 of Chapter - 5)

[15]

END...